

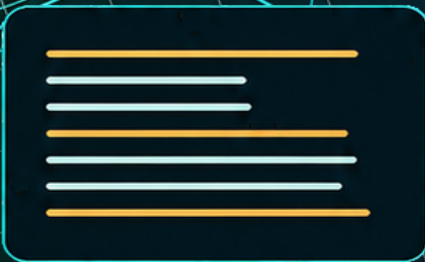
Go Essencial

Volume 1 — Fundamentos

Fundamentos da linguagem Go

Rudson Alves

Junho de 2026



Licença

Copyright (c) 2026 Rudson Alves.

Este livro utiliza licença dupla, separando o conteúdo editorial do código-fonte dos exemplos.

Conteúdo do livro

O conteúdo textual do livro, imagens, diagramas, arquivos Markdown, PDFs e demais materiais editoriais são licenciados sob a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0).

Você pode copiar, compartilhar e adaptar este material, desde que:

- atribua a autoria a Rudson Alves;
- não utilize o material para fins comerciais sem autorização prévia;
- distribua obras derivadas sob a mesma licença.

Texto oficial da licença:

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Código-fonte dos exemplos

Os exemplos de código em Go localizados em `vol_1-fundamentos/codigo` são licenciados sob a licença MIT.

O texto completo da licença MIT aplicada aos exemplos está disponível em:

`vol_1-fundamentos/codigo/LICENSE`

Agradecimentos

Meu agradecimento aos meus pais, Maria José R. Alves e João Walmiro Alves (†1998), que, mesmo com recursos modestos, nunca deixaram faltar apoio à minha formação. Deles recebi a oportunidade de estudar e construir o caminho que me levou ao mestrado na UNICAMP, em 1991. Ambos me ensinaram, pelo exemplo, o valor da dedicação e do trabalho bem feito. Em meu pai, porém, essas qualidades se manifestavam de forma extraordinária, e seu perfeccionismo tornou-se uma inspiração permanente para minha vida e minha carreira.

Meus sinceros agradecimentos à minha saudosa ex-esposa, Jeâni Daniela Bortolini (†2025), que compartilhou comigo anos decisivos de minha formação e de minha vida acadêmica. Minha eterna gratidão pelo trabalho extraordinário que realizou na educação de nosso querido filho, Gabriel Bortolini Alves, e pelos inesquecíveis momentos que compartilhamos como família.

À minha amada esposa, Juliana do E. S. Boasquevisque, por transformar minha vida com seu amor, sua força e sua presença constante. Foi ao seu lado que reencontrei a vontade de viver, de aceitar novos desafios e de construir novos objetivos. Seu apoio incondicional me permitiu permanecer de pé nos momentos mais difíceis e nunca desistir de seguir em frente. Este livro também é fruto da inspiração que encontro diariamente em você.

Apresentação

Por volta de 2022, iniciei minha migração da docência em física para a área de desenvolvimento de software. Trazia comigo muitos anos de estudo e ensino de cálculo, métodos matemáticos, fundamentos da física e física moderna, e essa formação moldou profundamente minha maneira de pensar, estudar e organizar conhecimento.

Sempre aprendi melhor lendo e escrevendo. Essa foi, desde cedo, minha forma mais natural de absorver ideias, reconstruí-las e organizar conhecimento. Como estudante, nunca gostei muito de assistir aulas; achava o ritmo lento e, muitas vezes, repetitivo demais. Ainda assim, a sala de aula acabou sendo meu destino por 23 anos, em cursos de engenharia, ciência da computação e acústica aplicada. Com o tempo, lecionar me mostrou que a repetição nem sempre é defeito: muitas vezes, ela é uma necessidade da própria dinâmica de sala. Alunos diferentes chegam aos conceitos por caminhos diferentes, e retornar ao mesmo ponto por outros ângulos treina o professor a apresentar o conteúdo de formas mais variadas, em vez de apenas repetir o que já sabe.

Quando comecei a estudar Go, em 2022, registrei minhas primeiras descobertas em uma série de artigos em meu blog: <https://rralves.dev.br/series/golang/>. Eram textos de estudo, mais próximos de anotações e experimentos pessoais do que de um material planejado como percurso progressivo pela linguagem. Na mesma época, Leomar Viegas Jr. (<https://www.linkedin.com/in/leomarviegas/>) me convidou para apresentar Go a alguns estagiários de sua empresa. A preparação dessas aulas reforçou meu contato com a linguagem; os arquivos daquele material se perderam, mas a experiência permaneceu como parte do meu aprendizado.

Ainda assim, eu era bastante iniciante em programação profissional. Tinha pouca familiaridade com *design patterns*, arquitetura de software, desenvolvimento backend e construção de sistemas robustos. Essa falta de base dificultou minha evolução em Go e, por algum tempo, a linguagem acabou ficando em pausa. O caminho, porém, não parou ali. Nesse período, meus estudos em Flutter continuaram avançando; em aplicativos menores, pude praticar arquitetura, padrões de desenvolvimento e organização de software, amadurecendo uma base que mais tarde seria importante para retomar Go com outro olhar.

Em 2025, voltei a me aproximar do desenvolvimento backend durante um projeto com o Vaden (<https://doc.vaden.dev/>), um framework para criação de APIs usando Dart. Esse contato aconteceu em um dos treinamentos que acompanhei na Flutterando com Jacob Moura (<https://www.linkedin.com/in/jacob-moura/>) e reacendeu meu interesse pela construção de APIs, trazendo-me pouco a pouco de volta ao Go.

Após sair da Aurix, em abril de 2026, comecei a desenvolver o BankLab (<https://github.com/rudsonalves/banklab>), um laboratório open source para estudar engenharia aplicada a sistemas finan-

ceiros, com backend em Go e aplicativo mobile em Flutter. Para iniciar o projeto, revisei meus artigos antigos do blog, mas logo ficou claro que esse “revisitar” era insuficiente. Eu precisava de uma revisão mais séria, organizada e honesta dos fundamentos, especialmente porque ferramentas de assistência de código mascaram facilmente lacunas do conhecimento ao sugerir soluções antes que o entendimento esteja consolidado.

Minha primeira tentativa foi reorganizar os artigos antigos na forma de um livro, com a intenção de ordenar meu próprio estudo de maneira mais progressiva. Depois de algumas frustrações, percebi que seria melhor recomeçar do zero. A ideia passou a ser mais pragmática: construir uma base de revisão, com explicações diretas, exemplos simples e espaço para consulta posterior.

Foi assim que este livro nasceu.

Tracei uma estrutura inicial e comecei a escrever. Aos poucos, ela cresceu mais do que eu imaginava e apontou para uma coleção em volumes. Este primeiro volume, **Go Essencial — Fundamentos**, reúne a base conceitual da linguagem em um percurso de revisão pensado para consulta, prática e amadurecimento.

Cada capítulo foi organizado para servir tanto como leitura sequencial quanto como referência durante o desenvolvimento.

A proposta, porém, não é listar apenas sintaxe. A revisão procura compreender Go a partir das ideias que moldaram a linguagem: simplicidade, composição, ferramentas integradas, tratamento explícito de erros, concorrência como parte natural do modelo de programação e preferência constante por código claro.

Este texto tem natureza de revisão e não pretende ser uma obra original no sentido estrito. Muitas ideias, explicações e exemplos simples foram inspirados, reorganizados ou adaptados a partir de documentação oficial, artigos, exemplos da comunidade, experiências de estudo e outros materiais. O valor pretendido está na curadoria, na organização progressiva e no esforço de tornar os fundamentos acessíveis para leitura, consulta e revisão.

Este livro também contou com o uso de ferramentas de inteligência artificial no processo de revisão gramatical, polimento de trechos, reorganização de ideias, pesquisa de apoio e formulação de estruturas de apresentação. Ainda assim, a seleção dos temas, o percurso de estudo, as decisões editoriais, a revisão crítica e a responsabilidade final pelo texto permanecem humanas.

O material é distribuído gratuitamente e com permissões explícitas para leitura, compartilhamento e adaptação, respeitando os termos apresentados na seção de licença. O conteúdo editorial segue a licença CC BY-NC-SA 4.0, enquanto os exemplos de código em Go são disponibilizados sob licença MIT.

No fundo, este volume ainda carrega meu modo pessoal de estudo: escrever para entender, reorganizar para aprender e voltar aos fundamentos sempre que a prática mostra que eles ainda precisam ficar mais sólidos.

Índice

Licença	iii
Conteúdo do livro	iii
Código-fonte dos exemplos	iii
Agradecimentos	v
Apresentação	vii
1 Introdução	1
1.1 História da Linguagem Go	2
1.2 Linguagem de Propósito Geral	3
1.3 A Filosofia por Trás de Go	5
2 Instalação	7
2.1 Linux	7
2.2 Windows	8
2.3 macOS	8
2.4 Verificando o ambiente	8
2.5 Um ambiente moderno	9
2.6 O editor de desenvolvimento	9
2.7 Primeiro Projeto	9
2.8 Executando Programas	11
2.9 Compilando Programas	11
2.10 Ferramentas do Go	11
3 Variáveis e Tipos	13
3.1 Variáveis	13
3.1.1 Valores Zero	14
3.1.2 Declaração Curta	15
3.1.3 Escopo	16
3.1.4 Nomes de Variáveis	17
3.1.5 Variáveis e Memória	17
3.2 Tipos Fundamentais	18
3.2.1 Inteiros	18
3.2.2 Números de Ponto Flutuante	19
3.2.3 Números Complexos	20
3.2.4 Booleanos	20

3.2.5	Strings	21
3.2.6	Bytes e Runes	22
3.2.7	Valores Literais	23
3.2.8	Resumo	23
3.3	Constantes	24
3.3.1	Constantes Tipadas	24
3.3.2	Constantes Não Tipadas	25
3.3.3	Tipos Padrão	25
3.3.4	Expressões Constantes	26
3.3.5	Constantes Numéricas	26
3.3.6	iota	27
3.3.7	Constantes Exportadas	28
3.3.8	Quando Usar Constantes	29
3.3.9	Resumo	29
3.4	Inferência de Tipos	29
3.4.1	Inferência com Declaração Curta	30
3.4.2	Tipos Inferidos a partir de Constantes	31
3.4.3	Inferência e Clareza	31
3.4.4	Inferência Não é Conversão Automática	32
3.4.5	Quando Usar Inferência	32
3.5	Conversão de Tipos	32
3.5.1	Conversões Explícitas	32
3.5.2	Não Existem Conversões Implícitas	33
3.5.3	Conversões Entre Tipos Numéricos	33
3.5.4	Perda de Informação	34
3.5.5	Conversão de Strings	34
3.5.6	O Pacote strconv	35
3.5.7	Conversão Entre Strings e Bytes	35
3.5.8	Conversão e Tipos Definidos pelo Usuário	36
3.5.9	Conversão Não é Type Assertion	36
3.5.10	Resumo	36
4	Controle de Fluxo	37
4.1	if	37
4.1.1	Expressões booleanas	38
4.1.2	if e else	39
4.1.3	Múltiplas condições	39
4.1.4	Inicialização no if	39
4.1.5	Escopo	40
4.1.6	Ausência de parênteses	41
4.1.7	Resumo	41
4.2	switch	41
4.2.1	Múltiplos valores em um caso	42
4.2.2	O caso default	43
4.2.3	switch sem expressão	43
4.2.4	Inicialização no switch	43
4.2.5	Avaliação dos casos	44
4.2.6	fallthrough	44

4.2.7	Expressões nos casos	45
4.2.8	Resumo	46
4.3	for	46
4.3.1	for como while	47
4.3.2	Laços infinitos	47
4.3.3	Percorrendo coleções com range	47
4.3.4	Iterando sobre strings	48
4.3.5	Variáveis do laço	49
4.3.6	Múltiplas variáveis	49
4.3.7	Laços aninhados	49
4.3.8	Ausência de parênteses	50
4.3.9	Resumo	50
4.4	break e continue	51
4.4.1	A instrução break	51
4.4.2	Encerrando laços infinitos	51
4.4.3	A instrução continue	52
4.4.4	Filtrando valores	53
4.4.5	break em switch	53
4.4.6	Laços aninhados	54
4.4.7	Labels	54
4.4.8	break, continue e legibilidade	55
4.4.9	Resumo	56
4.5	Desafios	56
5	Coleções	63
5.1	Arrays	64
5.1.1	Inicialização	64
5.1.2	Acessando Elementos	64
5.1.3	Alterando Elementos	65
5.1.4	Comprimento	65
5.1.5	Percorrendo Arrays	66
5.1.6	Arrays São Valores	66
5.1.7	Arrays Multidimensionais	67
5.1.8	Quando Usar Arrays	68
5.1.9	Resumo	68
5.2	Slices	69
5.2.1	Inicialização	69
5.2.2	Comprimento	69
5.2.3	Criando com make	70
5.2.4	Append	71
5.2.5	Capacidade	71
5.2.6	Fatiamento	72
5.2.7	Compartilhamento de Dados	72
5.2.8	Copiando Slices	73
5.2.9	Percorrendo Slices	73
5.2.10	Slices São Descritores	74
5.2.11	Quando Usar Slices	74
5.2.12	Resumo	75

5.3	Maps	75
5.3.1	Criando Maps	75
5.3.2	Acessando Valores	76
5.3.3	Chaves Inexistentes	76
5.3.4	Alterando Valores	77
5.3.5	Removendo Elementos	77
5.3.6	Comprimento	78
5.3.7	Percorrendo Maps	78
5.3.8	Ordem de Iteração	78
5.3.9	Maps São Tipos de Referência	79
5.3.10	Comparação	79
5.3.11	Chaves Válidas	80
5.3.12	Quando Usar Maps	80
5.3.13	Resumo	80
5.4	Desafios	81
6	Funções	87
6.1	Funções	87
6.2	Funções Variádicas	90
6.3	Funções Anônimas	92
6.4	Closures	94
6.5	Desafios	98
7	Ponteiros	103
7.1	Memória	104
7.2	Stack e Heap	106
7.3	Passagem por Valor	107
7.4	Endereços e Ponteiros	110
7.5	Desafios	114
8	Structs	117
8.1	Estruturas e Modelagem de Dados	118
8.2	Declaração e Inicialização	121
8.3	Zero Value	124
8.4	Struct Tags	127
8.5	Embedding	129
9	Métodos	135
9.1	Receivers	135
9.2	Value Receivers	138
9.3	Pointer Receivers	142
9.4	Desafios	146
10	Interfaces	151
10.1	Interfaces como Contratos	152
10.2	Satisfação Implícita	155
10.2.1	Interfaces e Pointer Receivers	158
10.2.2	Conclusão	161
10.3	Composição de Interfaces	161

10.4 Interfaces na Biblioteca Padrão	165
10.5 Verificação Explícita de Implementação	166
10.6 Desafios	167
11 Erros	169
11.1 A interface error	169
11.2 errors.New	171
11.2.1 Quando utilizar	172
11.2.2 Erros reutilizáveis	173
11.2.3 Limitações	173
11.2.4 Boas práticas	173
11.3 fmt.Errorf	174
11.3.1 Verbos de formatação	175
11.3.2 Quando utilizar	176
11.3.3 Comparando com errors.New	176
11.3.4 Preparando o terreno para <i>wrapping</i>	177
11.4 <i>Wrapping</i>	177
11.4.1 Motivação	177
11.4.2 Adicionando contexto	178
11.4.3 <i>Wrapping</i> em múltiplas camadas	179
11.4.4 Por que utilizar %w?	179
11.4.5 Quando utilizar	180
11.5 errors.Is	180
11.5.1 Comparando erros	180
11.5.2 Comparação direta após <i>wrapping</i>	181
11.5.3 Cadeias de encapsulamento	182
11.5.4 Quando utilizar	182
11.5.5 Evite comparar mensagens	183
11.6 errors.As	183
11.6.1 Motivação	183
11.6.2 Obtendo o erro concreto	184
11.6.3 Por que passar um ponteiro?	185
11.6.4 Comparando com errors.Is	185
11.6.5 Um exemplo da biblioteca padrão	186
11.6.6 Quando utilizar	186
11.7 panic	187
11.7.1 Exemplo básico	187
11.7.2 Interrompendo a execução	187
11.7.3 panic com um erro	188
11.7.4 Quando utilizar	188
11.7.5 panic na biblioteca padrão	189
11.7.6 panic não substitui error	189
11.8 recover	190
11.8.1 Utilizando recover	190
11.8.2 Por que utilizar defer?	191
11.8.3 Recuperando a execução	191
11.8.4 Um uso comum	192
11.8.5 recover não substitui o tratamento de erros	193

11.8.6	Resumo	193
12	Packages	195
12.1	Exportação	195
12.1.1	Exemplo	197
12.1.2	Conclusão	198
12.2	Organização	198
12.3	Visibilidade	199
12.4	Desafios	200
12.4.1	Antes de iniciar os exercícios	200
12.4.2	Exercícios	201
13	Módulos	203
13.1	go.mod	203
13.2	Dependências	204
13.3	O diretório de trabalho do Go	205
13.4	go get	206
13.5	go install	207
13.6	Desafio	208
14	Biblioteca Padrão	211
14.1	fmt	212
14.1.1	Exibindo valores	212
14.1.2	Formatação com Printf	213
14.1.3	Verbos de formatação mais comuns	213
14.1.4	Criando strings formatadas	214
14.1.5	Lendo dados do usuário	215
14.1.6	Resumo	216
14.2	strings	216
14.2.1	Verificando a presença de um texto	216
14.2.2	Verificando prefixos e sufixos	217
14.2.3	Convertendo para maiúsculas e minúsculas	217
14.2.4	Removendo espaços	218
14.2.5	Substituindo texto	218
14.2.6	Dividindo uma string	218
14.2.7	Unindo strings	219
14.2.8	Repetindo texto	219
14.2.9	Contando ocorrências	220
14.2.10	Resumo	220
14.3	strconv	221
14.3.1	Convertendo string para inteiro	221
14.3.2	Convertendo inteiro para string	222
14.3.3	Convertendo números de ponto flutuante	223
14.3.4	Convertendo booleanos	223
14.3.5	Formatando valores	224
14.3.6	Interpretando números em diferentes bases	224
14.3.7	Resumo	225
14.4	time	225

14.4.1	Obtendo a data e hora atuais	225
14.4.2	Acessando componentes da data	226
14.4.3	Criando datas específicas	227
14.4.4	Formatando datas	227
14.4.5	Convertendo texto em datas	228
14.4.6	Somando e subtraindo tempo	229
14.4.7	Medindo tempo de execução	229
14.4.8	Pausando a execução	230
14.4.9	Trabalhando com durações	230
14.4.10	Resumo	230
14.5	math	231
14.5.1	Constantes matemáticas	231
14.5.2	Valor absoluto	232
14.5.3	Potência e raiz quadrada	232
14.5.4	Arredondamentos	232
14.5.5	Máximo e mínimo	233
14.5.6	Funções trigonométricas	234
14.5.7	Logaritmos	234
14.5.8	Valores especiais	234
14.5.9	Resumo	235
14.6	os	235
14.6.1	Argumentos da linha de comando	236
14.6.2	Variáveis de ambiente	237
14.6.3	Obtendo o diretório atual	237
14.6.4	Criando diretórios	238
14.6.5	Removendo arquivos e diretórios	238
14.6.6	Criando arquivos	238
14.6.7	Lendo arquivos	239
14.6.8	Gravando arquivos	239
14.6.9	Encerrando o programa	240
14.6.10	Resumo	240
14.7	io	241
14.7.1	A interface Reader	241
14.7.2	A interface Writer	241
14.7.3	Copiando dados	242
14.7.4	Lendo todo o conteúdo	242
14.7.5	Limitando a leitura	243
14.7.6	Descartando dados	244
14.7.7	Detectando o fim da leitura	244
14.7.8	Resumo	244
14.8	bufio	245
14.8.1	Lendo uma linha do teclado	245
14.8.2	Lendo um arquivo linha por linha	246
14.8.3	Alterando o separador	246
14.8.4	Escrevendo com buffer	247
14.8.5	Resumo	247
14.9	encoding/json	248
14.9.1	Convertendo uma estrutura para JSON	248

14.9.2 Personalizando os nomes dos campos	249
14.9.3 Convertendo JSON para uma estrutura	249
14.9.4 Gerando JSON formatado	250
14.9.5 Ignorando campos	251
14.9.6 Gravando e lendo JSON em arquivos	251
14.9.7 Resumo	253
14.10 Desafios	254
15 Concorrência	257
15.1 O que é concorrência	258
15.2 Goroutines	259
15.2.1 A função main também é uma goroutine	260
15.2.2 Funções anônimas	261
15.2.3 Goroutines são leves	262
15.3 Sincronização com sync	262
15.3.1 WaitGroup	263
15.3.2 Mutex	264
15.3.3 RWMutex	266
15.3.4 Once	268
15.4 Channels	270
15.4.1 Criando canais	270
15.4.2 Enviando e recebendo valores	270
15.4.3 Canais sem buffer bloqueiam	271
15.4.4 Fechando canais	273
15.4.5 Canais direcionais	274
15.5 Buffered channels	276
15.5.1 Criando canais com buffer	276
15.5.2 Quando o envio bloqueia	278
15.5.3 Quando o recebimento bloqueia	279
15.5.4 Tamanho e capacidade	279
15.5.5 Usando buffer com produtores e consumidores	280
15.6 select	281
15.6.1 Recebendo do primeiro canal disponível	281
15.6.2 Usando select dentro de um loop	283
15.6.3 Timeout com time.After	285
15.6.4 default	286
15.7 context	287
15.8 Worker pools	288
15.9 Desafios	290
16 Testes	293
16.1 testing	293
16.2 Table Driven Tests	296
16.3 Benchmarks	299
16.4 Coverage	301
16.5 Desafios	303
16.5.1 Antes de iniciar os exercícios	303
16.5.2 Exercícios	303

17 Interfaces Avançadas	309
17.1 any	309
17.2 Type Assertions	312
17.3 Type Switch	314
17.4 Compile-time Interface Assertions	317
17.5 Boas Práticas	319
17.5.1 Prefira interfaces pequenas	319
17.5.2 Aceite interfaces e retorne structs	319
17.5.3 Defina interfaces onde elas são utilizadas	320
17.5.4 Evite utilizar any desnecessariamente	320
17.5.5 Evite <i>type assertions</i> frequentes	321
17.5.6 Utilize <i>compile-time interface assertions</i> em bibliotecas	321
17.5.7 Interfaces representam comportamento	322
17.5.8 Não crie interfaces cedo demais	322
17.6 Desafios	322
17.6.1 Exercícios	323

Capítulo 1

Introdução

Antes de estudar sintaxe, tipos, funções ou estruturas de controle, é importante compreender o contexto em que Go surgiu e quais problemas a linguagem se propõe a resolver. Uma linguagem de programação não é apenas um conjunto de regras e palavras-chave. Ela também expressa escolhas de projeto, influências históricas e necessidades práticas de uma determinada época.

Go nasceu em um momento no qual o desenvolvimento de software enfrentava mudanças importantes. Sistemas tornavam-se maiores, equipes cresciam, processadores multicore substituíam o antigo aumento contínuo da frequência dos processadores e a complexidade das linguagens existentes começava a afetar produtividade, manutenção e evolução dos sistemas.

Nesse cenário, seus criadores não buscaram apenas adicionar novos recursos a mais uma linguagem. A intenção era construir uma ferramenta simples, eficiente e adequada à construção de software de longa duração. Muitas características marcantes de Go, como a ausência de herança tradicional, o tratamento explícito de erros, as ferramentas integradas e o suporte nativo à concorrência, são consequências diretas dessa visão.

Este capítulo prepara a leitura do restante do livro. Em vez de apresentar detalhes técnicos isolados, ele procura responder a três perguntas fundamentais:

- Como e por que Go surgiu?
- O que caracteriza Go como linguagem de programação?
- Quais princípios orientaram suas principais decisões de projeto?

Para isso, começaremos pela história da linguagem, observando o ambiente técnico em que ela foi criada. Em seguida, veremos como Go pode ser classificada e quais características a aproximam ou a afastam de outras linguagens. Por fim, discutiremos a filosofia que orienta muitas de suas decisões de projeto.

Compreender esses pontos ajuda a evitar uma leitura apenas mecânica da linguagem. A sintaxe, os tipos, as interfaces, os erros e os recursos de concorrência estudados nos próximos capítulos não são elementos soltos. Eles fazem parte de uma forma específica de pensar software: simples, explícita, pragmática e orientada à manutenção.

1.1 História da Linguagem Go

No início dos anos 2000, a indústria de software enfrentava uma mudança importante. Durante décadas, grande parte do aumento de desempenho dos computadores vinha do crescimento da frequência dos processadores. Programas escritos de forma sequencial se beneficiavam naturalmente de CPUs cada vez mais rápidas.¹

Entretanto, limitações físicas relacionadas ao consumo de energia e à dissipação de calor impediram que essa estratégia continuasse avançando indefinidamente. A solução encontrada pelos fabricantes foi mudar o caminho: em vez de aumentar apenas a frequência, passaram a aumentar o número de núcleos de processamento.

Nascia a era dos processadores multicore.

Essa mudança trouxe um novo desafio. O hardware continuava evoluindo, mas escrever programas capazes de aproveitar bem múltiplos núcleos não era simples. Grande parte das linguagens e ferramentas disponíveis havia sido projetada em um contexto no qual a execução sequencial ainda era o caso dominante.

- C e C++ ofereciam alto desempenho, mas a programação concorrente era complexa e propensa a erros.
- Java possuía boas bibliotecas, porém com APIs extensas e um modelo baseado em threads relativamente pesadas.
- Linguagens interpretadas, como Python, privilegiavam produtividade, mas apresentavam limitações para aproveitar múltiplos núcleos em determinados tipos de aplicação.

Ao mesmo tempo, sistemas cada vez maiores eram desenvolvidos por equipes numerosas. Dentro do Google, onde Go começou a ser criada, era comum lidar com bases de código extensas, compilações demoradas e sistemas distribuídos que precisavam evoluir continuamente.²

Projetos desse tipo exigiam:

- Compilações rápidas;
- Código simples de manter;
- Boa produtividade;
- Suporte natural à concorrência.

Foi nesse contexto que, em 2007, Robert Griesemer, Rob Pike e Ken Thompson iniciaram, no Google, o desenvolvimento de uma nova linguagem.³

O objetivo não era criar uma linguagem com o maior número possível de recursos. Pelo contrário, a intenção era construir uma linguagem pequena, prática e adequada ao desenvolvimento de software em larga escala.

Go deveria ser:

- Simples;
- Rápida para compilar;

¹The Go Programming Language, *Frequently Asked Questions (FAQ)*, seção “Origins”: <https://go.dev/doc/faq#Origins>.

²Rob Pike, *Go at Google: Language Design in the Service of Software Engineering*: <https://go.dev/talks/2012/splash.article>.

³The Go Programming Language, *Frequently Asked Questions (FAQ)*, seção “What is the history of the project?”: <https://go.dev/doc/faq#history>.

- Eficiente;
- Fácil de manter;
- Natural para programação concorrente.

Inspirada nas ideias de CSP (*Communicating Sequential Processes*), estudadas por Tony Hoare e utilizadas anteriormente em linguagens como Newsqueak, Alef e Limbo, Go introduziu um modelo de concorrência baseado em:⁴

- **goroutines**, unidades leves de execução gerenciadas pelo *runtime*;
- **channels**, mecanismos de comunicação entre goroutines.

Rob Pike resumiu essa filosofia em uma frase famosa:⁵

Don't communicate by sharing memory; share memory by communicating.

A primeira versão pública da linguagem foi apresentada em 2009, e em 2012 foi lançado o Go 1.0, acompanhado de uma promessa que permanece até hoje:^{6 7}

Programas escritos para Go 1 continuarão compatíveis com versões futuras.

Essa promessa de compatibilidade foi uma decisão essencial. Ela indicava que Go não pretendia ser apenas uma linguagem experimental, mas uma ferramenta estável para a construção de sistemas de longa duração.

Desde então, Go tornou-se uma das principais linguagens para desenvolvimento de serviços distribuídos, ferramentas de infraestrutura, aplicações de backend e sistemas ligados à computação em nuvem. Projetos como Docker, Kubernetes, Terraform e Prometheus ajudaram a consolidar sua adoção.⁸

A história de Go, portanto, não começa apenas com uma nova sintaxe. Ela nasce de problemas práticos: programas cada vez maiores, equipes maiores, máquinas multicore, necessidade de compilação rápida e desejo por código simples de manter. Esses problemas explicam muitas das decisões que serão estudadas ao longo deste livro.

1.2 Linguagem de Propósito Geral

Go é uma linguagem de programação de propósito geral. Isso significa que ela não foi criada para resolver apenas uma classe específica de problemas, mas para ser utilizada em diferentes

⁴C. A. R. Hoare, *Communicating Sequential Processes*, Communications of the ACM, 1978: <https://doi.org/10.1145/359576.359585>. Ver também a seção “Why build concurrency on the ideas of CSP?” no FAQ oficial de Go: <https://go.dev/doc/faq#csp>.

⁵Andrew Gerrand, *Share Memory By Communicating*, The Go Blog: <https://go.dev/blog/share-memory-by-communicating>.

⁶The Go Programming Language, *Frequently Asked Questions (FAQ)*, seção “What is the history of the project?”: <https://go.dev/doc/faq#history>.

⁷The Go Programming Language, *Go 1 and the Future of Go Programs*: <https://go.dev/doc/go1compat>.

⁸The Go Programming Language, *Frequently Asked Questions (FAQ)*, seção “Usage”: <https://go.dev/doc/faq#Usage>.

domínios, desde pequenas ferramentas de linha de comando até sistemas distribuídos executados em milhares de máquinas.⁹

Ao longo dos anos, Go mostrou-se particularmente adequada para aplicações de backend, infraestrutura e computação em nuvem, mas seu uso não se restringe a essas áreas. Programas em Go são frequentemente utilizados na construção de APIs, aplicações web, ferramentas de automação, serviços de rede, sistemas concorrentes e utilitários de desenvolvimento.

Do ponto de vista técnico, Go é uma linguagem compilada. O código-fonte é transformado em um programa executável antes da execução. Essa característica favorece bom desempenho e simplifica a distribuição das aplicações, pois, em muitos casos, o resultado final pode ser entregue como um único binário.

Go também é estaticamente tipada. Isso significa que os tipos das variáveis, expressões e funções são verificados durante a compilação. Muitos erros podem ser detectados antes que o programa seja executado, sem impedir que a linguagem continue relativamente concisa por meio de inferência de tipos em situações comuns.¹⁰

Outra classificação importante é que Go é uma linguagem imperativa e estruturada. O programador descreve explicitamente as operações que devem ser realizadas, organizando o código por meio de funções, estruturas de controle, pacotes e tipos definidos pelo usuário.

Embora possua métodos e interfaces, Go não adota o modelo clássico de orientação a objetos baseado em classes e herança. Não existem classes, construtores especiais ou uma hierarquia obrigatória entre tipos. Em vez disso, a linguagem privilegia composição e abstrações baseadas em comportamento.¹¹

Essa decisão é importante. Em Go, um tipo pode ter métodos, satisfazer interfaces e participar de abstrações sem precisar declarar formalmente que pertence a uma determinada árvore de classes. O foco está menos em “o que o tipo é” e mais em “o que o tipo é capaz de fazer”.

Uma das características mais marcantes da linguagem é o suporte nativo à concorrência. Diferentemente de muitas linguagens, nas quais mecanismos concorrentes são oferecidos principalmente por bibliotecas externas ou APIs complexas, Go incorpora esse modelo à própria linguagem por meio de goroutines e channels. Essa decisão reflete uma das motivações centrais que levaram ao seu desenvolvimento: aproveitar de forma eficiente a crescente disponibilidade de processadores multicore.¹²

Outra característica prática é a produção de binários independentes. Em muitos casos, um programa em Go pode ser distribuído como um único arquivo executável, sem necessidade de instalar máquinas virtuais ou grandes conjuntos de dependências adicionais. Essa simplicidade tornou a linguagem particularmente atraente para aplicações de infraestrutura, ferramentas de linha de comando, serviços de rede e ambientes distribuídos.

Embora seja frequentemente comparada a linguagens como C, C++, Java e Python, Go ocupa uma posição própria. Ela procura combinar o desempenho e a eficiência das linguagens compiladas com a simplicidade e a produtividade normalmente associadas às linguagens interpretadas.

⁹The Go Programming Language Specification, introdução: <https://go.dev/ref/spec#Introduction>.

¹⁰The Go Programming Language Specification, seção “Types”: <https://go.dev/ref/spec#Types>.

¹¹The Go Programming Language, *Frequently Asked Questions (FAQ)*, seção “Is Go an object-oriented language?": https://go.dev/doc/faq#Is_Go_an_object-oriented_language.

¹²The Go Programming Language Specification, introdução: <https://go.dev/ref/spec#Introduction>.

Em resumo, Go é uma linguagem de propósito geral, compilada, estaticamente tipada, imperativa e estruturada, com suporte nativo à concorrência e um modelo de abstração baseado em composição e interfaces. Muitas das decisões discutidas nas próximas seções, como tratamento explícito de erros, interfaces implícitas e ferramentas integradas, são consequências naturais dessas características.

1.3 A Filosofia por Trás de Go

As características técnicas de uma linguagem não surgem por acaso. Por trás de cada decisão de projeto existe uma visão sobre como o software deve ser construído, mantido e evoluído. Em Go, essa visão é tão importante quanto a própria sintaxe.

Uma das ideias centrais por trás de Go é que programas são escritos para pessoas e apenas incidentalmente executados por computadores. Em outras palavras, um programa costuma ser lido muito mais vezes do que é escrito. Consequentemente, simplicidade, legibilidade e facilidade de manutenção foram colocadas acima da sofisticação e da quantidade de recursos disponíveis.¹³

Essa filosofia explica por que Go evita incorporar funcionalidades consideradas excessivamente complexas. Diferentemente de outras linguagens, Go não possui herança de classes, sobrecarga de métodos, operador ternário ou mecanismos sofisticados de metaprogramação. A ausência desses recursos não representa uma limitação acidental, mas uma decisão consciente.

Rob Pike sintetizou essa ideia em uma formulação curta:¹⁴

Less can be more.

Outro princípio importante é que o código deve ser explícito. Em Go, clareza é considerada mais importante do que esperteza. Isso aparece de forma muito clara no tratamento de erros. Em vez de utilizar exceções como mecanismo principal para propagar falhas, a linguagem favorece um fluxo de execução explícito, no qual erros são tratados como valores comuns.¹⁵

Essa abordagem pode tornar o código mais verboso, especialmente para quem vem de linguagens baseadas em exceções. Por outro lado, ela torna o fluxo do programa mais previsível: ao ler uma função, é possível enxergar onde um erro pode ocorrer e como ele será tratado.

Go também privilegia composição em vez de hierarquias complexas. Enquanto linguagens orientadas a objetos tradicionais se apoiam fortemente em herança, Go procura construir abstrações a partir da combinação de componentes menores e independentes. Da mesma forma, interfaces descrevem comportamentos, e não relações hierárquicas entre tipos. O resultado é um modelo que favorece desacoplamento e reduz dependências desnecessárias.

Essa busca por simplicidade estende-se às próprias ferramentas de desenvolvimento. Em muitos ecossistemas, diferentes equipes adotam diferentes formatadores, convenções e sistemas de construção. Go segue uma direção diferente. Ferramentas como `go fmt`, `go test`, `go doc` e `go`

¹³The Go Programming Language, *Effective Go*: https://go.dev/doc/effective_go.

¹⁴Rob Pike, *Less is exponentially more*: <https://commandcenter.blogspot.com/2012/06/less-is-exponentially-more.html>.

¹⁵Rob Pike, *Errors are values*, The Go Blog: <https://go.dev/blog/errors-are-values>.

vet fazem parte da distribuição oficial da linguagem.¹⁶

Isso tem uma consequência prática importante: existe uma forma comum de formatar, testar, documentar e analisar programas. Essa padronização reduz discussões desnecessárias e torna projetos escritos por equipes diferentes surpreendentemente semelhantes.

Outro aspecto fundamental da linguagem é o tratamento da concorrência. Go nasceu em uma época em que processadores multicore tornaram a programação concorrente uma necessidade prática. Em vez de oferecer apenas bibliotecas para manipulação de threads e mecanismos de sincronização, Go incorporou a concorrência ao próprio modelo da linguagem. Goroutines e channels refletem uma filosofia baseada na comunicação entre processos concorrentes, sintetizada por uma frase famosa de Rob Pike:

Don't communicate by sharing memory; share memory by communicating.

Por fim, Go foi concebida para a construção de software de longa duração. A linguagem privilegia estabilidade e compatibilidade. Desde a versão 1.0, um compromisso importante foi estabelecido: programas escritos para Go 1 deveriam continuar funcionando nas versões futuras.¹⁷

Essa preocupação revela que Go não foi projetada apenas para resolver problemas imediatos. Ela foi pensada para permitir a evolução contínua de sistemas ao longo dos anos, sem obrigar equipes a reescrever grandes bases de código a cada mudança da linguagem.

Mais do que uma coleção de recursos, Go representa uma determinada forma de pensar sobre software. Simplicidade, clareza, composição, ferramentas integradas, concorrência e compatibilidade são manifestações visíveis de uma filosofia maior. Compreender esses princípios é essencial para compreender a própria linguagem, pois muitas das características estudadas nos próximos capítulos são consequências naturais dessas ideias.

¹⁶The Go Programming Language, documentação do comando go: <https://pkg.go.dev/cmd/go>.

¹⁷The Go Programming Language, *Go 1 and the Future of Go Programs*: <https://go.dev/doc/go1compat>.

Capítulo 2

Instalação

Uma das características mais interessantes de Go é a simplicidade do ambiente de desenvolvimento. Diferentemente de outras plataformas, nas quais é comum instalar máquinas virtuais, gerenciadores de dependências e várias ferramentas adicionais antes de escrever o primeiro programa, a distribuição oficial de Go já fornece praticamente tudo o que é necessário para começar.

O compilador, a biblioteca padrão e as principais ferramentas de desenvolvimento fazem parte da própria distribuição. Essa integração é consequência direta da filosofia da linguagem: reduzir complexidade e oferecer um ambiente uniforme para diferentes sistemas operacionais e equipes.

A distribuição oficial pode ser obtida no site do projeto:

```
https://go.dev/dl/
```

São disponibilizados instaladores para Linux, Windows e macOS. A recomendação é utilizar sempre a versão estável mais recente.

As instruções a seguir acompanham as orientações da página oficial de instalação, disponível em go.dev/doc/install. No momento desta revisão, a versão estável mais recente é a 1.26.4, publicada em 18 de junho de 2026.

2.1 Linux

No Linux, a instalação pode ser realizada pelos pacotes fornecidos pela própria distribuição ou pelo pacote oficial disponível no site do projeto.

Ao utilizar o pacote oficial, após baixar o arquivo correspondente à arquitetura do sistema, remova uma instalação anterior e extraia o novo conteúdo para `/usr/local`:

```
sudo rm -rf /usr/local/go
```

```
sudo tar -C /usr/local -xzf go1.26.4.linux-amd64.tar.gz
```

Em seguida, adicione o diretório dos binários de Go à variável de ambiente `PATH`:

```
export PATH=$PATH:/usr/local/go/bin
```

Normalmente essa configuração é colocada em um arquivo de inicialização do shell, como `.bashrc`, `.zshrc` ou outro arquivo equivalente.

Após recarregar o terminal, a instalação pode ser verificada através do comando:

```
go version
```

2.2 Windows

No Windows, a instalação é mais direta. Basta executar o instalador disponibilizado no site oficial e seguir as instruções apresentadas. O próprio instalador configura as variáveis necessárias para utilização da linguagem.

Após a instalação, abra o Prompt de Comando ou o PowerShell e execute:

```
go version
```

2.3 macOS

No macOS, a instalação pode ser realizada com o pacote oficial disponibilizado no site do projeto ou, se o Homebrew já estiver sendo utilizado no sistema, pelo comando:

```
brew install go
```

Em seguida, a versão instalada pode ser verificada com:

```
go version
```

2.4 Verificando o ambiente

Independentemente do sistema operacional, dois comandos merecem atenção especial.

O primeiro é:

```
go version
```

Ele informa a versão instalada da linguagem e a plataforma utilizada.

O segundo é:

```
go env
```

Ele exibe as configurações do ambiente Go. Entre as informações apresentadas estão:

- sistema operacional (GOOS);
- arquitetura do processador (GOARCH);
- localização da instalação da linguagem (GOROOT);
- diretório utilizado para armazenamento de módulos e binários (GOPATH).

Embora essas variáveis sejam importantes, na maior parte do tempo não será necessário alterá-las manualmente.

2.5 Um ambiente moderno

Nas primeiras versões da linguagem, os projetos precisavam ser organizados dentro de uma estrutura especial chamada GOPATH. Essa restrição deixou de ser necessária com a introdução dos módulos, tornando possível criar projetos em praticamente qualquer diretório do sistema.

Hoje, um projeto Go moderno é identificado pela presença de um arquivo chamado `go.mod`, responsável por definir o módulo e registrar suas dependências.

Essa mudança simplificou consideravelmente a organização dos projetos e aproximou Go dos sistemas modernos de gerenciamento de dependências.

2.6 O editor de desenvolvimento

Embora seja possível utilizar qualquer editor de texto, uma combinação bastante popular é o Visual Studio Code com a extensão oficial da linguagem.

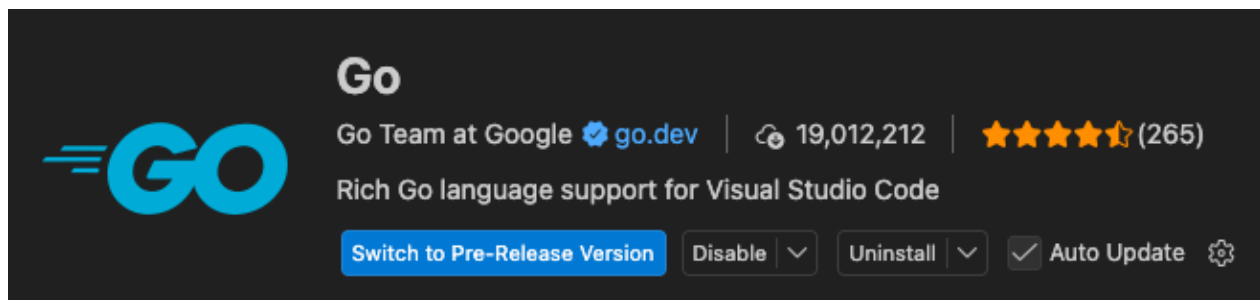


Figura 2.1: GoLang VSCode Extension

Essa extensão integra e, quando necessário, auxilia na instalação de ferramentas importantes, como:

- `gopls`, servidor de linguagem responsável por autocompletar, navegação e diagnósticos;
- `goimports`, responsável pela organização automática dos imports;
- `dlv`, depurador oficial da linguagem;
- ferramentas de análise estática utilizadas durante o desenvolvimento.

Com isso, o ambiente está preparado para a criação dos primeiros programas.

Mais importante do que a instalação em si é perceber que ela reflete uma das ideias centrais de Go: permitir que o desenvolvedor passe mais tempo escrevendo software e menos tempo configurando ferramentas. Essa preocupação com simplicidade e padronização acompanhará toda a linguagem e se tornará evidente nos próximos capítulos.

2.7 Primeiro Projeto

Com o ambiente devidamente instalado, estamos prontos para criar nosso primeiro programa em Go.

Inicialmente, criaremos um diretório para o projeto:

```
mkdir hello  
cd hello
```

Em seguida, utilizaremos o comando:

```
go mod init hello
```

Esse comando cria um arquivo chamado `go.mod`, responsável por identificar o módulo e registrar suas dependências.

Nosso diretório passa a ter a seguinte estrutura:

```
hello/  
└─ go.mod
```

Agora criaremos um arquivo chamado `main.go`:

```
hello/  
├─ go.mod  
└─ main.go
```

com o seguinte conteúdo:

```
1 package main  
2  
3 import "fmt"  
4  
5 func main() {  
6     fmt.Println("Hello, Go!")  
7 }
```

Embora ainda não tenhamos estudado a sintaxe da linguagem, é possível identificar alguns elementos importantes.

A primeira linha:

```
1 package main
```

indica que este arquivo pertence ao pacote principal do programa.

Em seguida:

```
3 import "fmt"
```

torna disponível o pacote `fmt`, pertencente à biblioteca padrão, responsável por funções de formatação e impressão.

Por fim, encontramos a função:

```
5 func main() {  
6     fmt.Println("Hello, Go!")  
7 }
```

Todo programa executável em Go começa sua execução pela função `main`.

A instrução:

```
fmt.Println("Hello, Go!")
```

escreve uma mensagem na saída padrão.

Nos próximos capítulos estudaremos cada um desses elementos em detalhes. Neste momento, nosso objetivo é apenas compreender a estrutura geral de um programa Go.

2.8 Executando Programas

Podemos executar o programa diretamente através do comando:

```
go run .
```

A saída será:

```
Hello, Go!
```

O comando `go run` compila o programa temporariamente e executa o resultado gerado.

2.9 Compilando Programas

Também podemos gerar um executável através do comando:

```
go build
```

No Linux ou macOS, será criado um executável chamado:

```
hello
```

No Windows, o executável terá a extensão `.exe`:

```
hello.exe
```

Esse arquivo pode ser executado diretamente pelo sistema operacional.

Uma característica importante de Go é que, em muitos casos, um único executável contém tudo o que é necessário para a execução do programa, sem depender de máquinas virtuais ou runtimes externos instalados separadamente.

2.10 Ferramentas do Go

A linguagem acompanha diversas ferramentas integradas:

- `go fmt`: formata automaticamente o código.
- `go doc`: consulta a documentação.
- `go test`: executa testes.
- `go vet`: realiza análise estática.
- `go build`: compila programas.
- `go env`: exibe informações sobre o ambiente.

Em Go, ferramentas não são acessórios adicionados posteriormente. Elas fazem parte do fluxo normal de desenvolvimento. Essa padronização reduz discussões desnecessárias, facilita a colaboração entre equipes e contribui para que programas escritos por diferentes desenvolvedores apresentem uma estrutura surpreendentemente uniforme.

Capítulo 3

Variáveis e Tipos

No capítulo anterior, preparamos o ambiente de desenvolvimento e criamos nosso primeiro programa em Go. A partir de agora, começamos a estudar os elementos básicos usados para construir programas reais.

Programas reais fazem mais do que imprimir mensagens na tela. Eles recebem, armazenam, transformam e produzem informações. Essas informações podem representar números, textos, datas, estados, valores monetários, configurações ou qualquer outro dado relevante para a aplicação.

Para lidar com esses dados, Go utiliza dois conceitos fundamentais: variáveis e tipos.

Uma variável é um nome associado a um valor armazenado durante a execução do programa. Esse valor pode ser lido, utilizado em expressões e, quando permitido, substituído por outro valor compatível. O tipo, por sua vez, define a natureza desse valor: quais dados podem ser armazenados, quais operações fazem sentido e como o compilador deve interpretar seu uso.

Essa relação entre variável e tipo é especialmente importante em Go porque a linguagem é estaticamente tipada. Isso significa que o tipo de cada variável e expressão é conhecido durante a compilação. Como consequência, muitos erros podem ser identificados antes da execução do programa, tornando o código mais previsível e mais fácil de manter.

Ao mesmo tempo, Go procura evitar excesso de cerimônia. Em muitas situações, não é necessário escrever explicitamente o tipo de uma variável, pois o compilador consegue inferi-lo a partir do valor utilizado na inicialização. Essa combinação entre segurança estática e concisão é uma das características que tornam a linguagem simples de ler e prática de usar.

Neste capítulo, estudaremos como declarar variáveis, quais são os principais tipos fornecidos pela linguagem, como funcionam os valores zero, o papel das constantes, os mecanismos de inferência de tipos e as conversões entre diferentes tipos.

3.1 Variáveis

Em Go, variáveis são declaradas com um tipo definido e sempre possuem um valor inicial. Essa combinação elimina variáveis não inicializadas e permite que o compilador verifique, ainda em tempo de compilação, se as operações realizadas sobre cada valor são compatíveis com o seu tipo.

A forma explícita de declaração utiliza a palavra-chave `var`:

```
var age int
```

A instrução acima declara `age` como uma variável do tipo `int`.

A declaração também pode incluir um valor inicial:

```
var age int = 30
```

Quando o tipo pode ser inferido a partir do valor inicial, ele pode ser omitido:

```
var age = 30  
var name = "Maria"
```

Nesse caso, `age` será inferida como `int` e `name` como `string`. A inferência ocorre apenas no momento da declaração; depois disso, o tipo da variável permanece fixo.

Também é possível declarar múltiplas variáveis em uma mesma instrução:

```
var width, height int
```

ou agrupá-las em um bloco:

```
var (  
    name string  
    age int  
)
```

O valor de uma variável pode ser alterado por atribuição, desde que o novo valor seja compatível com o tipo declarado:

```
var age int = 30  
  
age = 31
```

Uma atribuição incompatível resulta em erro de compilação:

```
var age int = 30  
  
age = "trinta" // erro de compilação
```

3.1.1 Valores Zero

Toda variável declarada em Go recebe automaticamente um valor inicial. Quando a declaração não informa um valor explícito, o valor utilizado é o valor zero do tipo.

Considere as declarações:

```
var age int
var price float64
var active bool
var name string
```

Os valores iniciais correspondentes são:

```
age    = 0
price  = 0.0
active = false
name   = ""
```

Cada tipo possui um valor zero bem definido. Essa regra simplifica o modelo da linguagem e evita uma classe inteira de problemas associados a variáveis não inicializadas.

3.1.2 Declaração Curta

Dentro de funções, a forma mais comum de declarar variáveis é a declaração curta, usando o operador `:=`:

```
name := "Maria"
age  := 30
active := true
```

A declaração curta combina declaração, inicialização e inferência de tipo. Ela só pode ser usada dentro de funções; no nível do pacote, utiliza-se `var`.

É importante distinguir declaração de atribuição. O operador `:=` introduz pelo menos uma variável nova. Para alterar o valor de uma variável já existente, usa-se `=`:

```
age := 30
age = 31
```

Quando a declaração curta envolve mais de um identificador, é permitido reutilizar uma variável já existente, desde que pelo menos uma das variáveis do lado esquerdo seja nova:

```
name, age := "Paulo", 35

name, height := "Paulo Eduardo", 1.73
```

Na segunda declaração, `name` já existia, mas `height` é uma variável nova. Por isso, a instrução é válida: `name` recebe um novo valor e `height` é declarada.

Se nenhum identificador novo for introduzido, a declaração curta não é permitida:

```
name, age := "Paulo", 35

name, age := "Sandra", 45 // erro de compilação
```

Esse comportamento aparece com frequência no uso de `err`, especialmente em funções que executam várias operações sequenciais:

```
file, err := os.Open("data.txt")
data, err := io.ReadAll(file)
```

Na segunda linha, `err` é reutilizada, mas `data` é uma variável nova. Essa forma é válida e bastante comum em código Go.

3.1.3 Escopo

O escopo define a região do programa em que um identificador pode ser utilizado.

No exemplo:

```
func main() {
    name := "Maria"

    fmt.Println(name)
}
```

`name` está disponível apenas dentro da função `main`.

Variáveis declaradas dentro de blocos possuem escopo restrito ao próprio bloco:

```
if age >= 18 {
    status := "adult"

    fmt.Println(status)
}
```

Nesse caso, `status` só pode ser usada dentro do bloco `if`.

Variáveis declaradas no nível do pacote ficam disponíveis para os arquivos que pertencem ao mesmo pacote:

```
var appName = "BankLab"

func main() {
    fmt.Println(appName)
}
```

Variáveis de pacote devem ser usadas com critério, especialmente quando representam estado

mutável. Valores compartilhados em muitos pontos do programa aumentam o acoplamento e dificultam testes e manutenção.

3.1.4 Nomes de Variáveis

Go utiliza a convenção *camelCase* para identificadores compostos:

```
customerName  
accountBalance  
isActive  
createdAt
```

Nomes devem comunicar o papel do valor no contexto em que aparecem. Em escopos curtos, identificadores pequenos são comuns:

```
i  
err  
ctx
```

Em escopos maiores, nomes mais descritivos tendem a ser preferíveis. A regra prática é simples: quanto mais longe o leitor precisa ir para entender um identificador, mais claro seu nome deve ser.

3.1.5 Variáveis e Memória

Uma variável corresponde a um valor armazenado em memória e acessado por meio de um identificador.

Por exemplo:

```
age := 30
```

A instrução acima associa o identificador `age` ao valor 30.

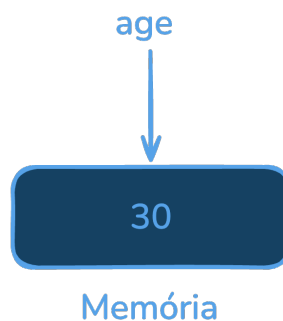


Figura 3.1: Variável `age` na memória

A localização física desse valor e a estratégia de alocação utilizada são decisões do compilador e do *runtime*.

Nos capítulos sobre ponteiros e gerenciamento de memória, veremos como Go decide se determinados valores permanecem na pilha, escapam para o heap ou são acessados indiretamente por meio de ponteiros.

Por enquanto, basta observar que variáveis são o ponto de partida para a manipulação de valores nomeados em Go.

3.2 Tipos Fundamentais

Toda variável em Go possui um tipo conhecido em tempo de compilação. O tipo define a representação do valor, as operações permitidas e as regras de compatibilidade em atribuições, chamadas de função e expressões.

Os tipos fundamentais da linguagem podem ser agrupados em algumas categorias principais.

3.2.1 Inteiros

Os tipos inteiros representam valores numéricos sem parte fracionária.

Tipo	Descrição
<code>int</code>	Inteiro com tamanho nativo da arquitetura (32 ou 64 bits).
<code>int8</code>	Inteiro com sinal de 8 bits (−128 a 127).
<code>int16</code>	Inteiro com sinal de 16 bits (−32768 a 32767).
<code>int32</code>	Inteiro com sinal de 32 bits.
<code>int64</code>	Inteiro com sinal de 64 bits.
<code>uint</code>	Inteiro sem sinal com tamanho nativo da arquitetura (32 ou 64 bits).
<code>uint8</code>	Inteiro sem sinal de 8 bits (0 a 255).
<code>uint16</code>	Inteiro sem sinal de 16 bits.
<code>uint32</code>	Inteiro sem sinal de 32 bits.
<code>uint64</code>	Inteiro sem sinal de 64 bits.
<code>uintptr</code>	Inteiro sem sinal capaz de armazenar a representação de um ponteiro.

Os tipos precedidos por `u` (*unsigned*) representam apenas valores não negativos.

Os tipos `int` e `uint` possuem tamanho dependente da arquitetura do sistema. Em geral, são usados quando não existe necessidade explícita de controlar a largura do tipo.

```
var age int = 30
var quantity uint = 100
```

Quando a largura importa, por exemplo em formatos binários, protocolos de rede, interoperabilidade ou controle preciso de memória, os tipos com tamanho explícito são preferíveis.

Os operadores aritméticos mais comuns são:

Operador	Descrição	Exemplo
+	Soma	a + b
-	Subtração	a - b
*	Multiplicação	a * b
/	Divisão inteira entre inteiros	a / b
%	Resto da divisão inteira	a % b

Por exemplo:

```
a := 10
b := 3

fmt.Println(a + b) // 13
fmt.Println(a - b) // 7
fmt.Println(a * b) // 30
fmt.Println(a / b) // 3
fmt.Println(a % b) // 1
```

A divisão entre inteiros produz outro valor inteiro. Para obter um resultado fracionário, pelo menos um dos operandos deve ser de ponto flutuante.

```
fmt.Println(7 / 2)      // 3
fmt.Println(7 / 2.0)   // 3.5
fmt.Println(7.0 / 2)   // 3.5
fmt.Println(7.0 / 2.0) // 3.5
```

Tipos inteiros distintos não são intercambiáveis automaticamente:

```
var x int = 10
var y int64 = x // erro de compilação
```

Quando necessário, a conversão deve ser explícita:

```
var x int = 10
var y int64 = int64(x)
```

3.2.2 Números de Ponto Flutuante

Go fornece dois tipos de ponto flutuante:

Tipo	Descrição
float32	Número de ponto flutuante de 32 bits.
float64	Número de ponto flutuante de 64 bits.

O tipo `float64` é o padrão para literais de ponto flutuante e costuma ser a escolha mais comum.

```
price := 12.75
pi := 3.141592653589793
```

Operações aritméticas seguem a mesma forma geral dos inteiros:

```
radius := 10.0
area := 3.14159 * radius * radius
```

Valores de ponto flutuante são representações aproximadas. Nem todo número decimal possui representação binária exata, o que pode produzir resultados aparentemente inesperados.

```
fmt.Println(0.1 + 0.2)
```

A saída pode não ser exatamente:

```
0.3
```

Esse comportamento não é específico de Go; ele decorre da forma como números de ponto flutuante são representados em hardware.

3.2.3 Números Complexos

Go possui suporte nativo a números complexos.

Tipo	Descrição
<code>complex64</code>	Número complexo com partes real e imaginária <code>float32</code> .
<code>complex128</code>	Número complexo com partes real e imaginária <code>float64</code> .

Um número complexo pode ser escrito diretamente com a unidade imaginária `i`:

```
z := 2 + 3i
```

As funções pré-declaradas `real` e `imag` acessam seus componentes:

```
fmt.Println(real(z)) // 2
fmt.Println(imag(z)) // 3
```

Embora pouco utilizados em aplicações convencionais de backend, números complexos são úteis em áreas como processamento de sinais, física e computação científica.

3.2.4 Booleanos

O tipo booleano é `bool`.

Os únicos valores possíveis são:

Valor	Descrição
true	Valor lógico verdadeiro.
false	Valor lógico falso.

Exemplo:

```
active := true
blocked := false
```

Os operadores lógicos são:

Operador	Descrição	Exemplo
&&	E lógico	a && b
	Ou lógico	a b
!	Negação	!a

Por exemplo:

```
adult := true
active := false

fmt.Println(adult && active)
fmt.Println(adult || active)
fmt.Println(!active)
```

Go não converte números automaticamente para booleanos. Expressões condicionais devem produzir valores do tipo bool.

3.2.5 Strings

O tipo string representa uma sequência imutável de bytes.

```
name := "Maria"
```

Strings podem ser concatenadas com o operador +:

```
firstName := "Maria"
lastName := "Silva"

fullName := firstName + " " + lastName
```

O comprimento de uma string é obtido pela função len, mas esse comprimento é medido em bytes, não em caracteres Unicode.

```
fmt.Println(len(fullName))
```

Literais de string em Go são codificados em UTF-8. Como alguns caracteres ocupam mais de um byte, o número de bytes pode ser diferente do número de caracteres percebidos pelo leitor.

Por exemplo:

```
word := "ação"

fmt.Println(len(word))
```

A saída é:

```
6
```

A palavra possui quatro caracteres, mas ç e ã ocupam dois bytes cada em UTF-8.

O tratamento detalhado de texto Unicode será retomado quando discutirmos strings, bytes e runes em mais profundidade.

3.2.6 Bytes e Runes

Os tipos byte e rune são aliases para tipos inteiros:

Tipo	Alias para	Descrição
byte	uint8	Usado para representar dados binários.
rune	int32	Representa um ponto de código Unicode.

byte é frequente na manipulação de dados binários, buffers e conteúdo lido de arquivos ou conexões:

```
var data []byte
```

rune representa um ponto de código Unicode:

```
letter := 'ç'
```

É importante distinguir os delimitadores de texto em Go:

Delimitador	Tipo retornado	Comportamento
'a'	rune	Representa um ponto de código Unicode.
"texto"	string	String interpretada; aceita sequências de escape como \n.
`texto`	string	String bruta (<i>raw string</i>); não interpreta sequências de escape.

Por exemplo:


```
fmt.Printf("%c\n", letter)
fmt.Printf("%d\n", letter)
```

A saída é:

```
ç
231
```

A distinção entre bytes e runes é essencial para manipular corretamente texto em UTF-8.

3.2.7 Valores Literais

Valores escritos diretamente no código são literais:

```
42
3.14
true
"Go"
'G'
```

Literais podem ser tipados ou não tipados, dependendo do contexto. Em muitos casos, o compilador adia a escolha do tipo concreto até que o valor seja usado em uma declaração, atribuição, chamada de função ou expressão que exija um tipo específico.

Esse comportamento será discutido com mais detalhes nas seções sobre constantes, inferência e conversões de tipos.

3.2.8 Resumo

Os tipos fundamentais fornecem os blocos básicos para representação de dados em Go.

Categoria	Tipos
Inteiros	int, int8, int16, int32, int64
Inteiros sem sinal	uint, uint8, uint16, uint32, uint64
Ponto flutuante	float32, float64
Complexos	complex64, complex128
Booleanos	bool
Texto	string
Caracteres Unicode	rune
Dados binários	byte

Esses tipos constituem a base sobre a qual são construídas estruturas mais complexas da linguagem, como arrays, slices, mapas e structs.

3.3 Constantes

Constantes representam valores fixos conhecidos em tempo de compilação e, diferentemente de variáveis, não podem ter seu valor modificado após a declaração.

Em Go, constantes são declaradas com a palavra-chave `const`:

```
const pi = 3.14159
const language = "Go"
const active = true
```

A tentativa de alterar uma constante resulta em erro de compilação:

```
const maxRetries = 3

maxRetries = 5 // erro de compilação
```

Constantes podem ser declaradas individualmente ou em blocos:

```
const timeout = 30

const (
    minAge = 18
    maxAge = 120
    appName = "BankLab"
)
```

3.3.1 Constantes Tipadas

Uma constante pode possuir tipo explícito:

```
const port int = 8080
const rate float64 = 0.15
const enabled bool = true
```

Nesse caso, o valor da constante passa a estar associado ao tipo informado.

```
const age int = 30

var value int64 = age // erro de compilação
```

Embora `age` contenha um valor numericamente compatível com `int64`, seu tipo é `int`, e Go não realiza conversões implícitas entre tipos numéricos distintos.

Para atribuir o valor a uma variável de outro tipo, é necessário converter explicitamente:

```
var value int64 = int64(age)
```

3.3.2 Constantes Não Tipadas

Quando o tipo não é informado, a constante permanece não tipada até ser utilizada em um contexto que exige um tipo concreto.

```
const age = 30
```

A constante acima representa o valor 30, mas ainda não está vinculada a um tipo específico como `int`, `int32` ou `int64`.

Por isso, ela pode ser utilizada em diferentes contextos numéricos compatíveis:

```
const age = 30

var a int = age
var b int32 = age
var c int64 = age
```

Constantes não tipadas tornam o código mais flexível sem comprometer a verificação estática de tipos.

O mesmo comportamento ocorre com literais de ponto flutuante, complexos, booleanos e strings:

```
const rate = 0.15
const title = "Go"
const enabled = true
```

O tipo concreto será determinado pelo contexto de uso.

3.3.3 Tipos Padrão

Quando uma constante não tipada é utilizada em uma declaração que exige inferência de tipo, Go aplica um tipo padrão.

```
const age = 30

value := age
```

Nesse caso, `value` recebe o tipo `int`.

Os principais tipos padrão são:

Categoria	Tipo padrão
Inteiro	<code>int</code>
Ponto flutuante	<code>float64</code>

Categoria	Tipo padrão
Complexo	complex128
Rune	rune
String	string
Booleano	bool

Exemplos:

```
i := 10      // int
f := 3.14    // float64
c := 2 + 3i  // complex128
r := 'A'     // rune
s := "Go"    // string
b := true    // bool
```

Esse comportamento será retomado na seção sobre inferência de tipos.

3.3.4 Expressões Constantes

Constantes podem ser definidas a partir de expressões constantes.

```
const secondsPerMinute = 60
const minutesPerHour = 60
const secondsPerHour = secondsPerMinute * minutesPerHour
```

A expressão deve poder ser avaliada em tempo de compilação.

Operações envolvendo chamadas de função, leitura de arquivos, acesso à rede ou qualquer informação disponível apenas em tempo de execução não podem ser usadas para inicializar constantes.

```
const now = time.Now() // erro de compilação
```

Constantes podem utilizar operadores aritméticos, lógicos e relacionais, desde que todos os operandos também sejam constantes.

```
const limit = 100
const valid = limit > 0
const message = "Go" + " Language"
```

3.3.5 Constantes Numéricas

Constantes numéricas não tipadas em Go possuem precisão arbitrária. Enquanto permanecem sem um tipo explícito, seus valores não estão limitados pelos tamanhos dos tipos numéricos da linguagem.

```
const big = 1_000_000_000_000_000_000
```

Nesse momento, `big` ainda não é um `int`, `int64` ou qualquer outro tipo concreto. O tipo é determinado apenas quando a constante é utilizada em um contexto que exige uma representação específica.

```
const big = 1_000_000_000_000_000_000

var a int64 = big
var b int8 = big // erro: constant 1000000000000000000 overflows int8
```

A atribuição para `a` é válida porque o valor pode ser representado por um `int64`. Já a atribuição para `b` produz um erro de compilação, pois o valor excede a faixa permitida para um `int8`.

O mesmo comportamento ocorre com constantes de ponto flutuante:

```
const rate = 1.0 / 3.0
```

Enquanto permanecer não tipada, a constante conserva uma precisão superior à de um `float64`. A representação concreta só é definida quando ela é utilizada em um contexto tipado.

```
var x float64 = rate // 0.3333333333333333
var y float32 = rate // 0.33333334
```

Nesse momento, o valor é convertido para o tipo correspondente, sujeitando-se às limitações de precisão e representação desse tipo.

3.3.6 `iota`

Go fornece o identificador pré-declarado `iota` para criação de sequências constantes.

Dentro de um bloco `const`, `iota` começa em 0 e é incrementado a cada linha do bloco.

```
const (
    Low = iota
    Medium
    High
)
```

As constantes acima recebem os seguintes valores:

```
Low    = 0
Medium = 1
High   = 2
```

Esse recurso é frequentemente utilizado para definir conjuntos de valores relacionados.

```
type Priority int

const (
    PriorityLow Priority = iota
    PriorityMedium
    PriorityHigh
)
```

Nesse exemplo, `Priority` define um novo tipo baseado em `int`, e as constantes representam valores válidos para esse domínio.

Também é possível descartar valores com o identificador em branco `_`:

```
const (
    _ = iota
    KB = 1 << (10 * iota)
    MB
    GB
)
```

Os valores resultantes são:

```
KB = 1024
MB = 1048576
GB = 1073741824
```

Esse padrão é comum para representar escalas baseadas em potências de dois.

3.3.7 Constantes Exportadas

As regras de visibilidade em Go também se aplicam a constantes.

Constantes cujo nome começa com letra maiúscula são exportadas e podem ser utilizadas por outros pacotes:

```
const DefaultTimeout = 30
```

Constantes iniciadas com letra minúscula permanecem restritas ao próprio pacote:

```
const defaultTimeout = 30
```

A decisão de exportar uma constante deve ser tomada com cuidado. Uma constante exportada passa a fazer parte da interface pública do pacote e pode ser utilizada por outros programas. Por esse motivo, valores exportados devem representar conceitos estáveis, cuja alteração não comprometa a compatibilidade do código que depende do pacote.

3.3.8 Quando Usar Constantes

Constantes devem ser utilizadas para representar valores fixos, conhecidos em tempo de compilação e semanticamente estáveis.

Exemplos comuns incluem:

```
const maxRetries = 3
const defaultPageSize = 20
const appName = "BankLab"
const taxRate = 0.15
```

Para valores definidos por configuração, ambiente, banco de dados ou entrada do usuário, utilize variáveis.

```
var serverPort string
var databaseURL string
```

Mesmo que um valor não seja alterado durante a execução, ele não deve ser uma constante se só for conhecido em tempo de execução.

3.3.9 Resumo

Constantes em Go representam valores imutáveis avaliados em tempo de compilação.

Conceito	Descrição
const	Declara um valor constante.
Constante tipada	Possui tipo explícito na declaração.
Constante não tipada	Assume um tipo concreto apenas no contexto de uso.
Tipo padrão	Tipo atribuído quando uma constante não tipada precisa ser materializada.
Expressão constante	Expressão avaliável em tempo de compilação.
iota	Identificador usado para gerar sequências constantes.

Constantes são especialmente importantes em Go porque combinam imutabilidade, verificação estática e flexibilidade por meio de valores não tipados.

3.4 Inferência de Tipos

Go é uma linguagem estaticamente tipada. Isso significa que toda variável possui um tipo definido em tempo de compilação. Entretanto, nem sempre é necessário escrever esse tipo explicitamente no código.

Em muitas situações, o compilador consegue inferir o tipo da variável a partir do valor utilizado na sua inicialização.

```
var name = "Go"  
var version = 1.22  
var stable = true
```

Nesse exemplo, o compilador infere os tipos das variáveis:

```
var name string = "Go"  
var version float64 = 1.22  
var stable bool = true
```

A inferência de tipos não torna Go uma linguagem dinamicamente tipada. O tipo continua sendo fixo após a declaração da variável.

```
var count = 10  
  
count = 20 // válido  
count = "vinte" // erro de compilação
```

A variável `count` foi inferida como `int`, portanto só pode receber valores compatíveis com esse tipo.

3.4.1 Inferência com Declaração Curta

A forma mais comum de inferência de tipos em Go ocorre com a declaração curta de variáveis, usando o operador `:=`.

```
name := "Go"  
version := 1.22  
stable := true
```

Essa forma só pode ser usada dentro de funções.

```
1 package main  
2  
3 import "fmt"  
4  
5 func main() {  
6     language := "Go"  
7     year := 2009  
8  
9     fmt.Println(language, year)  
10 }
```

O operador `:=` declara a variável e infere seu tipo a partir do valor atribuído. No exemplo anterior, `language` é inferida como `string` e `year` como `int`.

3.4.2 Tipos Inferidos a partir de Constantes

Quando uma variável é inicializada com uma constante não tipada, o compilador escolhe um tipo padrão para representar aquele valor.

```
x := 10
y := 3.14
z := 'A'
```

Nesse caso, os tipos inferidos são:

```
x // int
y // float64
z // rune
```

Isso acontece porque constantes numéricas não tipadas precisam receber um tipo concreto quando são usadas para inicializar uma variável.

```
const value = 10

var a int64 = value
b := value
```

A variável `a` tem tipo `int64`, pois o tipo foi informado explicitamente. Já `b` terá tipo `int`, pois esse é o tipo padrão escolhido para constantes inteiras não tipadas.

3.4.3 Inferência e Clareza

A inferência de tipos reduz a repetição no código, mas não deve prejudicar a clareza.

Em muitos casos, o tipo é evidente:

```
name := "Gabriel"
age := 30
active := true
```

Nesses exemplos, escrever o tipo explicitamente não acrescentaria muita informação.

Por outro lado, quando o valor inicial não deixa claro o tipo pretendido, a declaração explícita pode tornar o código mais legível.

```
var timeout int64 = 30
var ratio float32 = 0.75
```

Nesses casos, o tipo faz parte da intenção do código. Ao escrevê-lo explicitamente, evitamos depender apenas da escolha padrão feita pelo compilador.

3.4.4 Inferência Não é Conversão Automática

A inferência de tipos não realiza conversões automáticas entre tipos diferentes.

```
var x int = 10
var y int64 = x // erro de compilação
```

Embora `int` e `int64` sejam tipos inteiros, eles são tipos distintos. A conversão precisa ser explícita:

```
var x int = 10
var y int64 = int64(x)
```

Essa regra torna o código mais previsível. Go evita conversões implícitas que poderiam esconder perda de informação, mudanças de representação ou erros difíceis de perceber.

3.4.5 Quando Usar Inferência

A inferência de tipos é adequada quando o tipo da variável é evidente a partir do valor inicial ou quando o tipo exato não é essencial para a leitura imediata do código.

```
message := "Hello, Go!"
count := 3
enabled := false
```

A declaração explícita é preferível quando o tipo escolhido é relevante para o significado do programa.

```
var maxRetries int8 = 3
var amount float32 = 19.90
var timeoutSeconds int64 = 60
```

Em resumo, a inferência de tipos em Go é uma ferramenta de concisão, não de ambiguidade. Ela permite escrever menos código sem abrir mão da segurança da tipagem estática.

3.5 Conversão de Tipos

Em Go, tipos diferentes são considerados incompatíveis, mesmo quando representam valores semelhantes. Por esse motivo, conversões entre tipos devem ser realizadas explicitamente.

Essa decisão faz parte da filosofia da linguagem: evitar conversões implícitas que possam ocultar perda de informação ou produzir comportamentos inesperados.

3.5.1 Conversões Explícitas

A conversão é realizada utilizando o tipo desejado como uma função:

```
var x int = 10
var y int64 = int64(x)
```

Nesse exemplo, o valor armazenado em x é convertido para int64 antes de ser atribuído a y. Da mesma forma:

```
var a float64 = 3.14
var b int = int(a)
```

Após a conversão, b receberá o valor 3. A parte fracionária é descartada.

3.5.2 Não Existem Conversões Implícitas

Go não realiza conversões automáticas entre tipos numéricos.

```
var x int = 10
var y int64 = 20

var z = x + y // erro de compilação
```

Embora ambos sejam inteiros, int e int64 são tipos distintos. A conversão deve ser feita explicitamente:

```
var x int = 10
var y int64 = 20

var z = int64(x) + y
```

Essa regra torna o código mais previsível e deixa explícitas possíveis mudanças de representação.

3.5.3 Conversões Entre Tipos Numéricos

É possível converter entre diferentes tipos inteiros e de ponto flutuante.

```
var a int = 10
var b float64 = float64(a)

var c float64 = 3.14
var d int = int(c)
```

Entretanto, a conversão não altera o valor original:

```
var x int = 10
var y = float64(x)
```

Após a conversão, x continua sendo um int.

3.5.4 Perda de Informação

Nem toda conversão preserva exatamente o valor original.

Ao converter um número de ponto flutuante para um inteiro, a parte decimal é descartada:

```
var x float64 = 7.9
var y int = int(x)

fmt.Println(y)
```

Saída:

```
7
```

Conversões entre inteiros de tamanhos diferentes também podem provocar perda de informação:

```
var x int16 = 300
var y int8 = int8(x)

fmt.Println(y)
```

Saída:

```
44
```

O valor armazenado em `y` é diferente do valor original, pois 300 não pode ser representado por um `int8`.

3.5.5 Conversão de Strings

A conversão entre strings e valores numéricos não é automática.

O código abaixo produz um erro de compilação:

```
var age int = 25

var text string = string(age) // não converte para "25"
```

Nesse caso, `string(age)` interpreta o valor como um ponto de código Unicode:

```
fmt.Println(string(65))
```

Saída:

```
A
```

Para converter números em texto ou texto em números, utilizamos funções da biblioteca padrão.

3.5.6 O Pacote strconv

O pacote `strconv` fornece funções para conversão entre strings e tipos numéricos.

Convertendo uma string para inteiro:

```
import "strconv"

text := "42"

value, err := strconv.Atoi(text)
if err != nil {
    fmt.Println("valor inválido")
}

fmt.Println(value)
```

Convertendo um inteiro para string:

```
import "strconv"

value := 42

text := strconv.Itoa(value)

fmt.Println(text)
```

Também existem funções para outros tipos:

Função	Descrição
<code>strconv.ParseFloat()</code>	Converte uma string para <code>float64</code> .
<code>strconv.ParseBool()</code>	Converte uma string para <code>bool</code> .
<code>strconv.FormatFloat()</code>	Converte um <code>float64</code> para string.
<code>strconv.FormatBool()</code>	Converte um <code>bool</code> para string.

3.5.7 Conversão Entre Strings e Bytes

Strings em Go são imutáveis. Entretanto, é possível convertê-las para slices de bytes.

```
text := "Go"

bytes := []byte(text)
```

Também é possível realizar a operação inversa:

```
bytes := []byte{'G', 'o'}  
  
text := string(bytes)
```

Essas conversões são bastante utilizadas em operações de entrada e saída, arquivos e comunicação em rede.

3.5.8 Conversão e Tipos Definidos pelo Usuário

Tipos definidos a partir de outros tipos possuem identidade própria.

```
type Age int  
  
var x int = 30  
  
var y Age = Age(x)
```

Embora Age tenha como tipo subjacente `int`, a conversão explícita continua sendo necessária.

3.5.9 Conversão Não é Type Assertion

Conversões transformam um valor em outro tipo compatível.

```
var x int = 10  
  
var y float64 = float64(x)
```

Já type assertions são utilizadas com interfaces e serão estudadas posteriormente.

3.5.10 Resumo

Go exige que conversões entre tipos sejam explícitas. Essa decisão torna o código mais previsível e evita mudanças de representação ocultas. Embora muitas conversões sejam simples, é importante lembrar que algumas podem provocar perda de informação ou exigir o uso de funções da biblioteca padrão, como as fornecidas pelo pacote `strconv`.

Capítulo 4

Controle de Fluxo

Até este ponto, estudamos como representar informações por meio de variáveis, tipos e constantes. Entretanto, programas reais precisam fazer mais do que armazenar dados. Eles precisam tomar decisões, repetir operações e alterar seu comportamento de acordo com diferentes situações.

Em outras palavras, um programa precisa controlar o fluxo de execução.

Por padrão, as instruções em Go são executadas sequencialmente, de cima para baixo. Contudo, em muitos casos é necessário executar um bloco de código apenas quando uma condição é satisfeita, escolher entre múltiplos caminhos possíveis ou repetir um conjunto de instruções várias vezes. Essas tarefas são realizadas pelas estruturas de controle da linguagem.

Go adota uma abordagem simples e deliberadamente pequena. Diferentemente de algumas linguagens, que oferecem diversas construções semelhantes para resolver os mesmos problemas, Go concentra-se em um conjunto reduzido de mecanismos capazes de expressar a maior parte dos fluxos de execução de forma clara e previsível.

Neste capítulo estudaremos:

- a estrutura `if`, utilizada para execução condicional;
- a instrução `switch`, que permite selecionar diferentes caminhos de execução;
- o laço `for`, único mecanismo de repetição da linguagem;
- as instruções `break` e `continue`, responsáveis por controlar a interrupção ou a continuação de laços e estruturas de seleção.

Ao final deste capítulo, você será capaz de construir algoritmos que não apenas manipulam dados, mas também reagem a condições, repetem tarefas e controlam a sequência de execução do programa.

Como veremos, a filosofia de simplicidade de Go também se manifesta nas estruturas de controle. A linguagem oferece poucos mecanismos, mas eles são suficientemente expressivos para a construção de programas claros, legíveis e fáceis de manter.

4.1 `if`

Uma das tarefas mais comuns em um programa consiste em tomar decisões. Dependendo do valor de uma variável ou do resultado de uma expressão, diferentes instruções podem ser executadas.

Em Go, esse controle é realizado por meio da estrutura `if`.

Sua forma mais simples é:

```
if condition {  
    // instruções  
}
```

O bloco delimitado por chaves é executado apenas quando a expressão condicional resulta em `true`.

Por exemplo:

```
age := 20  
  
if age >= 18 {  
    fmt.Println("Maior de idade")  
}
```

Como `age >= 18` é verdadeiro, a mensagem será exibida.

Caso a condição seja falsa, o bloco é simplesmente ignorado.

4.1.1 Expressões booleanas

A condição de um `if` deve produzir um valor do tipo `bool`.

São comuns expressões relacionais:

```
x := 10  
  
if x > 0 {  
    fmt.Println("Positivo")  
}
```

e expressões compostas utilizando operadores lógicos:

```
age := 20  
active := true  
  
if age >= 18 && active {  
    fmt.Println("Acesso permitido")  
}
```

Ao contrário de linguagens como C, valores numéricos não são convertidos implicitamente para booleanos.

O código abaixo é inválido:


```
count := 10

if count { // erro de compilação
    fmt.Println("válido")
}
```

A expressão deve resultar explicitamente em `true` ou `false`.

4.1.2 `if` e `else`

Quando é necessário executar um bloco alternativo, utiliza-se a cláusula `else`:

```
balance := 100

if balance >= 0 {
    fmt.Println("Saldo positivo")
} else {
    fmt.Println("Saldo negativo")
}
```

Somente um dos blocos será executado.

4.1.3 Múltiplas condições

A cláusula `else if` permite testar condições adicionais:

```
grade := 7

if grade >= 9 {
    fmt.Println("Excelente")
} else if grade >= 7 {
    fmt.Println("Aprovado")
} else {
    fmt.Println("Reprovado")
}
```

As condições são avaliadas em ordem. A primeira condição verdadeira interrompe a sequência e seu bloco correspondente é executado.

Embora seja possível encadear muitos `else if`, cadeias longas tendem a dificultar a leitura. Em situações envolvendo múltiplos casos independentes, a estrutura `switch`, estudada na próxima seção, costuma produzir código mais claro.

4.1.4 Inicialização no `if`

Uma característica particular de Go é a possibilidade de executar uma instrução de inicialização antes da condição.

A sintaxe geral é:

```
if initialization; condition {  
    // instruções  
}
```

Por exemplo:

```
if value := rand.Intn(10); value > 5 {  
    fmt.Println("Maior que cinco:", value)  
}
```

A variável `value` existe apenas dentro do `if`.

Esse recurso é especialmente útil no tratamento de erros:

```
if err := saveUser(user); err != nil {  
    fmt.Println(err)  
}
```

Nesse caso, a variável `err` possui escopo limitado ao próprio `if`, evitando que ela permaneça visível após o término da estrutura.

Essa forma é extremamente comum em programas escritos em Go.

4.1.5 Escopo

Variáveis declaradas dentro do bloco de um `if` possuem escopo local:

```
if age >= 18 {  
    status := "adulto"  
    fmt.Println(status)  
}  
  
fmt.Println(status) // erro de compilação
```

O identificador `status` só existe dentro do bloco delimitado pelas chaves.

O mesmo vale para variáveis criadas na cláusula de inicialização:

```
if n := 10; n > 5 {  
    fmt.Println(n)  
}  
  
fmt.Println(n) // erro de compilação
```

Limitar o escopo das variáveis contribui para reduzir dependências e tornar o código mais fácil de compreender.

4.1.6 Ausência de parênteses

Diferentemente de muitas linguagens, Go não exige parênteses em torno da condição:

```
if age >= 18 {  
    fmt.Println("Adulto")  
}
```

Embora o uso de parênteses seja permitido em expressões internas, a forma abaixo é pouco comum e considerada desnecessária:

```
if (age >= 18) {  
    fmt.Println("Adulto")  
}
```

Essa escolha faz parte da filosofia da linguagem de evitar elementos sintáticos redundantes.

4.1.7 Resumo

A estrutura `if` permite executar diferentes caminhos de acordo com condições avaliadas em tempo de execução.

Os principais recursos oferecidos são:

- execução condicional por meio de `if`;
- escolha entre dois caminhos utilizando `else`;
- múltiplas condições com `else if`;
- declaração e inicialização de variáveis antes da condição;
- escopo limitado às chaves do bloco;
- obrigatoriedade de expressões booleanas;
- ausência de parênteses na condição.

Embora seja uma construção simples, o `if` está presente em praticamente todos os programas e constitui a base para a tomada de decisões em Go.

4.2 switch

Em muitos programas, uma decisão não envolve apenas duas possibilidades. Frequentemente é necessário escolher entre vários caminhos de execução. Embora isso possa ser realizado por meio de uma sequência de `if` e `else if`, em muitos casos a estrutura `switch` produz um código mais claro e mais fácil de manter.

Sua forma mais comum é:

```
switch expression {  
  case value1:  
    // instruções  
  case value2:  
    // instruções  
  default:  
    // instruções  
}
```

A expressão é avaliada e comparada com cada um dos valores definidos nas cláusulas case. Quando ocorre uma correspondência, o bloco correspondente é executado.

Por exemplo:

```
day := 3  
  
switch day {  
  case 1:  
    fmt.Println("Segunda-feira")  
  case 2:  
    fmt.Println("Terça-feira")  
  case 3:  
    fmt.Println("Quarta-feira")  
  default:  
    fmt.Println("Dia inválido")  
}
```

Neste caso, a saída será:

Quarta-feira

Após executar um caso, o switch é encerrado automaticamente.

4.2.1 Múltiplos valores em um caso

Uma mesma cláusula pode conter vários valores:

```
switch day {  
  case 6, 7:  
    fmt.Println("Fim de semana")  
  case 1, 2, 3, 4, 5:  
    fmt.Println("Dia útil")  
}
```

Essa forma evita repetições desnecessárias.

4.2.2 O caso default

A cláusula default é opcional e é executada quando nenhum dos casos anteriores é satisfeito.

```
status := 500

switch status {
case 200:
    fmt.Println("Sucesso")
case 404:
    fmt.Println("Não encontrado")
default:
    fmt.Println("Erro desconhecido")
}
```

Embora opcional, o uso de default normalmente torna o código mais robusto, pois explicita o tratamento para situações não previstas.

4.2.3 switch sem expressão

Em Go, a expressão após switch é opcional.

Quando omitida, a expressão true é assumida implicitamente:

```
grade := 7

switch {
case grade >= 9:
    fmt.Println("Excelente")
case grade >= 7:
    fmt.Println("Aprovado")
default:
    fmt.Println("Reprovado")
}
```

Essa forma pode substituir longas cadeias de if e else if, frequentemente produzindo código mais organizado.

Na prática, é bastante comum encontrar esse padrão em programas Go.

4.2.4 Inicialização no switch

Assim como ocorre com if, é possível executar uma instrução de inicialização antes da expressão:

```
switch value := rand.Intn(10); value {
case 0:
    fmt.Println("Zero")
case 1:
    fmt.Println("Um")
default:
    fmt.Println("Outro valor")
}
```

A variável `value` possui escopo limitado ao próprio `switch`.

4.2.5 Avaliação dos casos

Os casos são avaliados de cima para baixo.

A primeira correspondência encontrada é executada, e os demais casos são ignorados.

```
number := 10

switch {
case number > 0:
    fmt.Println("Positivo")
case number > 5:
    fmt.Println("Maior que cinco")
}
```

A saída será:

Positivo

Embora ambas as condições sejam verdadeiras, apenas o primeiro caso correspondente é executado.

Por esse motivo, a ordem dos casos pode ser importante.

4.2.6 fallthrough

Diferentemente de linguagens como C, Go não continua automaticamente para o próximo caso.

Considere:

```
switch n := 1; n {
case 1:
    fmt.Println("Um")
case 2:
    fmt.Println("Dois")
}
```

A saída será:

Um

O caso seguinte não é executado.

Entretanto, existe a instrução `fallthrough`, que força a execução do próximo caso:

```
switch n := 1; n {  
case 1:  
    fmt.Println("Um")  
    fallthrough  
case 2:  
    fmt.Println("Dois")  
}
```

Saída:

Um
Dois

O uso de `fallthrough` é relativamente raro em Go. Em geral, sua necessidade pode indicar que outra estrutura seria mais apropriada.

4.2.7 Expressões nos casos

As cláusulas `case` não precisam conter apenas constantes.

Elas podem conter expressões:

```
x := 20  
  
switch {  
case x%2 == 0:  
    fmt.Println("Par")  
default:  
    fmt.Println("Ímpar")  
}
```

Também é possível utilizar chamadas de função:

```
switch {  
case strings.HasPrefix(name, "A"):  
    fmt.Println("Começa com A")  
default:  
    fmt.Println("Outro nome")  
}
```

4.2.8 Resumo

A estrutura `switch` permite selecionar entre diferentes caminhos de execução de forma clara e organizada.

Seus principais recursos incluem:

- seleção entre múltiplos casos;
- agrupamento de vários valores em um mesmo `case`;
- cláusula opcional `default`;
- utilização sem expressão, substituindo cadeias de `if`;
- inicialização antes do teste;
- interrupção automática após cada caso;
- uso opcional de `fallthrough`;
- avaliação sequencial dos casos.

Em Go, o `switch` é uma das estruturas mais versáteis da linguagem e frequentemente produz código mais legível do que longas sequências de `if` e `else if`.

4.3 for

Repetição é uma das operações mais comuns em programação. Muitas vezes é necessário executar o mesmo conjunto de instruções diversas vezes, seja para percorrer uma coleção, processar dados ou repetir uma tarefa até que determinada condição seja satisfeita.

Em Go, toda repetição é realizada por meio da estrutura `for`.

Diferentemente de outras linguagens, Go não possui construções separadas como `while` e `do while`. Um único mecanismo é capaz de expressar os principais tipos de laços de repetição.

A forma clássica do `for` é:

```
for initialization; condition; post {  
    // instruções  
}
```

Por exemplo:

```
for i := 1; i <= 5; i++ {  
    fmt.Println(i)  
}
```

A saída será:

```
1  
2  
3  
4  
5
```

A execução ocorre em três etapas:

1. A instrução de inicialização é executada uma única vez.
2. A condição é avaliada antes de cada iteração.
3. Ao final de cada repetição, a expressão de atualização é executada.

4.3.1 for como while

Os elementos de inicialização e atualização são opcionais.

Assim, o for pode ser utilizado como um while:

```
count := 1

for count <= 5 {
    fmt.Println(count)
    count++
}
```

A condição é avaliada antes de cada iteração. Quando ela se torna falsa, o laço é encerrado.

Essa forma é bastante comum quando não existe uma variável de controle explícita.

4.3.2 Laços infinitos

Quando nenhuma condição é especificada, o laço torna-se infinito:

```
for {
    fmt.Println("Executando...")
}
```

Normalmente, laços infinitos são interrompidos por meio de um break, de um retorno da função ou do encerramento do programa.

Um exemplo típico é o processamento contínuo em servidores:

```
for {
    request := waitRequest()
    handleRequest(request)
}
```

4.3.3 Percorrendo coleções com range

Go fornece a cláusula range, que simplifica a iteração sobre estruturas como arrays, slices, strings, mapas e canais.

Por exemplo:

```
names := []string{"Ana", "Maria", "Carlos"}

for i, name := range names {
    fmt.Println(i, name)
}
```

Saída:

```
0 Ana
1 Maria
2 Carlos
```

O primeiro valor corresponde ao índice e o segundo ao elemento.

Caso apenas o valor seja necessário, pode-se ignorar o índice utilizando o identificador em branco:

```
for _, name := range names {
    fmt.Println(name)
}
```

Se apenas o índice for necessário:

```
for i := range names {
    fmt.Println(i)
}
```

O uso de range será estudado novamente nos capítulos dedicados a arrays, slices e mapas.

4.3.4 Iterando sobre strings

Strings em Go são codificadas em UTF-8. Ao utilizar range, a iteração ocorre sobre runes (pontos de código Unicode), e não sobre bytes individuais.

```
word := "ação"

for i, r := range word {
    fmt.Println(i, r)
}
```

Uma forma mais conveniente de visualizar o caractere é:

```
for i, r := range word {
    fmt.Printf("%d %c\n", i, r)
}
```

Saída:

```
0 a
1 ç
3 ã
5 o
```

Observe que os índices representam posições em bytes, não necessariamente em caracteres.

4.3.5 Variáveis do laço

É comum utilizar variáveis de controle:

```
for i := 0; i < 10; i++ {
    fmt.Println(i)
}
```

A variável `i` possui escopo restrito ao próprio laço:

```
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

fmt.Println(i) // erro de compilação
```

Limitar o escopo das variáveis reduz a possibilidade de erros e melhora a legibilidade.

4.3.6 Múltiplas variáveis

É possível utilizar várias variáveis na inicialização e atualização:

```
for i, j := 0, 10; i < j; i, j = i+1, j-1 {
    fmt.Println(i, j)
}
```

Saída:

```
0 10
1 9
2 8
3 7
4 6
```

Embora seja válido, expressões muito complexas na cláusula do `for` tendem a prejudicar a clareza.

4.3.7 Laços aninhados

Um laço pode conter outro laço:

```
for i := 1; i <= 3; i++ {  
    for j := 1; j <= 2; j++ {  
        fmt.Println(i, j)  
    }  
}
```

Saída:

```
1 1  
1 2  
2 1  
2 2  
3 1  
3 2
```

Laços aninhados são comuns no processamento de matrizes e algoritmos envolvendo múltiplas dimensões.

4.3.8 Ausência de parênteses

Assim como ocorre com `if`, Go não exige parênteses na condição:

```
for i < 10 {  
    i++  
}
```

Essa escolha faz parte da filosofia da linguagem de evitar elementos sintáticos desnecessários.

4.3.9 Resumo

A estrutura `for` é o único mecanismo de repetição de Go e pode assumir diferentes formas.

Entre suas principais aplicações estão:

- laços tradicionais com inicialização, condição e atualização;
- repetições condicionais, equivalentes ao `while`;
- laços infinitos;
- iteração sobre coleções utilizando `range`;
- processamento de strings Unicode;
- utilização de múltiplas variáveis de controle;
- construção de loops aninhados.

A existência de apenas uma estrutura de repetição reflete uma característica recorrente de Go: oferecer poucos mecanismos, mas suficientemente gerais para expressar diferentes formas de controle de fluxo de maneira simples e consistente.

4.4 break e continue

Em muitos casos, não basta apenas repetir um conjunto de instruções. Também é necessário controlar o comportamento do laço durante sua execução, interrompendo-o prematuramente ou ignorando determinadas iterações.

Em Go, esse controle é realizado principalmente por meio das instruções `break` e `continue`.

Embora sejam frequentemente utilizadas em conjunto com laços `for`, a instrução `break` também pode ser empregada em estruturas `switch`. Já `continue` atua sobre laços, mesmo quando aparece dentro de um `switch` contido em um `for`.

4.4.1 A instrução `break`

A instrução `break` encerra imediatamente a execução do laço mais interno.

Considere o exemplo:

```
for i := 1; i <= 10; i++ {  
    if i == 5 {  
        break  
    }  
  
    fmt.Println(i)  
}
```

A saída será:

```
1  
2  
3  
4
```

Quando `i` assume o valor 5, a instrução `break` é executada e o laço é encerrado.

Sem o `break`, o laço continuaria até `i == 10`.

4.4.2 Encerrando laços infinitos

Uma utilização comum de `break` é em laços infinitos:

```
count := 1

for {
    fmt.Println(count)

    if count == 5 {
        break
    }

    count++
}
```

Saída:

```
1
2
3
4
5
```

Embora o laço seja potencialmente infinito, a condição de parada é controlada internamente por meio do `break`.

Esse padrão é comum em servidores, consumidores de filas e programas interativos.

4.4.3 A instrução `continue`

A instrução `continue` interrompe apenas a iteração atual e faz o laço prosseguir para a próxima repetição.

Por exemplo:

```
for i := 1; i <= 5; i++ {
    if i == 3 {
        continue
    }

    fmt.Println(i)
}
```

Saída:

```
1
2
4
5
```

Quando `i` é igual a 3, o restante do bloco é ignorado, mas o laço continua normalmente.

4.4.4 Filtrando valores

continue é frequentemente utilizado para descartar elementos que não interessam ao processamento.

```
numbers := []int{1, 2, 3, 4, 5, 6}

for _, n := range numbers {
    if n%2 != 0 {
        continue
    }

    fmt.Println(n)
}
```

Saída:

```
2
4
6
```

Nesse caso, os números ímpares são ignorados.

Esse padrão geralmente produz um código mais simples do que grandes blocos de código aninhados:

```
for _, n := range numbers {
    if n%2 == 0 {
        fmt.Println(n)
    }
}
```

Em muitos casos, utilizar continue permite reduzir níveis de indentação e melhorar a legibilidade.

4.4.5 break em switch

Em Go, o switch termina automaticamente após executar um caso.

Por isso, normalmente não é necessário escrever:

```
switch day {
case 1:
    fmt.Println("Segunda")
    break
case 2:
    fmt.Println("Terça")
}
```

O break acima é redundante.

Entretanto, ele pode ser utilizado para sair antecipadamente de um switch em situações mais complexas.

4.4.6 Laços aninhados

Quando existem vários laços aninhados, um break afeta apenas o laço mais interno.

```
for i := 1; i <= 3; i++ {  
    for j := 1; j <= 3; j++ {  
        if j == 2 {  
            break  
        }  
  
        fmt.Println(i, j)  
    }  
}
```

Saída:

```
1 1  
2 1  
3 1
```

Observe que apenas o laço interno é interrompido.

O laço externo continua sua execução normalmente.

4.4.7 Labels

Go permite associar rótulos (labels) aos laços, possibilitando controlar qual estrutura deve ser interrompida.

Por exemplo:

```
outer:  
for i := 1; i <= 3; i++ {  
    for j := 1; j <= 3; j++ {  
        if i*j == 4 {  
            break outer  
        }  
  
        fmt.Println(i, j)  
    }  
}
```

Saída:

```
1 1  
1 2
```



```
1 3
2 1
```

Quando a condição é satisfeita, ambos os laços são encerrados.

Da mesma forma, é possível utilizar *continue* com labels:

```
outer:
for i := 1; i <= 3; i++ {
    for j := 1; j <= 3; j++ {
        if j == 2 {
            continue outer
        }

        fmt.Println(i, j)
    }
}
```

Saída:

```
1 1
2 1
3 1
```

Nesse caso, a execução passa diretamente para a próxima iteração do laço externo.

Embora labels sejam um recurso poderoso, seu uso é relativamente raro e deve ser reservado para situações em que realmente simplifiquem o código.

4.4.8 **break**, **continue** e legibilidade

As instruções *break* e *continue* podem tornar o código mais claro ao eliminar níveis desnecessários de aninhamento.

Por exemplo:

```
for _, user := range users {
    if !user.Active {
        continue
    }

    process(user)
}
```

Frequentemente essa forma é mais legível do que:

```
for _, user := range users {  
    if user.Active {  
        process(user)  
    }  
}
```

Por outro lado, o uso excessivo dessas instruções pode dificultar a compreensão do fluxo do programa.

Como em muitos aspectos de Go, o objetivo é favorecer clareza e simplicidade.

4.4.9 Resumo

As instruções `break` e `continue` permitem controlar o comportamento de laços. Além disso, `break` também pode interromper estruturas `switch`.

- `break` encerra o laço ou `switch` mais interno;
- `continue` interrompe apenas a iteração atual e inicia a próxima;
- ambos podem ser utilizados com labels para controlar laços externos;
- `break` é frequentemente empregado em laços infinitos;
- `continue` é útil para eliminar casos indesejados e reduzir níveis de indentação.

Essas instruções complementam a estrutura `for`, permitindo construir fluxos de repetição mais flexíveis e expressivos, sem aumentar a complexidade da linguagem.

4.5 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando as estruturas de controle estudadas aqui. Em muitos casos, existem várias soluções possíveis. Priorize programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui significativamente para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar a uma resposta correta.

1. Considere uma variável inteira contendo um número qualquer. Determine se o valor armazenado é positivo, negativo ou igual a zero e exiba uma mensagem correspondente.
2. Considere uma variável inteira representando a idade de uma pessoa. Determine se ela é maior ou menor de idade, assumindo que a maioridade é atingida aos 18 anos.
3. Considere uma variável inteira e determine se o número armazenado é par ou ímpar.
4. Considere duas variáveis inteiras contendo valores distintos. Compare os dois números e determine qual deles é o maior.

5. Considere três variáveis inteiras. Determine qual delas contém o maior valor e exiba esse valor.

6. Considere uma variável contendo uma nota entre 0 e 10. Classifique o desempenho do aluno como reprovado, em recuperação ou aprovado, de acordo com os seguintes critérios:

- notas inferiores a 6: Reprovado;
- notas entre 6 e 7,9: Recuperação;
- notas iguais ou superiores a 8: Aprovado.

7. Considere uma variável inteira contendo um valor entre 1 e 7. Determine qual dia da semana é representado por esse número e exiba o seu nome correspondente.

8. Considere uma variável inteira contendo um valor entre 1 e 12, representando um mês do ano. Determine quantos dias possui o mês correspondente.

9. Considere duas variáveis numéricas e uma terceira variável indicando uma das quatro operações aritméticas básicas (+, -, * ou /). Determine o resultado da operação indicada e exiba o valor obtido.

10. Utilizando uma estrutura de repetição, imprima todos os números inteiros de 1 até 100 em ordem crescente.

11. Utilizando uma estrutura de repetição, imprima todos os números inteiros de 100 até 1 em ordem decrescente.

12. Considere uma variável inteira positiva N. Calcule a soma de todos os números inteiros entre 1 e N, inclusive.

Por exemplo, para:

$N = 5$

o resultado esperado é:

$1 + 2 + 3 + 4 + 5 = 15$

13. Considere uma variável inteira contendo um número qualquer. Exiba a sua tabuada, apresentando os resultados das multiplicações pelos números de 1 a 10.

Por exemplo, para:

7

a saída deve conter:

$7 \times 1 = 7$

$7 \times 2 = 14$

...

$7 \times 10 = 70$

14. Considere uma variável inteira positiva. Calcule o fatorial do número armazenado.

Por exemplo:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

15. Considere duas variáveis inteiras positivas, base e expoente. Calcule o valor de base elevado a expoente, sem utilizar funções da biblioteca padrão.

Por exemplo, para:

```
base = 2  
expoente = 5
```

o resultado esperado é:

```
25 = 32
```

16. Determine o menor múltiplo de 17 que seja maior que 1000. Utilize um laço de repetição e interrompa a busca assim que o valor for encontrado.

17. Calcule a soma de todos os números pares entre 1 e 100. Os números ímpares devem ser ignorados durante o processamento.

18. Considere uma variável inteira positiva. Determine quantos divisores inteiros positivos esse número possui.

Por exemplo, para:

```
12
```

os divisores positivos são:

```
1, 2, 3, 4, 6 e 12
```

Portanto, o resultado esperado é:

```
6
```

Os exercícios a seguir continuam praticando controle de fluxo, mas também utilizam coleções como slices, arrays e mapas. Se você ainda não estudou esses recursos, volte a eles depois do próximo capítulo.

19. Considere um slice de inteiros. Percorra seus elementos e determine qual deles possui o maior valor.

Por exemplo:

```
numbers := []int{15, 7, 42, 18, 9}
```

Resultado esperado:

```
42
```

20. Considere um slice de inteiros e calcule a média aritmética de seus elementos.

Por exemplo:

```
numbers := []int{10, 20, 30, 40}
```

Resultado esperado:

```
25
```

21. Considere o seguinte slice:

```
numbers := []int{1, 2, 3, 4, 5}
```

Inverta a ordem dos elementos, de modo que o resultado seja:

```
[]int{5, 4, 3, 2, 1}
```

A solução deve modificar o próprio slice, sem criar um segundo slice para armazenar os valores.

22. Considere um slice de inteiros. Percorra seus elementos e determine quantos deles possuem valor par.

Por exemplo:

```
numbers := []int{3, 8, 5, 12, 7, 10}
```

Resultado esperado:

3

23. Considere um slice contendo valores positivos e negativos. Construa um novo slice contendo apenas os elementos positivos.

Por exemplo:

```
numbers := []int{-5, 8, -2, 10, 0, 7, -1}
```

Resultado esperado:

```
[]int{8, 10, 7}
```

24. Considere uma matriz 3×3 de inteiros. Percorra todos os seus elementos e calcule a soma total dos valores armazenados.

Por exemplo:

```
matrix := [3][3]int{
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
}
```

Resultado esperado:

45

25. Considere uma matriz quadrada. Calcule separadamente a soma dos elementos da diagonal principal e da diagonal secundária.

Por exemplo:

```
matrix := [3][3]int{
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
}
```

Diagonal principal:

1 + 5 + 9 = 15

Diagonal secundária:

3 + 5 + 7 = 15

26. Considere uma string. Determine quantas vezes cada caractere aparece e armazene essas informações em um `map[rune] int`.

Por exemplo, para a string:

banana

o resultado esperado é:

b -> 1
a -> 3
n -> 2

27. Considere uma frase formada por palavras separadas por espaços. Determine quantas vezes cada palavra aparece e armazene essas informações em um `map[string] int`.

Por exemplo, para a frase:

go é simples e go é rápido

o resultado esperado é:

go -> 2
é -> 2
simples -> 1
e -> 1
rápido -> 1

28. Considere uma string. Determine qual caractere ocorre com maior frequência.

Por exemplo, para a string:

banana

o resultado esperado é:

a -> 3

29. Considere uma variável inteira positiva. Determine se o número armazenado é primo.

Um número primo é divisível apenas por 1 e por ele mesmo.

Por exemplo:

7

é primo, enquanto:

12

não é.

30. Escolha um número inteiro e simule tentativas sucessivas até que o valor seja encontrado. Utilize uma estrutura de repetição para representar as tentativas e interrompa a execução assim que o número correto for alcançado.

Por exemplo, se o número escolhido for:

7

e as tentativas forem realizadas em ordem crescente, a sequência será:

1
2
3
4
5
6
7

encerrando o laço quando o valor procurado for encontrado.

Capítulo 5

Coleções

Nos capítulos anteriores, estudamos como representar informações por meio de variáveis e como controlar a execução dos programas utilizando estruturas condicionais e laços de repetição. Entretanto, na prática, raramente trabalhamos com valores isolados. Em vez disso, normalmente precisamos manipular conjuntos de dados.

Uma aplicação pode precisar armazenar uma lista de nomes, processar uma sequência de valores numéricos, manter uma coleção de registros ou associar chaves a determinados valores. Para lidar com essas situações, Go fornece diferentes estruturas de dados conhecidas genericamente como coleções.

As coleções permitem agrupar múltiplos valores e tratá-los como uma única entidade lógica. Elas constituem uma das bases para a construção de programas mais elaborados, pois tornam possível armazenar, percorrer e organizar grandes quantidades de informação de maneira eficiente.

Neste capítulo estudaremos três das principais coleções da linguagem:

- **Arrays**, estruturas de tamanho fixo cujos elementos possuem o mesmo tipo;
- **Slices**, estruturas flexíveis e dinâmicas construídas sobre arrays, que constituem uma das abstrações mais importantes de Go;
- **Maps**, coleções que associam chaves a valores, permitindo consultas rápidas e eficientes.

Embora os arrays sejam a estrutura fundamental sobre a qual os slices são construídos, na prática os slices são muito mais utilizados em programas Go. Por esse motivo, compreender a relação entre arrays e slices é essencial para entender como a linguagem representa e manipula sequências de dados.

Ao longo deste capítulo veremos não apenas como declarar e utilizar essas coleções, mas também algumas das ideias que orientaram seu projeto. Como ocorre em diversos aspectos da linguagem, Go procura oferecer mecanismos simples, mas suficientemente poderosos para atender à maioria das necessidades do desenvolvimento cotidiano.

Ao final deste capítulo, seremos capazes de armazenar, percorrer e manipular conjuntos de dados, construindo estruturas que servirão de base para os tipos compostos e abstrações mais avançadas estudados nos capítulos seguintes.

5.1 Arrays

Uma das formas mais simples de armazenar vários valores do mesmo tipo consiste em agrupá-los em uma sequência ordenada. Em Go, essa estrutura é denominada **array**.

Um array representa uma coleção de elementos do mesmo tipo armazenados em posições consecutivas da memória. Cada elemento é identificado por um índice, iniciado em zero.

Por exemplo:

```
var numbers [5]int
```

Essa declaração cria um array capaz de armazenar cinco valores do tipo `int`.

Como ocorre com qualquer variável em Go, todos os elementos são inicializados automaticamente com seus respectivos valores zero:

```
fmt.Println(numbers)
```

Saída:

```
[0 0 0 0 0]
```

5.1.1 Inicialização

Um array pode ser inicializado durante a declaração:

```
numbers := [5]int{10, 20, 30, 40, 50}
```

Os valores são armazenados na ordem em que aparecem.

Também é possível permitir que o compilador determine o tamanho do array:

```
numbers := [...]int{10, 20, 30, 40, 50}
```

Nesse caso, o comprimento é inferido automaticamente a partir do número de elementos.

5.1.2 Acessando Elementos

Os elementos são acessados por meio de índices.

```
numbers := [5]int{10, 20, 30, 40, 50}

fmt.Println(numbers[0])
fmt.Println(numbers[3])
```

Saída:

```
10  
40
```

Os índices variam entre 0 e `len(array)-1`.

Tentativas de acesso fora desses limites produzem um erro em tempo de execução:

```
fmt.Println(numbers[5])
```

```
panic: runtime error: index out of range
```

5.1.3 Alterando Elementos

Os elementos de um array podem ser modificados individualmente:

```
numbers := [5]int{10, 20, 30, 40, 50}  
  
numbers[2] = 100  
  
fmt.Println(numbers)
```

Saída:

```
[10 20 100 40 50]
```

5.1.4 Comprimento

A função `len` retorna o número de elementos do array:

```
numbers := [5]int{10, 20, 30, 40, 50}  
  
fmt.Println(len(numbers))
```

Saída:

```
5
```

O comprimento de um array faz parte do seu tipo.

Assim:

```
var a [3]int  
var b [5]int
```

definem dois tipos distintos.

Por esse motivo, a atribuição abaixo é inválida:

```
var a [3]int
var b [5]int

a = b // erro de compilação
```

Embora ambos contenham elementos do tipo `int`, seus comprimentos são diferentes.

5.1.5 Percorrendo Arrays

Arrays são frequentemente percorridos utilizando um laço `for`.

Com índices explícitos:

```
numbers := [5]int{10, 20, 30, 40, 50}

for i := 0; i < len(numbers); i++ {
    fmt.Println(numbers[i])
}
```

Mais frequentemente, utiliza-se `range`:

```
numbers := [5]int{10, 20, 30, 40, 50}

for i, value := range numbers {
    fmt.Println(i, value)
}
```

Saída:

```
0 10
1 20
2 30
3 40
4 50
```

Caso o índice não seja necessário:

```
for _, value := range numbers {
    fmt.Println(value)
}
```

5.1.6 Arrays São Valores

Uma característica importante dos arrays em Go é que eles possuem semântica de valor.

Considere:

```
a := [3]int{1, 2, 3}
b := a

b[0] = 100

fmt.Println(a)
fmt.Println(b)
```

Saída:

```
[1 2 3]
[100 2 3]
```

A atribuição `b := a` produz uma cópia completa do array.

Alterações em `b` não afetam `a`.

Da mesma forma, quando um array é passado para uma função, uma cópia é realizada:

```
func change(a [3]int) {
    a[0] = 100
}
```

```
numbers := [3]int{1, 2, 3}

change(numbers)

fmt.Println(numbers)
```

Saída:

```
[1 2 3]
```

O array original permanece inalterado.

Mais adiante estudaremos como ponteiros podem ser utilizados para evitar essa cópia.

5.1.7 Arrays Multidimensionais

Arrays podem conter outros arrays:

```
matrix := [2][3]int{
    {1, 2, 3},
    {4, 5, 6},
}
```

Os elementos são acessados por meio de múltiplos índices:

```
fmt.Println(matrix[1][2])
```

Saída:

```
6
```

Uma forma comum de percorrer uma matriz é utilizando laços aninhados:

```
for i := range matrix {  
    for j := range matrix[i] {  
        fmt.Println(matrix[i][j])  
    }  
}
```

5.1.8 Quando Usar Arrays

Embora arrays sejam estruturas fundamentais da linguagem, eles são relativamente pouco utilizados diretamente.

Isso ocorre porque:

- possuem tamanho fixo;
- o tamanho faz parte do tipo;
- atribuições produzem cópias completas.

Na prática, a maior parte dos programas utiliza **slices**, que oferecem maior flexibilidade e são construídos sobre arrays.

Entretanto, compreender arrays é importante porque eles constituem a base sobre a qual os slices são implementados.

5.1.9 Resumo

Arrays representam sequências ordenadas de elementos do mesmo tipo.

Suas principais características são:

- tamanho fixo;
- índices iniciados em zero;
- comprimento faz parte do tipo;
- semântica de valor;
- armazenamento contíguo em memória;
- suporte a múltiplas dimensões.

Embora seu uso direto seja relativamente raro, arrays desempenham um papel fundamental na linguagem e servem de base para uma das estruturas mais importantes de Go: os slices.

5.2 Slices

Embora arrays sejam a estrutura fundamental utilizada pela linguagem para representar sequências de elementos, na prática eles são utilizados diretamente com pouca frequência. A maioria dos programas em Go utiliza **slices**, uma estrutura mais flexível construída sobre arrays.

Um slice representa uma visão sobre uma sequência de elementos. Diferentemente dos arrays, seu tamanho não faz parte do tipo, o que permite trabalhar com coleções cujo número de elementos pode variar ao longo da execução do programa.

Uma declaração simples é:

```
var numbers []int
```

Nesse caso, numbers é um slice de inteiros.

O valor zero de um slice é nil:

```
var numbers []int

fmt.Println(numbers)
fmt.Println(numbers == nil)
fmt.Println(len(numbers))
```

Saída:

```
[]
true
0
```

5.2.1 Inicialização

Um slice pode ser inicializado com valores:

```
numbers := []int{10, 20, 30, 40, 50}
```

Ao contrário dos arrays, não é necessário especificar um tamanho:

```
fmt.Printf("%T\n", numbers)
```

Saída:

```
[]int
```

5.2.2 Comprimento

A função len retorna a quantidade de elementos do slice:

```
numbers := []int{10, 20, 30}

fmt.Println(len(numbers))
```

Saída:

```
3
```

Os elementos são acessados pelos mesmos índices utilizados em arrays:

```
numbers := []int{10, 20, 30}

fmt.Println(numbers[0])
fmt.Println(numbers[2])
```

Saída:

```
10
30
```

5.2.3 Criando com make

Uma forma bastante comum de criar slices é por meio da função make:

```
numbers := make([]int, 5)

fmt.Println(numbers)
```

Saída:

```
[0 0 0 0 0]
```

Todos os elementos são inicializados com seus respectivos valores zero.

Também é possível especificar a capacidade:

```
numbers := make([]int, 5, 10)
```

Nesse caso:

- comprimento (len) = 5;
- capacidade (cap) = 10.

```
fmt.Println(len(numbers))
fmt.Println(cap(numbers))
```

Saída:

```
5
10
```


5.2.4 Append

Uma das características mais importantes dos slices é a possibilidade de adicionar novos elementos por meio da função `append`.

```
numbers := []int{10, 20}

numbers = append(numbers, 30)

fmt.Println(numbers)
```

Saída:

```
[10 20 30]
```

É possível adicionar vários elementos de uma vez:

```
numbers = append(numbers, 40, 50, 60)
```

ou até mesmo concatenar slices:

```
a := []int{1, 2}
b := []int{3, 4}

a = append(a, b...)
```

Resultado:

```
[1 2 3 4]
```

5.2.5 Capacidade

Além do comprimento, um slice possui uma capacidade.

```
numbers := make([]int, 3, 5)

fmt.Println(len(numbers))
fmt.Println(cap(numbers))
```

Saída:

```
3
5
```

A capacidade representa quantos elementos podem ser armazenados antes que seja necessário alocar um novo array interno.

Quando a capacidade é excedida, o runtime cria um novo array e copia os elementos existentes.

Esse processo é transparente para o programador:

```
numbers := []int{1, 2, 3}

fmt.Println(len(numbers), cap(numbers))

numbers = append(numbers, 4)

fmt.Println(len(numbers), cap(numbers))
```

5.2.6 Fatiamento

Slices podem ser obtidos a partir de outros slices ou arrays por meio da operação de fatiamento:

```
numbers := []int{10, 20, 30, 40, 50}

part := numbers[1:4]

fmt.Println(part)
```

Saída:

```
[20 30 40]
```

Os limites seguem a convenção:

```
[start:end]
```

incluindo start e excluindo end.

Por exemplo:

```
numbers[:3]
numbers[2:]
numbers[:]
```

produzem:

```
[10 20 30]
[30 40 50]
[10 20 30 40 50]
```

5.2.7 Compartilhamento de Dados

Uma característica importante é que slices compartilham o mesmo array subjacente.

Considere:

```
numbers := []int{10, 20, 30, 40, 50}

part := numbers[1:4]

part[0] = 200

fmt.Println(numbers)
fmt.Println(part)
```

Saída:

```
[10 200 30 40 50]
[200 30 40]
```

Alterações realizadas em `part` afetam o slice original, pois ambos compartilham os mesmos dados.

5.2.8 Copiando Slices

Quando é necessário criar uma cópia independente, utiliza-se a função `copy`:

```
a := []int{10, 20, 30}

b := make([]int, len(a))

copy(b, a)

b[0] = 100

fmt.Println(a)
fmt.Println(b)
```

Saída:

```
[10 20 30]
[100 20 30]
```

Agora os dois slices são independentes.

5.2.9 Percorrendo Slices

Slices são frequentemente percorridos utilizando `range`:

```
numbers := []int{10, 20, 30}

for i, value := range numbers {
    fmt.Println(i, value)
}
```

Saída:

```
0 10
1 20
2 30
```

5.2.10 Slices São Descritores

Um slice não contém os elementos em si.

Ele é apenas uma pequena estrutura composta por:

- ponteiro para o array subjacente;
- comprimento (len);
- capacidade (cap).

Por esse motivo, atribuições não copiam os dados:

```
a := []int{1, 2, 3}

b := a

b[0] = 100

fmt.Println(a)
fmt.Println(b)
```

Saída:

```
[100 2 3]
[100 2 3]
```

Ambos os slices apontam para o mesmo array.

Essa característica é uma das diferenças mais importantes em relação aos arrays.

5.2.11 Quando Usar Slices

Slices são a principal estrutura utilizada para representar sequências em Go.

São apropriados quando:

- o tamanho da coleção pode variar;
- é necessário adicionar elementos dinamicamente;
- deseja-se compartilhar dados entre diferentes partes do programa;
- a eficiência de memória é importante.

Na prática, a maior parte das APIs da biblioteca padrão trabalha com slices.

5.2.12 Resumo

Slices representam sequências flexíveis de elementos e constituem uma das estruturas mais importantes da linguagem.

Suas principais características são:

- tamanho variável;
- suporte à função `append`;
- comprimento (`len`) e capacidade (`cap`);
- possibilidade de fatiamento;
- compartilhamento do array subjacente;
- cópia explícita por meio de `copy`;
- semântica baseada em descritores, e não em cópia de valores.

Compreender slices é fundamental para escrever programas idiomáticos em Go, pois eles aparecem em praticamente toda a biblioteca padrão e em grande parte do código produzido na linguagem.

5.3 Maps

Em muitas situações, armazenar elementos em uma sequência ordenada não é suficiente. Frequentemente precisamos associar um valor a uma chave, permitindo consultas rápidas sem a necessidade de percorrer toda uma coleção.

Por exemplo, podemos desejar associar nomes a idades, identificadores a usuários ou códigos a descrições. Em Go, esse tipo de estrutura é representado pelos **maps**.

Um map é uma coleção de pares chave-valor. Cada chave é única e permite acessar diretamente o valor associado.

Uma declaração simples é:

```
var ages map[string]int
```

Nesse exemplo, as chaves são strings e os valores são inteiros.

Entretanto, essa declaração produz um map com valor `nil`:

```
fmt.Println(ages == nil)
```

Saída:

```
true
```

Um map `nil` pode ser consultado, mas não permite inserções.

5.3.1 Criando Maps

A forma mais comum de criar um map é por meio da função `make`:

```
ages := make(map[string]int)
```

Agora é possível adicionar elementos:

```
ages["Maria"] = 30
ages["João"] = 25

fmt.Println(ages)
```

Saída:

```
map[João:25 Maria:30]
```

Também é possível inicializar um map diretamente:

```
ages := map[string]int{
    "Maria": 30,
    "João": 25,
    "Ana": 40,
}
```

5.3.2 Acessando Valores

Os valores são obtidos por meio da chave:

```
ages := map[string]int{
    "Maria": 30,
    "João": 25,
}

fmt.Println(ages["Maria"])
```

Saída:

```
30
```

5.3.3 Chaves Inexistentes

Quando uma chave não existe, o valor zero do tipo é retornado:

```
fmt.Println(ages["Pedro"])
```

Saída:

```
0
```

Esse comportamento pode tornar ambíguo distinguir entre:

- uma chave inexistente;
- uma chave cujo valor seja efetivamente zero.

Para resolver esse problema, Go fornece uma segunda variável indicando se a chave foi encontrada:

```
age, ok := ages["Pedro"]  
  
fmt.Println(age)  
fmt.Println(ok)
```

Saída:

```
0  
false
```

Quando a chave existe:

```
age, ok := ages["Maria"]  
  
fmt.Println(age)  
fmt.Println(ok)
```

Saída:

```
30  
true
```

Esse padrão é extremamente comum em Go.

5.3.4 Alterando Valores

Atribuições substituem valores existentes:

```
ages["Maria"] = 31
```

Caso a chave ainda não exista, ela será criada automaticamente.

5.3.5 Removendo Elementos

A função `delete` remove uma entrada do map:

```
delete(ages, "João")
```

Se a chave não existir, nenhuma ação é realizada.

```
delete(ages, "Pedro")
```

não produz erro.

5.3.6 Comprimento

A função `len` retorna a quantidade de elementos:

```
fmt.Println(len(ages))
```

Saída:

```
2
```

5.3.7 Percorrendo Maps

Maps podem ser percorridos utilizando `range`:

```
ages := map[string]int{
    "Maria": 30,
    "João": 25,
    "Ana": 40,
}

for key, value := range ages {
    fmt.Println(key, value)
}
```

A saída pode ser:

```
Maria 30
João 25
Ana 40
```

ou:

```
João 25
Ana 40
Maria 30
```

5.3.8 Ordem de Iteração

Uma característica importante dos maps em Go é que a ordem de iteração não é definida.

Portanto, o código abaixo:

```
for key, value := range ages {
    fmt.Println(key, value)
}
```

não garante nenhuma ordem específica.

Caso seja necessário percorrer os elementos em uma ordem determinada, normalmente as chaves são copiadas para um slice e ordenadas:


```
keys := make([]string, 0, len(ages))

for key := range ages {
    keys = append(keys, key)
}

sort.Strings(keys)

for _, key := range keys {
    fmt.Println(key, ages[key])
}
```

5.3.9 Maps São Tipos de Referência

Assim como slices, maps não possuem semântica de valor.

Considere:

```
a := map[string]int{
    "Maria": 30,
}

b := a

b["Maria"] = 40

fmt.Println(a)
fmt.Println(b)
```

Saída:

```
map[Maria:40]
map[Maria:40]
```

Ambas as variáveis referenciam a mesma estrutura interna.

Por esse motivo, modificações realizadas em uma variável tornam-se visíveis por meio da outra.

5.3.10 Comparação

Maps não podem ser comparados entre si:

```
a := map[string]int{}
b := map[string]int{}

fmt.Println(a == b)
```

Esse código produz erro de compilação.

A única comparação permitida é com `nil`:

```
var ages map[string]int

fmt.Println(ages == nil)
```

5.3.11 Chaves Válidas

Nem todo tipo pode ser utilizado como chave.

As chaves precisam ser comparáveis.

Por exemplo:

```
map[string]int
map[int]string
map[bool]float64
```

são válidos.

Entretanto:

```
map[[]int]string
```

é inválido, pois slices não são comparáveis.

5.3.12 Quando Usar Maps

Maps são apropriados quando é necessário:

- associar chaves a valores;
- realizar buscas rápidas;
- representar tabelas de consulta;
- armazenar configurações ou índices;
- modelar conjuntos de elementos.

São amplamente utilizados em praticamente todos os tipos de aplicações.

5.3.13 Resumo

Maps representam coleções de pares chave-valor.

Suas principais características são:

- acesso por meio de chaves;
- inserção e atualização por atribuição;
- remoção com `delete`;
- consulta com retorno opcional por meio do padrão `value, ok`;
- tamanho obtido por `len`;
- ordem de iteração indefinida;

- compartilhamento da estrutura interna entre variáveis;
- comparação permitida apenas com `nil`.

Ao lado dos slices, os maps constituem uma das estruturas de dados mais utilizadas em Go e estão presentes em praticamente toda a biblioteca padrão e em grande parte dos programas escritos na linguagem.

5.4 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Em muitos casos, existem várias soluções possíveis. Priorize escrever programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui significativamente para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar à resposta correta.

1. Considere um array de inteiros contendo dez elementos. Determine se todos os valores armazenados são diferentes entre si.

Por exemplo:

```
numbers := [10]int{3, 7, 5, 9, 1, 4, 8, 2, 6, 10}
```

Resultado esperado:

Todos os elementos são distintos.

2. Considere um slice de inteiros. Desloque todos os seus elementos uma posição para a direita, fazendo com que o último elemento passe a ocupar a primeira posição.

Por exemplo:

```
numbers := []int{10, 20, 30, 40, 50}
```

Resultado esperado:

```
[]int{50, 10, 20, 30, 40}
```

A modificação deve ser realizada no próprio slice.

3. Considere dois slices de inteiros de mesmo tamanho. Construa um terceiro slice contendo a soma dos elementos de mesma posição.

Por exemplo:

```
a := []int{1, 2, 3, 4}
b := []int{10, 20, 30, 40}
```

Resultado esperado:

```
[]int{11, 22, 33, 44}
```

4. Considere uma matriz quadrada de inteiros. Determine se ela é uma matriz diagonal, isto é, se todos os elementos fora da diagonal principal são iguais a zero.

Por exemplo:

```
matrix := [3][3]int{
    {2, 0, 0},
    {0, 5, 0},
    {0, 0, 8},
}
```

Resultado esperado:

A matriz é diagonal.

5. Considere um slice de inteiros. Construa um map [int] int em que cada chave represente um valor do slice e o valor associado seja a posição da primeira ocorrência desse elemento.

Por exemplo:

```
numbers := []int{7, 3, 5, 7, 2, 5}
```

Resultado esperado:

```
7 -> 0
3 -> 1
5 -> 2
2 -> 4
```

6. Considere um slice de inteiros. Verifique se os elementos estão ordenados em ordem crescente.

Por exemplo:

```
numbers := []int{3, 7, 10, 15, 18}
```

Resultado esperado:

O slice está ordenado.

7. Considere um slice de inteiros. Desloque todos os elementos uma posição para a esquerda, fazendo com que o primeiro elemento passe a ocupar a última posição.

Por exemplo:

```
numbers := []int{10, 20, 30, 40, 50}
```

Resultado esperado:

```
[]int{20, 30, 40, 50, 10}
```

A modificação deve ser realizada no próprio slice.

8. Considere uma matriz quadrada de inteiros. Determine se todos os elementos da diagonal principal possuem o mesmo valor.

Por exemplo:

```
matrix := [3][3]int{
    {5, 1, 2},
    {3, 5, 4},
    {7, 8, 5},
}
```

Resultado esperado:

Todos os elementos da diagonal principal são iguais.

9. Considere um `map[string]int` associando produtos às suas quantidades em estoque. Determine quais produtos possuem quantidade inferior a 10 unidades.

Por exemplo:

```
stock := map[string]int{
    "Arroz": 20,
    "Feijão": 8,
    "Macarrão": 5,
    "Café": 15,
}
```

Resultado esperado:

Feijão
Macarrão

10. Considere dois slices de inteiros. Determine se eles contêm exatamente os mesmos elementos, independentemente da ordem em que aparecem.

Por exemplo:

```
a := []int{3, 7, 2, 5}
b := []int{5, 2, 7, 3}
```

Resultado esperado:

Os slices possuem os mesmos elementos.

11. Considere uma matriz 3×3 de inteiros. Construa sua matriz transposta.

Por exemplo:

```
matrix := [3][3]int{
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
}
```

Resultado esperado:

```
[3][3]int{
    {1, 4, 7},
    {2, 5, 8},
    {3, 6, 9},
}
```

12. Considere um slice de inteiros. Construa um novo slice contendo apenas os valores que aparecem mais de uma vez.

Por exemplo:

```
numbers := []int{3, 7, 3, 2, 5, 7, 9}
```

Resultado esperado:

```
[]int{3, 7}
```

13. Considere um map[string]int associando nomes de cidades às suas populações. Determine a média das populações armazenadas.

14. Considere uma string. Construa um map[rune]int contendo apenas os caracteres que aparecem exatamente uma vez.

Por exemplo:

banana

Resultado esperado:

b -> 1

Como apenas b aparece uma única vez, o mapa resultante deverá conter somente essa entrada.

15. Considere um slice de inteiros. Determine o comprimento da maior sequência de elementos consecutivos iguais.

Por exemplo:

```
numbers := []int{3, 3, 3, 7, 7, 2, 5, 5, 5, 5, 1}
```

Resultado esperado:

4

Capítulo 6

Funções

À medida que os programas crescem, torna-se cada vez mais importante organizar o código de forma clara e evitar repetições desnecessárias. Escrever toda a lógica de um programa como uma única sequência de instruções rapidamente leva a códigos difíceis de compreender, testar e manter.

As funções são um dos mecanismos fundamentais para a construção de programas bem estruturados. Elas permitem agrupar operações relacionadas sob um mesmo nome, dividir problemas complexos em partes menores e reutilizar código de maneira consistente.

Em Go, funções ocupam uma posição central. Além de receber parâmetros e retornar resultados, elas podem produzir múltiplos valores, aceitar quantidades variáveis de argumentos, ser atribuídas a variáveis e ser criadas durante a execução do programa.

Essa flexibilidade torna as funções mais do que simples sub-rotinas. Na linguagem, funções também são valores: podem ser passadas como argumentos, retornadas por outras funções e utilizadas para encapsular comportamentos específicos.

Neste capítulo, estudaremos como definir e utilizar funções em Go, explorando características como múltiplos retornos, parâmetros variádicos, funções anônimas e closures. Esses recursos ampliam as possibilidades de composição do código e servem de base para diversos padrões e abstrações encontrados na biblioteca padrão e em aplicações idiomáticas escritas em Go.

6.1 Funções

Uma função é um bloco de código responsável por executar uma determinada tarefa. Seu principal objetivo é permitir a reutilização de código e dividir problemas maiores em partes menores e mais fáceis de compreender.

Em Go, uma função é declarada com a palavra-chave `func`, seguida pelo nome da função, pela lista de parâmetros e, opcionalmente, pelo tipo do valor retornado.

Por exemplo:

```
func square(x int) int {  
    return x * x  
}
```

Nesse caso, a função `square` recebe um valor do tipo `int` e retorna outro valor do tipo `int`.

Depois de declarada, a função pode ser chamada em outras partes do programa:

```
func main() {  
    result := square(5)  
  
    fmt.Println(result)  
}
```

Saída:

25

Funções também podem receber mais de um parâmetro:

```
func sum(a int, b int) int {  
    return a + b  
}
```

Quando parâmetros consecutivos possuem o mesmo tipo, é possível omitir a repetição:

```
func sum(a, b int) int {  
    return a + b  
}
```

Nesse exemplo, `a` e `b` são parâmetros do tipo `int`.

Nem toda função precisa retornar um valor. Funções que executam apenas uma ação podem omitir o tipo de retorno:

```
func greet(name string) {  
    fmt.Println("Olá,", name)  
}
```

Até este ponto, todas as funções apresentadas retornam no máximo um valor. Entretanto, uma característica importante de Go é a possibilidade de retornar múltiplos valores em uma mesma função.

Por exemplo:

```
func divide(a, b float64) (float64, bool) {  
    if b == 0 {  
        return 0, false  
    }  
  
    return a / b, true  
}
```

A função acima retorna dois valores. O primeiro corresponde ao resultado da divisão, enquanto o segundo informa se a operação pôde ser realizada.

Os valores retornados podem ser armazenados em variáveis distintas:

```
quotient, ok := divide(10, 2)  
  
if !ok {  
    fmt.Println("divisão inválida")  
    return  
}  
  
fmt.Println(quotient)
```

Saída:

5

Múltiplos retornos são amplamente utilizados em Go. O exemplo mais comum aparece no tratamento de erros:

```
func strconv.Atoi(s string) (int, error)
```

Essa função retorna dois valores: o número convertido e um possível erro produzido durante a conversão.

```
value, err := strconv.Atoi("42")  
  
if err != nil {  
    fmt.Println("valor inválido")  
    return  
}  
  
fmt.Println(value)
```

Caso algum dos valores retornados não seja necessário, ele pode ser descartado utilizando o identificador em branco (_):

```
value, _ := strconv.Atoi("42")

fmt.Println(value)
```

O suporte a múltiplos retornos é uma das características mais marcantes da linguagem. Ele permite que funções retornem não apenas o resultado principal de uma operação, mas também informações adicionais, tornando explícitas situações como erros, estados ou resultados complementares.

6.2 Funções Variádicas

Em alguns problemas, a quantidade de argumentos recebidos por uma função não é conhecida antecipadamente. Uma função que calcula uma soma, por exemplo, pode ser chamada com dois, três ou dezenas de valores.

Para essas situações, Go fornece as funções variádicas.

Uma função variádica recebe uma quantidade variável de argumentos de um mesmo tipo. A sintaxe utiliza três pontos (...) antes do tipo do último parâmetro.

Por exemplo:

```
func sum(numbers ...int) int {
    total := 0

    for _, value := range numbers {
        total += value
    }

    return total
}
```

Nesse caso, o parâmetro `numbers` pode receber zero ou mais argumentos do tipo `int`.

A função pode ser utilizada de diferentes maneiras:

```
fmt.Println(sum())
fmt.Println(sum(10))
fmt.Println(sum(10, 20))
fmt.Println(sum(10, 20, 30, 40))
```

Saída:

```
0
10
30
100
```

Embora a chamada da função utilize argumentos independentes, dentro da função o parâmetro

variádico é tratado como um slice.

Por exemplo:

```
func show(numbers ...int) {  
    fmt.Printf("%T\n", numbers)  
}
```

Ao executar:

```
show(1, 2, 3)
```

a saída será:

```
[]int
```

Como o parâmetro é um slice, ele pode ser percorrido normalmente:

```
func show(numbers ...int) {  
    for _, value := range numbers {  
        fmt.Println(value)  
    }  
}
```

Uma função pode combinar parâmetros convencionais com um parâmetro variádico. Entretanto, o parâmetro variádico deve ser sempre o último da lista.

Por exemplo:

```
func repeat(times int, words ...string) {  
    for i := 0; i < times; i++ {  
        for _, word := range words {  
            fmt.Println(word)  
        }  
    }  
}
```

Uso:

```
repeat(2, "Go", "é", "simples")
```

Também é possível utilizar um slice já existente como argumento de uma função variádica. Para isso, usa-se novamente o operador

```
numbers := []int{10, 20, 30, 40}  
  
result := sum(numbers...)
```

Sem os três pontos, o código produziria um erro de compilação, pois `numbers` é um slice, e não uma sequência de argumentos individuais.

Funções variádicas são frequentemente utilizadas na biblioteca padrão. Um exemplo conhecido é a função `fmt.Println`:

```
fmt.Println("Go", "é", "uma", "linguagem", "simples")
```

Internamente, sua assinatura é semelhante a:

```
func Println(a ...any) (n int, err error)
```

O parâmetro variádico permite que `Println` receba qualquer quantidade de argumentos, incluindo valores de tipos diferentes.

Funções variádicas oferecem uma forma flexível de definir operações cujo número de argumentos não é conhecido previamente. Embora sejam convenientes, devem ser utilizadas quando a quantidade variável de parâmetros faz parte natural do problema, evitando tornar a interface da função desnecessariamente ambígua.

6.3 Funções Anônimas

Até agora, todas as funções apresentadas tinham um nome:

```
func sum(a, b int) int {  
    return a + b  
}
```

Entretanto, em Go, funções também podem ser criadas sem nome. Essas funções são chamadas de **funções anônimas**.

Uma função anônima possui a mesma estrutura de uma função comum, mas não apresenta um identificador após a palavra-chave `func`:

```
func(a, b int) int {  
    return a + b  
}
```

Como não possui nome, essa função não pode ser chamada posteriormente por um identificador próprio. Para reutilizá-la, normalmente ela é atribuída a uma variável:

```
sum := func(a, b int) int {  
    return a + b  
}  
  
fmt.Println(sum(10, 20))
```

Saída:

```
30
```

Nesse exemplo, a variável `sum` passa a armazenar uma função. Isso é possível porque, em Go, funções são valores e podem ser atribuídas a variáveis da mesma forma que números, strings ou structs.

Uma variável também pode ser declarada explicitamente com um tipo de função:

```
var operation func(int, int) int
```

Posteriormente, qualquer função com a mesma assinatura pode ser atribuída a essa variável:

```
operation = func(a, b int) int {  
    return a + b  
}  
  
fmt.Println(operation(5, 7))
```

Saída:

```
12
```

Funções anônimas também podem ser executadas imediatamente após sua definição. Esse padrão é conhecido como *Immediately Invoked Function Expression* (IIFE), ou expressão de função imediatamente invocada.

Por exemplo:

```
func() {  
    fmt.Println("Olá, Go!")  
}()
```

Saída:

```
Olá, Go!
```

Nesse caso, os parênteses finais fazem a chamada imediata da função.

Funções anônimas podem receber parâmetros:

```
func(name string) {  
    fmt.Println("Olá,", name)  
}("Maria")
```

Saída:

```
Olá, Maria
```

Uma das aplicações mais comuns das funções anônimas é usá-las como argumentos de outras

funções.

Considere a seguinte função:

```
func apply(a, b int, operation func(int, int) int) int {  
    return operation(a, b)  
}
```

Podemos utilizá-la da seguinte forma:

```
result := apply(10, 20, func(a, b int) int {  
    return a + b  
})  
  
fmt.Println(result)
```

Saída:

```
30
```

Nesse exemplo, a função responsável pela soma é criada diretamente no momento da chamada, sem necessidade de uma declaração separada.

O fato de funções serem valores permite construir abstrações mais flexíveis e serve de base para diversos padrões utilizados em Go. No próximo tópico, veremos como funções anônimas podem acessar variáveis definidas em seu contexto externo, formando estruturas conhecidas como *closures*.

6.4 Closures

Uma função anônima pode acessar variáveis declaradas no ambiente em que foi criada.

Considere o exemplo:

```
func main() {  
    name := "Maria"  
  
    greet := func() {  
        fmt.Println("Olá,", name)  
    }  
  
    greet()  
}
```

Saída:

```
Olá, Maria
```

Embora `name` tenha sido declarada fora da função anônima, ela pode ser utilizada normalmente

dentro dela.

Esse comportamento é chamado de *closure*. A função mantém acesso às variáveis que estavam disponíveis no momento em que foi criada.

Mais importante ainda: a função não recebe uma cópia dessas variáveis. Ela continua acessando as variáveis originais.

Por exemplo:

```
func main() {  
    name := "Maria"  
  
    greet := func() {  
        fmt.Println("Olá,", name)  
    }  
  
    greet()  
  
    name = "João"  
  
    greet()  
}
```

Saída:

```
Olá, Maria  
Olá, João
```

Observe que a função foi criada quando `name` valia "Maria", mas a segunda chamada imprime "João".

Isso acontece porque a função não armazenou uma cópia do valor. Ela continuou acessando a mesma variável `name`.

Esse comportamento pode ser visualizado da seguinte forma:

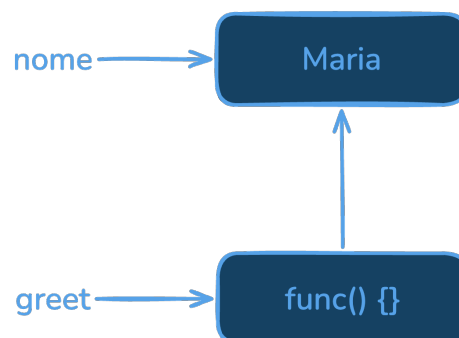


Figura 6.1: Closure 01

Quando executamos:

```
name = "João"
```

apenas o conteúdo da variável é alterado:

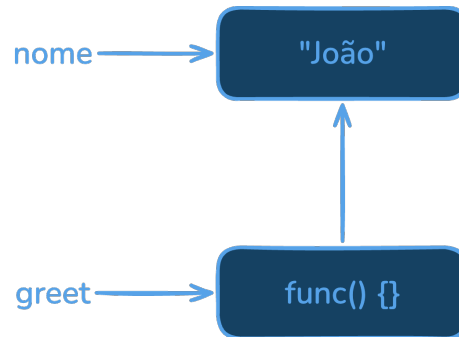


Figura 6.2: Closure 02

A função continua apontando para a mesma variável.

O mesmo mecanismo permite que uma função anônima modifique variáveis do escopo externo:

```
func main() {  
    count := 0  
  
    increment := func() {  
        count++  
    }  
  
    increment()  
    increment()  
    increment()  
  
    fmt.Println(count)  
}
```

Saída:

```
3
```

Embora `count` pertença ao escopo de `main`, a função `increment` continua tendo acesso à mesma variável.

Em outras palavras, não existem duas variáveis:

Existe apenas uma:

Por isso, modificações realizadas dentro da função são visíveis fora dela.

Quando uma função anônima continua tendo acesso às variáveis do ambiente em que foi criada, dizemos que ela forma um *closure*.

Closures são frequentemente utilizados com callbacks e funções passadas como argumentos. Em

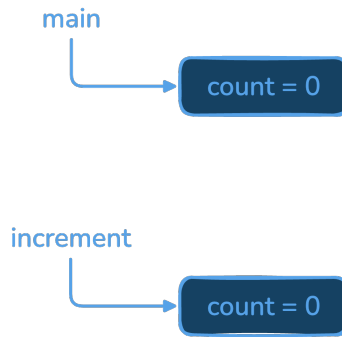


Figura 6.3: main e increment

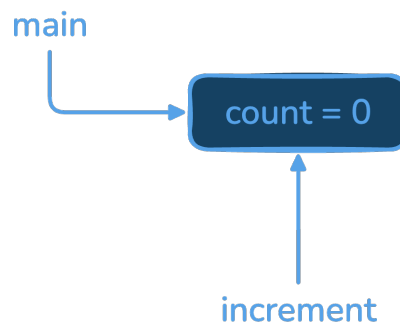


Figura 6.4: main - increment

alguns casos, também são usados para encapsular pequenos estados internos, como no exemplo abaixo:

```
func counter() func() int {
    count := 0

    return func() int {
        count++
        return count
    }
}
```

Uso:

```
next := counter()

fmt.Println(next())
fmt.Println(next())
fmt.Println(next())
```

Saída:

```
1  
2  
3
```

Cada chamada de `counter` cria uma nova variável `count` e retorna uma função associada a ela. Dessa forma, diferentes contadores mantêm estados independentes.

Embora sejam poderosos, closures devem ser usados com cuidado. Em muitos problemas, structs e métodos oferecem soluções mais simples e mais fáceis de compreender.

6.5 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Em muitos casos, há mais de uma solução possível. Priorize escrever programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar à resposta correta.

1. Escreva uma função chamada `minMax` que receba um slice de inteiros não vazio e retorne dois valores: o menor e o maior elemento presentes no slice.

Por exemplo:

```
numbers := []int{15, 7, 42, 18, 9}
```

Resultado esperado:

```
7  
42
```

2. Escreva uma função variádica chamada `average` que receba uma quantidade arbitrária de números reais e retorne a média aritmética desses valores. Caso nenhum argumento seja informado, a função deve retornar `0`.

Por exemplo:

```
average(10, 20, 30, 40)
```

Resultado esperado:

```
25
```

3. Escreva uma função chamada `divide` que receba dois números inteiros e retorne dois valores: o quociente da divisão inteira e um valor booleano indicando se a divisão é válida.

Por exemplo:

```
divide(10, 2)
```

Resultado esperado:

```
5 true
```

e para:

```
divide(10, 0)
```

Resultado esperado:

```
0 false
```

4. Escreva uma função chamada `countEvenOdd` que receba um slice de inteiros e retorne dois valores: a quantidade de números pares e a quantidade de números ímpares.

Por exemplo:

```
numbers := []int{3, 8, 5, 12, 7, 10}
```

Resultado esperado:

```
3 3
```

5. Escreva uma função variádica chamada `concat` que receba uma quantidade arbitrária de strings e retorne uma única string contendo todas elas concatenadas e separadas por espaços. A string final não deve terminar com espaço extra.

Por exemplo:

```
concat("Go", "é", "simples")
```

Resultado esperado:

```
Go é simples
```

6. Escreva uma função chamada `splitEvenOdd` que receba um slice de inteiros e retorne dois slices: o primeiro contendo apenas os números pares e o segundo contendo apenas os números ímpares.

Por exemplo:

```
numbers := []int{3, 8, 5, 12, 7, 10}
```

Resultado esperado:

```
[]int{8, 12, 10}  
[]int{3, 5, 7}
```

7. Escreva uma função chamada `apply` que receba dois inteiros e uma função com assinatura:

```
func(int, int) int
```

A função recebida deve ser aplicada aos dois valores, e o resultado deve ser retornado.

Por exemplo:

```
result := apply(10, 20, func(a, b int) int {  
    return a + b  
})
```

Resultado esperado:

30

Teste também utilizando multiplicação e subtração.

8. Escreva uma função chamada `filter` que receba um slice de inteiros e uma função com assinatura:

```
func(int) bool
```

A função deve retornar um novo slice contendo apenas os elementos para os quais a função recebida retorne `true`.

Por exemplo:

```
numbers := []int{3, 8, 5, 12, 7, 10}
```

utilizando:

```
func(n int) bool {  
    return n%2 == 0  
}
```

Resultado esperado:

```
[]int{8, 12, 10}
```

9. Escreva uma função chamada `counter` que retorne outra função. A função retornada deve manter internamente um contador e, a cada chamada, retornar o próximo número da sequência.

Por exemplo:

```
next := counter()

fmt.Println(next())
fmt.Println(next())
fmt.Println(next())
```

Resultado esperado:

```
1
2
3
```

Crie dois contadores distintos e verifique que eles mantêm estados independentes.

10. Escreva uma função chamada `memoizeSquare` que retorne outra função responsável por calcular o quadrado de um número inteiro.

Na primeira vez que um valor for solicitado, o quadrado deve ser calculado normalmente. Nas chamadas seguintes, caso o mesmo valor seja solicitado novamente, o resultado deve ser obtido a partir de um `map[int] int` interno, evitando o recálculo.

Por exemplo:

```
square := memoizeSquare()

fmt.Println(square(5))
fmt.Println(square(8))
fmt.Println(square(5))
```

Resultado esperado:

```
25
64
25
```

O mapa utilizado para armazenar os resultados deve permanecer encapsulado dentro do closure.

Capítulo 7

Ponteiros

Nos capítulos anteriores, estudamos como variáveis armazenam valores e como funções permitem organizar o código em partes menores e reutilizáveis. Até este ponto, tratamos os dados principalmente por meio de seus valores, sem nos preocuparmos muito com a forma como eles são representados e manipulados na memória do computador.

Por trás de cada variável existe uma região de memória onde seus dados são armazenados. Em muitos casos, trabalhar apenas com os valores é suficiente. Em outros, é necessário acessar o local onde esses valores estão armazenados ou permitir que diferentes partes do programa compartilhem e modifiquem os mesmos dados.

É nesse contexto que surgem os ponteiros.

Embora o conceito de ponteiro seja frequentemente associado a complexidade e erros difíceis de detectar, em Go seu uso é significativamente mais simples do que em linguagens como C e C++. A linguagem elimina diversas operações consideradas perigosas e oferece um modelo mais seguro e previsível para trabalhar com endereços de memória.

Além de compreender o que são ponteiros, é importante desenvolver uma visão mais clara sobre como os dados são armazenados e transferidos durante a execução do programa. Conceitos como passagem por valor, endereços de memória e mecanismos de alocação ajudam a explicar muitos comportamentos da linguagem e serão fundamentais para compreender, nos capítulos seguintes, estruturas, métodos e interfaces.

Neste capítulo, estudaremos como os valores são armazenados na memória, apresentaremos uma visão geral das regiões conhecidas como stack e heap, discutiremos o mecanismo de passagem por valor utilizado por Go e, finalmente, veremos como ponteiros permitem acessar e manipular dados por meio de seus endereços.

O objetivo deste capítulo não é aprofundar os detalhes internos do compilador ou do gerenciador de memória, mas construir uma compreensão suficiente para utilizar ponteiros de forma consciente e entender melhor o comportamento dos programas escritos em Go.

7.1 Memória

Todo programa em execução manipula dados. Esses dados precisam ser armazenados em algum lugar, e esse lugar é a memória do computador.

Quando declaramos uma variável em Go:

```
age := 30
```

um objeto é criado na memória para armazenar o valor 30, e o identificador age passa a ser usado para acessar esse objeto.

Podemos imaginar a situação da seguinte forma:

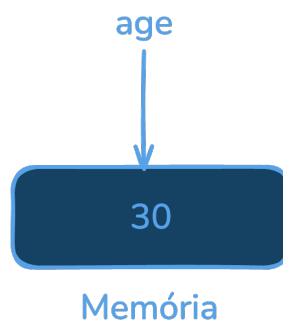


Figura 7.1: Memória age

Embora seja comum dizer que “a variável contém o valor”, uma descrição mais precisa é afirmar que a variável é um nome associado a uma região da memória onde esse valor está armazenado.

Quando atribuímos outro valor à variável:

```
age = 31
```

o conteúdo armazenado é atualizado:

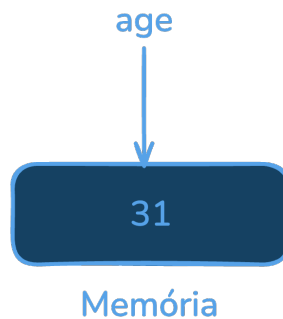


Figura 7.2: Memória age

O programador normalmente não precisa se preocupar com o local exato em que os dados são armazenados. O compilador e o ambiente de execução são responsáveis por gerenciar a memória

e determinar onde cada objeto será alocado.

Valores de diferentes tipos ocupam quantidades diferentes de memória.

Por exemplo:

```
var a int8
var b int64
var c bool
var d float64
```

Cada tipo possui uma representação interna própria, adequada ao conjunto de valores que precisa representar.

Também é importante observar o que acontece quando atribuímos o valor de uma variável a outra.

Considere:

```
a := 10
b := a
```

Podemos representar a situação da seguinte forma:

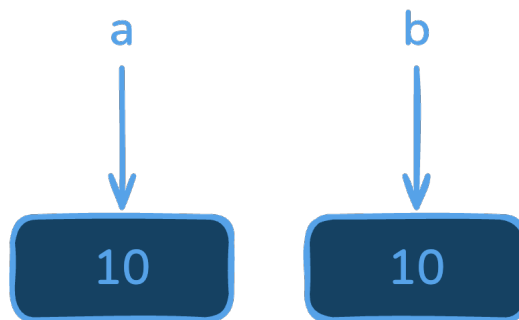


Figura 7.3: Memórias de a e b

Embora ambas possuam o mesmo valor, a e b representam objetos distintos na memória.

Consequentemente, modificar uma delas não altera a outra:

```
b = 20

fmt.Println(a)
fmt.Println(b)
```

Saída:

```
10
20
```

Em Go, a maioria das operações trabalha com valores. Atribuições, passagem de parâmetros para funções e retornos normalmente envolvem cópias dos dados. Essa característica torna o

comportamento dos programas mais previsível e será importante para compreender o mecanismo de passagem por valor estudado nas próximas seções.

Embora o programador raramente precise manipular diretamente a memória, compreender que cada variável corresponde a um objeto armazenado em algum lugar é fundamental para entender o funcionamento dos ponteiros e a forma como os dados podem ser compartilhados entre diferentes partes do programa.

7.2 Stack e Heap

Quando um programa é executado, os objetos criados em memória precisam ser armazenados em algum lugar. Em termos gerais, duas regiões costumam aparecer nessa discussão: a stack e a heap.

A stack (pilha) é uma área de memória utilizada principalmente para armazenar informações relacionadas às chamadas de funções, como parâmetros, valores de retorno e algumas variáveis locais.

Por exemplo:

```
func square(x int) int {  
    result := x * x  
  
    return result  
}
```

Durante a execução da função, os valores associados a `x` e `result` precisam ser armazenados em memória. Frequentemente, esse armazenamento ocorre na stack.

Podemos imaginar a situação da seguinte forma:

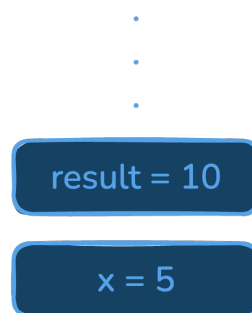


Figura 7.4: Stack

Quando a função termina, a região usada por aquela chamada deixa de ser necessária e pode ser reutilizada.

Por outro lado, alguns objetos precisam continuar existindo mesmo após o término da função que os criou. Nesses casos, eles podem ser armazenados em outra região da memória chamada heap.

Considere:

```
func createCounter() *int {  
    count := 0  
  
    return &count  
}
```

Embora `count` tenha sido criada dentro da função, o valor associado a ela continua sendo utilizado após o retorno da função. Portanto, o compilador precisa garantir que esse objeto permaneça válido.

De forma simplificada, podemos representar a situação assim:

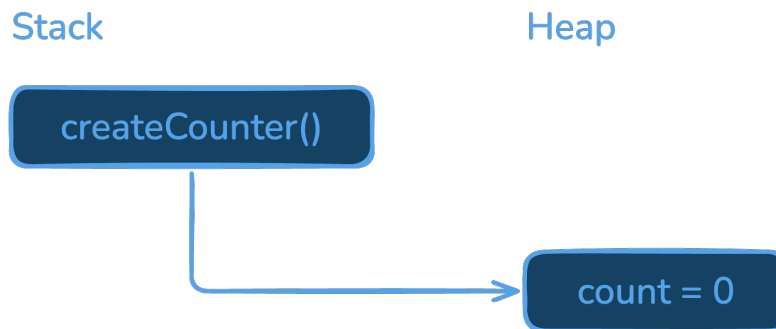


Figura 7.5: Heap

Em Go, a decisão de armazenar um objeto na stack ou na heap é responsabilidade do compilador. O programador normalmente não controla essa escolha diretamente.

Essa análise é conhecida como *escape analysis*. Se o compilador perceber que um objeto não precisa sobreviver ao término da função, ele pode armazená-lo na stack. Caso contrário, pode alocá-lo na heap.

É importante destacar que essa decisão é uma otimização interna da linguagem. Em muitos casos, dois programas com código semelhante podem produzir estratégias de alocação diferentes sem que isso altere seu comportamento.

Por esse motivo, em Go raramente é necessário pensar explicitamente em stack e heap durante a escrita do programa. O mais importante é compreender que os objetos precisam existir enquanto ainda estiverem sendo utilizados, e cabe ao compilador determinar onde cada um deles será armazenado.

Nos próximos tópicos, veremos que, independentemente de onde os objetos estejam alocados, Go continua trabalhando principalmente com valores. Essa característica explica por que atribuições e chamadas de funções normalmente produzem cópias dos dados, um comportamento conhecido como passagem por valor.

7.3 Passagem por Valor

Uma característica fundamental de Go é que funções recebem argumentos por valor.

Isso significa que, quando uma variável é passada para uma função, o valor armazenado nela é copiado para o parâmetro correspondente.

Considere:

```
func double(x int) {  
    x = x * 2  
}  
  
func main() {  
    number := 10  
  
    double(number)  
  
    fmt.Println(number)  
}
```

Saída:

10

Embora a função modifique o parâmetro `x`, a variável `number` permanece inalterada.

Isso acontece porque a função recebeu uma cópia do valor:

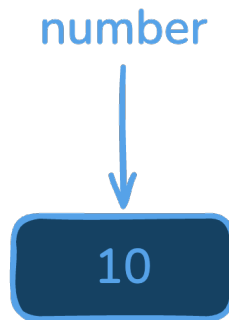


Figura 7.6: Number

Durante a chamada:

São dois objetos distintos. Alterações em `x` não afetam `number`.

O mesmo ocorre com atribuições:

```
a := 10  
b := a  
  
b = 20  
  
fmt.Println(a)  
fmt.Println(b)
```

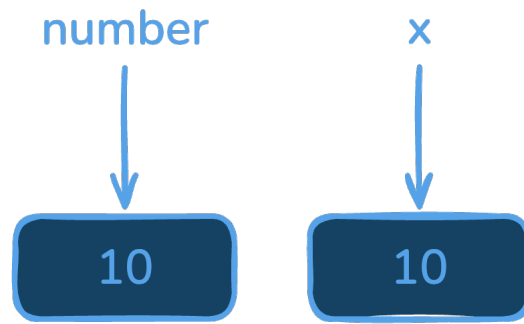


Figura 7.7: Number

Saída:

```
10  
20
```

A variável `b` recebeu uma cópia do valor armazenado em `a`.

Esse comportamento é conhecido como passagem por valor. Ele torna os programas mais previsíveis, pois funções normalmente não modificam as variáveis utilizadas em suas chamadas.

Em algumas situações, porém, desejamos justamente permitir que uma função altere os dados originais.

Considere uma função para incrementar uma variável:

```
func increment(x int) {  
    x++  
}
```

Utilizando:

```
count := 0  
  
increment(count)  
  
fmt.Println(count)
```

a saída será:

```
0
```

Novamente, a variável original não é alterada porque apenas uma cópia do valor foi passada para a função.

Como, então, permitir que a função modifique o objeto original?

A resposta está nos ponteiros.

Em vez de fornecer uma cópia do valor, podemos fornecer o endereço onde esse valor está arma-

zenado. Dessa forma, diferentes partes do programa podem acessar e modificar o mesmo objeto em memória.

No próximo tópico, veremos como obter endereços de memória e como utilizá-los por meio de ponteiros.

7.4 Endereços e Ponteiros

Até agora, vimos que funções recebem cópias dos valores e que modificações realizadas nos parâmetros não afetam as variáveis originais.

Em algumas situações, porém, desejamos justamente permitir que uma função altere dados armazenados em outro lugar do programa. Para isso, precisamos de uma forma de identificar onde um objeto está na memória.

Cada objeto possui um endereço de memória. Em Go, podemos obter esse endereço usando o operador `&`.

Por exemplo:

```
func main() {  
    number := 10  
  
    fmt.Println(&number)  
}
```

A saída será semelhante a:

```
0xc0000120c0
```

O valor exibido representa o endereço do objeto associado à variável `number`.

Podemos imaginar a situação da seguinte forma:



Figura 7.8: Endereço

Um ponteiro é uma variável cujo valor é justamente o endereço de outro objeto.

Por exemplo:


```
func main() {  
    number := 10  
  
    ptr := &number  
}
```

Podemos representar essa situação assim:

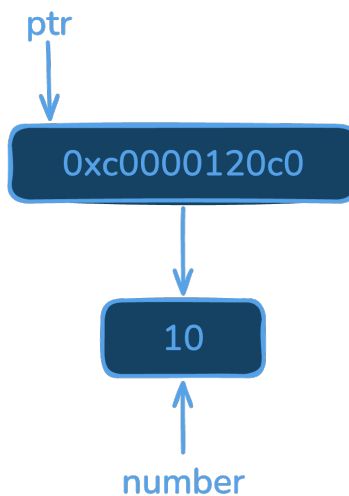


Figura 7.9: Endereço na memória

O tipo de um ponteiro é indicado pelo símbolo `*`.

Por exemplo:

```
var ptr *int
```

Nesse caso, `ptr` é capaz de armazenar o endereço de uma variável do tipo `int`.

Uma vez que possuímos o endereço de um objeto, podemos acessar seu conteúdo por meio do operador `*`, chamado operador de indireção ou desreferenciamento.

Por exemplo:

```
func main() {  
    number := 10  
  
    ptr := &number  
  
    fmt.Println(*ptr)  
}
```

Saída:

10

O operador `*ptr` significa:

Acesse o valor armazenado no endereço contido em `ptr`.

Também é possível modificar esse valor:

```
func main() {  
    number := 10  
  
    ptr := &number  
  
    *ptr = 20  
  
    fmt.Println(number)  
}
```

Saída:

20

Podemos visualizar essa operação da seguinte forma:

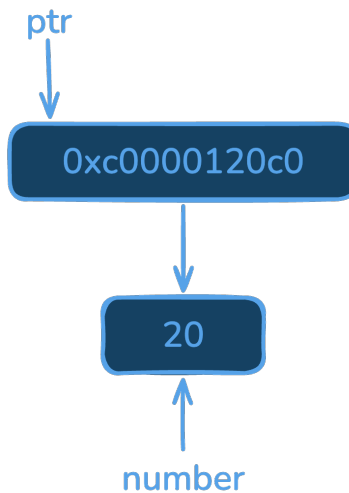


Figura 7.10: Endereço na memória

Observe que existe apenas um objeto contendo o valor 20. Tanto `number` quanto `ptr` permitem acessá-lo.

Esse mecanismo permite que diferentes partes do programa compartilhem e modifiquem os mesmos dados.

Por exemplo:

```
func increment(x *int) {  
    *x = *x + 1  
}  
  
func main() {  
    count := 0  
  
    increment(&count)  
  
    fmt.Println(count)  
}
```

Saída:

1

Nesse caso, a função não recebe uma cópia do valor de count, mas o endereço onde ele está armazenado.

É importante notar que Go continua utilizando passagem por valor. O que é copiado para o parâmetro x é o valor do ponteiro, isto é, o endereço da variável count.

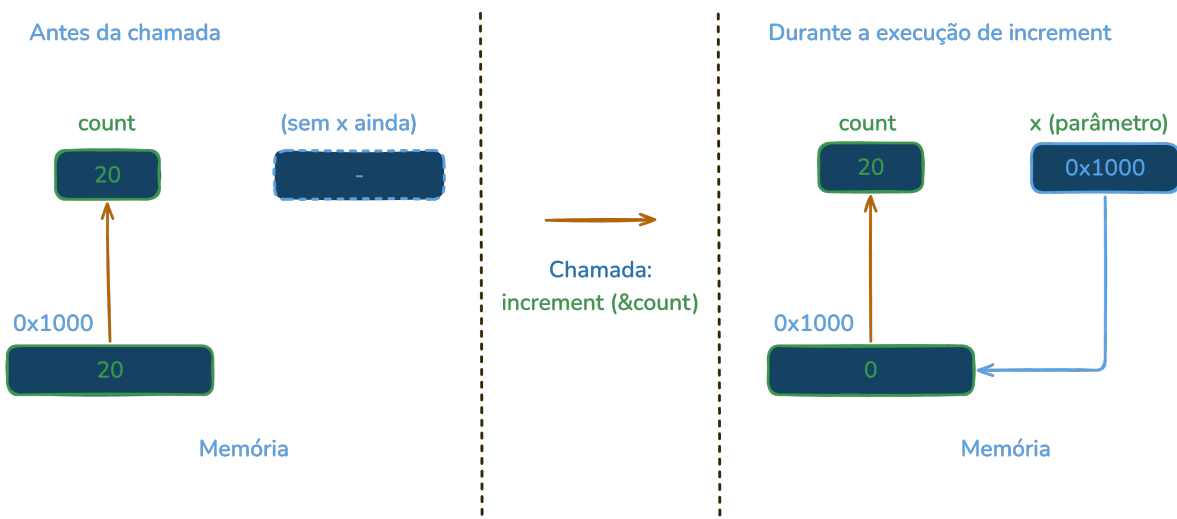


Figura 7.11: Operando com ponteiros e endereço

Assim, tanto count quanto x permitem chegar ao mesmo objeto na memória.

Ponteiros são uma ferramenta fundamental em Go. Eles permitem compartilhar dados entre diferentes partes do programa e serão utilizados extensivamente nos próximos capítulos, especialmente em structs e métodos. A linguagem, entretanto, restringe diversas operações presentes em outras linguagens, tornando seu uso mais simples e seguro.

7.5 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Em muitos casos, há mais de uma solução possível. Priorize escrever programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar à resposta correta.

1. Escreva uma função chamada `swap` que receba dois ponteiros para inteiros e troque os valores armazenados nas variáveis apontadas.

Por exemplo:

```
a := 10  
b := 20
```

Após a chamada da função, os valores devem ser:

```
a = 20  
b = 10
```

2. Escreva uma função chamada `zero` que receba um ponteiro para um inteiro e altere o valor apontado para zero.

Por exemplo:

```
number := 42
```

Após a chamada da função:

```
0
```

3. Escreva uma função chamada `increment` que receba um ponteiro para um inteiro e incremente o valor apontado em uma unidade.

Por exemplo:

```
count := 5
```

Após três chamadas consecutivas:

```
8
```

4. Escreva uma função chamada `maxPtr` que receba dois ponteiros para inteiros e retorne um ponteiro para a variável que contém o maior valor.

Por exemplo:

```
a := 15  
b := 42
```

O ponteiro retornado deve apontar para b.

Verifique que, ao modificar o valor por meio do ponteiro retornado, a variável original também é modificada.

5. Considere o código abaixo:

```
func double(x int) {  
    x *= 2  
}  
  
func main() {  
    number := 10  
  
    double(number)  
  
    fmt.Println(number)  
}
```

Modifique a função `double` para que ela altere efetivamente a variável `number`, utilizando ponteiros.

6. Escreva uma função chamada `sum` que receba dois ponteiros para inteiros e retorne a soma dos valores apontados por eles.

Por exemplo:

```
a := 10  
b := 20
```

Resultado esperado:

```
30
```

7. Escreva uma função chamada `minMax` que receba um slice de inteiros não vazio e dois ponteiros para inteiros. A função deve armazenar, nas variáveis apontadas, o menor e o maior elemento do slice.

Por exemplo:

```
numbers := []int{15, 7, 42, 18, 9}
```

Resultado esperado:

```
min = 7  
max = 42
```

8. Considere o código:

```
func create() *int {  
    value := 10  
  
    return &value  
}
```

Sem executar o programa, responda:

- O objeto associado a `value` deixa de existir ao final da função?
- Por que o ponteiro retornado continua válido em Go?
- Em qual região de memória esse objeto provavelmente será armazenado?

9. Escreva uma função chamada `divide` que receba dois inteiros e três ponteiros: um para armazenar o quociente da divisão inteira, outro para armazenar o resto da divisão e outro para indicar se a operação é válida.

Por exemplo:

```
a := 17  
b := 5
```

Resultado esperado:

```
quociente = 3  
resto = 2  
ok = true
```

Caso o divisor seja zero, `ok` deve receber `false`.

10. Implemente uma função chamada `reverse` que receba um ponteiro para um slice de inteiros e inverta a ordem dos elementos do próprio slice, sem criar um segundo slice.

Por exemplo:

```
numbers := []int{1, 2, 3, 4, 5}
```

Resultado esperado:

```
[]int{5, 4, 3, 2, 1}
```

Capítulo 8

Structs

Nos capítulos anteriores, estudamos os tipos fundamentais da linguagem, coleções, funções e ponteiros. Esses recursos são suficientes para resolver diversos problemas, mas programas reais raramente trabalham apenas com valores isolados. Em geral, é necessário representar entidades compostas por vários atributos relacionados.

Por exemplo, um cliente pode possuir nome, idade e endereço. Uma conta bancária pode possuir número, saldo e data de criação. Um produto pode possuir código, descrição e preço. Embora seja possível armazenar cada informação em variáveis independentes, essa abordagem rapidamente se torna difícil de organizar e manter.

É nesse contexto que surgem as structs.

Uma struct é um tipo composto capaz de agrupar diferentes valores em uma única unidade lógica. Em vez de tratar cada informação separadamente, podemos modelar entidades do domínio da aplicação de forma mais clara e organizada.

Além de simplificar a organização dos dados, structs constituem um dos mecanismos mais importantes da linguagem. Em Go, elas ocupam parte do papel que classes ocupam em outras linguagens, mas sem herança tradicional e sem métodos declarados dentro do próprio tipo. Grande parte da biblioteca padrão e praticamente todas as aplicações escritas em Go utilizam structs para representar seus principais conceitos.

Combinadas com ponteiros e métodos, structs permitem construir abstrações expressivas, mantendo a simplicidade característica da linguagem.

Neste capítulo, estudaremos como utilizar structs para modelar dados, veremos diferentes formas de declaração e inicialização, compreenderemos o conceito de zero value e exploraremos recursos adicionais, como struct tags e embedding, frequentemente usados em conjunto com serialização, bancos de dados e composição de tipos.

Ao final do capítulo, estaremos preparados para avançar para métodos e, posteriormente, para interfaces, completando os principais mecanismos utilizados para construção de abstrações em Go.

8.1 Estruturas e Modelagem de Dados

Nos capítulos anteriores, trabalhamos principalmente com tipos fundamentais, arrays, slices e mapas. Esses recursos são adequados para armazenar valores individuais ou coleções de elementos do mesmo tipo. Entretanto, aplicações reais normalmente precisam representar entidades compostas por informações de naturezas diferentes.

Considere, por exemplo, um cliente de um banco. Além do nome, pode ser necessário armazenar sua idade, e-mail e saldo da conta.

Uma abordagem possível seria utilizar variáveis independentes:

```
name := "Maria"
age  := 35
email := "maria@example.com"
balance := 1500.0
```

Embora funcione, essa solução apresenta alguns problemas. Os dados relacionados ao cliente ficam espalhados pelo programa, e nada impede que informações de clientes diferentes sejam misturadas.

Por exemplo:

```
name1 := "Maria"
age1  := 35

name2 := "João"
age2  := 42
```

À medida que a quantidade de atributos aumenta, a manutenção do código se torna mais difícil.

Uma alternativa seria utilizar um slice:

```
customer := []any{
    "Maria",
    35,
    1500.0,
}
```

Entretanto, essa solução apresenta outro problema: a posição de cada elemento passa a ter significado.

```
name := customer[0].(string)
age  := customer[1].(int)
balance := customer[2].(float64)
```

Além de pouco legível, essa abordagem é frágil. Uma mudança na ordem dos elementos ou um tipo inesperado pode causar erros difíceis de perceber.

Structs resolvem esse tipo de problema. Elas permitem agrupar informações relacionadas em uma única unidade lógica.

Por exemplo:

```
type Customer struct {  
    Name    string  
    Age     int  
    Email   string  
    Balance float64  
}
```

Agora podemos representar um cliente como um único valor:

```
customer := Customer{  
    Name:    "Maria",  
    Age:     35,  
    Email:   "maria@example.com",  
    Balance: 1500.0,  
}
```

Os atributos são acessados pelo nome:

```
fmt.Println(customer.Name)  
fmt.Println(customer.Balance)
```

Saída:

```
Maria  
1500
```

Essa forma de organizar os dados torna o código mais claro e reduz a possibilidade de erros.

Mais importante ainda, structs permitem modelar conceitos do domínio da aplicação.

Por exemplo:

```
type Product struct {  
    ID    int  
    Name  string  
    Price float64  
}  
  
type Employee struct {  
    Name    string  
    Salary  float64  
}  
  
type Account struct {  
    Number  string  
    Balance float64  
}
```

Cada struct representa uma entidade específica do problema que está sendo resolvido.

Esse processo é conhecido como modelagem de dados. Em vez de pensar apenas em variáveis e tipos primitivos, procuramos identificar os conceitos presentes no domínio da aplicação e representar suas características por meio de structs.

Por exemplo, em um sistema acadêmico poderíamos ter:

```
type Student struct {  
    Name string  
    Age  int  
}  
  
type Course struct {  
    Name string  
    Code string  
}
```

Em uma aplicação bancária:

```
type Customer struct {  
    Name string  
    CPF  string  
}  
  
type Account struct {  
    Number  string  
    Balance float64  
}
```

Observe que structs descrevem apenas os dados. O comportamento associado a esses dados será introduzido no próximo capítulo, por meio dos métodos.

Por esse motivo, é comum afirmar que structs ocupam em Go parte do papel das classes em outras linguagens. Entretanto, uma struct é apenas um tipo composto. Ela não possui herança, construtores especiais ou métodos incorporados automaticamente.

As structs constituem o principal mecanismo para modelagem de dados em Go. Grande parte da biblioteca padrão e praticamente todos os programas escritos na linguagem utilizam structs para representar entidades e organizar informações de forma clara e consistente.

8.2 Declaração e Inicialização

Uma struct é um tipo definido pelo usuário, composto por um conjunto de campos. A declaração de uma struct utiliza a palavra-chave `type`, seguida do nome do novo tipo e da palavra-chave `struct`.

Por exemplo:

```
type Customer struct {  
    Name    string  
    Age     int  
    Balance float64  
}
```

Essa declaração define um novo tipo chamado `Customer`, composto por três campos: `Name`, `Age` e `Balance`.

Uma vez definido o tipo, podemos criar variáveis desse tipo:

```
var customer Customer
```

A variável `customer` agora representa um objeto capaz de armazenar as informações definidas pela struct.

Os campos podem ser acessados por meio do operador `.`:

```
customer.Name = "Maria"  
customer.Age  = 35  
customer.Balance = 1500.0  
  
fmt.Println(customer.Name)  
fmt.Println(customer.Balance)
```

Saída:

```
Maria  
1500
```

Uma forma comum de inicializar uma struct é utilizar um literal composto:

```
customer := Customer{
    Name: "Maria",
    Age: 35,
    Balance: 1500.0,
}
```

Essa forma é conhecida como inicialização nomeada (*keyed initialization*).

Os campos podem aparecer em qualquer ordem:

```
customer := Customer{
    Balance: 1500.0,
    Name: "Maria",
    Age: 35,
}
```

Como os nomes dos campos estão presentes, a ordem em que eles aparecem não é importante.

Também é possível inicializar uma struct sem informar os nomes dos campos:

```
customer := Customer{
    "Maria",
    35,
    1500.0,
}
```

Nesse caso, os valores são associados aos campos de acordo com sua posição na declaração da struct.

Essa forma é menos recomendada, pois uma alteração na ordem dos campos da struct pode exigir modificações em diversos pontos do programa.

Por esse motivo, a inicialização nomeada é geralmente preferida. Também é possível inicializar apenas alguns campos:

```
customer := Customer{
    Name: "Maria",
}
```

Os campos não especificados recebem automaticamente seus valores zero:

```
fmt.Println(customer.Name)
fmt.Println(customer.Age)
fmt.Println(customer.Balance)
```

Saída:

```
Maria
0
0
```

Structs também podem ser declaradas e inicializadas em uma única instrução:

```
customer := Customer{
    Name: "Maria",
    Age: 35,
    Balance: 1500.0,
}
```

Também é possível criar ponteiros para structs usando o operador &:

```
customer := &Customer{
    Name: "Maria",
    Age: 35,
    Balance: 1500.0,
}
```

Nesse caso, customer possui tipo *Customer.

Apesar disso, os campos continuam sendo acessados da mesma forma:

```
fmt.Println(customer.Name)
```

e não:

```
fmt.Println((*customer).Name)
```

Embora a segunda forma seja válida, Go realiza automaticamente o desreferenciamento quando o operador . é usado sobre ponteiros para structs.

Também é possível declarar structs anônimas, sem criar um novo tipo:

```
user := struct {
    Name string
    Age int
}{
    Name: "Maria",
    Age: 35,
}
```

Structs anônimas são úteis quando uma estrutura de dados é necessária apenas em um contexto específico e não faz sentido criar um tipo reutilizável.

As diferentes formas de declaração e inicialização tornam as structs bastante flexíveis. Entretanto, independentemente da forma utilizada, todos os campos sempre recebem valores válidos. Esse

comportamento é consequência do mecanismo de zero value, estudado na próxima seção.

8.3 Zero Value

Uma característica importante de Go é que todos os objetos são inicializados automaticamente. Isso significa que, mesmo quando não fornecemos valores explicitamente, todos os campos de uma struct recebem valores definidos.

Considere a seguinte definição:

```
type Customer struct {  
    Name    string  
    Age     int  
    Balance float64  
    Active  bool  
}
```

Podemos declarar uma variável desse tipo sem fornecer uma inicialização explícita:

```
var customer Customer
```

Nesse momento, todos os campos da struct já possuem valores:

```
fmt.Println(customer.Name)  
fmt.Println(customer.Age)  
fmt.Println(customer.Balance)  
fmt.Println(customer.Active)
```

Saída:

```
0  
0  
false
```

Cada tipo possui um valor zero (*zero value*) bem definido:

Tipo	Zero value
int	0
float64	0
bool	false
string	""
Ponteiros	nil
Slices	nil
Maps	nil
Interfaces	nil

Por exemplo:

```
type Person struct {  
    Name  string  
    Age   int  
    Active bool  
}
```

Ao declarar:

```
var p Person
```

podemos imaginar:

```
p  
├── Name   = ""  
├── Age    = 0  
└── Active = false
```

Isso reduz um problema comum em outras linguagens: objetos com campos sem valor definido.

Mesmo uma inicialização parcial é complementada automaticamente:

```
customer := Customer{  
    Name: "Maria",  
}
```

Os demais campos recebem seus valores zero:

```
fmt.Println(customer.Name)  
fmt.Println(customer.Age)  
fmt.Println(customer.Balance)  
fmt.Println(customer.Active)
```

Saída:

```
Maria  
0  
0  
false
```

Esse comportamento também se aplica a structs aninhadas:

```
type Address struct {  
    City string  
    Zip  string  
}  
  
type Customer struct {  
    Name    string  
    Address Address  
}
```

Ao declarar:

```
var customer Customer
```

temos:

```
customer  
├─ Name = ""  
└─ Address  
    ├─ City = ""  
    └─ Zip  = ""
```

Campos que são ponteiros possuem `nil` como valor zero:

```
type Customer struct {  
    Name    string  
    Manager *Customer  
}
```

Então:

```
var customer Customer  
  
fmt.Println(customer.Manager == nil)
```

Saída:

```
true
```

O fato de todos os campos possuírem valores definidos faz com que, em Go, raramente sejam necessários construtores apenas para garantir inicializações básicas.

Por exemplo, a declaração:

```
var customer Customer
```

já produz um objeto válido.

Por esse motivo, um princípio importante na linguagem é:

O zero value de um tipo deve ser útil.

Diversos tipos da biblioteca padrão seguem essa filosofia. Por exemplo, um `sync.Mutex`, um `bytes.Buffer` ou um `sync.WaitGroup` podem ser declarados e utilizados imediatamente, sem necessidade de funções de inicialização.

Essa preocupação com zero values úteis simplifica a construção de programas e reduz a quantidade de código necessária para criar objetos em um estado inicial seguro.

8.4 Struct Tags

Além do nome e do tipo dos campos, uma struct pode conter informações adicionais chamadas *tags*.

Uma tag é uma sequência de caracteres associada a um campo e armazenada como uma string literal. Essas informações não alteram diretamente o comportamento da linguagem, mas podem ser lidas por bibliotecas e ferramentas para controlar diferentes aspectos do processamento dos dados.

Considere a seguinte struct:

```
type Customer struct {  
    Name    string `json:"name"`  
    Age     int    `json:"age"`  
    Balance float64 `json:"balance"`  
}
```

As expressões entre crases constituem as tags associadas a cada campo.

Essas tags são frequentemente utilizadas em operações de serialização para formatos como JSON e XML, além de bibliotecas de acesso a bancos de dados.

Por exemplo, ao converter uma struct para JSON:

```
customer := Customer{  
    Name:    "Maria",  
    Age:     35,  
    Balance: 1500.0,  
}  
  
data, _ := json.Marshal(customer)  
  
fmt.Println(string(data))
```

Saída:

```
{  
  "name": "Maria",
```

```
"age": 35,  
"balance": 1500  
}
```

Observe que os nomes das chaves foram obtidos a partir das tags, e não dos nomes dos campos da struct.

Sem as tags:

```
type Customer struct {  
    Name    string  
    Age     int  
    Balance float64  
}
```

a saída seria:

```
{  
  "Name": "Maria",  
  "Age": 35,  
  "Balance": 1500  
}
```

As tags permitem controlar diversos aspectos da serialização.

Por exemplo, o modificador `omitempty` faz com que campos contendo o zero value sejam omitidos:

```
type Customer struct {  
    Name    string `json:"name"`  
    Age     int    `json:"age,omitempty"`  
    Balance float64 `json:"balance,omitempty"`  
}
```

Se:

```
customer := Customer{  
    Name: "Maria",  
}
```

a saída será:

```
{  
  "name": "Maria"  
}
```

Também é possível impedir que um campo seja serializado:

```
type Customer struct {  
    Name      string `json:"name"`  
    Password  string `json:"-"`  
}
```

Nesse caso, o campo Password será ignorado pelo pacote encoding/json.

Uma mesma tag pode conter várias informações:

```
type Customer struct {  
    Name string `json:"name" db:"customer_name"`  
}
```

Cada biblioteca interpreta apenas as tags que lhe interessam.

É importante observar que as tags não possuem significado especial para a linguagem. Para Go, elas são apenas strings associadas aos campos. O significado dessas informações é definido pelas bibliotecas que as utilizam.

As tags também podem ser acessadas em tempo de execução por meio do pacote reflect:

```
type Customer struct {  
    Name string `json:"name"`  
}  
  
t := reflect.TypeOf(Customer{})  
  
field, _ := t.FieldByName("Name")  
  
fmt.Println(field.Tag.Get("json"))
```

Saída:

```
name
```

Embora seja possível manipular tags diretamente usando reflexão, isso é relativamente incomum em aplicações do dia a dia. Na prática, as tags são utilizadas principalmente para fornecer metadados consumidos por bibliotecas como encoding/json, ORMs e frameworks.

Struct tags constituem um mecanismo simples e flexível para associar informações adicionais aos campos de uma struct, permitindo integrar tipos definidos pelo usuário com diferentes formatos de dados, bibliotecas e ferramentas.

8.5 Embedding

Em muitas situações, diferentes structs compartilham alguns campos. Uma possibilidade seria repetir esses campos em todas as definições, mas isso torna o código mais difícil de manter.

Por exemplo:

```
type Customer struct {  
    ID    int  
    Name  string  
    Age   int  
}  
  
type Employee struct {  
    ID      int  
    Name    string  
    Salary  float64  
}
```

Observe que ambas as structs possuem os campos ID e Name.

Uma forma de evitar essa repetição é definir uma struct separada:

```
type Person struct {  
    ID    int  
    Name  string  
}
```

e utilizá-la como um campo convencional:

```
type Customer struct {  
    Person Person  
    Age    int  
}  
  
type Employee struct {  
    Person Person  
    Salary  float64  
}
```

O acesso aos campos é realizado por meio da struct interna:

```
customer := Customer{  
    Person: Person{  
        ID: 1,  
        Name: "Maria",  
    },  
    Age: 35,  
}  
  
fmt.Println(customer.Person.Name)
```

Saída:

Maria

Go também oferece um mecanismo mais conveniente chamado *embedding*.

Em vez de declarar um campo com nome explícito, podemos incorporar diretamente um tipo:

```
type Customer struct {  
    Person  
    Age int  
}  
  
type Employee struct {  
    Person  
    Salary float64  
}
```

Agora podemos criar um objeto normalmente:

```
customer := Customer{  
    Person: Person{  
        ID: 1,  
        Name: "Maria",  
    },  
    Age: 35,  
}
```

Embora Name pertença à struct Person, ele pode ser acessado diretamente:

```
fmt.Println(customer.Name)
```

Saída:

Maria

Internamente, a struct incorporada continua existindo:

```
fmt.Println(customer.Person.Name)
```

também produz:

Maria

Podemos imaginar a seguinte estrutura:

```
customer  
├── Person  
│   ├── ID  
│   └── Name  
└── Age
```

O embedding promove os campos e métodos do tipo incorporado, tornando-os acessíveis como se fossem campos da própria struct externa. Eles continuam pertencendo ao tipo incorporado, mas podem ser acessados por um caminho mais curto.

Por exemplo:

```
type Address struct {  
    City string  
}  
  
type Customer struct {  
    Name string  
    Address  
}  
  
customer := Customer{  
    Name: "Maria",  
    Address: Address{  
        City: "Vitória",  
    },  
}  
  
fmt.Println(customer.City)
```

Saída:

Vitória

Se a struct externa possuir um campo com o mesmo nome de um campo promovido, o campo mais externo terá prioridade:

```
type A struct {  
    Name string  
}  
  
type B struct {  
    A  
    Name string  
}
```

Nesse caso:

```
b := B{
    A: A{
        Name: "Interno",
    },
    Name: "Externo",
}

fmt.Println(b.Name)
```

Saída:

Externo

O campo interno continua acessível:

```
fmt.Println(b.A.Name)
```

Saída:

Interno

É importante observar que embedding não é herança.

Em linguagens orientadas a objetos tradicionais, herança estabelece uma relação do tipo “é um” (*is-a*). Em Go, embedding representa composição: um valor contém outro valor.

Por exemplo:

```
type Person struct {
    Name string
}

type Employee struct {
    Person
    Salary float64
}
```

Nesse modelo, Employee não herda de Person; ele contém um Person.

Essa filosofia de composição em vez de herança é um dos princípios fundamentais da linguagem. Nos próximos capítulos, veremos que métodos definidos no tipo incorporado também são promovidos, permitindo construir abstrações expressivas sem a necessidade de hierarquias complexas.

Capítulo 9

Métodos

No capítulo anterior, estudamos como as structs permitem modelar entidades e organizar informações relacionadas em uma única unidade lógica. Entretanto, uma entidade raramente é definida apenas pelos dados que armazena. Em geral, também é necessário descrever comportamentos associados a esses dados.

Por exemplo, uma conta bancária não é caracterizada apenas pelo número da conta e pelo saldo disponível. Ela também deve ser capaz de realizar operações como depósitos, saques e transferências. Da mesma forma, um retângulo pode possuir largura e altura, mas também pode calcular sua área ou perímetro.

Em Go, esses comportamentos são implementados por meio de métodos.

Um método é uma função associada a um tipo. Embora a sintaxe seja semelhante à de uma função comum, os métodos permitem expressar de forma mais natural a relação entre os dados de uma struct e as operações realizadas sobre eles.

Ao contrário de muitas linguagens orientadas a objetos, Go não possui classes. A combinação de structs e métodos fornece um mecanismo simples e expressivo para associar comportamento a dados, sem a necessidade de hierarquias complexas ou herança.

Além disso, a escolha entre métodos que operam sobre cópias dos dados e métodos que acessam diretamente o objeto original introduz um dos conceitos mais importantes da linguagem: a diferença entre *value receivers* e *pointer receivers*. Essa distinção está diretamente relacionada aos conceitos de passagem por valor e ponteiros estudados no capítulo anterior.

Neste capítulo, estudaremos como declarar métodos, compreenderemos o papel dos receivers e analisaremos as diferenças entre métodos que operam sobre valores e métodos que operam sobre ponteiros. Esses conceitos serão fundamentais para os próximos capítulos, especialmente para o estudo das interfaces, um dos recursos mais importantes e característicos de Go.

9.1 Receivers

Uma função é um bloco de código independente que pode receber parâmetros e retornar valores.

Por exemplo:

```
func area(width, height float64) float64 {  
    return width * height  
}
```

Essa função pode ser utilizada da seguinte forma:

```
result := area(10, 5)
```

Embora essa abordagem seja perfeitamente válida, existe uma relação natural entre os dados de um retângulo e a operação de calcular sua área. Em vez de manter dados e operações sempre separados, podemos associar esse comportamento ao próprio tipo.

Considere a seguinte struct:

```
type Rectangle struct {  
    Width float64  
    Height float64  
}
```

Podemos definir uma função associada a esse tipo:

```
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}
```

Essa construção define um método.

A parte:

```
(r Rectangle)
```

é chamada de *receiver*.

O receiver é um parâmetro especial que indica a qual tipo o método está associado. Nesse exemplo, o método `Area` pertence ao tipo `Rectangle`.

Podemos utilizá-lo da seguinte forma:

```
rect := Rectangle{  
    Width: 10,  
    Height: 5,  
}  
  
fmt.Println(rect.Area())
```

Saída:

50

Observe que a chamada:

```
rect.Area()
```

é uma forma mais conveniente de escrever algo conceitualmente semelhante a:

```
Area(rect)
```

A principal diferença é que o comportamento agora está associado ao próprio tipo.

Métodos permitem expressar de forma mais natural as operações relacionadas a uma entidade.

Por exemplo:

```
type Account struct {  
    Balance float64  
}  
  
func (a Account) IsEmpty() bool {  
    return a.Balance == 0  
}
```

Uso:

```
account := Account{  
    Balance: 100,  
}  
  
fmt.Println(account.IsEmpty())
```

Saída:

```
false
```

O identificador usado no receiver não é uma palavra reservada da linguagem. Ele é escolhido pelo programador.

Por exemplo, as declarações abaixo são equivalentes:

```
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}
```

e

```
func (rectangle Rectangle) Area() float64 {  
    return rectangle.Width * rectangle.Height  
}
```

Entretanto, é comum utilizar nomes curtos, normalmente uma ou duas letras, derivados do nome do tipo.

Por exemplo:

```
func (c Customer) String() string  
func (a Account) Deposit(amount float64)  
func (r Rectangle) Area() float64
```

Assim como funções, métodos podem receber parâmetros adicionais:

```
type Account struct {  
    Balance float64  
}  
  
func (a Account) CanWithdraw(amount float64) bool {  
    return amount <= a.Balance  
}
```

Uso:

```
account := Account{  
    Balance: 100,  
}  
  
fmt.Println(account.CanWithdraw(50))
```

Saída:

```
true
```

Do ponto de vista da linguagem, métodos são funções associadas a tipos. Essa associação permite organizar melhor o código e torna mais explícita a relação entre os dados e os comportamentos que operam sobre eles.

Nos próximos tópicos, veremos que o receiver pode receber uma cópia do objeto ou um ponteiro para ele. Essa escolha determina se o método opera sobre uma cópia dos dados ou sobre o próprio objeto original.

9.2 Value Receivers

Nos exemplos anteriores, os métodos foram declarados da seguinte forma:

```
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}
```

Observe que o receiver é uma variável do tipo `Rectangle`, e não um ponteiro para `Rectangle`. Isso significa que, ao chamar o método, uma cópia do objeto é passada para o receiver.

Por exemplo:

```
type Counter struct {  
    Value int  
}  
  
func (c Counter) Increment() {  
    c.Value++  
}  
  
func main() {  
    counter := Counter{  
        Value: 10,  
    }  
  
    counter.Increment()  
  
    fmt.Println(counter.Value)  
}
```

Saída:

```
10
```

Embora o método execute:

```
c.Value++
```

o valor original permanece inalterado.

Isso acontece porque `c` é uma cópia de `counter`.

Podemos representar a situação da seguinte forma:



Figura 9.1: Receiver - 01

Quando o método altera `c.Value`, apenas a cópia é modificada:



Figura 9.2: Receiver - 01

Ao término da execução do método, a cópia é descartada.

Por esse motivo, métodos com value receivers normalmente são utilizados quando o método apenas consulta os dados da struct, sem modificá-los.

Por exemplo:

```
type Rectangle struct {
    Width float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Width + r.Height)
}
```

Uso:

```
rect := Rectangle{
    Width: 10,
    Height: 5,
}

fmt.Println(rect.Area())
fmt.Println(rect.Perimeter())
```

Saída:

```
50
30
```

Esses métodos apenas leem os dados do objeto, portanto não há necessidade de utilizar ponteiros.

Conceitualmente, um value receiver é semelhante a uma função que recebe uma struct como parâmetro:

```
func Area(r Rectangle) float64 {  
    return r.Width * r.Height  
}
```

e:

```
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}
```

possuem comportamentos semelhantes.

Como structs são passadas por valor, o receiver também é passado por valor.

É importante observar que modificar um campo da cópia não altera o objeto original:

```
type Customer struct {  
    Name string  
}  
  
func (c Customer) Rename(name string) {  
    c.Name = name  
}  
  
func main() {  
    customer := Customer{  
        Name: "Maria",  
    }  
  
    customer.Rename("João")  
  
    fmt.Println(customer.Name)  
}
```

Saída:

Maria

Entretanto, isso não significa que o método seja incapaz de modificar qualquer dado.

Se a struct possuir campos que referenciam outros objetos, como slices, maps ou ponteiros, os dados referenciados poderão ser alterados:

```
type Numbers struct {  
    Values []int  
}  
  
func (n Numbers) SetFirst(value int) {  
    n.Values[0] = value  
}
```

Uso:

```
numbers := Numbers{  
    Values: []int{1, 2, 3},  
}  
  
numbers.SetFirst(10)  
  
fmt.Println(numbers.Values)
```

Saída:

```
[10 2 3]
```

Embora `n` seja uma cópia da struct, tanto a cópia quanto o objeto original contêm um slice que aponta para o mesmo array subjacente.

Em geral, value receivers são apropriados quando:

- o método não precisa modificar a struct;
- a struct é pequena;
- a semântica natural do tipo é de imutabilidade.

No próximo tópico, veremos como utilizar ponteiros como receivers, permitindo que os métodos operem diretamente sobre o objeto original.

9.3 Pointer Receivers

Nos exemplos anteriores, os métodos recebiam uma cópia da struct. Consequentemente, alterações realizadas no receiver não afetavam o objeto original.

Em muitos casos, porém, desejamos justamente modificar o estado do objeto. Para isso, utilizamos ponteiros como receivers.

Considere a seguinte struct:

```
type Counter struct {  
    Value int  
}
```

Se declararmos o método utilizando um value receiver:


```
func (c Counter) Increment() {  
    c.Value++  
}
```

o valor original não será alterado:

```
counter := Counter{  
    Value: 10,  
}  
  
counter.Increment()  
  
fmt.Println(counter.Value)
```

Saída:

```
10
```

Isso ocorre porque c é uma cópia de counter.

Para modificar o objeto original, utilizamos um ponteiro como receiver:

```
func (c *Counter) Increment() {  
    c.Value++  
}
```

Agora:

```
counter := Counter{  
    Value: 10,  
}  
  
counter.Increment()  
  
fmt.Println(counter.Value)
```

Saída:

```
11
```

Podemos representar a situação da seguinte forma:

Tanto counter quanto c permitem acessar o mesmo objeto.

Por isso, quando executamos:

```
c.Value++
```

o conteúdo do objeto original é modificado:

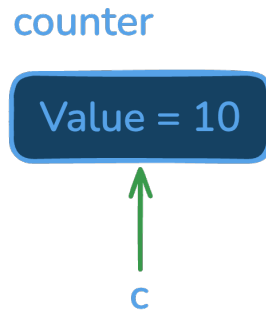


Figura 9.3: Receiver - 03

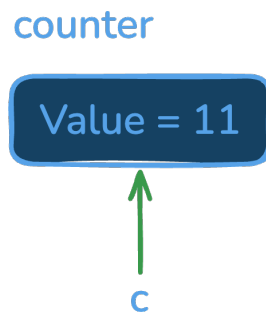


Figura 9.4: Receiver - 04

Do ponto de vista conceitual, um método com pointer receiver é semelhante à seguinte função:

```
func Increment(c *Counter) {  
    c.Value++  
}
```

A principal diferença é que o comportamento permanece associado ao tipo.

Embora o receiver seja um ponteiro, Go permite chamar o método diretamente sobre uma variável endereçável:

```
counter.Increment()
```

A linguagem interpreta essa chamada de forma equivalente a:

```
(&counter).Increment()
```

Da mesma forma, se tivermos:

```
ptr := &counter
```

podemos escrever:

```
ptr.Increment()
```

sem necessidade de desreferenciar explicitamente o ponteiro.

Esse comportamento torna o uso de pointer receivers bastante natural.

Métodos com pointer receivers são frequentemente utilizados quando:

- o método precisa modificar a struct;
- a struct é grande e desejamos evitar cópias;
- o tipo possui algum estado interno que deve ser compartilhado.

Por exemplo:

```
type Account struct {  
    Balance float64  
}  
  
func (a *Account) Deposit(amount float64) {  
    a.Balance += amount  
}  
  
func (a *Account) Withdraw(amount float64) bool {  
    if amount > a.Balance {  
        return false  
    }  
  
    a.Balance -= amount  
  
    return true  
}
```

Uso:

```
account := Account{  
    Balance: 100,  
}  
  
account.Deposit(50)  
  
fmt.Println(account.Balance)
```

Saída:

```
150
```

Embora seja possível misturar value receivers e pointer receivers em um mesmo tipo, isso deve ser feito com cuidado.

Por exemplo:

```
type Counter struct {  
    Value int  
}  
  
func (c Counter) Value() int {  
    return c.Value  
}  
  
func (c *Counter) Increment() {  
    c.Value++  
}
```

Embora o código seja válido, a coexistência de ambos os estilos pode tornar a interface do tipo menos consistente.

Por esse motivo, é comum adotar uma estratégia consistente para os métodos do tipo.

Uma regra prática bastante utilizada é:

Se algum método precisa de um pointer receiver, normalmente os demais métodos desse tipo também devem utilizar pointer receivers.

Nos próximos capítulos, veremos que a escolha entre value receivers e pointer receivers possui consequências importantes na implementação de interfaces. Por esse motivo, compreender essa distinção é fundamental para escrever código idiomático em Go.

9.4 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Em muitos casos, há mais de uma solução possível. Priorize escrever programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar à resposta correta.

1. Defina uma struct `Rectangle` contendo largura e altura. Implemente os métodos `Area()` e `Perimeter()` utilizando value receivers.

Por exemplo:

```
rect := Rectangle{
    Width: 10,
    Height: 5,
}
```

Resultado esperado:

```
50
30
```

2. Defina uma struct `Circle` contendo o raio. Implemente um método `Area()` que retorne a área do círculo.

Considere:

```
pi = 3.141592653589793
```

3. Defina uma struct `Temperature` contendo um valor em graus Celsius. Implemente os métodos:

- `ToFahrenheit() float64`
- `ToKelvin() float64`

Utilize value receivers.

4. Defina uma struct `Counter` contendo um campo inteiro chamado `Value`. Implemente um método `Increment()` utilizando um pointer receiver.

Por exemplo:

```
counter := Counter{}

counter.Increment()
counter.Increment()
counter.Increment()
```

Resultado esperado:

```
3
```

5. Defina uma struct `BankAccount` contendo um saldo (`Balance`). Implemente os métodos:

- `Deposit(amount float64)`
- `Withdraw(amount float64) bool`

O método `Withdraw` deve retornar `false` caso o saldo seja insuficiente. Caso contrário, deve descontar o valor e retornar `true`.

6. Defina uma struct `Student` contendo nome e três notas. Implemente os métodos:

- `Average() float64`
- `Approved() bool`

Considere aprovado o aluno cuja média seja maior ou igual a 6.

7. Defina uma struct `Person` contendo nome e idade. Implemente um método `Birthday()` utilizando um pointer receiver que incremente a idade da pessoa.

Por exemplo:

```
person := Person{
    Name: "Maria",
    Age: 35,
}

person.Birthday()
```

Resultado esperado:

36

8. Defina uma struct `Stack` contendo um slice de inteiros. Implemente os métodos:

- `Push(value int)`
- `Pop() (int, bool)`

O método `Pop` deve remover e retornar o elemento do topo da pilha. Caso a pilha esteja vazia, deve retornar `0, false`.

9. Defina uma struct `Statistics` contendo um slice de inteiros. Implemente os métodos:

- `Min() int`
- `Max() int`
- `Average() float64`

Utilize value receivers e considere que o slice terá pelo menos um elemento.

10. Defina uma struct `Timer` contendo um campo `Seconds`. Implemente os métodos:

- `Add(seconds int)`
- `Reset()`
- `String() string`

de modo que:

```
timer := Timer{}

timer.Add(30)
timer.Add(45)

fmt.Println(timer.String())

timer.Reset()

fmt.Println(timer.String())
```

produza:

75 seconds

0 seconds

11. Defina uma struct `TodoList` contendo um slice de strings. Implemente os métodos:

- `Add(task string)`
- `Remove(index int) bool`
- `Count() int`

O método `Remove` deve retornar `false` caso o índice seja inválido.

12. Defina uma struct `Product` contendo nome, preço e percentual de desconto. Implemente os métodos:

- `FinalPrice() float64`
- `ApplyDiscount(percent float64)`

O segundo método deve alterar permanentemente o desconto armazenado na struct.

13. Defina uma struct `Point` contendo as coordenadas X e Y. Implemente os métodos:

- `Move(dx, dy float64)`
- `Distance() float64`

onde `Distance()` retorna a distância até a origem `(0, 0)`.

14. Defina uma struct `Queue` contendo um slice de inteiros. Implemente os métodos:

- `Enqueue(value int)`
- `Dequeue() (int, bool)`
- `Size() int`

O método `Dequeue` deve retornar `0`, `false` caso a fila esteja vazia.

15. Defina uma struct `Text` contendo uma string. Implemente os métodos:

- `Upper() string`
- `Lower() string`
- `Replace(old, new string)`

Decida quais métodos devem utilizar value receivers e quais devem utilizar pointer receivers, justificando sua escolha.

Capítulo 10

Interfaces

Nos capítulos anteriores, estudamos como structs permitem modelar dados e como métodos possibilitam associar comportamentos a esses tipos. Até este ponto, porém, nossas funções e métodos normalmente trabalharam com tipos concretos específicos.

Por exemplo, uma função pode receber um `Rectangle`, uma `Account` ou um `Customer`. Embora essa abordagem seja simples, ela pode tornar o código excessivamente dependente de implementações específicas, dificultando sua reutilização.

Em muitos casos, o que realmente importa não é o tipo concreto de um valor, mas aquilo que ele é capaz de fazer.

Por exemplo, ao salvar informações, nem sempre é importante saber se os dados serão gravados em um arquivo físico, enviados pela rede ou armazenados em memória. O que importa é que exista uma forma de escrever dados nesse destino.

Da mesma forma, ao ler informações, pode ser irrelevante saber se os dados vêm de um arquivo, de uma conexão de rede ou de uma sequência de bytes armazenada em memória. O importante é que exista um mecanismo para realizar a leitura.

As interfaces surgem justamente para representar esse tipo de abstração.

Uma interface define um conjunto de comportamentos que um tipo deve fornecer. Em vez de depender de implementações concretas, o código passa a depender de contratos, tornando-se mais flexível e reutilizável.

Interfaces constituem um dos recursos mais importantes da linguagem Go. Diferentemente de muitas linguagens orientadas a objetos, Go não exige declarações explícitas para indicar que um tipo implementa uma interface. A relação entre tipos e interfaces é determinada automaticamente pelo conjunto de métodos disponíveis, uma característica conhecida como satisfação implícita.

Essa abordagem favorece a composição e reduz o acoplamento entre diferentes partes do sistema, permitindo construir programas mais simples e mais fáceis de evoluir.

Neste capítulo, estudaremos como interfaces definem contratos entre componentes, veremos como funciona a satisfação implícita, exploraremos mecanismos de composição e analisaremos duas das interfaces mais importantes da biblioteca padrão: `io.Reader` e `io.Writer`. Esses conceitos formam a base de grande parte da arquitetura da linguagem e são amplamente utilizados tanto na biblioteca padrão quanto em aplicações profissionais escritas em Go.

10.1 Interfaces como Contratos

Uma interface define um conjunto de comportamentos que um tipo deve fornecer.

Por exemplo, suponha que desejamos representar objetos capazes de calcular sua área:

```
type Rectangle struct {  
    Width float64  
    Height float64  
}  
  
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}
```

Também podemos ter um círculo:

```
type Circle struct {  
    Radius float64  
}  
  
func (c Circle) Area() float64 {  
    return math.Pi * c.Radius * c.Radius  
}
```

Embora sejam tipos diferentes, ambos possuem um comportamento em comum: a capacidade de calcular uma área.

Podemos representar esse comportamento por meio de uma interface:

```
type Shape interface {  
    Area() float64  
}
```

Uma interface não armazena dados e não contém implementação. Ela apenas descreve um conjunto de métodos.

Nesse caso, qualquer tipo que possua um método:

```
Area() float64
```

pode ser tratado como um Shape.

Podemos então escrever funções que dependem apenas desse contrato:

```
func PrintArea(s Shape) {  
    fmt.Println(s.Area())  
}
```

Uso:

```
rect := Rectangle{  
    Width: 10,  
    Height: 5,  
}  
  
circle := Circle{  
    Radius: 3,  
}  
  
PrintArea(rect)  
PrintArea(circle)
```

Saída:

```
50  
28.274333882308138
```

Observe que a função não precisa conhecer os detalhes de `Rectangle` ou `Circle`. Ela sabe apenas que o valor recebido possui um método `Area()`.

Podemos visualizar essa relação da seguinte forma:

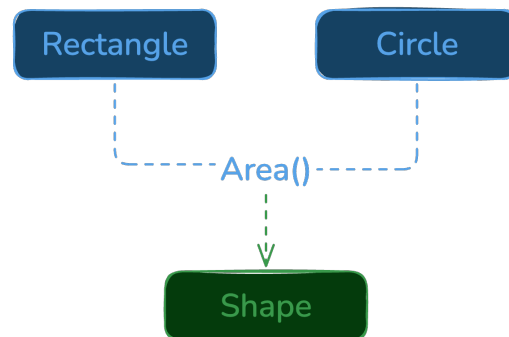


Figura 10.1: Interface

O foco deixa de ser o tipo concreto e passa a ser o comportamento oferecido.

Esse é o principal objetivo das interfaces: permitir que o código dependa de capacidades, e não de implementações específicas.

Por exemplo, considere um sistema de notificações:

```
type EmailNotifier struct{}  
type SMSNotifier struct{}  
type PushNotifier struct{}
```

Cada tipo pode implementar o método:

```
Notify(message string)
```

Podemos então definir o contrato:

```
type Notifier interface {  
    Notify(message string)  
}
```

e escrever funções independentes da implementação concreta:

```
func SendWelcome(n Notifier) {  
    n.Notify("Bem-vindo!")  
}
```

A função não precisa saber se a mensagem será enviada por e-mail, SMS ou notificação push.

Ela depende apenas do contrato definido pela interface.

Uma interface pode possuir vários métodos:

```
type Shape interface {  
    Area() float64  
    Perimeter() float64  
}
```

No entanto, em Go é comum priorizar interfaces pequenas:

```
type Reader interface {  
    Read([]byte) (int, error)  
}  
  
type Writer interface {  
    Write([]byte) (int, error)  
}
```

Quanto menor for a interface, maior tende a ser sua flexibilidade.

Por esse motivo, a biblioteca padrão utiliza frequentemente interfaces pequenas, muitas vezes contendo apenas um único método.

Um exemplo famoso é a interface `Stringer`:

```
type Stringer interface {  
    String() string  
}
```

Qualquer tipo que implemente esse método pode fornecer sua própria representação textual.

Interfaces permitem separar o que um valor faz de como ele realiza essa tarefa. Essa ideia de programação orientada a contratos é um dos pilares da linguagem Go e uma ferramenta importante para reduzir acoplamento e aumentar a reutilização de código.

No próximo tópico, veremos uma característica que diferencia Go de muitas outras linguagens: um tipo não precisa declarar explicitamente que implementa uma interface. Essa relação é determinada automaticamente por meio da satisfação implícita.

10.2 Satisfação Implícita

Em muitas linguagens orientadas a objetos, um tipo precisa declarar explicitamente que implementa uma determinada interface.

Por exemplo, em linguagens como Java, é comum encontrar declarações semelhantes a:

```
class Rectangle implements Shape {  
    ...  
}
```

Go adota uma abordagem diferente.

Um tipo implementa uma interface automaticamente quando possui todos os métodos exigidos por ela.

Considere a seguinte interface:

```
type Shape interface {  
    Area() float64  
    Perimeter() float64  
}
```

e a struct:

```
type Rectangle struct {  
    Width float64  
    Height float64  
}  
  
func (r Rectangle) Area() float64 {  
    return r.Width * r.Height  
}  
  
func (r Rectangle) Perimeter() float64 {  
    return 2 * (r.Width + r.Height)  
}
```

Não existe nenhuma declaração informando que `Rectangle` implementa `Shape`.

Mesmo assim, o compilador considera que o contrato foi satisfeito, pois o tipo possui todos os métodos exigidos pela interface.

Portanto:

```
var s Shape

s = Rectangle{
    Width: 10,
    Height: 5,
}
```

é perfeitamente válido.

Podemos utilizar normalmente:

```
fmt.Println(s.Area())
fmt.Println(s.Perimeter())
```

Saída:

```
50
30
```

O mesmo ocorre com qualquer outro tipo que possua um método compatível:

```
type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}
```

Também podemos escrever:

```
s = Circle{
    Radius: 3,
}
```

Sem qualquer modificação na interface ou no tipo concreto.

Podemos visualizar essa relação da seguinte forma:

```
Shape
├─ Area() float64
└─ Perimeter() float64
```

```
Rectangle → satisfaz Shape  
Circle → satisfaz Shape
```

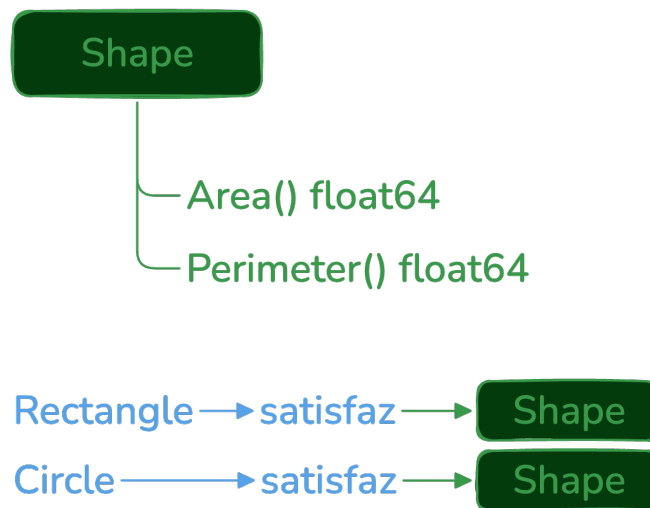


Figura 10.2: Implementação

A satisfação implícita reduz significativamente o acoplamento entre os componentes.

A interface não precisa conhecer os tipos concretos, e os tipos concretos também não precisam conhecer a interface.

Considere:

```
type Printer interface {  
    Print()  
}
```

Qualquer tipo que possua um método:

```
Print()
```

passará automaticamente a satisfazer a interface.

Por exemplo:

```
type Report struct{}  
  
func (Report) Print() {  
    fmt.Println("Imprimindo relatório")  
}
```

Agora:

```
var p Printer = Report{}
```

é válido.

Caso um método esteja ausente, o compilador gera um erro.

Por exemplo:

```
type Shape interface {
    Area() float64
    Perimeter() float64
}

type Square struct {
    Side float64
}

func (s Square) Area() float64 {
    return s.Side * s.Side
}
```

A atribuição:

```
var s Shape = Square{}
```

produzirá um erro semelhante a:

```
cannot use Square{} (value of struct type Square) as Shape value in variable declaratio
```

Essa verificação ocorre em tempo de compilação, tornando o sistema seguro sem necessidade de declarações explícitas.

10.2.1 Interfaces e Pointer Receivers

A relação entre interfaces e receivers merece uma atenção especial.

Considere a interface:

```
type Shape interface {
    Area() float64
    SetSide(value float64)
}
```

e a seguinte struct:


```
type Square struct {  
    Side float64  
}  
  
func (s Square) Area() float64 {  
    return s.Side * s.Side  
}  
  
func (s *Square) SetSide(value float64) {  
    s.Side = value  
}
```

À primeira vista, pode parecer que Square satisfaz a interface Shape, afinal, há métodos Area e SetSide associados a esse tipo.

Entretanto, a atribuição abaixo não é válida:

```
var s Shape = Square{}
```

O compilador produz um erro semelhante a:

```
Square does not implement Shape  
(method SetSide has pointer receiver)
```

Por outro lado:

```
var s Shape = &Square{}
```

é perfeitamente válido.

Para entender esse comportamento, é necessário lembrar que métodos definidos com pointer receivers pertencem ao conjunto de métodos do tipo ponteiro (*Square), e não ao conjunto de métodos do tipo valor (Square).

Podemos representar essa situação da seguinte forma:

Como a interface exige ambos os métodos:

apenas *Square satisfaz o contrato.

Esse comportamento costuma causar confusão porque a chamada abaixo é válida:

```
square := Square{  
  
square.SetSide(10)
```

Mesmo que SetSide tenha sido definido com um pointer receiver.

Nesse caso, Go realiza automaticamente a conversão:

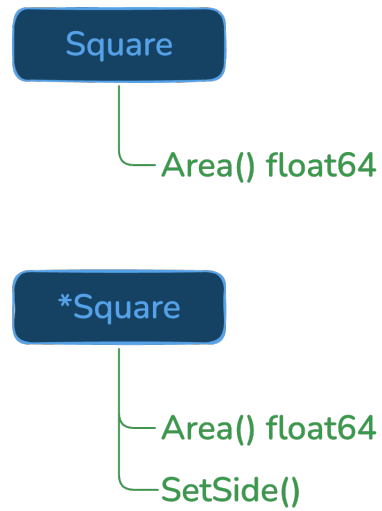


Figura 10.3: Pointer Receivers

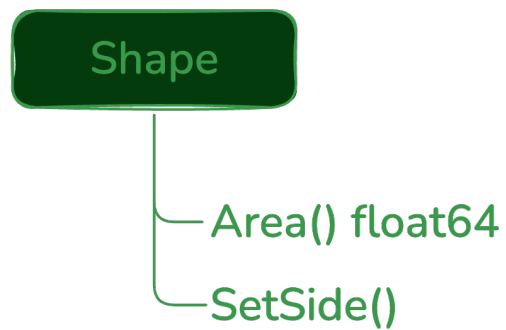


Figura 10.4: Pointer Receivers

```
(square).SetSide(10)
```

Entretanto, essa conveniência existe apenas durante a chamada de métodos.

Quando o compilador verifica se um tipo satisfaz uma interface, ele utiliza os conjuntos de métodos reais de cada tipo, sem realizar essa conversão automática.

Por esse motivo, a escolha entre value receivers e pointer receivers influencia diretamente quais tipos satisfazem uma interface.

10.2.2 Conclusão

A satisfação implícita é uma das características mais marcantes de Go. Em vez de declarar explicitamente que um tipo implementa uma interface, basta que ele forneça todos os métodos exigidos pelo contrato.

Essa abordagem reduz o acoplamento entre componentes, favorece a reutilização de código e permite que as interfaces sejam definidas por quem as utiliza, e não necessariamente por quem implementa os tipos concretos.

Também vimos que a forma como os métodos são declarados influencia diretamente a satisfação de interfaces. Métodos definidos com *pointer receivers* pertencem ao conjunto de métodos do tipo ponteiro, podendo fazer com que apenas **T* satisfaça uma determinada interface, enquanto *T* não.

No próximo tópico, veremos como interfaces pequenas podem ser combinadas para formar contratos mais complexos por meio da composição.

10.3 Composição de Interfaces

Uma das características mais importantes das interfaces em Go é a possibilidade de combiná-las para formar contratos mais completos.

Considere as seguintes interfaces:

```
type Reader interface {  
    Read([]byte) (int, error)  
}  
  
type Writer interface {  
    Write([]byte) (int, error)  
}
```

Cada uma representa uma capacidade específica.

- Reader representa a capacidade de ler dados.
- Writer representa a capacidade de escrever dados.

Em algumas situações, porém, precisamos de um valor que seja capaz de realizar ambas as operações.

Uma possibilidade seria criar uma nova interface repetindo os métodos:

```
type ReadWriter interface {  
    Read([]byte) (int, error)  
    Write([]byte) (int, error)  
}
```

Embora funcione, essa abordagem não aproveita as interfaces já existentes.

Em Go, podemos compor interfaces diretamente:

```
type ReadWriter interface {  
    Reader  
    Writer  
}
```

Essa declaração significa:

Um ReadWriter é qualquer tipo que satisfaça simultaneamente as interfaces Reader e Writer.

Podemos visualizar essa relação da seguinte forma:

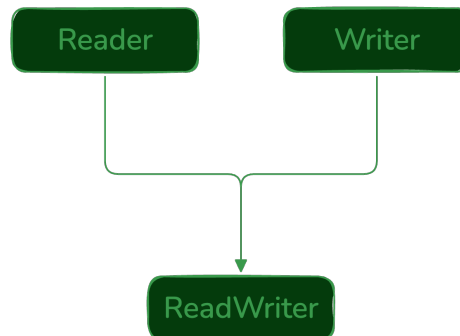


Figura 10.5: ReadWriter

Qualquer tipo que implemente ambos os métodos satisfará automaticamente a nova interface.

Por exemplo:

```
type Buffer struct {  
    data []byte  
}  
  
func (b *Buffer) Read(p []byte) (int, error) {  
    return 0, nil  
}  
  
func (b *Buffer) Write(p []byte) (int, error) {  
    return len(p), nil  
}
```

Agora:

```
var rw ReadWriter  
  
rw = &Buffer{}
```

é válido.

Observe que não existe nenhuma declaração explícita informando que Buffer implementa ReadWriter, Reader ou Writer.

Tudo é determinado automaticamente por meio da satisfação implícita.

Essa capacidade de combinar interfaces pequenas é amplamente utilizada na biblioteca padrão.

Por exemplo, o pacote io define:

```
type Reader interface {  
    Read([]byte) (int, error)  
}  
  
type Writer interface {  
    Write([]byte) (int, error)  
}
```

e posteriormente:

```
type ReadWriter interface {  
    Reader  
    Writer  
}
```

Também existem composições maiores:

```
type ReadWriterCloser interface {  
    Reader  
    Writer  
    Closer  
}
```

onde:

```
type Closer interface {  
    Close() error  
}
```

Nesse caso, um `ReadWriteCloser` representa qualquer objeto capaz de:

- ler dados;
- escrever dados;
- liberar recursos por meio de `Close`.

Um aspecto importante da filosofia de Go é a preferência por interfaces pequenas.

Em vez de criar interfaces extensas contendo muitos métodos:

```
type FileManager interface {  
    Read()  
    Write()  
    Close()  
    Seek()  
    Stat()  
    Sync()  
}
```

é comum dividir responsabilidades:

```
type Reader interface {  
    Read([]byte) (int, error)  
}  
  
type Writer interface {  
    Write([]byte) (int, error)  
}  
  
type Closer interface {  
    Close() error  
}
```

e compô-las quando necessário.

Essa abordagem oferece maior flexibilidade, pois os tipos podem implementar apenas os compor-

tamentos realmente necessários.

Por esse motivo, um princípio bastante difundido na comunidade Go é:

Interfaces devem descrever comportamentos pequenos e bem definidos.

A composição permite construir abstrações mais completas sem abrir mão dessa simplicidade.

Nos próximos tópicos, veremos duas das interfaces mais importantes da biblioteca padrão, `io.Reader` e `io.Writer`, que servem de base para grande parte das operações de entrada e saída realizadas em programas Go.

10.4 Interfaces na Biblioteca Padrão

Até este ponto, estudamos os conceitos fundamentais das interfaces. A biblioteca padrão de Go utiliza esses mesmos conceitos em diversos pacotes, normalmente por meio de interfaces pequenas que representam um único comportamento.

Um dos exemplos mais conhecidos é o pacote `io`, que define as interfaces `Reader` e `Writer`:

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}  
  
type Writer interface {  
    Write(p []byte) (n int, err error)  
}
```

Embora possuam apenas um método cada, essas interfaces são utilizadas em grande parte da biblioteca padrão.

A interface `io.Reader` representa qualquer tipo capaz de fornecer dados para leitura.

Podemos visualizar alguns exemplos da seguinte forma:

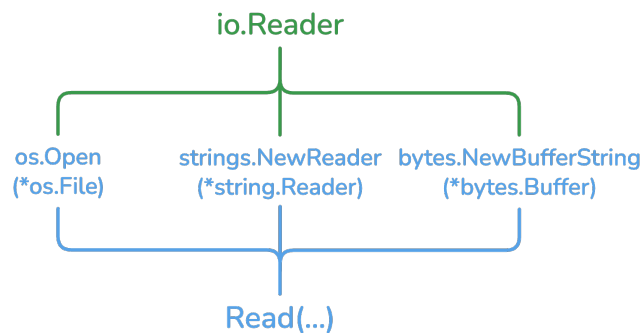


Figura 10.6: `io.Reader`

Da mesma forma, `io.Writer` representa qualquer tipo capaz de receber dados para escrita.

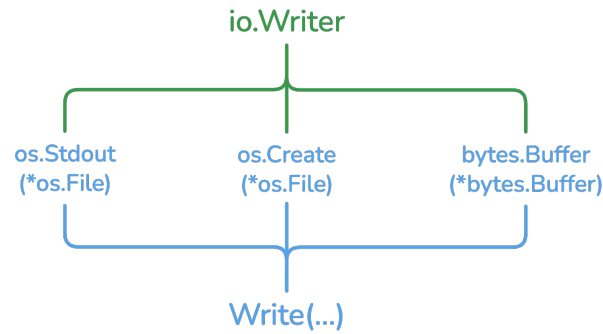


Figura 10.7: io.Writer

Observe que os tipos concretos podem ser completamente diferentes. Ainda assim, todos satisfazem as interfaces correspondentes quando implementam os métodos exigidos.

Esse desacoplamento permite que funções trabalhem apenas com os comportamentos necessários, sem depender de implementações específicas.

Por exemplo:

```
func WriteMessage(w io.Writer) {
    fmt.Fprintln(w, "Olá, Go!")
}
```

A mesma função pode escrever no terminal, em um arquivo ou em um buffer de memória:

```
WriteMessage(os.Stdout)

buffer := &bytes.Buffer{}
WriteMessage(buffer)
```

A função não precisa saber qual é o destino concreto dos dados. Ela depende apenas do contrato definido por `io.Writer`.

Essa é uma das principais vantagens das interfaces em Go: permitir que componentes independentes cooperem por meio de contratos simples e bem definidos.

10.5 Verificação Explícita de Implementação

Embora Go utilize satisfação implícita, em alguns casos pode ser útil solicitar ao compilador uma verificação explícita de que um determinado tipo satisfaz uma interface.

Uma forma simples de fazer isso é realizar uma atribuição para uma variável do tipo da interface:

```
var _ Shape = Square{}
```

Essa declaração não possui finalidade prática durante a execução do programa. Seu único objetivo

é fazer com que o compilador verifique se `Square` implementa todos os métodos exigidos por `Shape`.

Caso algum método esteja ausente ou possua uma assinatura incompatível, o código deixará de compilar.

Por exemplo, se `Square` não implementar o método `Perimeter`, será produzido um erro semelhante a:

```
Square does not implement Shape
(missing method Perimeter)
```

O identificador `_`, conhecido como *blank identifier*, indica que o valor da variável será descartado. Assim, a declaração existe apenas para provocar a verificação realizada pelo compilador.

Na prática, também é comum encontrar uma técnica semelhante para verificar o tipo ponteiro:

```
var _ Shape = (*Square)(nil)
```

Essa forma verifica se `*Square` satisfaz `Shape`. Ela é especialmente útil quando os métodos necessários foram definidos com pointer receivers.

Portanto, a escolha entre as duas formas depende do tipo que deve satisfazer a interface:

```
var _ Shape = Square{}           // verifica Square
var _ Shape = (*Square)(nil)     // verifica *Square
```

Essa sintaxe utiliza recursos da linguagem que ainda não foram apresentados em detalhes neste texto. Eles serão retomados posteriormente, quando abordarmos conversões de tipos, o valor `nil` e outros padrões comuns encontrados em projetos Go.

10.6 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Em muitos casos, há mais de uma solução possível. Priorize escrever programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar à resposta correta.

1. Crie uma interface chamada `Describer` contendo o método `Description() string`. Em seguida, defina as structs `Person` e `Product` e implemente esse método para ambas. Escreva uma função que receba um `Describer` e exiba a descrição correspondente, demonstrando que a mesma função pode trabalhar com diferentes tipos.

2. Crie uma interface chamada `Shape` com o método `Area() float64`. Implemente essa interface para um retângulo e para um círculo. Em seguida, escreva uma função que receba um `Shape` e exiba sua área.
3. Crie uma interface chamada `Speaker` contendo o método `Speak() string`. Implemente essa interface para as structs `Dog`, `Cat` e `Bird`. Armazene instâncias desses tipos em um único slice de `Speaker` e exiba a mensagem produzida por cada uma.
4. Crie uma interface chamada `Discountable` com o método `Discount() float64`. Implemente essa interface para os tipos `Book` e `Electronic`, considerando regras de desconto diferentes para cada um. Em seguida, escreva uma função que calcule o valor final de qualquer item que satisfaça essa interface.
5. Considere uma aplicação que trabalha com diferentes formas de armazenamento. Crie a interface:

```
type Repository interface {  
    Save(string) error  
    Load(id string) (string, error)  
}
```

Implemente essa interface em dois tipos distintos: um que armazene os dados em memória e outro que apenas exiba mensagens simulando uma operação em banco de dados. Escreva uma função que utilize apenas a interface, sem conhecer a implementação concreta.

6. Crie uma interface chamada `Logger` contendo o método `Log(message string)`. Implemente duas versões: uma que escreva as mensagens no terminal e outra que armazene as mensagens em uma slice. Escreva uma função que receba um `Logger` e registre diversas mensagens.
7. Crie uma interface chamada `PaymentMethod` com o método `Pay(amount float64) error`. Implemente essa interface para os tipos `CreditCard`, `Pix` e `Cash`. Em seguida, desenvolva uma função responsável por processar um pagamento utilizando qualquer implementação da interface.
8. Implemente uma função chamada `CopyData` que receba um `io.Reader` e um `io.Writer`. A função deve ler os dados do leitor e escrevê-los no destino utilizando `io.Copy`. Teste a função copiando o conteúdo de uma string para um `bytes.Buffer` e, em seguida, para a saída padrão (`os.Stdout`).
9. Considere o seguinte contrato:

```
type Notifier interface {  
    Send(message string) error  
}
```

Implemente duas structs, `EmailNotifier` e `SMSNotifier`, que satisfaçam essa interface. Em seguida, escreva uma função que receba um slice de `Notifier` e envie a mesma mensagem utilizando todos os notificadores disponíveis.

Capítulo 11

Erros

Até este ponto, desenvolvemos programas capazes de armazenar informações, controlar o fluxo de execução, organizar dados por meio de structs e reutilizar código com funções e interfaces. Entretanto, ainda assumimos que todas as operações sempre produzem o resultado esperado.

Na prática, isso raramente acontece.

Um arquivo pode não existir, uma conexão de rede pode ser interrompida, uma conversão de dados pode falhar ou um usuário pode fornecer informações inválidas. Programas robustos precisam estar preparados para lidar com essas situações de forma previsível e segura.

Em muitas linguagens de programação, essas condições são tratadas por meio de exceções (*exceptions*), que interrompem o fluxo normal da execução e transferem o controle para mecanismos específicos de captura.

Go adota uma filosofia diferente.

Em vez de utilizar exceções para representar falhas esperadas, a linguagem trata erros como valores comuns. Uma função que pode falhar retorna um valor do tipo `error`, permitindo que o código chamador decida como reagir à situação.

Essa abordagem torna o fluxo de tratamento de erros explícito, facilita a compreensão do programa e reduz comportamentos inesperados durante a execução.

Isso não significa que Go não possua mecanismos para lidar com falhas graves. Situações excepcionais, das quais o programa não consegue se recuperar normalmente, podem envolver as funções `panic` e `recover`. Entretanto, esses mecanismos possuem objetivos diferentes do tratamento convencional de erros e devem ser utilizados com cautela.

Neste capítulo, estudaremos como representar erros utilizando a interface `error`, como criar novos erros, adicionar contexto às mensagens, encapsular erros preservando sua origem e verificar suas causas por meio das funções `errors.Is` e `errors.As`. Ao final, conheceremos os mecanismos `panic` e `recover` e compreenderemos em quais situações seu uso é apropriado.

11.1 A interface `error`

O tratamento de erros em Go é baseado em uma interface extremamente simples.

Ela é definida na biblioteca padrão como:

```
type error interface {  
    Error() string  
}
```

Qualquer tipo que implemente o método:

```
Error() string
```

passa automaticamente a implementar a interface `error`.

Isso significa que erros, em Go, são valores como quaisquer outros. Eles podem ser atribuídos a variáveis, retornados por funções, armazenados em structs e passados como argumentos.

Por exemplo, considere uma função responsável por realizar uma divisão:

```
func Divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, errors.New("divisão por zero")  
    }  
  
    return a / b, nil  
}
```

Observe que essa função retorna dois valores:

- o resultado da operação;
- um possível erro.

Quando a operação é realizada com sucesso, o erro retornado é `nil`, indicando ausência de falha.

Caso contrário, a função retorna um valor que implementa a interface `error`.

O uso dessa função normalmente segue o padrão:

```
result, err := Divide(10, 2)  
  
if err != nil {  
    fmt.Println(err)  
    return  
}  
  
fmt.Println(result)
```

Saída:

5

Já:

```
result, err := Divide(10, 0)

if err != nil {
    fmt.Println(err)
    return
}

fmt.Println(result)
```

produz:

divisão por zero

Observe que o tratamento do erro é explícito.

A função que realiza a divisão apenas informa que uma falha ocorreu.

Cabe ao código que a chamou decidir como reagir: exibir uma mensagem ao usuário, tentar novamente, registrar um log ou interromper a operação.

Esse padrão aparece constantemente em programas escritos em Go:

```
value, err := SomeFunction()

if err != nil {
    // tratamento do erro
}
```

Por esse motivo, é comum encontrar funções retornando múltiplos valores, sendo o último deles um erro.

Essa convenção tornou-se um padrão da linguagem e está presente em praticamente toda a biblioteca padrão.

Embora a interface possua apenas um método, ela representa um dos conceitos mais importantes de Go. Ao tratar erros como valores comuns, a linguagem evita mecanismos ocultos de controle de fluxo e torna o comportamento do programa mais previsível e fácil de compreender.

Na próxima seção veremos como criar valores de erro utilizando a função `errors.New`.

11.2 errors.New

Nem todo erro precisa carregar informações adicionais. Em muitos casos, basta indicar que uma determinada condição ocorreu. Para essas situações, a biblioteca padrão fornece a função `errors.New`, que cria um valor do tipo `error` a partir de uma mensagem fixa.

Sua assinatura é bastante simples:

```
func New(text string) error
```

O argumento recebido é uma string que descreve o problema. O valor retornado implementa a interface `error`.

```
1 package main
2
3 import (
4     "errors"
5     "fmt"
6 )
7
8 func dividir(a, b float64) (float64, error) {
9     if b == 0 {
10         return 0, errors.New("divisão por zero")
11     }
12
13     return a / b, nil
14 }
15
16 func main() {
17     resultado, err := dividir(10, 0)
18     if err != nil {
19         fmt.Println(err)
20         return
21     }
22
23     fmt.Println(resultado)
24 }
```

Saída:

```
divisão por zero
```

Nesse exemplo, quando o divisor é igual a zero, a função retorna um erro criado com `errors.New`. O código que chamou a função verifica esse erro e decide como tratá-lo.

11.2.1 Quando utilizar

`errors.New` é indicado quando:

- o erro representa uma condição fixa;
- não há necessidade de incluir valores dinâmicos na mensagem;
- deseja-se criar um erro simples e direto.

Por exemplo:

```
var ErrUnauthorized = errors.New("usuário não autorizado")
var ErrInvalidPassword = errors.New("senha inválida")
var ErrFileNotFound = errors.New("arquivo não encontrado")
```

Essas variáveis podem ser reutilizadas em diferentes partes da aplicação.

11.2.2 Erros reutilizáveis

Uma prática comum em Go é declarar erros como variáveis de pacote.

```
1 package calculator
2
3 import "errors"
4
5 var ErrDivisionByZero = errors.New("divisão por zero")
```

Em seguida, basta retornar essa variável sempre que necessário.

```
func Divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, ErrDivisionByZero
    }

    return a / b, nil
}
```

Isso torna o código mais consistente e evita a criação de vários valores de erro representando exatamente a mesma condição.

11.2.3 Limitações

Como `errors.New` recebe apenas uma string, ele é mais adequado para mensagens fixas. Quando a mensagem precisa incluir informações específicas da execução, a chamada deixa de ser suficiente.

Por exemplo, suponha que seja necessário informar qual arquivo não pôde ser aberto:

```
return errors.New("erro ao abrir arquivo")
```

Essa mensagem informa que ocorreu um problema, mas não diz qual arquivo causou a falha.

Quando for necessário incorporar valores dinâmicos à mensagem, a abordagem adequada é utilizar `fmt.Errorf`, que será apresentada na próxima seção.

11.2.4 Boas práticas

Ao criar erros com `errors.New`:

- utilize mensagens curtas e objetivas;
- escreva a mensagem em letras minúsculas;
- evite pontuação no final da frase;
- descreva o problema, e não a ação que deve ser tomada.

Exemplos:

```
errors.New("arquivo não encontrado")
errors.New("credenciais inválidas")
errors.New("conexão encerrada")
```

Evite mensagens como:

```
errors.New("Erro!")
errors.New("Ocorreu um erro durante a execução.")
errors.New("Você deve verificar o arquivo.")
```

Mensagens claras e consistentes facilitam o entendimento do código e tornam os erros mais úteis quando são propagados pela aplicação.

11.3 `fmt.Errorf`

Em muitas situações, uma mensagem fixa não é suficiente para descrever um problema. Frequentemente é necessário incluir informações específicas da execução, como o nome de um arquivo, um valor inválido ou o identificador de um recurso.

Para esses casos, a biblioteca padrão oferece a função `fmt.Errorf`, que permite criar erros com a mesma sintaxe de formatação usada por `fmt.Printf` e `fmt.Sprintf`.

Sua assinatura é:

```
func Errorf(format string, a ...any) error
```

O primeiro argumento é uma string de formatação, seguida pelos valores que serão inseridos na mensagem. O valor retornado implementa a interface `error`.


```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func abrirArquivo(nome string) error {
8     return fmt.Errorf("não foi possível abrir o arquivo %q", nome)
9 }
10
11 func main() {
12     err := abrirArquivo("config.json")
13     if err != nil {
14         fmt.Println(err)
15     }
16 }
```

Saída:

```
não foi possível abrir o arquivo "config.json"
```

Observe que a mensagem contém o nome do arquivo que provocou a falha, tornando o diagnóstico muito mais simples.

11.3.1 Verbos de formatação

Como `fmt.Errorf` utiliza o mesmo mecanismo de formatação do pacote `fmt`, todos os verbos conhecidos podem ser empregados.

Alguns dos mais utilizados são:

Verbo	Descrição
%s	String
%d	Inteiro decimal
%f	Número de ponto flutuante
%v	Valor no formato padrão
%q	String entre aspas
%T	Tipo do valor

Exemplo:

```
1 package main
2
3 import "fmt"
4
5 func validarIdade(idade int) error {
6     if idade < 0 {
7         return fmt.Errorf("idade inválida: %d", idade)
8     }
9
10    return nil
11 }
12
13 func main() {
14     err := validarIdade(-5)
15     if err != nil {
16         fmt.Println(err)
17     }
18 }
```

Saída:

```
idade inválida: -5
```

11.3.2 Quando utilizar

`fmt.Errorf` deve ser utilizado quando a mensagem de erro depende de informações conhecidas em tempo de execução.

Por exemplo:

```
return fmt.Errorf("usuário %d não encontrado", id)
```

```
return fmt.Errorf("arquivo %q não existe", nome)
```

```
return fmt.Errorf("valor %v é inválido", valor)
```

Em todos esses casos, usar `errors.New` obrigaria a criar uma mensagem genérica, perdendo informações importantes para o diagnóstico do problema.

11.3.3 Comparando com `errors.New`

Quando a mensagem é fixa, `errors.New` continua sendo a opção mais simples.

```
return errors.New("senha inválida")
```

Quando a mensagem precisa incorporar valores, `fmt.Errorf` é a escolha adequada.

```
return fmt.Errorf("senha inválida para o usuário %q", usuario)
```

A escolha entre uma função e outra depende da necessidade de incluir valores dinâmicos na mensagem.

11.3.4 Preparando o terreno para *wrapping*

Além de criar mensagens formatadas, `fmt.Errorf` possui outra funcionalidade importante: a capacidade de encapsular erros já existentes utilizando o verbo `%w`.

Esse recurso permite preservar o erro original enquanto adiciona mais contexto à mensagem, e é muito utilizado na construção de aplicações maiores.

O encapsulamento de erros será apresentado na próxima seção.

11.4 *Wrapping*

À medida que uma aplicação cresce, é comum que uma função chame outra, que por sua vez chame uma terceira. Quando um erro ocorre em uma das camadas mais internas, simplesmente retorná-lo pode dificultar a identificação do local onde o problema aconteceu.

Uma boa prática é adicionar contexto ao erro à medida que ele é propagado pelas diferentes camadas da aplicação. Em Go, isso é conhecido como *error wrapping*, ou encapsulamento de erros.

O encapsulamento de erros é realizado utilizando o verbo `%w` da função `fmt.Errorf`.

```
return fmt.Errorf("mensagem adicional: %w", err)
```

Dessa forma, uma nova mensagem é adicionada sem descartar o erro original.

11.4.1 Motivação

Considere duas funções. A primeira realiza uma divisão e a segunda apenas a utiliza.

```
1 package main
2
3 import (
4     "errors"
5     "fmt"
6 )
7
8 var ErrDivisionByZero = errors.New("divisão por zero")
9
10 func divide(a, b float64) (float64, error) {
11     if b == 0 {
12         return 0, ErrDivisionByZero
13     }
14
15     return a / b, nil
16 }
17
18 func calcularMedia(a, b float64) (float64, error) {
19     resultado, err := divide(a, b)
20     if err != nil {
21         return 0, err
22     }
23
24     return resultado, nil
25 }
26
27 func main() {
28     _, err := calcularMedia(10, 0)
29     if err != nil {
30         fmt.Println(err)
31     }
32 }
```

Saída:

```
divisão por zero
```

Embora o erro informe o problema, ele não diz em que contexto ocorreu. Se diversas funções utilizarem `divide`, será difícil identificar qual chamada gerou a falha.

11.4.2 Adicionando contexto

Com *wrapping*, cada camada pode acrescentar informações úteis antes de retornar o erro.

```
func calcularMedia(a, b float64) (float64, error) {
    resultado, err := divide(a, b)
    if err != nil {
        return 0, fmt.Errorf("erro ao calcular média: %w", err)
    }

    return resultado, nil
}
```

Agora a saída passa a ser:

```
erro ao calcular média: divisão por zero
```

Observe que a mensagem ficou mais descritiva, mas o erro original continua preservado na cadeia de erros.

11.4.3 *Wrapping* em múltiplas camadas

Cada função pode adicionar seu próprio contexto.

```
func gerarRelatorio() error {
    _, err := calcularMedia(10, 0)
    if err != nil {
        return fmt.Errorf("falha ao gerar relatório: %w", err)
    }

    return nil
}
```

A mensagem produzida será semelhante a:

```
falha ao gerar relatório: erro ao calcular média: divisão por zero
```

Perceba que cada camada acrescenta uma informação relevante sobre o caminho percorrido até a falha.

11.4.4 Por que utilizar %w?

Pode parecer tentador apenas construir uma nova mensagem usando o texto do erro recebido.

```
return fmt.Errorf("erro ao calcular média: %v", err)
```

Embora a saída visual seja parecida, existe uma diferença importante.

Com %v, apenas o texto do erro é incorporado à nova mensagem.

Com %w, além do texto, o erro original também é preservado na cadeia.

Essa preservação permite que outras partes da aplicação identifiquem o erro original utilizando funções como `errors.Is` e `errors.As`, que serão apresentadas nas próximas seções.

11.4.5 Quando utilizar

O encapsulamento de erros é recomendado sempre que uma função:

- recebe um erro de outra função;
- adiciona uma informação útil sobre o contexto em que o erro ocorreu;
- continua propagando esse erro para o chamador.

Essa prática produz mensagens mais informativas sem perder a capacidade de identificar o erro original.

Nos próximos tópicos veremos como aproveitar esse erro encapsulado utilizando `errors.Is` e `errors.As`.

11.5 errors.Is

Com o uso de *error wrapping*, tornou-se necessário um mecanismo para verificar se um erro encapsulado representa uma determinada condição, independentemente de quantas camadas de contexto tenham sido adicionadas.

Para isso, o pacote `errors` fornece a função `errors.Is`.

Sua assinatura é:

```
func Is(err, target error) bool
```

Ela percorre a cadeia de erros encapsulados procurando o erro informado em `target`.

11.5.1 Comparando erros

Considere o seguinte exemplo:

```
1 package main
2
3 import (
4     "errors"
5     "fmt"
6 )
7
8 var ErrDivisionByZero = errors.New("divisão por zero")
9
10 func divide(a, b float64) (float64, error) {
11     if b == 0 {
12         return 0, ErrDivisionByZero
13     }
14
15     return a / b, nil
16 }
17
18 func calcularMedia(a, b float64) (float64, error) {
19     resultado, err := divide(a, b)
20     if err != nil {
21         return 0, fmt.Errorf("erro ao calcular média: %w", err)
22     }
23
24     return resultado, nil
25 }
26
27 func main() {
28     _, err := calcularMedia(10, 0)
29
30     if errors.Is(err, ErrDivisionByZero) {
31         fmt.Println("não é possível dividir por zero")
32     }
33 }
```

Saída:

```
não é possível dividir por zero
```

Embora `ErrDivisionByZero` esteja encapsulado dentro de outro erro, `errors.Is` consegue identificá-lo.

11.5.2 Comparação direta após wrapping

Após um erro ser encapsulado, ele deixa de ser exatamente o mesmo valor retornado originalmente.

Por isso, a comparação direta normalmente falha.

```
if err == ErrDivisionByZero {  
    fmt.Println("divisão por zero")  
}
```

Nesse caso, `err` representa um novo erro criado por `fmt.Errorf`, e não diretamente `ErrDivisionByZero`.

Já `errors.Is` percorre a cadeia de erros até encontrar o erro procurado.

```
if errors.Is(err, ErrDivisionByZero) {  
    fmt.Println("divisão por zero")  
}
```

Essa é a forma recomendada de verificar erros que podem ter sido encapsulados.

11.5.3 Cadeias de encapsulamento

O número de camadas não importa.

```
err := fmt.Errorf("camada 3: %w",  
    fmt.Errorf("camada 2: %w",  
        fmt.Errorf("camada 1: %w",  
            ErrDivisionByZero)))
```

Mesmo nesse caso:

```
errors.Is(err, ErrDivisionByZero)
```

retorna:

```
true
```

A função percorre automaticamente toda a cadeia.

11.5.4 Quando utilizar

`errors.Is` deve ser utilizado quando o objetivo for verificar se um erro representa uma condição específica.

Exemplos comuns:

- verificar se um arquivo não existe;
- identificar falha de autenticação;
- detectar um erro de validação;
- verificar erros definidos pela própria aplicação.

Por exemplo:


```
if errors.Is(err, ErrUnauthorized) {  
    // solicitar novo login  
}  
  
if errors.Is(err, ErrInvalidPassword) {  
    // informar senha inválida  
}
```

Observe que o código verifica a natureza do erro, não sua mensagem.

11.5.5 Evite comparar mensagens

Um erro bastante comum é verificar diretamente o texto retornado por `Error()`.

```
if err.Error() == "divisão por zero" {  
    // ...  
}
```

Essa abordagem apresenta diversos problemas:

- a mensagem pode mudar futuramente;
- mensagens podem ser traduzidas para outros idiomas;
- erros encapsulados alteram o texto exibido;
- diferentes erros podem possuir mensagens semelhantes.

Sempre que possível, verifique o próprio erro utilizando `errors.Is`.

Essa abordagem é mais segura, mais robusta e permanece funcionando mesmo quando os erros são encapsulados por várias camadas da aplicação.

11.6 errors.As

Nem todos os erros podem ser identificados apenas comparando seu valor. Em muitas situações, um erro possui informações adicionais armazenadas em uma estrutura específica, como o nome de um arquivo, um código de erro ou outros dados relevantes.

Para acessar essas informações, o pacote `errors` oferece a função `errors.As`.

Sua assinatura é:

```
func As(err error, target any) bool
```

A função percorre a cadeia de erros encapsulados procurando um erro compatível com o tipo informado em `target`.

11.6.1 Motivação

Considere uma aplicação que define um tipo próprio de erro.

```
1 package main
2
3 import "fmt"
4
5 type ValidationError struct {
6     Field string
7 }
8
9 func (e ValidationError) Error() string {
10     return fmt.Sprintf("campo %q é inválido", e.Field)
11 }
```

Além da mensagem, esse erro armazena qual campo provocou o problema.

Uma função pode retorná-lo normalmente.

```
func validarNome(nome string) error {
    if nome == "" {
        return ValidationError{
            Field: "nome",
        }
    }

    return nil
}
```

11.6.2 Obtendo o erro concreto

Suponha que esse erro seja encapsulado.

```
func cadastrar(nome string) error {
    err := validarNome(nome)
    if err != nil {
        return fmt.Errorf("erro ao cadastrar usuário: %w", err)
    }

    return nil
}
```

Ainda é possível acessar esse erro concreto utilizando `errors.As`.

```
func main() {
    err := cadastrar("")

    var validationErr ValidationError

    if errors.As(err, &validationErr) {
        fmt.Println(validationErr.Field)
    }
}
```

Saída:

```
nome
```

Observe que `errors.As` encontrou o erro encapsulado e preencheu a variável `validationErr` com o valor correspondente.

11.6.3 Por que passar um ponteiro?

O segundo argumento de `errors.As` deve ser um ponteiro para a variável que receberá o erro compatível.

```
var validationErr ValidationError

errors.As(err, &validationErr)
```

A função precisa modificar essa variável caso encontre um erro do tipo esperado.

Sem o ponteiro, isso não seria possível.

11.6.4 Comparando com errors.Is

Embora ambas as funções percorram a cadeia de erros, seus objetivos são diferentes.

Utilize `errors.Is` quando desejar verificar se o erro representa uma condição específica.

```
if errors.Is(err, ErrDivisionByZero) {
    // ...
}
```

Utilize `errors.As` quando precisar acessar um erro de determinado tipo para utilizar suas informações.

```
var validationErr ValidationError

if errors.As(err, &validationErr) {
    fmt.Println(validationErr.Field)
}
```

Em resumo:

- `errors.Is` responde à pergunta: “este erro representa esta condição?”;
- `errors.As` responde à pergunta: “existe um erro deste tipo na cadeia?”.

11.6.5 Um exemplo da biblioteca padrão

Diversos pacotes da biblioteca padrão retornam erros que carregam informações adicionais.

Por exemplo, ao abrir um arquivo inexistente, a função `os.Open` retorna um erro do tipo `*os.PathError`.

```
1 package main
2
3 import (
4     "errors"
5     "fmt"
6     "os"
7 )
8
9 func main() {
10     _, err := os.Open("arquivo.txt")
11
12     var pathErr *os.PathError
13
14     if errors.As(err, &pathErr) {
15         fmt.Println(pathErr.Path)
16     }
17 }
```

Saída:

```
arquivo.txt
```

Nesse caso, além da mensagem de erro, é possível descobrir qual caminho provocou a falha.

11.6.6 Quando utilizar

`errors.As` é recomendado quando o erro contém informações adicionais além da mensagem.

Isso é comum em:

- erros definidos pela própria aplicação;
- erros retornados pela biblioteca padrão;
- bibliotecas que implementam seus próprios tipos de erro.

Sempre que for necessário acessar esses dados, utilize `errors.As` em vez de analisar a mensagem retornada por `Error()`. Dessa forma, o código permanece mais robusto, seguro e independente da forma como a mensagem é escrita.

11.7 panic

Até este ponto, todos os erros foram tratados utilizando o valor retornado pela interface `error`. Essa é a abordagem recomendada em Go e deve ser utilizada na grande maioria das situações.

Entretanto, existem casos excepcionais em que não faz sentido continuar a execução do programa. Para essas situações, Go fornece a função `panic`.

Quando um `panic` é executado, a execução normal da função é interrompida imediatamente. O processo passa a desempilhar (*unwind*) a pilha de chamadas até encontrar um mecanismo de recuperação ou, caso contrário, encerra a aplicação exibindo uma mensagem de erro e o *stack trace*.

Sua assinatura é:

```
func panic(v any)
```

O argumento pode ser qualquer valor, embora seja comum utilizar uma string ou um valor de erro.

11.7.1 Exemplo básico

```
1 package main
2
3 func X() {
4     panic("erro inesperado")
5 }
6
7 func main() {
8     X()
9 }
```

Saída:

```
panic: erro inesperado

goroutine 1 [running]:
main.X(...)
    /tmp/sandbox3923525807/prog.go:6
main.main()
    /tmp/sandbox3923525807/prog.go:10 +0x25

Program exited.
```

Observe que, além da mensagem, Go exibe a pilha de chamadas até o ponto em que o problema ocorreu.

11.7.2 Interrompendo a execução

Após a execução de um `panic`, nenhuma instrução posterior é executada.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Início")
7
8     panic("erro fatal")
9
10    fmt.Println("Fim")
11 }
```

Saída:

```
Início
panic: erro fatal
```

A última chamada a `fmt.Println` nunca é executada.

11.7.3 panic com um erro

Embora seja possível utilizar qualquer valor, muitas vezes o argumento é um erro.

```
1 package main
2
3 import (
4     "errors"
5 )
6
7 func main() {
8     panic(errors.New("arquivo de configuração inválido"))
9 }
```

Saída:

```
panic: arquivo de configuração inválido
```

Isso torna o valor associado ao `panic` mais consistente com o restante do mecanismo de tratamento de erros da linguagem.

11.7.4 Quando utilizar

O uso de `panic` deve ser reservado para situações excepcionais, nas quais a execução do programa não pode prosseguir de forma segura.

Alguns exemplos incluem:

- violação de uma suposição interna do programa;
- estado inconsistente da aplicação;

- erros de inicialização que impedem o funcionamento do sistema;
- situações que indicam falhas de programação.

Por outro lado, erros esperados durante a execução normalmente **não** devem gerar um `panic`.

Por exemplo:

```
func carregarArquivo(nome string) error {  
    // ...  
}
```

Se um arquivo não existir, isso representa uma condição que pode ser tratada normalmente pelo código chamador.

Da mesma forma:

- uma senha incorreta;
- um usuário inexistente;
- uma conexão recusada;
- uma entrada inválida.

Todos esses casos devem retornar um `error`, não provocar um `panic`.

11.7.5 `panic` na biblioteca padrão

Algumas funções da biblioteca padrão utilizam `panic` quando recebem argumentos inválidos que representam erro de programação.

Por exemplo:

```
1 package main  
2  
3 import "fmt"  
4  
5 func main() {  
6     numbers := []int{1, 2, 3}  
7  
8     fmt.Println(numbers[10])  
9 }
```

Saída:

```
panic: runtime error: index out of range [10] with length 3
```

Nesse caso, acessar uma posição inexistente da slice representa um erro de programação, não uma condição esperada de execução.

11.7.6 `panic` não substitui `error`

Um erro comum entre iniciantes é utilizar `panic` como mecanismo principal de tratamento de erros.

```
func dividir(a, b float64) float64 {  
    if b == 0 {  
        panic("divisão por zero")  
    }  
  
    return a / b  
}
```

Essa implementação é inadequada, pois uma divisão por zero pode ser tratada por quem chamou a função.

Uma abordagem melhor é retornar um erro.

```
func dividir(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, errors.New("divisão por zero")  
    }  
  
    return a / b, nil  
}
```

Como regra geral:

- utilize `error` para situações esperadas que podem ser tratadas;
- utilize `panic` apenas para falhas excepcionais das quais a aplicação não consegue se recuperar com segurança.

Na próxima seção veremos que, embora um `panic` interrompa a execução normal do programa, ainda existe um mecanismo capaz de interceptá-lo: a função `recover`.

11.8 recover

Como vimos na seção anterior, quando ocorre um `panic`, a execução normal do programa é interrompida. Entretanto, Go oferece um mecanismo que permite interceptar esse `panic` antes que a aplicação seja encerrada: a função `recover`.

Sua assinatura é:

```
func recover() any
```

Quando chamada durante o tratamento de um `panic`, `recover` retorna o valor passado para `panic` e interrompe sua propagação. Caso não exista um `panic` em andamento, retorna `nil`.

11.8.1 Utilizando `recover`

Para que `recover` funcione, ele deve ser chamado dentro de uma função adiada com `defer`.


```
1 package main
2
3 import "fmt"
4
5 func main() {
6     defer func() {
7         if r := recover(); r != nil {
8             fmt.Println("panic recuperado:", r)
9         }
10    }()
11
12    panic("algo deu errado")
13
14    fmt.Println("esta linha nunca será executada")
15 }
```

Saída:

```
panic recuperado: algo deu errado
```

Observe que, embora o panic tenha ocorrido, o programa não foi encerrado abruptamente.

11.8.2 Por que utilizar defer?

Quando um panic acontece, Go inicia o desempilhamento da pilha de chamadas. Durante esse processo, todas as funções registradas com defer são executadas.

É nesse momento que recover pode interceptar o panic.

Fora desse contexto, recover não possui efeito prático.

```
func main() {
    r := recover()

    fmt.Println(r)
}
```

Saída:

```
<nil>
```

Nesse exemplo, nenhum panic estava sendo tratado; por isso, recover apenas retornou nil.

11.8.3 Recuperando a execução

Após recuperar um panic, a função em que o defer foi executado é encerrada normalmente.

Considere o exemplo:

```
1 package main
2
3 import "fmt"
4
5 func executar() {
6     defer func() {
7         if recover() != nil {
8             fmt.Println("erro recuperado")
9         }
10    }()
11
12    panic("erro")
13 }
14
15 func main() {
16     executar()
17
18     fmt.Println("programa continua")
19 }
```

Saída:

```
erro recuperado
programa continua
```

O panic interrompeu a execução de executar, mas não encerrou toda a aplicação.

11.8.4 Um uso comum

Um dos usos mais frequentes de recover é impedir que uma falha em uma única requisição encerre todo um servidor HTTP.

De forma simplificada:

```
func handler() {
    defer func() {
        if r := recover(); r != nil {
            // registrar o erro
            // retornar HTTP 500
        }
    }()

    // processamento da requisição
}
```

Assim, caso um panic ocorra durante o atendimento de uma requisição, apenas aquela requisição será interrompida. O servidor continuará processando as demais normalmente.

É justamente por esse motivo que diversos frameworks e bibliotecas implementam mecanismos

automáticos de recuperação de panic.

11.8.5 recover não substitui o tratamento de erros

Assim como panic, recover não deve ser utilizado como mecanismo comum de controle de fluxo.

Por exemplo, não faz sentido escrever código como:

```
func dividir(a, b float64) float64 {  
    defer func() {  
        recover()  
    }()  
  
    if b == 0 {  
        panic("divisão por zero")  
    }  
  
    return a / b  
}
```

Nesse caso, retornar um error continua sendo a solução correta.

recover existe para lidar com situações excepcionais, geralmente próximas às fronteiras da aplicação, como servidores, *workers* ou processos em segundo plano.

11.8.6 Resumo

Neste capítulo foram apresentados os principais mecanismos de tratamento de erros disponíveis em Go:

- a interface `error`, utilizada para representar falhas de forma explícita;
- `errors.New`, para criar erros simples;
- `fmt.Errorf`, para construir mensagens formatadas;
- *error wrapping*, para adicionar contexto preservando o erro original;
- `errors.Is`, para verificar condições específicas;
- `errors.As`, para acessar erros de um determinado tipo;
- `panic`, para sinalizar falhas irrecuperáveis;
- `recover`, para interceptar um `panic` e evitar o encerramento da aplicação.

Na maior parte do tempo, aplicações Go utilizam apenas o retorno de `error`. O uso de `panic` e `recover` deve ser restrito a situações excepcionais, preservando uma das principais características da linguagem: o tratamento explícito e previsível de erros.

Capítulo 12

Packages

Até este ponto, muitos exemplos foram escritos em um único arquivo pertencente ao pacote `main`. Essa abordagem é suficiente para aprender os conceitos fundamentais da linguagem, mas rapidamente se torna limitada à medida que os programas crescem.

Em aplicações reais, o código costuma ser dividido em múltiplos arquivos e pacotes, cada um responsável por uma parte específica do sistema. Essa organização facilita a manutenção, reduz o acoplamento entre componentes e torna o código mais reutilizável.

Em Go, todo arquivo-fonte pertence obrigatoriamente a um pacote. O compilador utiliza essa estrutura para organizar o código, controlar a visibilidade dos identificadores e definir como diferentes partes de um programa podem interagir.

Ao longo deste capítulo, veremos como os pacotes são organizados, quais elementos podem ser acessados por outros pacotes e como Go implementa um modelo simples de encapsulamento. Diferentemente de muitas linguagens orientadas a objetos, Go não utiliza modificadores de acesso como `public`, `private` ou `protected`. Em vez disso, a visibilidade é determinada por uma convenção baseada na capitalização dos identificadores.

Ao final deste capítulo, você será capaz de organizar aplicações em múltiplos pacotes, compreender as regras de exportação de identificadores e estruturar projetos de forma clara, modular e idiomática.

12.1 Exportação

Em Go, funções, tipos, variáveis, constantes, interfaces, campos e métodos podem ser acessados por outros pacotes apenas quando são exportados.

A regra de exportação é simples: identificadores cujo nome começa com uma letra maiúscula são exportados; aqueles que começam com letra minúscula permanecem visíveis apenas dentro do próprio pacote.

```
1 package geometry
2
3 func Area(radius float64) float64 {
4     // ...
5 }
6
7 func perimeter(radius float64) float64 {
8     // ...
9 }
```

Nesse exemplo, `Area` pode ser utilizada por qualquer outro pacote que importe `geometry`, enquanto `perimeter` só pode ser chamada por arquivos pertencentes ao próprio pacote.

A mesma regra se aplica a todos os identificadores declarados em um pacote:

```
1 package config
2
3 const Version = "1.0.0"
4
5 var MaxConnections = 100
6
7 type Server struct {
8     Address string
9     port    int
10 }
11
12 type logger struct {
13     level int
14 }
```

Neste exemplo:

- `Version` pode ser acessada por outros pacotes.
- `MaxConnections` também é exportada.
- `Server` é um tipo exportado.
- `Address` é um campo exportado de `Server`.
- `port` é um campo restrito ao pacote `config`.
- `logger` permanece restrito ao pacote `config`.

Essa convenção substitui palavras-chave como `public`, `private` ou `protected`, presentes em outras linguagens. O compilador determina automaticamente a visibilidade de um identificador observando a primeira letra de seu nome.

Essa simplicidade é uma das características marcantes da linguagem. Ao ler um código Go, é possível identificar imediatamente quais elementos fazem parte da interface pública de um pacote e quais representam detalhes internos de implementação.

12.1.1 Exemplo

Considere a seguinte estrutura de diretórios:

```
projeto/  
├── go.mod  
├── main.go  
└── geometry/  
    └── circle.go
```

Antes de criar os arquivos do exemplo, inicialize o projeto como um módulo Go:

```
go mod init projeto
```

Esse comando cria o arquivo `go.mod`, necessário para que o pacote `geometry` possa ser importado pelo pacote `main`. O funcionamento desse arquivo e do sistema de módulos será estudado em detalhes no próximo capítulo.

Arquivo `geometry/circle.go`:

```
1 package geometry  
2  
3 import "math"  
4  
5 func Area(radius float64) float64 {  
6     return math.Pi * radius * radius  
7 }  
8  
9 func perimeter(radius float64) float64 {  
10    return 2 * math.Pi * radius  
11 }
```

Arquivo `main.go`:

```
1 package main  
2  
3 import (  
4     "fmt"  
5  
6     "projeto/geometry"  
7 )  
8  
9 func main() {  
10    fmt.Println(geometry.Area(5))  
11 }
```

A função `Area` pode ser chamada normalmente porque é exportada. Já a função `perimeter` não pode ser acessada pelo pacote `main`:

```
fmt.Println(geometry.perimeter(5))
```

A adição dessa linha à função `main` não compila, pois `perimeter` não foi exportada pelo pacote `geometry`.

12.1.2 Conclusão

A exportação é um dos mecanismos fundamentais de encapsulamento em Go. Em vez de depender de diversos modificadores de acesso, a linguagem adota uma convenção simples e consistente, tornando o código mais fácil de ler e compreender.

Nos próximos tópicos veremos como essa regra se integra à organização dos pacotes e como ela influencia a estrutura de aplicações Go de qualquer porte.

12.2 Organização

À medida que um programa cresce, manter todo o código em um único arquivo ou em um único pacote torna a manutenção cada vez mais difícil. Dividir a aplicação em pacotes permite separar responsabilidades, facilitar a reutilização de código e tornar o projeto mais fácil de compreender.

Em Go, os arquivos de um mesmo diretório normalmente formam um pacote. Todos os arquivos `.go` desse diretório devem declarar o mesmo pacote na cláusula `package`.

Considere a seguinte estrutura:

```
projeto/
├── main.go
├── geometry/
│   ├── circle.go
│   └── rectangle.go
├── mathutil/
│   └── max.go
```

O arquivo `main.go` pertence ao pacote `main`:

```
1 package main
```

Os arquivos `circle.go` e `rectangle.go` pertencem ao pacote `geometry`:

```
1 package geometry
```

Já o arquivo `max.go` pertence ao pacote `mathutil`:

```
1 package mathutil
```

Embora estejam em arquivos diferentes, todos os elementos declarados dentro do mesmo pacote podem ser utilizados entre si, independentemente de serem exportados.

Por exemplo, suponha que `circle.go` contenha a seguinte função:

```
1 package geometry
2
3 func Area(radius float64) float64 {
4     return area(radius)
5 }
```

E que `helper.go`, no mesmo diretório, contenha:

```
1 package geometry
2
3 import "math"
4
5 func area(radius float64) float64 {
6     return math.Pi * radius * radius
7 }
```

Mesmo sendo uma função não exportada, `area` pode ser chamada por `Area`, pois ambas pertencem ao mesmo pacote.

A divisão em múltiplos arquivos ajuda a organizar o código, mas não cria novas fronteiras de visibilidade dentro do pacote.

Para o compilador, todos os arquivos pertencentes ao mesmo pacote são tratados como uma única unidade.

Uma boa prática é agrupar em um mesmo pacote elementos que possuam uma responsabilidade comum. Pacotes muito grandes tendem a concentrar responsabilidades demais, enquanto pacotes excessivamente pequenos podem tornar a navegação do projeto mais complexa do que o necessário. O objetivo é encontrar uma organização clara, coesa e fácil de manter.

12.3 Visibilidade

A exportação determina quais identificadores podem ser acessados por outros pacotes. A visibilidade, por sua vez, define em quais partes do programa cada identificador pode ser utilizado.

Em Go, existem apenas dois níveis de visibilidade:

- visibilidade interna ao próprio pacote;
- visibilidade externa, para pacotes que importem o pacote em que o identificador foi declarado.

Não existem níveis intermediários, como `protected` ou acesso por classes derivadas, comuns em algumas linguagens orientadas a objetos.

Considere o exemplo a seguir:

```
1 package geometry
2
3 import "math"
4
5 func Area(radius float64) float64 {
6     return area(radius)
7 }
8
9 func area(radius float64) float64 {
10    return math.Pi * radius * radius
11 }
```

A função `area` possui visibilidade apenas dentro do pacote `geometry`. Ela pode ser utilizada por qualquer arquivo pertencente a esse pacote, mas não pode ser chamada por outros pacotes.

Já a função `Area` é exportada e pode ser utilizada por outros pacotes que importem `geometry`.

Essa distinção permite ocultar detalhes de implementação e expor apenas aquilo que faz parte da interface pública do pacote. Alterações em funções, tipos ou variáveis não exportados podem ser realizadas sem impactar os demais pacotes da aplicação.

Ao projetar um pacote, é recomendável exportar apenas os identificadores que realmente precisam ser utilizados externamente. Manter a interface pública pequena reduz o acoplamento entre os componentes e torna o código mais simples de manter e evoluir.

Em Go, o encapsulamento é realizado no nível do pacote. Assim, funções, tipos e demais identificadores não exportados permanecem acessíveis por todos os arquivos do mesmo pacote, mas ficam ocultos para os demais pacotes da aplicação.

12.4 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Em muitos casos, existem várias soluções possíveis. Priorize programas corretos, claros e fáceis de compreender.

Antes de consultar a solução ou recorrer a ferramentas de inteligência artificial, dedique um tempo para tentar resolver o problema por conta própria. O processo de análise, tentativa e correção faz parte do aprendizado e contribui significativamente para o desenvolvimento da capacidade de programação.

Se um exercício parecer difícil, divida-o em partes menores, valide cada etapa e teste seu programa com diferentes entradas. Desenvolver o hábito de experimentar, depurar e refatorar o código é tão importante quanto chegar a uma resposta correta.

12.4.1 Antes de iniciar os exercícios

Antes de iniciar os desafios, crie um diretório para o projeto e inicialize um módulo Go:

```
mkdir practice-01
cd practice-01
go mod init practice-01
```

Esse comando cria o arquivo `go.mod`, necessário para que os pacotes criados dentro do projeto possam ser importados pelo pacote `main`. O sistema de módulos será estudado em detalhes no próximo capítulo.

Os pacotes criados dentro desse projeto deverão ser importados a partir do nome do módulo.

Por exemplo, se o projeto possuir um pacote `mathutil`, o `import` no arquivo `main.go` deverá ser:

```
import "practice-01/mathutil"
```

12.4.2 Exercícios

1. Crie um pacote chamado `mathutil` contendo as funções `Max(a, b int) int` e `Min(a, b int) int`. Em seguida, desenvolva um programa no pacote `main` que importe esse pacote e utilize ambas as funções para comparar diferentes pares de números.

2. Desenvolva um pacote chamado `geometry` para representar figuras geométricas.

Nesse pacote, crie uma interface chamada `Shape` contendo os métodos:

```
Area() float64
Perimeter() float64
```

Em seguida, implemente essa interface para três structs:

```
Circle
Square
Rectangle
```

Cada figura deve armazenar apenas os dados necessários para seus cálculos. Por exemplo, `Circle` deve armazenar o raio, `Square` deve armazenar o lado e `Rectangle` deve armazenar largura e altura.

Como essas structs serão utilizadas a partir do pacote `main`, defina uma forma clara de criar seus valores. Uma opção simples, adequada para este exercício, é utilizar campos exportados:

```
type Circle struct {
    Radius float64
}
```

Organize a implementação em múltiplos arquivos, por exemplo:

```
geometry/
├── shape.go
└── circle.go
```

```
| square.go  
| rectangle.go
```

No pacote `main`, crie um slice de `geometry.Shape` contendo diferentes figuras. Percorra esse slice e exiba a área e o perímetro de cada elemento.

O objetivo deste exercício é observar que diferentes tipos concretos podem ser tratados de maneira uniforme quando implementam a mesma interface.

Capítulo 13

Módulos

Nos capítulos anteriores aprendemos a dividir uma aplicação em pacotes, organizando o código de forma clara e modular. Entretanto, essa organização, por si só, não resolve outro desafio importante: como compartilhar código entre diferentes projetos e gerenciar bibliotecas externas.

Para atender a essa necessidade, Go utiliza o conceito de módulos. Um módulo representa uma unidade de distribuição e versionamento de código, permitindo que um projeto declare suas dependências de forma explícita e reproduzível.

Desde o Go 1.11, os módulos passaram a substituir o antigo modelo baseado na variável de ambiente GOPATH. Atualmente, eles constituem o mecanismo oficial para criação de projetos, gerenciamento de versões e obtenção de bibliotecas desenvolvidas por terceiros.

O funcionamento dos módulos é definido principalmente pelo arquivo `go.mod`, que identifica o módulo e registra as dependências necessárias para sua compilação. A ferramenta Go utiliza essas informações para localizar, baixar e manter as versões corretas dos pacotes utilizados pelo projeto.

Neste capítulo, conheceremos a estrutura do arquivo `go.mod`, aprenderemos como adicionar e atualizar dependências e veremos a diferença entre os comandos `go get` e `go install`, dois recursos fundamentais para o desenvolvimento de aplicações em Go.

13.1 `go.mod`

Todo módulo Go é definido por um arquivo chamado `go.mod`, localizado na raiz do projeto. Esse arquivo identifica o módulo e informa à ferramenta Go quais dependências são utilizadas pela aplicação.

Antes de conhecer sua estrutura, é importante compreender a diferença entre um pacote e um módulo.

Um **pacote** é um conjunto de arquivos Go localizados em um mesmo diretório e que compartilham a mesma declaração `package`. Os pacotes são utilizados para organizar o código dentro de uma aplicação.

Um **módulo**, por outro lado, representa um projeto completo. Ele pode conter um ou vários pacotes e é identificado por um único arquivo `go.mod`.

Considere a seguinte estrutura:

```
calculator/  
├── go.mod  
├── main.go  
├── math/  
│   ├── add.go  
│   └── subtract.go  
└── geometry/  
    └── circle.go
```

Nesse exemplo:

- O projeto `calculator` corresponde a um módulo.
- O diretório `math` representa um pacote.
- O diretório `geometry` representa outro pacote.
- O arquivo `main.go` pertence ao pacote `main`.

Todo o módulo é identificado pelo arquivo `go.mod`, cujo conteúdo básico é semelhante ao seguinte:

```
module github.com/user/calculator  
  
go 1.26
```

A diretiva `module` define o identificador único do módulo. Esse nome será utilizado sempre que outro projeto desejar importar seus pacotes.

A diretiva `go` informa a versão mínima da linguagem para a qual o módulo foi desenvolvido. Ela também permite que a ferramenta Go utilize o comportamento adequado para aquela versão da linguagem.

O arquivo `go.mod` é criado automaticamente pelo comando:

```
go mod init github.com/user/calculator
```

Após sua criação, o projeto passa a ser reconhecido como um módulo Go, permitindo a utilização de dependências externas, gerenciamento de versões e todas as funcionalidades do sistema de módulos da linguagem.

Na maior parte dos projetos, o arquivo `go.mod` é mantido automaticamente pela própria ferramenta Go. Sempre que novas dependências são adicionadas ou removidas, esse arquivo é atualizado para refletir o estado atual do projeto.

13.2 Dependências

Raramente uma aplicação é desenvolvida utilizando apenas a biblioteca padrão da linguagem. Em muitos casos, é necessário utilizar bibliotecas criadas por terceiros para implementar funcionalidades como acesso a bancos de dados, comunicação HTTP, criptografia, geração de arquivos, entre muitas outras.

Os módulos Go permitem declarar essas dependências de forma explícita no arquivo `go.mod`.

Dessa forma, qualquer pessoa que obtenha o código do projeto poderá utilizar exatamente as mesmas versões das bibliotecas utilizadas durante o desenvolvimento.

Suponha que um projeto utilize o roteador HTTP `github.com/go-chi/chi/v5`. Após adicionar essa dependência, o arquivo `go.mod` poderá conter um trecho semelhante ao seguinte:

```
module github.com/user/calculator

go 1.26

require github.com/go-chi/chi/v5 v5.2.3
```

Cada linha da diretiva `require` informa:

- o caminho do módulo;
- a versão utilizada pelo projeto.

Sempre que o projeto for compilado, a ferramenta Go garantirá que essa versão esteja disponível no ambiente de desenvolvimento.

Além do arquivo `go.mod`, todo módulo possui um arquivo chamado `go.sum`.

```
go.mod
go.sum
```

Enquanto o `go.mod` registra quais módulos e versões são utilizados pelo projeto, o `go.sum` armazena verificações criptográficas (*checksums*) dessas dependências. Essas verificações permitem garantir que o conteúdo baixado seja exatamente o mesmo utilizado durante o desenvolvimento, aumentando a segurança e a reprodutibilidade das compilações.

Na maioria das situações, tanto o `go.mod` quanto o `go.sum` são atualizados automaticamente pela ferramenta Go. Por esse motivo, ambos devem ser mantidos sob controle de versão juntamente com o restante do projeto.

O gerenciamento automático de dependências é uma das características que tornam o ecossistema Go simples e confiável. O desenvolvedor informa apenas quais módulos deseja utilizar, enquanto a ferramenta Go se encarrega de localizar, baixar e manter as versões apropriadas.

13.3 O diretório de trabalho do Go

Além dos arquivos do próprio projeto, a ferramenta Go mantém um diretório de trabalho no ambiente do usuário. Na maioria das instalações, esse diretório é criado automaticamente em:

```
~/go
```

Ele é utilizado para armazenar módulos baixados da Internet, ferramentas instaladas e outros arquivos utilizados durante o desenvolvimento. Um diretório típico possui a seguinte estrutura:

```
~/go/
├─ bin/
├─ pkg/
```

```
└─ mod/  
└─ src/
```

Cada diretório possui uma finalidade específica:

- `bin/`: armazena programas instalados com `go install`.
- `pkg/mod/`: contém o cache local dos módulos baixados pela ferramenta Go.
- `src/`: era utilizado pelo antigo modelo baseado em `GOPATH` e atualmente raramente é utilizado em projetos que empregam módulos.

Sempre que um módulo é utilizado pela primeira vez, Go realiza seu download e o armazena em `pkg/mod`. Nas próximas compilações, esse mesmo módulo será reutilizado a partir do cache local, evitando novos downloads.

Esse diretório é gerenciado automaticamente pela ferramenta Go e, em condições normais, não deve ser modificado manualmente. Sua existência faz parte do funcionamento do sistema de módulos e não indica qualquer problema na instalação da linguagem.

13.4 go get

O comando `go get` é utilizado para adicionar, atualizar ou remover dependências de um módulo. Sempre que uma dependência é modificada, a ferramenta Go atualiza automaticamente os arquivos `go.mod` e `go.sum`.

Caso o módulo ainda não esteja disponível no ambiente local, a ferramenta Go realiza seu download e o armazena no cache de módulos, que por padrão está localizado em:

```
~/go/pkg/mod
```

Por exemplo, para adicionar o módulo `github.com/go-chi/chi/v5` ao projeto, basta executar:

```
go get github.com/go-chi/chi/v5
```

Após a execução do comando, a dependência será registrada no arquivo `go.mod` e estará disponível para utilização no código.

Também é possível utilizar uma versão específica:

```
go get github.com/go-chi/chi/v5@v5.2.3
```

A versão desejada é indicada acrescentando `@versão` ao final do caminho do módulo. Para atualizar uma dependência para sua versão mais recente:

```
go get github.com/go-chi/chi/v5@latest
```

Da mesma forma, é possível atualizar todas as dependências do projeto:

```
go get -u ./...
```

O padrão `./...` indica que a operação deve ser aplicada ao módulo atual e a todos os seus pacotes.

Após adicionar ou remover dependências, é recomendável executar o comando:


```
go mod tidy
```

Esse comando remove dependências que não são mais utilizadas, adiciona aquelas que estão faltando e mantém os arquivos `go.mod` e `go.sum` consistentes com o código-fonte do projeto.

Na prática, muitos desenvolvedores executam `go mod tidy` sempre que adicionam ou removem dependências, especialmente antes de realizar um commit.

Enquanto o `go get` gerencia as dependências utilizadas por um projeto, o `go install` é destinado à instalação de programas executáveis, normalmente ferramentas de desenvolvimento. Esse comando será apresentado na próxima seção.

13.5 go install

O comando `go install` é utilizado para compilar e instalar programas executáveis. Diferentemente do `go get`, ele não adiciona dependências ao projeto atual nem modifica os arquivos `go.mod` ou `go.sum`.

Esse comando é amplamente utilizado para instalar ferramentas de desenvolvimento, como analisadores estáticos, geradores de código, servidores de linguagem e outros utilitários utilizados durante a programação.

Por exemplo, para instalar a versão mais recente do servidor de linguagem Go (`gopls`), basta executar:

```
go install golang.org/x/tools/gopls@latest
```

Da mesma forma, é possível instalar uma versão específica:

```
go install golang.org/x/tools/gopls@v0.20.0
```

Após a instalação, o executável é armazenado, por padrão, no diretório:

```
~/go/bin
```

Para utilizar a ferramenta a partir de qualquer diretório do sistema, esse caminho deve estar presente na variável de ambiente `PATH`.

Diferentemente do `go get`, que gerencia as dependências de um projeto, o `go install` instala programas destinados ao ambiente de desenvolvimento. Assim, ferramentas como `gopls`, `golangci-lint`, `stringer` e `air` podem ser instaladas uma única vez e utilizadas em qualquer projeto Go.

Embora ambos os comandos possam realizar o download de módulos, seus objetivos são distintos: `go get` gerencia as dependências de uma aplicação, enquanto `go install` instala programas executáveis para uso no sistema.

Ao longo deste capítulo vimos como os módulos permitem organizar projetos, controlar versões e gerenciar dependências de forma simples e reproduzível. Compreender o funcionamento do `go.mod`, do `go.sum` e dos comandos `go get` e `go install` é fundamental para o desenvolvimento de aplicações Go modernas.

13.6 Desafio

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo. Procure resolver cada problema utilizando apenas os recursos estudados até aqui. Priorize escrever programas corretos, claros e fáceis de compreender.

1. Crie um módulo Go contendo uma aplicação de linha de comando chamada `quadratic`.

Essa aplicação deve resolver equações do segundo grau no formato:

$$ax^2 + bx + c = 0$$

O programa deve receber os coeficientes a , b e c como argumentos de linha de comando.

Por exemplo:

```
quadratic 1 -3 2
```

Para essa entrada, o programa deverá resolver:

$$x^2 - 3x + 2 = 0$$

e exibir as raízes da equação.

A aplicação deve tratar os seguintes casos:

- se a for igual a zero, informe que a equação não é do segundo grau;
- se o discriminante for negativo, informe que a equação não possui raízes reais;
- se o discriminante for igual a zero, exiba a única raiz real;
- se o discriminante for positivo, exiba as duas raízes reais.

Organize o projeto em pelo menos dois pacotes:

```
quadratic/  
├── go.mod  
├── main.go  
└── solver/  
    └── solver.go
```

O pacote `solver` deve conter a lógica de resolução da equação. O pacote `main` deve apenas ler os argumentos, converter os valores, chamar o pacote `solver` e exibir o resultado.

Para criar o módulo, utilize:

```
mkdir quadratic  
cd quadratic  
go mod init quadratic
```

Durante o desenvolvimento, execute o programa com:

```
go run . 1 -3 2
```

Ao final, instale a aplicação com:

```
go install
```

Depois de instalada, execute o programa diretamente pelo terminal:

```
quadratic 1 -3 2
```


Capítulo 14

Biblioteca Padrão

Até este ponto, estudamos os principais fundamentos da linguagem Go: tipos, controle de fluxo, coleções, funções, ponteiros, structs, métodos, interfaces, erros, pacotes e módulos. Esses recursos formam a base da linguagem. Entretanto, para construir programas reais, precisamos também interagir com textos, números, arquivos, datas, entrada e saída, formatos de dados e execução concorrente.

É nesse contexto que a biblioteca padrão assume um papel central.

A biblioteca padrão de Go é um dos pontos mais fortes da linguagem. Ela fornece um conjunto amplo, consistente e bem projetado de pacotes para resolver problemas comuns sem depender imediatamente de bibliotecas externas. Essa característica contribui para uma das ideias centrais de Go: permitir que programas úteis sejam escritos com poucas dependências e com comportamento previsível.

Diferentemente de ecossistemas nos quais tarefas básicas exigem a instalação de vários pacotes de terceiros, Go já oferece, em sua distribuição oficial, recursos para formatação de texto, manipulação de strings, conversão de tipos, tratamento de datas, operações matemáticas, acesso ao sistema operacional, leitura e escrita de dados, buffers, codificação JSON e sincronização entre goroutines.

Neste capítulo, estudaremos alguns dos pacotes mais utilizados da biblioteca padrão:

```
fmt
strings
strconv
time
math
os
io
bufio
encoding/json
```

O objetivo não é esgotar todos os detalhes desses pacotes, mas apresentar os recursos mais importantes para o desenvolvimento cotidiano. Muitos deles já apareceram em exemplos anteriores, como `fmt.Println`, `strconv.Atoi` e `json.Marshal`. Agora, porém, vamos estudá-los de forma mais organizada.

A biblioteca padrão também ajuda a consolidar a escrita idiomática em Go. Ao compreender como seus pacotes são organizados, como suas funções recebem e retornam valores, como erros são tratados e como interfaces como `io.Reader` e `io.Writer` são utilizadas, o programador passa a entender melhor o estilo da própria linguagem.

Ao final deste capítulo, seremos capazes de utilizar a biblioteca padrão para resolver tarefas frequentes de programação, escrevendo programas mais completos, práticos e alinhados com as convenções de Go.

14.1 fmt

O pacote `fmt` é um dos mais utilizados da biblioteca padrão de Go. Seu principal objetivo é fornecer funções para formatação de texto, entrada e saída de dados. Praticamente todo programa em Go utiliza esse pacote, seja para exibir informações no terminal, construir mensagens formatadas ou ler dados informados pelo usuário.

O nome `fmt` é uma abreviação de *format*. Grande parte de suas funções segue uma sintaxe inspirada na linguagem C, utilizando verbos de formatação para controlar como cada valor será representado.

Para utilizar o pacote, basta importá-lo:

```
import "fmt"
```

14.1.1 Exibindo valores

As funções mais comuns do pacote são `Print`, `Println` e `Printf`.

A função `Print` escreve os valores exatamente como são fornecidos, sem adicionar espaços ou uma quebra de linha ao final.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Print("Olá")
7     fmt.Print(" Mundo")
8 }
```

Saída:

```
Olá Mundo
```

A função `Println` adiciona automaticamente um espaço entre os argumentos e uma quebra de linha ao final da impressão.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Nome:", "Maria")
7     fmt.Println("Idade:", 28)
8 }
```

Saída:

```
Nome: Maria
Idade: 28
```

Na maioria dos exemplos deste livro, `Println` será a função utilizada por produzir uma saída mais legível.

14.1.2 Formatação com `Printf`

Quando é necessário controlar exatamente como um valor será exibido, utiliza-se `Printf`.

Nessa função, o primeiro argumento é uma string de formato, contendo verbos que indicam como os valores seguintes devem ser representados.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     name := "Carlos"
7     age := 35
8
9     fmt.Printf("Nome: %s\n", name)
10    fmt.Printf("Idade: %d anos\n", age)
11 }
```

Saída:

```
Nome: Carlos
Idade: 35 anos
```

Observe que `Printf` não adiciona uma quebra de linha automaticamente. Por isso, utilizamos `\n` quando desejamos iniciar uma nova linha.

14.1.3 Verbos de formatação mais comuns

O pacote `fmt` possui dezenas de verbos de formatação. Os mais utilizados são:

Verbo	Descrição	Exemplo
%v	Valor no formato padrão	10, true, [1 2 3]
%+v	Estruturas com nomes dos campos	{Name:Ana Age:20}
%#v	Representação em sintaxe Go	main.Person{Name:"Ana", Age:20}
%T	Tipo do valor	int, string
%d	Inteiro decimal	42
%f	Número de ponto flutuante	3.141593
%.2f	Float com duas casas decimais	3.14
%s	String	Go
%q	String entre aspas	"Go"
%t	Booleano	true
%p	Endereço de memória de um ponteiro	0xc0000120a0
%c	imprimir o caractere Unicode	("%c", 65) imprime o caractere de código ASCII 65, "A"

Exemplo:

```

1 package main
2
3 import "fmt"
4
5 func main() {
6     value := 3.14159265
7
8     fmt.Printf("Valor: %.2f\n", value)
9     fmt.Printf("Tipo: %T\n", value)
10 }
```

Saída:

```

Valor: 3.14
Tipo: float64
```

14.1.4 Criando strings formatadas

Nem sempre desejamos imprimir o resultado imediatamente. Muitas vezes é necessário construir uma string formatada para utilizá-la posteriormente.

Nesses casos, utiliza-se `Sprintf`.


```
1 package main
2
3 import "fmt"
4
5 func main() {
6     name := "João"
7     score := 98.5
8
9     message := fmt.Sprintf("%s obteve %.1f pontos.", name, score)
10
11    fmt.Println(message)
12 }
```

Saída:

João obteve 98.5 pontos.

14.1.5 Lendo dados do usuário

O pacote `fmt` também oferece funções para leitura de dados a partir da entrada padrão.

A função `Scan` lê valores separados por espaços.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var name string
7     var age int
8
9     fmt.Print("Nome: ")
10    fmt.Scan(&name)
11
12    fmt.Print("Idade: ")
13    fmt.Scan(&age)
14
15    fmt.Println(name, "possui", age, "anos.")
16 }
```

Observe que as variáveis são passadas utilizando o operador `&`, pois a função precisa modificar seus valores.

Embora `Scan` seja útil para exemplos simples, aplicações maiores normalmente utilizam o pacote `bufio`, que estudaremos mais adiante neste capítulo, por oferecer uma leitura mais flexível.

14.1.6 Resumo

O pacote `fmt` concentra as principais operações de entrada e saída de dados em Go. Entre suas funções mais utilizadas estão:

- `Print()` — escreve valores sem quebra de linha.
- `Println()` — escreve valores separados por espaços e adiciona uma quebra de linha.
- `Printf()` — escreve utilizando formatação personalizada.
- `Sprintf()` — retorna uma string formatada.
- `Scan()` — lê valores da entrada padrão.

Dominar essas funções é essencial, pois elas aparecem em praticamente todos os programas escritos em Go, desde pequenos exemplos até aplicações profissionais.

14.2 strings

O pacote `strings` reúne funções para manipulação de textos. Como as strings em Go são imutáveis, nenhuma dessas funções altera a string original. Em vez disso, cada operação retorna uma nova string contendo o resultado da transformação.

Esse pacote oferece recursos para pesquisas, substituições, divisão, junção, conversão de letras e diversas outras operações comuns no processamento de texto.

Para utilizá-lo, basta importá-lo:

```
import "strings"
```

14.2.1 Verificando a presença de um texto

A função `Contains` informa se uma string contém outra.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     text := "Aprendendo Go"
10
11     fmt.Println(strings.Contains(text, "Go"))
12     fmt.Println(strings.Contains(text, "Java"))
13 }
```

Saída:

```
true
false
```

14.2.2 Verificando prefixos e sufixos

As funções `HasPrefix` e `HasSuffix` verificam se uma string começa ou termina com determinado texto.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     file := "relatorio.pdf"
10
11     fmt.Println(strings.HasPrefix(file, "rela"))
12     fmt.Println(strings.HasSuffix(file, ".pdf"))
13 }
```

Saída:

```
true
true
```

14.2.3 Convertendo para maiúsculas e minúsculas

As funções `ToUpper` e `ToLower` convertem todas as letras de uma string.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     text := "GoLang"
10
11     fmt.Println(strings.ToUpper(text))
12     fmt.Println(strings.ToLower(text))
13 }
```

Saída:

```
GOLANG
golang
```

14.2.4 Removendo espaços

A função `TrimSpace` remove espaços em branco no início e no final da string.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     text := "  Olá, Go!  "
10
11     fmt.Println(strings.TrimSpace(text))
12 }
```

Saída:

Olá, Go!

14.2.5 Substituindo texto

A função `ReplaceAll` substitui todas as ocorrências de um trecho por outro.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     text := "Go é rápido. Go é simples."
10
11     result := strings.ReplaceAll(text, "Go", "Golang")
12
13     fmt.Println(result)
14 }
```

Saída:

Golang é rápido. Golang é simples.

14.2.6 Dividindo uma string

A função `Split` divide uma string em partes, retornando uma slice de strings.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     csv := "Ana,Carlos,João"
10
11     names := strings.Split(csv, ",")
12
13     fmt.Println(names)
14 }
```

Saída:

```
[Ana Carlos João]
```

14.2.7 Unindo strings

A função `Join` realiza a operação inversa de `Split`, unindo os elementos de uma slice.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     names := []string{"Ana", "Carlos", "João"}
10
11     result := strings.Join(names, " - ")
12
13     fmt.Println(result)
14 }
```

Saída:

```
Ana - Carlos - João
```

14.2.8 Repetindo texto

A função `Repeat` cria uma nova string repetindo o texto informado um determinado número de vezes.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     fmt.Println(strings.Repeat("-", 20))
10 }
```

Saída:

14.2.9 Contando ocorrências

A função `Count` informa quantas vezes um trecho aparece em uma string.

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 func main() {
9     text := "banana"
10
11     fmt.Println(strings.Count(text, "a"))
12 }
```

Saída:

3

14.2.10 Resumo

O pacote `strings` fornece diversas funções para manipulação de textos sem modificar a string original. Entre as mais utilizadas estão:

Função	Descrição
<code>Contains()</code>	Verifica se um texto está presente na string.
<code>HasPrefix()</code>	Verifica o início da string.
<code>HasSuffix()</code>	Verifica o final da string.
<code>ToUpper()</code>	Converte para letras maiúsculas.
<code>ToLower()</code>	Converte para letras minúsculas.

Função	Descrição
<code>TrimSpace()</code>	Remove espaços nas extremidades.
<code>ReplaceAll()</code>	Substitui todas as ocorrências de um texto.
<code>Split()</code>	Divide uma string em uma slice.
<code>Join()</code>	Une uma slice de strings.
<code>Repeat()</code>	Repete uma string várias vezes.
<code>Count()</code>	Conta ocorrências de um trecho.

O pacote `strings` está entre os mais utilizados da biblioteca padrão e aparece frequentemente em aplicações que processam arquivos, interpretam comandos, manipulam URLs, tratam dados recebidos pela rede ou realizam validações de entrada.

14.3 `strconv`

O pacote `strconv` fornece funções para conversão entre strings e tipos básicos da linguagem. Ele é amplamente utilizado quando programas precisam interpretar dados recebidos do usuário, ler arquivos de texto, processar parâmetros da linha de comando ou trabalhar com formatos como CSV e JSON.

Enquanto uma conversão de tipos em Go é feita por meio da sintaxe `T(valor)`, o pacote `strconv` é utilizado quando a conversão envolve uma representação textual.

Para utilizá-lo, basta importá-lo:

```
import "strconv"
```

14.3.1 Convertendo string para inteiro

A função `Atoi` (*ASCII to Integer*) converte uma string para um valor do tipo `int`.

```
1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     value, err := strconv.Atoi("42")
10    if err != nil {
11        fmt.Println(err)
12        return
13    }
14
15    fmt.Println(value)
16 }
```

Saída:

42

Caso a string não represente um número inteiro válido, a função retorna um erro.

```
value, err := strconv.Atoi("abc")
```

Nesse caso, err será diferente de nil.

14.3.2 Convertendo inteiro para string

A função `Itoa` (*Integer to ASCII*) realiza a operação inversa.

```
1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     text := strconv.Itoa(2026)
10
11    fmt.Println(text)
12 }
```

Saída:

2026

14.3.3 Convertendo números de ponto flutuante

Para converter uma string em um float64, utiliza-se ParseFloat.

```
1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     value, err := strconv.ParseFloat("3.14159", 64)
10    if err != nil {
11        fmt.Println(err)
12        return
13    }
14
15    fmt.Println(value)
16 }
```

Saída:

```
3.14159
```

O segundo argumento indica a precisão desejada (32 ou 64).

14.3.4 Convertendo booleanos

A função ParseBool interpreta representações textuais de valores booleanos.

```
1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     value, _ := strconv.ParseBool("true")
10
11    fmt.Println(value)
12 }
```

Saída:

```
true
```

Também são aceitos valores como "false", "1", "0", "t" e "f".

14.3.5 Formatando valores

Além de interpretar strings, o pacote também possui funções para produzir representações textuais.

```
1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     fmt.Println(strconv.FormatBool(true))
10    fmt.Println(strconv.FormatInt(255, 10))
11    fmt.Println(strconv.FormatFloat(3.14, 'f', 2, 64))
12 }
```

Saída:

```
true
255
3.14
```

Observe que `FormatInt` recebe a base numérica desejada. O valor 10 representa a base decimal, mas também podem ser utilizadas outras bases, como 2 (binário), 8 (octal) e 16 (hexadecimal).

14.3.6 Interpretando números em diferentes bases

Quando é necessário interpretar números escritos em outras bases, utiliza-se `ParseInt`.

```
1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     value, _ := strconv.ParseInt("1010", 2, 64)
10
11    fmt.Println(value)
12 }
```

Saída:

```
10
```

Nesse exemplo, "1010" é interpretado como um número binário.

14.3.7 Resumo

O pacote `strconv` concentra as funções de conversão entre strings e tipos básicos da linguagem.

Função	Descrição
<code>Atoi()</code>	Converte uma string para <code>int</code> .
<code>Itoa()</code>	Converte um <code>int</code> para string.
<code>ParseBool()</code>	Converte uma string para <code>bool</code> .
<code>ParseFloat()</code>	Converte uma string para <code>float32</code> ou <code>float64</code> .
<code>ParseInt()</code>	Converte uma string para um inteiro em uma base numérica específica.
<code>FormatBool()</code>	Converte um <code>bool</code> para string.
<code>FormatInt()</code>	Converte um inteiro para string em uma base escolhida.
<code>FormatFloat()</code>	Converte um número de ponto flutuante para string.

O pacote `strconv` é utilizado sempre que valores textuais precisam ser interpretados como dados numéricos ou booleanos, ou quando esses tipos precisam ser convertidos para sua representação textual. Ele aparece frequentemente em aplicações que processam entrada do usuário, arquivos, APIs, variáveis de ambiente e parâmetros da linha de comando.

14.4 time

O pacote `time` fornece recursos para trabalhar com datas, horários, durações e intervalos de tempo. Ele é amplamente utilizado em aplicações que registram eventos, medem desempenho, implementam temporizadores, agendam tarefas ou manipulam datas e horas.

Em Go, o tipo central desse pacote é `time.Time`, que representa um instante específico no tempo.

Para utilizar o pacote, basta importá-lo:

```
import "time"
```

14.4.1 Obtendo a data e hora atuais

A função `Now` retorna o instante atual.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     now := time.Now()
10
11     fmt.Println(now)
12 }
```

Saída (exemplo):

```
2026-06-26 14:30:15.123456 -0300 -03
```

Cada execução produzirá um horário diferente.

14.4.2 Acessando componentes da data

O tipo `time.Time` possui diversos métodos para acessar partes da data e da hora.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     now := time.Now()
10
11     fmt.Println("Ano:", now.Year())
12     fmt.Println("Mês:", now.Month())
13     fmt.Println("Dia:", now.Day())
14     fmt.Println("Hora:", now.Hour())
15     fmt.Println("Minuto:", now.Minute())
16     fmt.Println("Segundo:", now.Second())
17 }
```

Saída (exemplo):

```
Ano: 2026
Mês: June
Dia: 26
Hora: 14
Minuto: 30
```

Segundo: 15

Observe que o mês é representado pelo tipo `time.Month`, cuja impressão padrão utiliza o nome em inglês.

14.4.3 Criando datas específicas

A função `Date` cria um valor do tipo `time.Time` a partir de seus componentes.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     date := time.Date(
10        2026,
11        time.December,
12        25,
13        8,
14        30,
15        0,
16        0,
17        time.Local,
18    )
19
20    fmt.Println(date)
21 }
```

Saída:

2026-12-25 08:30:00 -0300 -03

14.4.4 Formatando datas

Em Go, a formatação de datas utiliza uma referência fixa:

02/01/2006 15:04:05

Cada número dessa data de referência representa uma parte da data e da hora.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     now := time.Now()
10
11     fmt.Println(now.Format("02/01/2006"))
12     fmt.Println(now.Format("02/01/2006 15:04"))
13 }
```

Saída (exemplo):

```
26/06/2026
26/06/2026 14:30
```

Esse modelo pode parecer incomum à primeira vista, mas torna a definição dos formatos bastante simples e flexível.

14.4.5 Convertendo texto em datas

A função `Parse` realiza a operação inversa, convertendo uma string para `time.Time`.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     date, err := time.Parse(
10         "02/01/2006",
11         "15/08/2026",
12     )
13
14     if err != nil {
15         fmt.Println(err)
16         return
17     }
18
19     fmt.Println(date)
20 }
```

14.4.6 Somando e subtraindo tempo

O método `Add` permite adicionar ou remover intervalos de tempo.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     now := time.Now()
10
11     fmt.Println(now)
12     fmt.Println(now.Add(48 * time.Hour))
13 }
```

Saída (exemplo):

```
2026-06-26 14:30:15 -0300 -03
2026-06-28 14:30:15 -0300 -03
```

Para subtrair tempo, basta utilizar uma duração negativa.

```
yesterday := time.Now().Add(-24 * time.Hour)
```

14.4.7 Medindo tempo de execução

Uma aplicação comum do pacote é medir o tempo gasto por uma operação.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     start := time.Now()
10
11     time.Sleep(2 * time.Second)
12
13     elapsed := time.Since(start)
14
15     fmt.Println(elapsed)
16 }
```

Saída (aproximada):

```
2.000341215s
```

Nesse exemplo, `Sleep` apenas pausa a execução por dois segundos.

14.4.8 Pausando a execução

A função `Sleep` interrompe a execução da goroutine atual durante o intervalo informado.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     fmt.Println("Início")
10
11     time.Sleep(3 * time.Second)
12
13     fmt.Println("Fim")
14 }
```

Entre as duas mensagens haverá uma pausa de aproximadamente três segundos.

14.4.9 Trabalhando com durações

O tipo `time.Duration` representa intervalos de tempo.

As constantes mais utilizadas são:

```
time.Nanosecond
time.Microsecond
time.Millisecond
time.Second
time.Minute
time.Hour
```

Essas constantes podem ser combinadas em operações aritméticas.

```
timeout := 5*time.Second + 500*time.Millisecond
```

14.4.10 Resumo

O pacote `time` fornece recursos completos para manipulação de datas e horários.

Função ou método	Descrição
<code>time.Now()</code>	Retorna a data e hora atuais.
<code>time.Date()</code>	Cria uma data específica.
<code>time.Parse()</code>	Converte uma string em <code>time.Time</code> .
<code>Time.Format()</code>	Formata uma data como string.
<code>Time.Add()</code>	Soma ou subtrai um intervalo de tempo.
<code>time.Since()</code>	Calcula o tempo decorrido desde um instante.
<code>time.Sleep()</code>	Pausa a execução da goroutine atual.

O pacote `time` está presente em praticamente todas as aplicações profissionais escritas em Go, sendo utilizado para registrar logs, controlar prazos, implementar expiração de sessões, medir desempenho, criar temporizadores e lidar com datas e horários de forma segura e eficiente.

14.5 math

O pacote `math` fornece funções e constantes matemáticas para operações com números de ponto flutuante. Ele inclui recursos para cálculos trigonométricos, exponenciais, logaritmos, arredondamentos, raízes, valores absolutos e diversas outras operações comuns em aplicações científicas, financeiras e de engenharia.

Todas as funções do pacote trabalham com valores do tipo `float64`. Caso seja necessário utilizar outros tipos numéricos, é preciso realizar a conversão antes da chamada.

Para utilizar o pacote, basta importá-lo:

```
import "math"
```

14.5.1 Constantes matemáticas

O pacote disponibiliza diversas constantes úteis, como π e a base dos logaritmos naturais.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println(math.Pi)
10    fmt.Println(math.E)
11 }
```

Saída:

```
3.141592653589793
2.718281828459045
```

14.5.2 Valor absoluto

A função Abs retorna o valor absoluto de um número.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println(math.Abs(-12.5))
10 }
```

Saída:

```
12.5
```

14.5.3 Potência e raiz quadrada

As funções Pow e Sqrt calculam potências e raízes quadradas.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println(math.Pow(2, 10))
10    fmt.Println(math.Sqrt(144))
11 }
```

Saída:

```
1024
12
```

14.5.4 Arredondamentos

O pacote oferece diferentes formas de arredondar valores.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     value := 3.7
10
11     fmt.Println(math.Floor(value))
12     fmt.Println(math.Ceil(value))
13     fmt.Println(math.Round(value))
14 }
```

Saída:

```
3
4
4
```

- Floor arredonda para baixo.
- Ceil arredonda para cima.
- Round arredonda para o inteiro mais próximo.

14.5.5 Máximo e mínimo

As funções Max e Min retornam, respectivamente, o maior e o menor entre dois valores.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println(math.Max(15, 20))
10    fmt.Println(math.Min(15, 20))
11 }
```

Saída:

```
20
15
```

14.5.6 Funções trigonométricas

O pacote também implementa as funções trigonométricas mais conhecidas.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     angle := math.Pi / 2
10
11     fmt.Println(math.Sin(angle))
12     fmt.Println(math.Cos(angle))
13 }
```

Saída (aproximada):

```
1
6.123233995736766e-17
```

Os ângulos devem ser informados em radianos.

14.5.7 Logaritmos

As funções Log e Log10 calculam logaritmos naturais e decimais.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println(math.Log(math.E))
10    fmt.Println(math.Log10(1000))
11 }
```

Saída:

```
1
3
```

14.5.8 Valores especiais

O pacote também define representações para infinito e valores indefinidos (*Not a Number*).

```

1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println(math.Inf(1))
10    fmt.Println(math.NaN())
11 }

```

Esses valores aparecem principalmente em cálculos científicos e numéricos.

14.5.9 Resumo

O pacote `math` reúne funções matemáticas de uso geral para valores do tipo `float64`.

Função ou constante	Descrição
<code>math.Pi</code>	Valor de π .
<code>math.E</code>	Base dos logaritmos naturais.
<code>math.Abs()</code>	Valor absoluto.
<code>math.Pow()</code>	Potência.
<code>math.Sqrt()</code>	Raiz quadrada.
<code>math.Floor()</code>	Arredonda para baixo.
<code>math.Ceil()</code>	Arredonda para cima.
<code>math.Round()</code>	Arredonda para o inteiro mais próximo.
<code>math.Max()</code>	Maior entre dois valores.
<code>math.Min()</code>	Menor entre dois valores.
<code>math.Sin()</code>	Seno de um ângulo em radianos.
<code>math.Cos()</code>	Cosseno de um ângulo em radianos.
<code>math.Log()</code>	Logaritmo natural.
<code>math.Log10()</code>	Logaritmo na base 10.

Embora muitos programas utilizem apenas algumas dessas funções, o pacote `math` é essencial para aplicações que envolvem cálculos numéricos, processamento gráfico, estatística, engenharia, física, simulações e algoritmos científicos.

14.6 os

O pacote `os` fornece funções para interação com o sistema operacional. Por meio dele, é possível acessar argumentos da linha de comando, variáveis de ambiente, criar e remover arquivos e diretórios, consultar informações do sistema de arquivos e controlar a execução do programa.

Grande parte das operações envolvendo arquivos e diretórios começa com recursos disponibilizados por esse pacote.

Para utilizá-lo, basta importá-lo:

```
import "os"
```

14.6.1 Argumentos da linha de comando

Os argumentos fornecidos ao executar um programa ficam disponíveis na variável `os.Args`.

Ela é uma slice de strings, na qual o primeiro elemento corresponde ao nome do programa.

```
1 package main
2
3 import (
4     "fmt"
5     "os"
6 )
7
8 func main() {
9     fmt.Println(os.Args)
10 }
```

Executando:

```
go run main.go João 25
```

Saída:

```
[/tmp/go-build.../main João 25]
```

Para acessar apenas os argumentos informados pelo usuário, normalmente utiliza-se `os.Args[1:]`.

```
1 package main
2
3 import (
4     "fmt"
5     "os"
6 )
7
8 func main() {
9     for _, arg := range os.Args[1:] {
10         fmt.Println(arg)
11     }
12 }
```

Saída:

```
João
25
```

14.6.2 Variáveis de ambiente

Variáveis de ambiente armazenam configurações do sistema ou da aplicação.

A função `Getenv` recupera seu valor.

```
1 package main
2
3 import (
4     "fmt"
5     "os"
6 )
7
8 func main() {
9     home := os.Getenv("HOME")
10
11     fmt.Println(home)
12 }
```

Caso a variável não exista, a função retorna uma string vazia.

Também é possível definir variáveis de ambiente durante a execução.

```
os.Setenv("APP_ENV", "development")
```

E removê-las:

```
os.Unsetenv("APP_ENV")
```

14.6.3 Obtendo o diretório atual

A função `Getwd` retorna o diretório de trabalho atual.

```
1 package main
2
3 import (
4     "fmt"
5     "os"
6 )
7
8 func main() {
9     dir, err := os.Getwd()
10    if err != nil {
11        fmt.Println(err)
12        return
13    }
14
15    fmt.Println(dir)
16 }
```

14.6.4 Criando diretórios

A função `Mkdir` cria um diretório.

```
err := os.Mkdir("dados", 0755)
```

O segundo argumento representa as permissões do diretório em sistemas compatíveis com Unix. Quando é necessário criar todos os diretórios inexistentes de um caminho, utiliza-se `MkdirAll`.

```
os.MkdirAll("logs/2026/junho", 0755)
```

14.6.5 Removendo arquivos e diretórios

A função `Remove` remove um arquivo ou um diretório vazio.

```
os.Remove("arquivo.txt")
```

Para remover um diretório juntamente com todo o seu conteúdo, utiliza-se `RemoveAll`.

```
os.RemoveAll("temp")
```

Essa função deve ser utilizada com cuidado, pois a remoção é permanente.

14.6.6 Criando arquivos

A função `Create` cria um novo arquivo ou substitui um arquivo existente.


```
1 package main
2
3 import (
4     "fmt"
5     "os"
6 )
7
8 func main() {
9     file, err := os.Create("dados.txt")
10    if err != nil {
11        fmt.Println(err)
12        return
13    }
14
15    defer file.Close()
16
17    file.WriteString("Olá, Go!\n")
18 }
```

O método `Close` deve ser chamado ao final do uso do arquivo. O uso de `defer` garante que isso ocorrerá mesmo que a função seja encerrada antecipadamente.

14.6.7 Lendo arquivos

Uma forma simples de ler todo o conteúdo de um arquivo é utilizando `ReadFile`.

```
1 package main
2
3 import (
4     "fmt"
5     "os"
6 )
7
8 func main() {
9     data, err := os.ReadFile("dados.txt")
10    if err != nil {
11        fmt.Println(err)
12        return
13    }
14
15    fmt.Println(string(data))
16 }
```

14.6.8 Gravando arquivos

A função `WriteFile` grava todo o conteúdo de um arquivo em uma única operação.

```
1 package main
2
3 import (
4     "os"
5 )
6
7 func main() {
8     text := []byte("Primeira linha\nSegunda linha\n")
9
10    os.WriteFile("dados.txt", text, 0644)
11 }
```

Caso o arquivo já exista, seu conteúdo será substituído.

14.6.9 Encerrando o programa

A função `Exit` encerra imediatamente a execução do programa.

```
1 package main
2
3 import "os"
4
5 func main() {
6     os.Exit(1)
7 }
```

Por convenção:

- 0 indica execução bem-sucedida.
- Valores diferentes de 0 indicam algum tipo de erro.

É importante observar que funções adiadas com `defer` **não** são executadas após uma chamada a `os.Exit`.

14.6.10 Resumo

O pacote `os` fornece recursos para interação com o sistema operacional.

Função ou variável	Descrição
<code>os.Args</code>	Argumentos da linha de comando.
<code>os.Getenv()</code>	Obtém o valor de uma variável de ambiente.
<code>os.Setenv()</code>	Define uma variável de ambiente.
<code>os.Unsetenv()</code>	Remove uma variável de ambiente.
<code>os.Getwd()</code>	Obtém o diretório de trabalho atual.
<code>os.Mkdir()</code>	Cria um diretório.
<code>os.MkdirAll()</code>	Cria todos os diretórios de um caminho.
<code>os.Remove()</code>	Remove um arquivo ou diretório vazio.

Função ou variável	Descrição
<code>os.RemoveAll()</code>	Remove um diretório e todo o seu conteúdo.
<code>os.Create()</code>	Cria um arquivo.
<code>os.ReadFile()</code>	Lê todo o conteúdo de um arquivo.
<code>os.WriteFile()</code>	Grava todo o conteúdo de um arquivo.
<code>os.Exit()</code>	Encerra imediatamente o programa.
<code>os.Open()</code>	Abre um arquivo para leitura/escrita

O pacote `os` é a principal porta de entrada para a interação entre um programa Go e o sistema operacional. Ele é amplamente utilizado em aplicações de linha de comando, servidores, ferramentas de automação e utilitários que manipulam arquivos, diretórios e configurações do ambiente.

14.7 io

O pacote `io` define as interfaces fundamentais para operações de entrada e saída de dados em Go. Em vez de fornecer implementações específicas para arquivos, conexões de rede ou memória, ele estabelece um conjunto de contratos que permitem que diferentes tipos de fontes e destinos de dados sejam utilizados de maneira uniforme.

Essa é uma das bases do projeto da biblioteca padrão. Diversos pacotes, como `os`, `bufio`, `net/http`, `encoding/json` e `compress/gzip`, utilizam as interfaces definidas em `io`, tornando seus componentes facilmente combináveis.

Para utilizar o pacote, basta importá-lo:

```
import "io"
```

14.7.1 A interface Reader

A interface `Reader` representa qualquer tipo capaz de fornecer dados para leitura.

Sua definição é bastante simples:

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```

O método `Read` copia dados para o buffer informado e retorna:

- a quantidade de bytes lidos;
- um erro, quando ocorrer.

Arquivos, conexões TCP, respostas HTTP e diversos outros tipos implementam essa interface.

14.7.2 A interface Writer

De forma semelhante, a interface `Writer` representa qualquer tipo capaz de receber dados.

```
type Writer interface {  
    Write(p []byte) (n int, err error)  
}
```

Arquivos, conexões de rede e até mesmo a saída padrão (`os.Stdout`) implementam essa interface.

14.7.3 Copiando dados

Uma das funções mais úteis do pacote é `io.Copy`, que copia todos os dados de um `Reader` para um `Writer`.

```
1 package main  
2  
3 import (  
4     "io"  
5     "os"  
6 )  
7  
8 func main() {  
9     src, _ := os.Open("origem.txt")  
10    defer src.Close()  
11  
12    dst, _ := os.Create("destino.txt")  
13    defer dst.Close()  
14  
15    io.Copy(dst, src)  
16 }
```

Nesse exemplo, todo o conteúdo de `origem.txt` é copiado para `destino.txt`.

Observe que `io.Copy` não precisa saber que está trabalhando com arquivos. Basta que um objeto implemente `io.Reader` e o outro implemente `io.Writer`.

14.7.4 Lendo todo o conteúdo

A função `ReadAll` lê todos os dados fornecidos por um `Reader`.

```
1 package main
2
3 import (
4     "fmt"
5     "io"
6     "strings"
7 )
8
9 func main() {
10     reader := strings.NewReader("Olá, Go!")
11
12     data, _ := io.ReadAll(reader)
13
14     fmt.Println(string(data))
15 }
```

Saída:

Olá, Go!

Essa função é conveniente para pequenas quantidades de dados. Em arquivos muito grandes, o ideal é processar o conteúdo em partes.

14.7.5 Limitando a leitura

A função `LimitReader` cria um novo `Reader` que permite ler apenas uma quantidade específica de bytes.

```
1 package main
2
3 import (
4     "fmt"
5     "io"
6     "strings"
7 )
8
9 func main() {
10     reader := strings.NewReader("Biblioteca padrão")
11
12     limited := io.LimitReader(reader, 10)
13
14     data, _ := io.ReadAll(limited)
15
16     fmt.Println(string(data))
17 }
```

Saída:

Biblioteca

Essa função é útil para evitar leituras excessivas de dados.

14.7.6 Descartando dados

Em algumas situações, é necessário consumir dados sem armazená-los.

O valor `io.Discard` implementa a interface `Writer`, mas simplesmente ignora todos os bytes recebidos.

```
io.Copy(io.Discard, reader)
```

Esse recurso é bastante utilizado em testes e em situações nas quais apenas interessa consumir o fluxo de dados.

14.7.7 Detectando o fim da leitura

Quando um `Reader` não possui mais dados disponíveis, ele retorna o erro especial `io.EOF`.

```
if err == io.EOF {  
    fmt.Println("Fim da leitura.")  
}
```

Esse valor não representa uma falha, mas apenas informa que todos os dados já foram lidos.

14.7.8 Resumo

O pacote `io` define as principais interfaces utilizadas para entrada e saída de dados em Go.

Interface ou função	Descrição
<code>io.Reader</code>	Interface para leitura de dados.
<code>io.Writer</code>	Interface para escrita de dados.
<code>io.Copy()</code>	Copia dados entre um <code>Reader</code> e um <code>Writer</code> .
<code>io.ReadAll()</code>	Lê todo o conteúdo de um <code>Reader</code> .
<code>io.LimitReader()</code>	Limita a quantidade de dados lidos.
<code>io.Discard</code>	Descarta todos os dados escritos.
<code>io.EOF</code>	Indica o fim de uma leitura.

O pacote `io` é um dos pilares da biblioteca padrão. Ao utilizar interfaces em vez de tipos concretos, ele permite que diferentes componentes trabalhem juntos de forma simples e flexível, seguindo um dos princípios fundamentais do projeto da linguagem Go: a composição por meio de interfaces.

14.8 bufio

O pacote `bufio` fornece operações de entrada e saída com *buffer*. Em vez de acessar diretamente arquivos, conexões ou a entrada padrão a cada leitura ou escrita, ele mantém uma área de memória intermediária, reduzindo a quantidade de acessos ao dispositivo e melhorando o desempenho.

Além dos ganhos de eficiência, `bufio` oferece recursos convenientes para leitura de linhas, palavras e outros padrões de texto.

Para utilizá-lo, basta importá-lo:

```
import "bufio"
```

14.8.1 Lendo uma linha do teclado

Uma das aplicações mais comuns de `bufio` é a leitura de texto digitado pelo usuário.

```
1 package main
2
3 import (
4     "bufio"
5     "fmt"
6     "os"
7 )
8
9 func main() {
10     reader := bufio.NewReader(os.Stdin)
11
12     fmt.Print("Nome: ")
13
14     name, err := reader.ReadString('\n')
15     if err != nil {
16         fmt.Println(err)
17         return
18     }
19
20     fmt.Println("Olá,", name)
21 }
```

O método `ReadString` lê caracteres até encontrar o delimitador informado. Nesse exemplo, a leitura termina quando o usuário pressiona **Enter** (`'\n'`).

É importante observar que a string retornada inclui o caractere de quebra de linha. Em muitos casos, utiliza-se `strings.TrimSpace` para removê-lo.

```
name = strings.TrimSpace(name)
```

14.8.2 Lendo um arquivo linha por linha

O tipo `Scanner` facilita a leitura sequencial de arquivos de texto.

```
1 package main
2
3 import (
4     "bufio"
5     "fmt"
6     "os"
7 )
8
9 func main() {
10     file, err := os.Open("dados.txt")
11     if err != nil {
12         fmt.Println(err)
13         return
14     }
15     defer file.Close()
16
17     scanner := bufio.NewScanner(file)
18
19     for scanner.Scan() {
20         fmt.Println(scanner.Text())
21     }
22
23     if err := scanner.Err(); err != nil {
24         fmt.Println(err)
25     }
26 }
```

O método `Scan` avança para o próximo elemento e retorna `false` quando não houver mais dados ou ocorrer algum erro.

O texto correspondente pode ser obtido por meio de `Text`.

14.8.3 Alterando o separador

Por padrão, `Scanner` lê uma linha por vez.

Também é possível alterar o critério de divisão utilizando uma função de separação (*split function*).

Por exemplo, para ler uma palavra por vez:

```
scanner.Split(bufio.ScanWords)
```

Nesse caso, cada chamada de `Scan` retorna a próxima palavra encontrada.

14.8.4 Escrevendo com buffer

O tipo `Writer` permite acumular dados em memória antes de enviá-los ao destino.

```
1 package main
2
3 import (
4     "bufio"
5     "os"
6 )
7
8 func main() {
9     file, _ := os.Create("saida.txt")
10    defer file.Close()
11
12    writer := bufio.NewWriter(file)
13
14    writer.WriteString("Primeira linha\n")
15    writer.WriteString("Segunda linha\n")
16
17    writer.Flush()
18 }
```

Os dados permanecem no buffer até que ele seja esvaziado.

Por isso, após concluir a escrita, é necessário chamar `Flush` para garantir que todo o conteúdo seja efetivamente gravado.

14.8.5 Resumo

O pacote `bufio` fornece leitura e escrita com buffer, tornando operações de entrada e saída mais eficientes.

Tipo ou função	Descrição
<code>bufio.NewReader()</code>	Cria um leitor com buffer.
<code>Reader.ReadString()</code>	Lê até encontrar um delimitador.
<code>bufio.NewScanner()</code>	Cria um leitor sequencial de texto.
<code>Scanner.Scan()</code>	Avança para o próximo elemento.
<code>Scanner.Text()</code>	Retorna o texto lido.
<code>Scanner.Split()</code>	Define como o texto será dividido.
<code>bufio.NewWriter()</code>	Cria um escritor com buffer.
<code>Writer.WriteString()</code>	Escreve uma string no buffer.
<code>Writer.Flush()</code>	Envia o conteúdo do buffer ao destino.

O pacote `bufio` é amplamente utilizado em programas que processam arquivos de texto, interpretam arquivos de configuração, implementam ferramentas de linha de comando e realizam leitura de dados fornecidos pelo usuário. Ele complementa o pacote `io`, oferecendo operações mais eficientes e recursos específicos para o processamento de texto.

14.9 encoding/json

O pacote `encoding/json` fornece recursos para converter dados entre estruturas Go e o formato JSON (*JavaScript Object Notation*). Esse formato é amplamente utilizado para troca de informações entre aplicações, especialmente em APIs, serviços web e arquivos de configuração.

As operações mais comuns consistem em transformar uma estrutura Go em JSON e realizar o processo inverso.

Para utilizar o pacote, basta importá-lo:

```
import "encoding/json"
```

14.9.1 Convertendo uma estrutura para JSON

A função `Marshal` converte um valor Go para uma sequência de bytes contendo sua representação em JSON.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type User struct {
9     Name string
10    Age  int
11 }
12
13 func main() {
14     user := User{
15         Name: "Maria",
16         Age:  30,
17     }
18
19     data, err := json.Marshal(user)
20     if err != nil {
21         fmt.Println(err)
22         return
23     }
24
25     fmt.Println(string(data))
26 }
```

Saída:

```
{"Name": "Maria", "Age": 30}
```

14.9.2 Personalizando os nomes dos campos

Normalmente, os nomes dos campos em um JSON seguem convenções diferentes das utilizadas em Go. Para isso, utilizam-se *struct tags*.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type User struct {
9     Name string `json:"name"`
10    Age  int   `json:"age"`
11 }
12
13 func main() {
14     user := User{
15         Name: "Maria",
16         Age:  30,
17     }
18
19     data, _ := json.Marshal(user)
20
21     fmt.Println(string(data))
22 }
```

Saída:

```
{"name": "Maria", "age": 30}
```

As *tags* permitem definir como cada campo será representado no JSON, sem alterar o nome do campo na estrutura.

14.9.3 Convertendo JSON para uma estrutura

A função `Unmarshal` realiza a operação inversa, convertendo um JSON para uma estrutura Go.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type User struct {
9     Name string `json:"name"`
10    Age  int    `json:"age"`
11 }
12
13 func main() {
14     data := []byte(`{"name":"Carlos","age":40}`)
15
16     var user User
17
18     err := json.Unmarshal(data, &user)
19     if err != nil {
20         fmt.Println(err)
21         return
22     }
23
24     fmt.Println(user)
25 }
```

Saída:

```
{Carlos 40}
```

Observe que `Unmarshal` recebe um ponteiro para a estrutura, pois ela será preenchida com os valores lidos do JSON.

14.9.4 Gerando JSON formatado

Para facilitar a leitura, especialmente durante o desenvolvimento, pode-se utilizar `MarshalIndent`.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type User struct {
9     Name string `json:"name"`
10    Age  int    `json:"age"`
11 }
12
13 func main() {
14     user := User{
15         Name: "Maria",
16         Age:  30,
17     }
18
19     data, _ := json.MarshalIndent(user, "", "  ")
20
21     fmt.Println(string(data))
22 }
```

Saída:

```
{
  "name": "Maria",
  "age": 30
}
```

14.9.5 Ignorando campos

Uma *struct tag* também pode indicar que determinado campo não deve aparecer no JSON.

```
type User struct {
    Name      string `json:"name"`
    Password  string `json:"-"`
}
```

Nesse exemplo, o campo Password será ignorado tanto durante a codificação quanto na decodificação.

14.9.6 Gravando e lendo JSON em arquivos

Além de converter valores para []byte com Marshal e Unmarshal, o pacote encoding/json também permite codificar e decodificar dados diretamente a partir de arquivos, conexões de rede ou qualquer outro valor que implemente as interfaces de entrada e saída da biblioteca padrão.

Para gravar JSON em um arquivo, utiliza-se `json.NewEncoder`.

```
1 package main
2
3 import (
4     "encoding/json"
5     "os"
6 )
7
8 type Todo struct {
9     Title string `json:"title"`
10    Done  bool  `json:"done"`
11 }
12
13 func main() {
14     file, err := os.OpenFile("todos.json", os.O_CREATE|os.O_WRONLY|os.O_APPEND, 064
15     if err != nil {
16         panic(err)
17     }
18     defer file.Close()
19
20     todo := Todo{
21         Title: "Estudar Go",
22         Done:  false,
23     }
24
25     err = json.NewEncoder(file).Encode(todo)
26     if err != nil {
27         panic(err)
28     }
29 }
```

O método `Encode` converte o valor para JSON e grava o resultado no arquivo. Ele também adiciona uma quebra de linha ao final, o que facilita gravar vários valores JSON no mesmo arquivo.

O arquivo gerado ficaria assim:

```
{"title":"Estudar Go","done":false}
```

Para ler o arquivo novamente, pode-se usar `json.NewDecoder`.

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6     "io"
7     "os"
8 )
9
10 type Todo struct {
11     Title string `json:"title"`
12     Done  bool  `json:"done"`
13 }
14
15 func main() {
16     file, err := os.Open("todos.json")
17     if err != nil {
18         panic(err)
19     }
20     defer file.Close()
21
22     decoder := json.NewDecoder(file)
23     for {
24         var todo Todo
25
26         err := decoder.Decode(&todo)
27         if err != nil {
28             if err == io.EOF {
29                 break
30             }
31
32             panic(err)
33         }
34
35         fmt.Println(todo)
36     }
37 }
```

Esse formato é útil quando cada chamada de `Encode` grava uma tarefa no arquivo. Depois, o `Decoder` consegue ler uma tarefa por vez até encontrar o final do arquivo, representado por `io.EOF`.

14.9.7 Resumo

O pacote `encoding/json` fornece funções para conversão entre estruturas Go e o formato JSON.

Função	Descrição
<code>json.Marshal()</code>	Converte um valor Go para JSON.
<code>json.Unmarshal()</code>	Converte um JSON para um valor Go.
<code>json.MarshalIndent()</code>	Gera um JSON formatado para leitura.
<code>json.NewEncoder()</code>	Grava JSON em um destino de saída.
<code>json.NewDecoder()</code>	Lê JSON a partir de uma entrada.
<code>json:"campo"</code>	Define o nome do campo no JSON.
<code>json:"-"</code>	Ignora o campo durante a conversão.

O pacote `encoding/json` está entre os mais utilizados da biblioteca padrão. Ele é fundamental no desenvolvimento de APIs REST, aplicações distribuídas, arquivos de configuração e qualquer sistema que utilize JSON como formato de intercâmbio de dados.

14.10 Desafios

Os exercícios a seguir têm como objetivo consolidar o uso de alguns pacotes da biblioteca padrão apresentados neste capítulo. Procure resolver cada problema de forma simples, priorizando clareza e familiaridade com as funções estudadas.

1. Utilize o pacote `fmt` para ler do teclado o nome, a idade e a altura de uma pessoa. Em seguida, exiba esses dados em uma única linha formatada.

Por exemplo, para os valores:

```
Maria
30
1.68
```

a saída esperada é:

```
Maria tem 30 anos e 1.68 m de altura.
```

2. Desenvolva um programa que sorteie internamente um número inteiro entre 0 e 99. Em seguida, peça ao usuário que tente adivinhar o número, lendo cada tentativa com o pacote `fmt`. Após cada tentativa, informe se o número sorteado é maior ou menor que o valor informado. O programa deve terminar quando o usuário acertar o número.

Dica: *pesquise na documentação da biblioteca padrão como utilizar o pacote `math/rand/v2` para gerar números aleatórios.*

3. Implemente uma função chamada `NormalizeName` que receba uma string contendo um nome completo e retorne uma versão normalizada seguindo as regras abaixo:

- Remova os espaços em branco no início e no final da string.
- Considere que as palavras podem estar separadas por um ou mais espaços.
- Converta a primeira letra de cada palavra para maiúscula e as demais para minúsculas.
- Retorne o nome com apenas um espaço entre cada palavra.

Por exemplo:

```
Entrada:
"  r0bErTo    sAnToS  "

Saída:
"Roberto Santos"
```

Utilize funções do pacote `strings` para remover espaços, dividir a string em palavras, converter letras para maiúsculas e minúsculas e reconstruir o resultado.

4. Desenvolva um programa que receba, pela linha de comando, um número inteiro representando um intervalo em segundos.

Ao iniciar, exiba o horário atual no formato HH:mm:ss. Em seguida, exiba novamente o horário atual a cada segundo até que o intervalo informado seja atingido.

Por exemplo, executando o programa com um intervalo de 5 segundos:

```
$ go run main.go 5

15:30:10
15:30:11
15:30:12
15:30:13
15:30:14
15:30:15
```

Dica: utilize os pacotes `os`, `strconv` e `time`.

5. Implemente uma pequena ferramenta de linha de comando para manipulação de arquivos.

O programa deve receber o comando como primeiro argumento e, em seguida, os caminhos necessários para a operação.

A estrutura geral de uso deve ser:

```
go run main.go comando arg1 arg2
```

Implemente os seguintes comandos:

```
cat caminho
```

Lê o conteúdo do arquivo informado e exibe no terminal.

```
cp origem destino
```

Copia o conteúdo do arquivo de origem para o arquivo de destino.

```
mv origem destino
```

Move ou renomeia o arquivo de origem para o destino.

```
rm caminho
```

Remove o arquivo informado.

Exemplos:

```
go run main.go cat mensagem.txt
go run main.go cp origem.txt destino.txt
go run main.go mv antigo.txt novo.txt
go run main.go rm mensagem.txt
```

Utilize os pacotes `os` e `fmt` para acessar os argumentos, manipular arquivos e exibir mensagens de erro quando o comando ou a quantidade de argumentos forem inválidos.

6. Desenvolva um programa que receba do usuário o nome e o preço de vários produtos. Ao final da entrada, armazene todos os produtos em uma `slice` e gere um arquivo chamado `products.json` contendo os dados em formato JSON.

Utilize o pacote `encoding/json` para serializar os dados e o pacote `os` para gravar o arquivo.

Capítulo 15

Concorrência

Desde o surgimento dos processadores com múltiplos núcleos, a execução de tarefas em paralelo tornou-se uma característica fundamental das aplicações modernas. Servidores web atendem milhares de requisições simultaneamente, bancos de dados processam diversas consultas ao mesmo tempo e aplicações interativas realizam operações em segundo plano sem interromper a interface do usuário.

Escrever programas concorrentes, entretanto, nem sempre é uma tarefa simples. Em muitas linguagens, a criação e sincronização de *threads* envolve APIs complexas e sujeitas a erros difíceis de identificar, como condições de corrida (*race conditions*), bloqueios (*deadlocks*) e problemas de sincronização.

Go foi projetada tendo a concorrência como um de seus pilares. Em vez de tratar a execução concorrente como um recurso avançado, a linguagem fornece mecanismos simples e integrados para criar e coordenar múltiplas tarefas que executam simultaneamente.

O principal desses mecanismos são as *goroutines*, unidades leves de execução gerenciadas pelo próprio ambiente de execução (*runtime*) da linguagem. Criar milhares de goroutines costuma ser muito mais eficiente do que criar milhares de *threads* do sistema operacional, permitindo que aplicações concorrentes sejam escritas com pouco código e excelente desempenho.

Além das goroutines, Go oferece diferentes mecanismos para coordenar sua execução. Em algumas situações, é necessário aguardar que várias tarefas terminem ou proteger dados compartilhados contra acessos simultâneos. Em outras, a comunicação entre as goroutines pode ser feita por meio de *channels*, permitindo a troca segura de informações sem a necessidade de compartilhar memória diretamente.

Neste capítulo, estudaremos os principais recursos utilizados na programação concorrente em Go. Começaremos pelas goroutines e pelos mecanismos de sincronização fornecidos pelo pacote *sync*. Em seguida, aprenderemos a utilizar *channels*, *buffered channels*, a instrução *select*, o pacote *context* e, por fim, o padrão *worker pool*.

Ao final deste capítulo, seremos capazes de escrever programas concorrentes simples, compreender os principais mecanismos de sincronização da linguagem e utilizar padrões amplamente adotados no desenvolvimento de aplicações profissionais.

15.1 O que é concorrência

Antes de estudarmos as ferramentas oferecidas pela linguagem, é importante compreender o que significa concorrência e como ela se diferencia de paralelismo. Embora esses termos sejam frequentemente utilizados como sinônimos, eles representam conceitos distintos.

Concorrência é uma forma de organizar um programa para que várias tarefas possam progredir de maneira independente. Essas tarefas não precisam estar sendo executadas simultaneamente; basta que o programa seja capaz de alternar entre elas conforme necessário. Já o **paralelismo** ocorre quando duas ou mais tarefas realmente executam ao mesmo tempo, normalmente utilizando diferentes núcleos do processador.

Em outras palavras, um programa pode ser concorrente sem ser paralelo, mas todo programa paralelo envolve a execução concorrente de múltiplas tarefas.

Considere, por exemplo, um servidor web. Enquanto aguarda a resposta do banco de dados para uma requisição, ele pode continuar processando outras conexões recebidas. Durante esse período, o processador não permanece ocioso, podendo executar outras tarefas do programa.

Em um computador com múltiplos núcleos, algumas dessas tarefas poderão ser executadas simultaneamente, caracterizando o paralelismo. Dessa forma, concorrência é uma forma de organizar o programa, enquanto paralelismo depende também dos recursos de hardware disponíveis.

Go foi projetada para facilitar a escrita de programas concorrentes. Em vez de exigir que o programador gerencie diretamente as *threads* do sistema operacional, a linguagem introduz as **goroutines**, unidades leves de execução administradas pelo próprio *runtime*.

Essa abordagem apresenta diversas vantagens. As goroutines são leves, possuem baixo custo de criação, consomem pouca memória e são escalonadas pelo *runtime*, que agenda sua execução sobre um conjunto de *threads* do sistema operacional, aproveitando os recursos disponíveis da máquina.

No entanto, é importante destacar que iniciar várias goroutines não garante que elas serão executadas em paralelo. Se houver apenas um núcleo de processamento disponível, o *runtime* alternará sua execução ao longo do tempo. Quando houver múltiplos núcleos, diferentes goroutines poderão ser executadas simultaneamente.

Outro aspecto importante é que Go incentiva um modelo de concorrência baseado na comunicação entre tarefas, em vez do compartilhamento de memória. Essa filosofia é resumida em uma das frases mais conhecidas da linguagem:

Do not communicate by sharing memory; instead, share memory by communicating.

Em português:

Não se comunique compartilhando memória; compartilhe memória comunicando-se.

Isso não significa que estruturas de sincronização baseadas em compartilhamento de memória

não existam ou não tenham utilidade. Elas continuam sendo importantes em várias situações. A ideia é compreender quando coordenar goroutines por meio de memória compartilhada e quando modelar o problema como comunicação por *channels*.

Nos próximos tópicos aprenderemos a criar goroutines, coordenar sua execução e utilizar os mecanismos oferecidos pela linguagem para construir aplicações concorrentes de forma segura, simples e eficiente.

15.2 Goroutines

Uma **goroutine** é uma unidade leve de execução utilizada para executar uma função de forma concorrente. Em Go, criar uma goroutine é extremamente simples: basta utilizar a palavra-chave `go` antes da chamada de uma função.

Considere o exemplo a seguir:

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func message() {
9     fmt.Println("Início da goroutine")
10    time.Sleep(500 * time.Millisecond)
11    fmt.Println("Fim da goroutine")
12 }
13
14 func main() {
15     fmt.Println("Antes da goroutine")
16
17     go message()
18
19     fmt.Println("Após iniciar a goroutine")
20
21     time.Sleep(time.Second)
22
23     fmt.Println("Fim da main")
24 }
```

A instrução abaixo inicia a função `message` em uma nova goroutine:

```
go message()
```

Observe que, após executar `go message()`, a função `main` não espera que `message` termine. A execução prossegue imediatamente para a próxima instrução, enquanto o *runtime* agenda a nova

goroutine para ser executada concorrentemente. Neste exemplo, utilizamos `time.Sleep` apenas para impedir que o programa seja encerrado antes que a goroutine conclua seu trabalho.

Uma possível saída para este código seria:

```
Antes da goroutine
Após iniciar a goroutine
Início da goroutine
Fim da goroutine
Fim da main
```

É importante observar que a ordem de execução entre goroutines não é previsível. O *runtime* decide quando cada uma será executada, podendo alternar entre elas conforme necessário.

15.2.1 A função main também é uma goroutine

Embora normalmente não pensemos nisso, a própria função `main` é executada em uma goroutine criada automaticamente pelo *runtime*. Quando escrevemos:

```
func main() {
    // ...
}
```

essa função já está sendo executada concorrentemente em relação às demais goroutines que vierem a ser criadas.

O programa é encerrado quando a goroutine que executa `main` termina sua execução. Nesse momento, todas as demais goroutines ainda em execução são interrompidas imediatamente.

Esse comportamento pode ser observado no exemplo abaixo:

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func worker() {
9     time.Sleep(time.Second)
10    fmt.Println("Trabalho concluído")
11 }
12
13 func main() {
14     go worker()
15
16     fmt.Println("Fim da função main")
17 }
```

Uma possível saída é:

Fim da função main

A mensagem "Trabalho concluído" não é exibida porque o programa termina assim que a função main retorna. A goroutine criada por worker não teve tempo suficiente para concluir sua execução.

Esse é um dos erros mais comuns entre iniciantes. Criar uma goroutine não faz com que o programa espere automaticamente por ela.

15.2.2 Funções anônimas

Assim como qualquer outra função, funções anônimas também podem ser executadas como goroutines.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     go func() {
10         fmt.Println("Olá da goroutine!")
11     }()
12
13     time.Sleep(time.Second)
14 }
```

O uso de funções anônimas é bastante comum quando a lógica da goroutine é pequena ou utilizada apenas naquele ponto do programa.

Também é possível capturar variáveis do escopo externo, da mesma forma que ocorre com qualquer *closure*.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     name := "Go"
10
11     go func() {
12         fmt.Println("Olá,", name)
13     }()
14
15     time.Sleep(time.Second)
16 }
```

15.2.3 Goroutines são leves

Uma das principais vantagens das goroutines é seu baixo custo de criação. Enquanto *threads* do sistema operacional normalmente exigem uma quantidade significativa de memória e possuem maior custo de gerenciamento, goroutines são muito menores e são administradas pelo próprio *runtime* da linguagem.

Isso permite que aplicações em Go utilizem milhares ou até milhões de goroutines simultaneamente, dependendo da memória disponível e da natureza da aplicação.

Essa característica torna Go especialmente adequada para servidores, sistemas distribuídos, processamento de eventos, aplicações de rede e outras situações em que diversas tarefas independentes precisam ser executadas concorrentemente.

Nos próximos tópicos veremos como coordenar a execução de múltiplas goroutines e permitir que elas troquem informações de forma segura.

15.3 Sincronização com sync

As goroutines permitem que diferentes partes de um programa sejam executadas de forma concorrente. Entretanto, criar goroutines é apenas parte da solução. Em muitos casos, também é necessário coordenar sua execução.

Por exemplo, o programa principal pode precisar aguardar que várias goroutines terminem antes de continuar. Em outras situações, diversas goroutines podem acessar uma mesma variável simultaneamente, tornando necessário controlar esse acesso para evitar resultados incorretos.

O pacote `sync` reúne os principais mecanismos de sincronização da biblioteca padrão para resolver esses problemas. Entre seus tipos mais utilizados estão:

- `WaitGroup`, que permite aguardar a conclusão de um conjunto de goroutines;
- `Mutex`, que garante acesso exclusivo a um recurso compartilhado;

- `RWMutex`, que permite múltiplas leituras simultâneas, mantendo exclusividade durante operações de escrita;
- `Once`, que assegura que uma função seja executada apenas uma vez durante toda a execução do programa.

Essas estruturas são fundamentais para programas que compartilham memória entre goroutines. Entretanto, sempre que possível, Go recomenda uma abordagem diferente: utilizar *channels* para que as goroutines se comuniquem, reduzindo a necessidade de sincronização explícita.

Nas próximas seções estudaremos cada um desses mecanismos e veremos em quais situações sua utilização é mais apropriada.

15.3.1 `WaitGroup`

Ao iniciar uma goroutine, sua execução ocorre de forma concorrente com a goroutine principal. Em muitos casos, é necessário aguardar que uma ou mais goroutines concluam seu trabalho antes que o programa continue sua execução.

Para essa finalidade, o pacote `sync` fornece o tipo `WaitGroup`.

Seu funcionamento é baseado em três métodos:

- `Add(n)`: informa quantas goroutines serão aguardadas.
- `Done()`: indica que uma goroutine terminou sua execução.
- `Wait()`: bloqueia a execução até que todas as goroutines tenham chamado `Done()`.

O exemplo a seguir inicia três goroutines e aguarda a conclusão de todas elas.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8 func task(id int, wg *sync.WaitGroup) {
9     defer wg.Done()
10
11     fmt.Println("Tarefa", id)
12 }
13
14 func main() {
15     var wg sync.WaitGroup
16
17     for i := 1; i <= 3; i++ {
18         wg.Add(1)
19         go task(i, &wg)
20     }
21
22     wg.Wait()
23
24     fmt.Println("Fim do programa")
25 }
```

Uma possível saída é:

```
Tarefa 2
Tarefa 1
Tarefa 3
Fim do programa
```

A ordem em que as goroutines são executadas pode variar a cada execução. O `WaitGroup` garante apenas que a função `Wait()` retornará depois que todas as goroutines tiverem chamado `Done()`.

Uma prática comum é utilizar `defer wg.Done()` no início da função executada pela goroutine. Dessa forma, o `WaitGroup` será notificado mesmo que a função termine antecipadamente por meio de um `return`.

15.3.2 Mutex

Quando duas ou mais goroutines acessam simultaneamente uma mesma variável, podem ocorrer resultados inesperados. Esse problema é conhecido como **condição de corrida** (*race condition*).

Para evitar esse tipo de situação, o pacote `sync` fornece o tipo `Mutex` (*Mutual Exclusion*), que garante que apenas uma goroutine por vez tenha acesso a um recurso compartilhado.

Um Mutex disponibiliza dois métodos principais:

- `Lock()`: obtém acesso exclusivo ao recurso.
- `Unlock()`: libera o recurso para que outras goroutines possam utilizá-lo.

No exemplo a seguir, várias goroutines incrementam um contador compartilhado. O uso do Mutex garante que apenas uma goroutine realize a atualização por vez.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8 func increment(id int, counter *int, wg *sync.WaitGroup, mu *sync.Mutex) {
9     defer wg.Done()
10
11     mu.Lock()
12     (*counter)++
13     current := *counter
14     mu.Unlock()
15
16     fmt.Printf("%d -> value %d\n", id, current)
17 }
18
19 func main() {
20     var (
21         counter int
22         wg       sync.WaitGroup
23         mu       sync.Mutex
24     )
25
26     for id := 1; id < 20; id++ {
27         wg.Add(1)
28         go increment(id, &counter, &wg, &mu)
29     }
30
31     wg.Wait()
32
33     fmt.Println("Todas as tarefas terminaram!")
34 }
```

Saída:

```
19 -> value 1
10 -> value 2
15 -> value 3
16 -> value 4
```

```
...
3 -> value 13
6 -> value 14
5 -> value 15
7 -> value 16
8 -> value 17
14 -> value 18
9 -> value 19
Todas as tarefas terminaram!
```

Sem o uso do Mutex, duas ou mais goroutines poderiam ler o mesmo valor de counter simultaneamente, fazendo com que alguns incrementos fossem perdidos. Como consequência, o valor final poderia ser menor que o esperado.

Nesse exemplo, apenas o acesso ao contador fica protegido pelo Mutex. Depois que o valor atualizado é copiado para a variável current, o bloqueio é liberado e a mensagem é exibida fora da região crítica. Em geral, é melhor manter regiões críticas pequenas.

Uma prática recomendada é liberar o Mutex utilizando defer, especialmente quando a região protegida contém mais de uma instrução:

```
mu.Lock()
defer mu.Unlock()

// Código que acessa o recurso compartilhado.
```

O defer é excelente quando a região crítica possui várias instruções ou retornos antecipados.

Por outro lado, para regiões críticas muito pequenas, como:

```
mu.Lock()
counter++
mu.Unlock()
```

é razoável não utilizar defer, evitando um pequeno custo adicional.

15.3.3 RWMutex

Em algumas aplicações, um recurso é consultado com muito mais frequência do que é modificado. Nesses casos, utilizar um Mutex pode limitar desnecessariamente a concorrência, pois tanto as operações de leitura quanto as de escrita são executadas de forma exclusiva.

Para essa situação, o pacote sync fornece o tipo RWMutex (*Read-Write Mutex*), que diferencia operações de leitura e escrita.

Ele disponibiliza quatro métodos principais:

- RLock(): obtém acesso compartilhado para leitura.
- RUnlock(): libera o acesso de leitura.
- Lock(): obtém acesso exclusivo para escrita.

- `Unlock()`: libera o acesso de escrita.

Enquanto uma ou mais goroutines estiverem realizando apenas leituras, todas poderão acessar o recurso simultaneamente. Entretanto, quando uma goroutine solicitar acesso para escrita, ela aguardará até que todas as leituras sejam concluídas e bloqueará novas leituras até terminar a modificação.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8 func read(id int, value *int, mu *sync.RWMutex, wg *sync.WaitGroup) {
9     defer wg.Done()
10
11     mu.RLock()
12     fmt.Printf("leitor %d leu %d\n", id, *value)
13     mu.RUnlock()
14 }
15
16 func write(value *int, mu *sync.RWMutex, wg *sync.WaitGroup) {
17     defer wg.Done()
18
19     mu.Lock()
20     (*value)++
21     mu.Unlock()
22 }
23
24 func main() {
25     var (
26         value int
27         mu     sync.RWMutex
28         wg     sync.WaitGroup
29     )
30
31     wg.Add(1)
32     go write(&value, &mu, &wg)
33
34     for i := 1; i <= 3; i++ {
35         wg.Add(1)
36         go read(i, &value, &mu, &wg)
37     }
38
39     wg.Wait()
40 }
```

Nesse exemplo, as leituras usam `RLock()` e podem ocorrer simultaneamente entre si. Já a escrita usa `Lock()`, exigindo acesso exclusivo ao valor compartilhado.

O `RWMutex` é recomendado quando há muitas operações de leitura e poucas operações de escrita. Caso contrário, um `Mutex` simples costuma ser suficiente e apresenta menor custo de sincronização.

15.3.4 `Once`

Em algumas situações, uma determinada operação deve ser executada apenas uma única vez durante toda a execução do programa, independentemente da quantidade de goroutines que tentem realizá-la.

Para esse tipo de cenário, o pacote `sync` fornece o tipo `Once`.

Seu principal método é:

- `Do(f)`: executa a função `f` apenas uma vez.

No exemplo a seguir, três goroutines tentam executar a mesma função de inicialização. Entretanto, apenas a primeira chamada realmente executa essa função.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8 func main() {
9     var (
10         once sync.Once
11         wg    sync.WaitGroup
12     )
13
14     initialize := func() {
15         fmt.Println("Iniciando...")
16     }
17
18     for i := 1; i <= 3; i++ {
19         wg.Add(1)
20
21         go func(id int) {
22             defer wg.Done()
23
24             once.Do(initialize)
25             fmt.Println("goroutine", id)
26         }(i)
27     }
28
29     wg.Wait()
30 }
```

Uma possível saída é:

```
Iniciando...
goroutine 3
goroutine 1
goroutine 2
```

Mesmo com três goroutines chamando `once.Do(initialize)`, a função `initialize` é executada apenas uma vez.

O tipo `Once` é frequentemente utilizado para inicializar recursos compartilhados, como configurações, conexões ou caches, garantindo que a inicialização ocorra uma única vez, mesmo quando diversas goroutines tentam executá-la simultaneamente.

15.4 Channels

Até este ponto, vimos como utilizar o pacote `sync` para coordenar a execução de goroutines e proteger recursos compartilhados. Embora essas estruturas sejam fundamentais, Go incentiva uma abordagem diferente para a construção de programas concorrentes: a comunicação entre goroutines por meio de *channels*.

Um *channel* é um mecanismo que permite o envio e o recebimento de valores entre goroutines de forma segura. Em vez de compartilhar uma variável e sincronizar seu acesso com um `Mutex`, uma goroutine pode produzir um valor e outra consumi-lo por meio de um canal.

Essa abordagem reduz a necessidade de compartilhamento de memória e torna o fluxo de dados entre as goroutines mais explícito.

Uma frase bastante associada ao modelo de concorrência de Go resume essa ideia:

Não comunique compartilhando memória; compartilhe memória comunicando.

Isso não significa que `Mutex`, `WaitGroup` e outras estruturas do pacote `sync` devam ser evitadas sempre. Elas continuam sendo importantes. A ideia é que, quando o problema puder ser modelado como troca de mensagens ou passagem de dados entre partes do programa, canais costumam produzir um desenho mais claro.

15.4.1 Criando canais

Um canal possui um tipo, que determina quais valores podem ser transmitidos por ele. Por exemplo, um canal de inteiros é declarado da seguinte forma:

```
ch := make(chan int)
```

Nesse exemplo, `ch` é capaz de transportar apenas valores do tipo `int`.

Também poderíamos criar canais para outros tipos:

```
names := make(chan string)
done := make(chan bool)
users := make(chan User)
```

O tipo do canal faz parte de sua definição. Um canal do tipo `chan int` só pode transportar valores `int`; um canal do tipo `chan string` só pode transportar valores `string`, e assim por diante.

15.4.2 Enviando e recebendo valores

O envio de um valor para um canal é realizado com o operador `<-`.

```
ch <- 10
```

Já o recebimento utiliza o mesmo operador, mas na direção oposta.


```
value := <-ch
```

Na prática, o envio e o recebimento normalmente ocorrem em goroutines diferentes.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     ch := make(chan string)
7
8     go func() {
9         ch <- "Olá, Go!"
10    }()
11
12    message := <-ch
13
14    fmt.Println(message)
15 }
```

Saída:

```
Olá, Go!
```

Nesse exemplo, a goroutine anônima envia uma mensagem para o canal, enquanto a goroutine principal aguarda o recebimento desse valor.

15.4.3 Canais sem buffer bloqueiam

Uma característica importante dos canais é que, por padrão, eles são **bloqueantes**. Isso significa que uma operação de envio aguarda até que outra goroutine esteja pronta para receber o valor. Da mesma forma, uma operação de recebimento permanece bloqueada até que algum valor seja enviado ao canal.

Esse comportamento permite sincronizar naturalmente a execução das goroutines, dispensando, em muitos casos, o uso explícito de estruturas como Mutex.

Considere o exemplo a seguir:

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     ch := make(chan int)
10
11     go func() {
12         fmt.Println("enviando valor")
13         time.Sleep(5 * time.Second)
14         ch <- 42
15         fmt.Println("valor enviado")
16     }()
17
18     fmt.Println("aguardando valor...")
19     value := <-ch
20
21     fmt.Println("valor recebido:", value)
22 }
```

Uma possível saída é:

```
enviando valor
aguardando valor...
valor enviado
valor recebido: 42
```

Nesse exemplo, o `time.Sleep` não é responsável pela sincronização. Ele apenas torna visível o período em que a goroutine principal fica aguardando em `<-ch`. Depois da pausa, a goroutine secundária envia o valor pelo canal.

A operação de envio em `ch <- 42` só é concluída quando a goroutine principal executa `<-ch`. Quando o recebimento acontece, o valor é transferido e as duas goroutines podem continuar.

Se tentarmos enviar um valor para um canal sem que exista alguém pronto para recebê-lo, o programa ficará bloqueado.

```
1 package main
2
3 func main() {
4     ch := make(chan int)
5
6     ch <- 10
7 }
```

Esse programa causa um erro em tempo de execução:

```
fatal error: all goroutines are asleep - deadlock!
```

O mesmo pode acontecer ao tentar receber de um canal sem que exista alguma goroutine enviando valores.

```
1 package main
2
3 func main() {
4     ch := make(chan int)
5
6     <-ch
7 }
```

Esse comportamento é esperado. Um canal sem buffer representa uma troca direta entre quem envia e quem recebe.

15.4.4 Fechando canais

Quando uma goroutine termina de enviar valores por um canal, ela pode fechá-lo usando a função `close`.

```
close(ch)
```

Fechar um canal indica que nenhum novo valor será enviado por ele. Isso é útil quando outra goroutine precisa consumir valores até que o canal seja encerrado.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     ch := make(chan int)
7
8     go func() {
9         for i := 1; i <= 3; i++ {
10             ch <- i
11         }
12
13         close(ch)
14     }()
15
16     for value := range ch {
17         fmt.Println(value)
18     }
19 }
```

Saída:

```
1
2
3
```

O `range` sobre um canal recebe valores até que o canal seja fechado. Sem o `close(ch)`, o laço ficaria aguardando novos valores indefinidamente.

Também é possível verificar se um canal ainda está aberto usando a forma com dois retornos:

```
value, ok := <-ch
```

Quando `ok` é `true`, o valor foi recebido normalmente. Quando `ok` é `false`, o canal foi fechado e não há mais valores a receber.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     ch := make(chan string)
7
8     go func() {
9         ch <- "Go"
10        close(ch)
11    }()
12
13    value, ok := <-ch
14    fmt.Println(value, ok)
15
16    value, ok = <-ch
17    fmt.Println(value, ok)
18 }
```

Saída:

```
Go true
false
```

No segundo recebimento, o canal já está fechado e vazio. Por isso, o valor recebido é o valor zero do tipo `string`, e `ok` é `false`.

Em geral, o canal deve ser fechado por quem envia os valores, não por quem recebe. Fechar um canal mais de uma vez causa `panic`, assim como tentar enviar um valor para um canal já fechado.

15.4.5 Canais direcionais

Em assinaturas de funções, é possível restringir se um canal será usado apenas para envio ou apenas para recebimento. Isso torna a intenção da função mais explícita e ajuda o compilador a impedir usos incorretos.

Um canal apenas para envio é declarado como `chan<- T`:

```
func send(ch chan<- string) {  
    ch <- "mensagem"  
}
```

Um canal apenas para recebimento é declarado como `<-chan T`:

```
func receive(ch <-chan string) {  
    fmt.Println(<-ch)  
}
```

Exemplo completo:

```
1 package main  
2  
3 import "fmt"  
4  
5 func send(ch chan<- string) {  
6     ch <- "Olá"  
7     close(ch)  
8 }  
9  
10 func receive(ch <-chan string) {  
11     for message := range ch {  
12         fmt.Println(message)  
13     }  
14 }  
15  
16 func main() {  
17     ch := make(chan string)  
18  
19     go send(ch)  
20     receive(ch)  
21 }
```

Nesse exemplo, `send` só pode enviar valores para o canal, enquanto `receive` só pode receber valores. A restrição melhora a leitura do código e reduz a chance de uso acidental do canal em uma direção errada.

Nos próximos tópicos veremos como criar canais com buffer, utilizar a instrução `select` para trabalhar com múltiplos canais e aplicar esses recursos na construção de programas concorrentes mais elaborados.

15.5 Buffered channels

Os canais apresentados até agora são chamados de **canais sem buffer** (*unbuffered channels*). Neles, uma operação de envio permanece bloqueada até que outra goroutine esteja pronta para receber o valor.

Em algumas situações, entretanto, é desejável permitir que alguns valores sejam enviados antes que outra goroutine os consuma. Para isso, Go oferece os **canais com buffer** (*buffered channels*).

Um canal com buffer possui uma pequena fila interna. Quando um valor é enviado, ele pode ficar armazenado nessa fila até que alguma goroutine o receba. Isso permite desacoplar parcialmente o ritmo de quem produz valores do ritmo de quem consome valores.

15.5.1 Criando canais com buffer

Um canal com buffer é criado informando sua capacidade como segundo argumento da função `make`.

```
ch := make(chan int, 3)
```

Nesse exemplo, o canal pode armazenar até três valores do tipo `int` antes que uma operação de envio fique bloqueada.

O exemplo a seguir usa um produtor mais rápido que o consumidor. O canal possui capacidade 2, então o produtor consegue adiantar duas tarefas. A terceira só é concluída quando o consumidor acorda e libera espaço no buffer.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func sendTask(tasks chan<- string, task string, delay time.Duration) {
9     time.Sleep(delay)
10
11     fmt.Println("enviando:", task)
12     tasks <- task
13     fmt.Println("enviada:", task)
14 }
15
16 func producer(tasks chan<- string) {
17     sendTask(tasks, "tarefa 1", 0)
18     sendTask(tasks, "tarefa 2", 500*time.Millisecond)
19     sendTask(tasks, "tarefa 3", 500*time.Millisecond)
20
21     close(tasks)
22 }
23
24 func consumer(tasks <-chan string, done chan<- struct{}) {
25     time.Sleep(2 * time.Second)
26
27     for task := range tasks {
28         fmt.Println("recebida:", task)
29         time.Sleep(1 * time.Second)
30     }
31
32     close(done)
33 }
34
35 func main() {
36     tasks := make(chan string, 2)
37     done := make(chan struct{})
38
39     go producer(tasks)
40     go consumer(tasks, done)
41
42     <-done
43 }
```

Uma saída típica é:

```
enviando: tarefa 1
enviada: tarefa 1
```

```
enviando: tarefa 2
enviada: tarefa 2
enviando: tarefa 3
recebida: tarefa 1
enviada: tarefa 3
recebida: tarefa 2
recebida: tarefa 3
```

Como o buffer possui capacidade para dois elementos, os dois primeiros envios são realizados imediatamente, mesmo com o consumidor esperando 2 segundos antes de começar a receber.

Quando o produtor tenta enviar tarefa 3, o buffer já está cheio. A linha `enviando: tarefa 3` aparece, mas `enviada: tarefa 3` só aparece depois que o consumidor recebe tarefa 1 e libera um espaço no canal.

15.5.2 Quando o envio bloqueia

Um canal com buffer não elimina o bloqueio. Ele apenas permite que uma quantidade limitada de valores seja armazenada temporariamente.

Quando o buffer fica cheio, novas operações de envio bloqueiam até que algum valor seja recebido.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     ch := make(chan int, 2)
7
8     ch <- 1
9     ch <- 2
10
11     fmt.Println("buffer cheio")
12
13     ch <- 3
14
15     fmt.Println("fim")
16 }
```

Esse programa causa um erro em tempo de execução:

```
fatal error: all goroutines are asleep - deadlock!
```

Os dois primeiros envios cabem no buffer. O terceiro envio bloqueia porque o canal já está cheio e nenhuma goroutine está recebendo valores.

Para que o terceiro envio possa continuar, algum valor precisa ser retirado do canal.


```
1 package main
2
3 import "fmt"
4
5 func main() {
6     ch := make(chan int, 2)
7
8     ch <- 1
9     ch <- 2
10
11     fmt.Println(<-ch)
12
13     ch <- 3
14
15     fmt.Println(<-ch)
16     fmt.Println(<-ch)
17 }
```

Saída:

```
1
2
3
```

Ao receber o primeiro valor, liberamos um espaço no buffer. Com isso, o envio de 3 pode ser realizado.

15.5.3 Quando o recebimento bloqueia

O recebimento em um canal com buffer bloqueia quando o canal está vazio.

```
1 package main
2
3 func main() {
4     ch := make(chan int, 2)
5
6     <-ch
7 }
```

Esse programa também causa um deadlock, pois não há valores armazenados no buffer e nenhuma goroutine enviando valores.

15.5.4 Tamanho e capacidade

Também é possível consultar a quantidade de elementos armazenados e a capacidade do canal utilizando as funções `len` e `cap`.

```
ch := make(chan int, 5)

ch <- 10
ch <- 20

fmt.Println(len(ch))
fmt.Println(cap(ch))
```

Saída:

```
2
5
```

Nesse exemplo, o canal contém dois elementos e possui capacidade total para armazenar cinco.

Em canais, `len(ch)` informa quantos valores estão atualmente armazenados no buffer, enquanto `cap(ch)` informa a capacidade total do buffer.

Essas funções são úteis para observação e depuração, mas raramente devem ser usadas como base para decisões importantes de sincronização. Em programas concorrentes, o estado de um canal pode mudar logo após a chamada de `len`.

15.5.5 Usando buffer com produtores e consumidores

Canais com buffer aparecem com frequência quando uma goroutine produz valores e outra os consome. O produtor pode enviar alguns valores sem depender imediatamente do consumidor, desde que ainda exista espaço no buffer.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     jobs := make(chan int, 3)
7
8     go func() {
9         for i := 1; i <= 5; i++ {
10             fmt.Println("enviando tarefa", i)
11             jobs <- i
12         }
13
14         close(jobs)
15     }()
16
17     for job := range jobs {
18         fmt.Println("processando tarefa", job)
19     }
20 }
```

Uma possível saída é:

```
enviando tarefa 1
enviando tarefa 2
enviando tarefa 3
enviando tarefa 4
processando tarefa 1
processando tarefa 2
processando tarefa 3
processando tarefa 4
enviando tarefa 5
processando tarefa 5
```

A ordem exata pode variar. O ponto importante é que o buffer permite ao produtor adiantar parte do trabalho, mas ele ainda será bloqueado se tentar enviar mais valores do que a capacidade disponível.

Os canais com buffer são úteis quando produtor e consumidor podem trabalhar de forma parcialmente independente, reduzindo o tempo em que as goroutines permanecem bloqueadas. Entretanto, definir um buffer muito grande pode aumentar o consumo de memória sem trazer benefícios significativos. Por esse motivo, a capacidade do canal deve ser escolhida de acordo com as necessidades da aplicação.

Como regra geral, o buffer deve representar uma necessidade real do fluxo de trabalho, e não ser usado apenas para esconder bloqueios. Se um programa só funciona com um buffer muito grande, provavelmente vale revisar o desenho da comunicação entre as goroutines.

15.6 *select*

Em programas concorrentes, é comum que uma goroutine precise aguardar dados provenientes de diferentes canais. Nesses casos, utilizar várias operações de recebimento sequenciais não é uma boa solução, pois a primeira operação pode bloquear a execução do programa.

Para resolver esse problema, Go fornece a instrução `select`.

Seu funcionamento é semelhante ao da instrução `switch`, mas, em vez de comparar valores, ela aguarda operações de envio ou recebimento em múltiplos canais. Quando uma dessas operações estiver pronta para ser executada, o respectivo bloco de código será executado.

15.6.1 Recebendo do primeiro canal disponível

O exemplo a seguir cria dois produtores, cada um enviando uma mensagem para um canal diferente depois de um tempo sorteado. O `select` aguarda até que um dos canais tenha um valor disponível.

```
1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6     "time"
7 )
8
9 func randomDelay(random *rand.Rand) time.Duration {
10     delay := time.Duration(random.Intn(3)+1) * time.Second
11     return delay
12 }
13
14 func sendAfter(ch chan<- string, message string, delay time.Duration) {
15     time.Sleep(delay)
16     ch <- message
17 }
18
19 func main() {
20     ch1 := make(chan string)
21     ch2 := make(chan string)
22     random := rand.New(rand.NewSource(time.Now().UnixNano()))
23
24     delay1 := randomDelay(random)
25     delay2 := randomDelay(random)
26     for delay2 == delay1 {
27         delay2 = randomDelay(random)
28     }
29
30     fmt.Println("canal 1 responderá em", delay1)
31     fmt.Println("canal 2 responderá em", delay2)
32
33     go sendAfter(ch1, "mensagem do canal 1", delay1)
34     go sendAfter(ch2, "mensagem do canal 2", delay2)
35
36     select {
37     case msg := <-ch1:
38         fmt.Println(msg)
39     case msg := <-ch2:
40         fmt.Println(msg)
41     }
42 }
```

Uma saída possível é:

```
canal 1 responderá em 3s
canal 2 responderá em 1s
mensagem do canal 2
```

Outra saída possível seria:

```
canal 1 responderá em 1s  
canal 2 responderá em 2s  
mensagem do canal 1
```

Nesse exemplo, cada goroutine espera por um tempo diferente antes de enviar sua mensagem. O `select` não escolhe pelo nome do canal nem pela ordem dos `case`; ele executa o primeiro `case` cuja operação estiver pronta.

Um `select` executa apenas um `case` por vez. Se for necessário continuar aguardando outros valores, ele normalmente aparece dentro de um `for`.

15.6.2 Usando `select` dentro de um `loop`

O exemplo a seguir recebe valores de dois canais até que ambos sejam fechados.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func count(ch chan<- int, start int, delay time.Duration) {
9     for i := start; i < start+3; i++ {
10         time.Sleep(delay)
11         ch <- i
12     }
13
14     close(ch)
15 }
16
17 func main() {
18     fast := make(chan int)
19     slow := make(chan int)
20
21     go count(fast, 1, 500*time.Millisecond)
22     go count(slow, 10, 1*time.Second)
23
24     for fast != nil || slow != nil {
25         select {
26             case value, ok := <-fast:
27                 if !ok {
28                     fast = nil
29                     continue
30                 }
31                 fmt.Println("fast:", value)
32
33             case value, ok := <-slow:
34                 if !ok {
35                     slow = nil
36                     continue
37                 }
38                 fmt.Println("slow:", value)
39             }
40         }
41     }
```

Uma saída possível é:

```
fast: 1
slow: 10
fast: 2
fast: 3
```

```
slow: 11
slow: 12
```

Quando um canal é fechado, a leitura retorna `ok == false`. Nesse exemplo, atribuímos `nil` ao canal fechado para removê-lo dos próximos `select`. Um canal `nil` nunca fica pronto para envio ou recebimento, então ele passa a ser ignorado pelo `select`.

Se mais de um case estiver pronto ao mesmo tempo, Go escolhe um deles de forma pseudoaleatória. Por isso, em programas concorrentes, a ordem exata das mensagens pode variar.

15.6.3 Timeout com `time.After`

O `select` também é muito usado para limitar quanto tempo uma goroutine deve esperar por uma resposta.

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func slowOperation(result chan<- string) {
9     time.Sleep(2 * time.Second)
10    result <- "operação concluída"
11 }
12
13 func main() {
14     result := make(chan string)
15
16     go slowOperation(result)
17
18     select {
19     case value := <-result:
20         fmt.Println(value)
21     case <-time.After(1 * time.Second):
22         fmt.Println("tempo limite excedido")
23     }
24 }
```

Saída:

```
tempo limite excedido
```

Nesse caso, `time.After` retorna um canal que recebe um valor depois do tempo informado. Como a operação demora 2 segundos, mas o timeout é de 1 segundo, o segundo case é executado primeiro.

Essa técnica é útil para evitar que uma goroutine fique bloqueada indefinidamente esperando uma

resposta.

15.6.4 default

Também é possível utilizar uma cláusula default. Ela será executada imediatamente caso nenhuma operação esteja pronta.

```
select {  
  case msg := <-ch:  
    fmt.Println(msg)  
  default:  
    fmt.Println("Nenhuma mensagem disponível.")  
}
```

A cláusula default impede que o select fique bloqueado aguardando um canal.

Isso pode ser útil quando queremos tentar receber um valor, mas continuar a execução caso não exista nada disponível naquele momento.

```
1 package main  
2  
3 import "fmt"  
4  
5 func main() {  
6     notifications := make(chan string)  
7  
8     select {  
9         case message := <-notifications:  
10             fmt.Println("mensagem recebida:", message)  
11         default:  
12             fmt.Println("nenhuma mensagem disponível")  
13     }  
14  
15     fmt.Println("continuando o programa")  
16 }
```

Saída:

```
nenhuma mensagem disponível  
continuando o programa
```

Sem o default, esse programa ficaria bloqueado esperando uma mensagem no canal notifications.

O select é uma das construções mais importantes da concorrência em Go. Ele permite coordenar múltiplos canais de forma simples e eficiente, sendo amplamente utilizado em conjunto com context, temporizadores e canais de controle para implementar cancelamento, comunicação e sincronização entre goroutines.

15.7 context

Em aplicações concorrentes, é comum que uma operação precise ser interrompida antes de sua conclusão. Isso pode ocorrer, por exemplo, quando um tempo limite é atingido, o usuário cancela uma requisição ou o programa precisa encerrar um conjunto de goroutines relacionadas.

Para coordenar esse tipo de situação, Go fornece o pacote `context`.

Um `Context` transporta informações sobre o ciclo de vida de uma operação, permitindo que diferentes goroutines saibam quando devem interromper seu trabalho.

O contexto não interrompe uma goroutine automaticamente. Ele apenas sinaliza que a operação foi cancelada ou que seu prazo expirou. Cabe ao código observar esse sinal e encerrar o trabalho de maneira cooperativa.

O exemplo a seguir cria um contexto que será cancelado automaticamente após dois segundos. A função `process` recebe esse contexto e usa `select` para decidir se conclui o trabalho ou se deve parar antes.

```
1 package main
2
3 import (
4     "context"
5     "fmt"
6     "time"
7 )
8
9 func process(ctx context.Context) {
10     select {
11     case <-time.After(3 * time.Second):
12         fmt.Println("Operação concluída.")
13     case <-ctx.Done():
14         fmt.Println("Operação cancelada:", ctx.Err())
15     }
16 }
17
18 func main() {
19     ctx, cancel := context.WithTimeout(
20         context.Background(),
21         2*time.Second,
22     )
23     defer cancel()
24
25     process(ctx)
26 }
```

Saída:

```
Operação cancelada: context deadline exceeded
```

Nesse exemplo, a operação levaria três segundos para terminar. Entretanto, como o contexto possui um tempo limite de dois segundos, o canal retornado por `ctx.Done()` é fechado antes da conclusão do trabalho. A função então percebe o cancelamento e retorna.

O método `Done()` retorna um canal que é fechado quando o contexto é cancelado. Dessa forma, uma goroutine pode utilizar `select` para interromper sua execução de maneira cooperativa.

O método `Err()` informa o motivo do encerramento do contexto. Quando o prazo expira, o erro é `context deadline exceeded`. Quando o contexto é cancelado explicitamente, o erro é `context canceled`.

Além do tempo limite, um contexto também pode ser cancelado explicitamente.

```
ctx, cancel := context.WithCancel(context.Background())

// ...

cancel()
```

Após a chamada de `cancel()`, as goroutines que observam esse contexto por meio de `ctx.Done()` serão notificadas.

Mesmo quando o contexto possui um tempo limite, é comum chamar `cancel()` logo após sua criação. Isso libera recursos associados ao contexto caso a operação termine antes do prazo.

Por convenção, funções que representam operações canceláveis recebem o contexto como primeiro parâmetro.

```
func fetchUser(ctx context.Context, id int) {
    // ...
}
```

O pacote `context` é amplamente utilizado na biblioteca padrão e em aplicações profissionais. Diversos pacotes, como `net/http`, `database/sql` e clientes de APIs, recebem um `Context` para controlar cancelamentos, prazos e o encerramento de operações concorrentes.

Embora seu uso seja mais comum em servidores e aplicações distribuídas, compreender seu funcionamento é importante para escrever programas concorrentes robustos e capazes de responder adequadamente a cancelamentos e limites de tempo.

15.8 Worker pools

Em algumas aplicações, é necessário processar um grande número de tarefas concorrentes. Criar uma goroutine para cada tarefa pode não ser a melhor solução, principalmente quando a quantidade de trabalho é muito elevada.

Uma abordagem bastante utilizada é o padrão **worker pool**. Nesse modelo, cria-se um número fixo de goroutines, chamadas *workers*, responsáveis por consumir tarefas enviadas por um canal.

Dessa forma, o número de goroutines permanece controlado, enquanto as tarefas são distribuídas

entre os *workers* disponíveis.

Esse padrão é útil quando queremos limitar a quantidade de trabalho executado ao mesmo tempo. Em vez de criar uma goroutine para cada tarefa, criamos poucos *workers* e enviamos as tarefas para eles.

O exemplo a seguir cria três *workers* para processar cinco tarefas.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6     "time"
7 )
8
9 func worker(id int, jobs <-chan int, wg *sync.WaitGroup) {
10     defer wg.Done()
11
12     for job := range jobs {
13         fmt.Printf("Worker %d processando tarefa %d\n", id, job)
14         time.Sleep(500 * time.Millisecond)
15     }
16 }
17
18 func main() {
19     jobs := make(chan int)
20     var wg sync.WaitGroup
21
22     for i := 0; i < 3; i++ {
23         wg.Add(1)
24         go worker(i+1, jobs, &wg)
25     }
26
27     for i := 0; i < 5; i++ {
28         jobs <- i + 1
29     }
30
31     close(jobs)
32
33     wg.Wait()
34 }
```

Uma possível saída é:

```
Worker 1 processando tarefa 1
Worker 3 processando tarefa 2
Worker 2 processando tarefa 3
Worker 1 processando tarefa 4
```

Worker 3 processando tarefa 5

A distribuição das tarefas entre os *workers* pode variar a cada execução. Cada tarefa será processada por apenas um *worker*, mas não há garantia sobre qual deles a receberá.

O canal `jobs` é usado para enviar tarefas aos *workers*. Como ele é recebido na função `worker` com o tipo `<-chan int`, essa função pode apenas receber valores do canal.

O laço `for job := range jobs` continua executando enquanto o canal estiver aberto. Por isso, depois de enviar todas as tarefas, a função `main` chama `close(jobs)`. Sem esse fechamento, os *workers* ficariam aguardando novas tarefas indefinidamente.

O `WaitGroup` é usado para fazer a goroutine principal esperar todos os *workers* terminarem. Cada *worker* chama `defer wg.Done()` quando encerra sua execução, e `wg.Wait()` bloqueia até que todos tenham finalizado.

O padrão *worker pool* é amplamente utilizado em aplicações que processam filas de trabalho, realizam operações de entrada e saída, executam tarefas em segundo plano ou atendem um grande número de solicitações simultâneas. Ao limitar a quantidade de goroutines ativas, ele permite controlar melhor o consumo de recursos e manter a aplicação eficiente mesmo sob alta carga.

15.9 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos de concorrência apresentados neste capítulo. Procure resolver cada problema de forma incremental, testando o programa a cada nova parte implementada.

Em programas concorrentes, a ordem exata das mensagens pode variar entre execuções. Quando isso acontecer, concentre-se no comportamento esperado do programa, não em uma ordem fixa de saída.

1. Crie um programa que inicie três goroutines. Cada goroutine deve exibir uma mensagem identificando seu número e aguardar um tempo diferente antes de terminar.

Use `time.Sleep` dentro de cada goroutine para simular o tempo de execução.

Exemplo de mensagens:

```
goroutine 1 iniciada
goroutine 2 iniciada
goroutine 3 iniciada
goroutine 2 finalizada
goroutine 1 finalizada
goroutine 3 finalizada
```

2. Reescreva o exercício anterior utilizando `sync.WaitGroup` para garantir que a função `main` espere todas as goroutines terminarem antes de encerrar o programa.

3. Crie um contador compartilhado entre 10 goroutines. Cada goroutine deve incrementar esse contador 100 vezes.

Como várias goroutines acessarão a mesma variável, proteja a atualização do contador utilizando um `sync.Mutex`.

Ao final, exiba o valor final do contador. O resultado esperado é:

1000

4. Crie uma função chamada `produce` que envie os números de 1 a 5 para um canal. Na função `main`, receba os valores desse canal e exiba cada número no terminal.

Feche o canal após o envio dos valores e utilize `range` para consumir os dados.

5. Crie um canal com buffer de capacidade 3. Envie três mensagens para esse canal sem iniciar nenhuma goroutine consumidora. Em seguida, leia e exiba as três mensagens.

Depois, modifique o programa para tentar enviar uma quarta mensagem antes de qualquer leitura. Observe o erro produzido e explique por que o programa entra em deadlock.

6. Crie dois canais, `fast` e `slow`. Cada um deve receber uma mensagem enviada por uma goroutine depois de um intervalo diferente.

Use `select` para exibir a primeira mensagem disponível.

Depois, adicione um terceiro `case` usando `time.After` para limitar a espera a 1 segundo.

7. Crie uma função chamada `countdown` que receba um `context.Context`.

A cada segundo, a função deve exibir o próximo número de uma contagem regressiva iniciando em 10. Caso o contexto seja cancelado antes do término da contagem, a função deve interromper sua execução e exibir o erro retornado por `ctx.Err()`.

Na função `main`, utilize `context.WithTimeout` para cancelar a contagem antes que ela termine.

8. Crie um sistema de atendimento.

Um canal deve representar a fila de clientes. Envie vinte clientes numerados de 1 a 20 para essa fila.

Inicie quatro atendentes (*workers*). Cada atendente deve retirar clientes da fila, simular o atendimento durante um intervalo aleatório entre 500 ms e 2 s e exibir qual cliente foi atendido.

Após enviar todos os clientes, feche a fila e aguarde o término de todos os atendentes utilizando um `sync.WaitGroup`.

9. Modifique o exercício anterior para que cada tarefa produza um resultado.

Crie um segundo canal chamado `results`. Cada *worker* deve receber um número, calcular o dobro desse número e enviar o resultado para `results`.

A função `main` deve receber e exibir todos os resultados produzidos.

10. Implemente uma versão simplificada do problema do jantar dos filósofos.

Considere três filósofos sentados ao redor de uma mesa: Sócrates, Platão e Aristóteles. Há apenas um garfo disponível. Para comer, um filósofo precisa utilizar o garfo. Como existe apenas um garfo, apenas um filósofo pode comer por vez.

Represente cada filósofo por uma goroutine e utilize um `sync.Mutex` para representar o garfo.

Cada filósofo deve repetir entre 7 e 10 vezes o seguinte ciclo:

- pensar durante um intervalo aleatório entre 500 ms e 2 s;
- aguardar o garfo;
- pegar o garfo;
- comer durante um intervalo aleatório entre 500 ms e 2 s;
- devolver o garfo.

Exiba mensagens indicando quando cada filósofo:

- está pensando;
- está aguardando o garfo;
- começou a comer;
- terminou de comer.

Utilize um `sync.WaitGroup` para aguardar o término de todas as goroutines.

11. Reescreva o exercício anterior substituindo o `sync.Mutex` por um canal.

Crie um canal chamado `fork` com buffer de capacidade 1. Esse canal deve representar o único garfo disponível na mesa.

Antes de comer, cada filósofo deve enviar um valor para o canal, indicando que pegou o garfo. Ao terminar de comer, deve receber um valor do canal, liberando o garfo para outro filósofo.

Mantenha o restante do comportamento igual ao exercício anterior:

- três filósofos: Sócrates, Platão e Aristóteles;
- cada filósofo deve repetir o ciclo entre 7 e 10 vezes;
- pensar durante um intervalo aleatório entre 500 ms e 2 s;
- aguardar o garfo;
- pegar o garfo;
- comer durante um intervalo aleatório entre 500 ms e 2 s;
- devolver o garfo;
- usar `sync.WaitGroup` para aguardar todas as goroutines.

Observe como um canal com buffer de capacidade 1 pode funcionar como um semáforo simples.

Capítulo 16

Testes

Até este ponto, desenvolvemos programas, bibliotecas e aplicações utilizando os principais recursos da linguagem Go. No entanto, escrever código que funciona em uma situação específica é apenas parte do trabalho de um desenvolvedor. À medida que um projeto cresce, torna-se cada vez mais importante garantir que alterações futuras não quebrem comportamentos que já estavam corretos.

Testes automatizados são uma das principais ferramentas para atingir esse objetivo. Eles permitem verificar o comportamento esperado de funções e componentes de maneira rápida, repetitiva e confiável. Com bons testes, é possível modificar o código com mais segurança, identificar regressões cedo e documentar, por meio de exemplos executáveis, como uma parte do programa deve se comportar.

Go trata testes como um recurso fundamental da linguagem. Desde sua primeira versão, a distribuição oficial inclui o pacote `testing` e a ferramenta `go test`, eliminando a necessidade de instalar frameworks externos para a maioria dos projetos. Essa simplicidade incentiva a criação de testes desde as primeiras etapas do desenvolvimento.

Neste capítulo, estudaremos como escrever testes unitários com o pacote `testing`, executar testes com `go test`, organizar cenários com *Table Driven Tests*, criar benchmarks para medir desempenho e gerar relatórios de cobertura para identificar quais partes do código são exercitadas pelos testes.

Ao final do capítulo, você será capaz de criar testes simples, estruturar múltiplos cenários de validação, executar testes específicos durante o desenvolvimento, avaliar o desempenho de trechos de código e interpretar relatórios de cobertura utilizando apenas ferramentas fornecidas pela própria linguagem Go.

16.1 `testing`

O pacote `testing` faz parte da biblioteca padrão de Go e fornece a infraestrutura necessária para escrever e executar testes automatizados. Em conjunto com o comando `go test`, ele permite validar o comportamento do código sem depender de bibliotecas externas.

Em Go, testes são escritos em arquivos cujo nome termina com o sufixo `_test.go`. Esses arquivos normalmente ficam no mesmo diretório do código testado e podem conter testes, *benchmarks* e

exemplos executáveis.

Considere a seguinte função responsável por somar dois números:

```
1 package calculator
2
3 func Sum(a, b int) int {
4     return a + b
5 }
```

Se essa função estiver em um arquivo chamado `calculator.go`, o teste correspondente pode ser criado em um arquivo chamado `calculator_test.go`:

```
1 package calculator
2
3 import "testing"
4
5 func TestSum(t *testing.T) {
6     got := Sum(2, 3)
7     want := 5
8
9     if got != want {
10         t.Errorf("Sum(2, 3) = %d; want %d", got, want)
11     }
12 }
```

Nesse exemplo, `got` representa o valor obtido pela execução da função, enquanto `want` representa o valor esperado. Essa convenção aparece com frequência em testes escritos em Go e ajuda a deixar a comparação mais clara.

Toda função de teste deve obedecer a algumas regras:

- Seu nome deve começar com `Test`.
- Deve receber um único parâmetro do tipo `*testing.T`.
- Deve estar localizada em um arquivo com o sufixo `_test.go`.

Por convenção, o nome após `Test` descreve o comportamento testado. Por exemplo, `TestSum`, `TestCreateUser` ou `TestParseDate`.

Quando um teste é executado, qualquer falha deve ser informada por meio dos métodos disponibilizados por `testing.T`, como `Error`, `Errorf`, `Fatal` e `Fatalf`.

Os métodos `Error` e `Errorf` registram a falha, mas permitem que o restante da função continue sendo executado. Já `Fatal` e `Fatalf` registram a falha e interrompem imediatamente a execução daquele teste.

```
if got != want {
    t.Fatalf("Sum(2, 3) = %d; want %d", got, want)
}
```


O uso de `Fatal` é comum quando não faz sentido continuar o teste depois da falha. Para verificações simples, `Errorf` costuma ser suficiente.

A execução dos testes do pacote atual é realizada com o comando:

```
go test
```

Caso todos os testes sejam aprovados, a saída será semelhante a:

```
PASS
ok      example/calculator    0.002s
```

Para obter mais informações durante a execução, utilize a opção `-v` (*verbose*):

```
go test -v
```

A saída passa a exibir cada teste executado:

```
=== RUN   TestSum
--- PASS: TestSum (0.00s)
PASS
ok      example/calculator    0.002s
```

Se o teste falhar, a saída indicará o nome do teste e a mensagem registrada por `t.Errorf` ou `t.Fatalf`.

```
=== RUN   TestSum
calculator_test.go:10: Sum(2, 3) = 6; want 5
--- FAIL: TestSum (0.00s)
FAIL
exit status 1
FAIL    example/calculator    0.002s
```

Também é possível executar apenas um teste específico utilizando a opção `-run`:

```
go test -run TestSum
```

O valor passado para `-run` é interpretado como uma expressão regular. Por isso, o comando também pode executar um grupo de testes com nomes relacionados.

Para executar os testes de todos os pacotes a partir do diretório atual, utilize:

```
go test ./...
```

Durante o desenvolvimento, é comum executar apenas os testes relacionados ao código que está sendo modificado. Antes de integrar uma alteração, porém, costuma ser uma boa prática executar uma suíte mais ampla, como `go test ./...`, para verificar se outros pacotes continuam funcionando.

Embora o exemplo apresentado contenha apenas uma única verificação, na prática uma função normalmente precisa ser testada com diferentes conjuntos de entrada, incluindo casos válidos, valores extremos e situações de erro. A próxima seção apresenta uma técnica bastante utilizada na comunidade Go para organizar esses cenários de maneira clara e escalável: os *Table Driven Tests*.

16.2 Table Driven Tests

À medida que uma função passa a ser testada com diferentes combinações de entradas e saídas, escrever um teste separado para cada cenário torna o código repetitivo e difícil de manter.

Em Go, uma abordagem muito utilizada para resolver esse problema é conhecida como *Table Driven Tests*. Nessa técnica, os casos de teste são organizados em uma tabela de dados, enquanto a lógica de execução permanece única.

Isso reduz duplicação, facilita a leitura e torna mais simples adicionar novos cenários.

Considere novamente a função Sum:

```
1 package calculator
2
3 func Sum(a, b int) int {
4     return a + b
5 }
```

Em vez de escrever vários testes independentes, podemos definir uma tabela contendo todos os casos:

```
1 package calculator
2
3 import "testing"
4
5 func TestSum(t *testing.T) {
6     tests := []struct {
7         a, b int
8         want int
9     }{
10         {2, 3, 5},
11         {10, 5, 15},
12         {-2, 2, 0},
13         {-5, -8, -13},
14         {0, 0, 0},
15     }
16
17     for _, test := range tests {
18         got := Sum(test.a, test.b)
19
20         if got != test.want {
21             t.Errorf(
22                 "Sum(%d, %d) = %d; want %d",
23                 test.a,
24                 test.b,
25                 got,
26                 test.want,
27             )
28         }
29     }
30 }
```

Cada elemento da slice representa um cenário diferente. O laço percorre todos os casos e executa a mesma lógica de verificação para cada um deles.

Nesse tipo de teste, os campos da estrutura costumam representar três grupos de informação:

- os valores de entrada;
- o resultado esperado;
- opcionalmente, um nome para identificar o cenário.

Uma melhoria bastante comum consiste em atribuir um nome para cada caso de teste. Isso facilita a identificação de falhas quando um conjunto grande de cenários é executado.

```
func TestSum(t *testing.T) {
    tests := []struct {
        name string
        a, b int
        want int
    }{
        {"positive numbers", 2, 3, 5},
        {"negative numbers", -5, -8, -13},
        {"different signs", -2, 2, 0},
        {"zero values", 0, 0, 0},
    }

    for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
            got := Sum(test.a, test.b)

            if got != test.want {
                t.Errorf(
                    "Sum(%d, %d) = %d; want %d",
                    test.a,
                    test.b,
                    got,
                    test.want,
                )
            }
        })
    }
}
```

A função `t.Run` cria um subteste para cada cenário. Dessa forma, cada caso passa a ser executado de forma independente e aparece individualmente na saída do comando `go test -v`:

```
=== RUN    TestSum
=== RUN    TestSum/positive_numbers
=== RUN    TestSum/negative_numbers
=== RUN    TestSum/different_signs
=== RUN    TestSum/zero_values
--- PASS: TestSum (0.00s)
    --- PASS: TestSum/positive_numbers (0.00s)
    --- PASS: TestSum/negative_numbers (0.00s)
    --- PASS: TestSum/different_signs (0.00s)
    --- PASS: TestSum/zero_values (0.00s)
PASS
```

Os nomes dos subtestes também podem ser usados com a opção `-run`. Por exemplo:

```
go test -run TestSum/negative_numbers
```

Isso é útil quando apenas um cenário específico está falhando e queremos executá-lo repetida-

mente durante a correção.

Table Driven Tests combinam simplicidade, organização e facilidade de manutenção. Conforme novas situações precisam ser testadas, basta adicionar uma nova entrada à tabela, sem alterar a lógica principal do teste.

Nos próximos tópicos veremos que o pacote `testing` também pode ser utilizado para medir o desempenho do código por meio de *benchmarks*.

16.3 Benchmarks

Além de verificar se um programa produz o resultado correto, muitas vezes também é importante avaliar seu desempenho. Em Go, isso é feito por meio de *benchmarks*, um recurso fornecido pelo próprio pacote `testing`.

Um benchmark mede quanto tempo uma operação leva para executar. Opcionalmente, também pode informar a quantidade de memória alocada durante essa execução.

Benchmarks são úteis para comparar diferentes implementações de um mesmo algoritmo, avaliar o impacto de otimizações e identificar possíveis regressões de desempenho ao longo da evolução do projeto.

Assim como os testes, os benchmarks são escritos em arquivos com o sufixo `_test.go`. A diferença está na assinatura da função: seu nome deve começar com `Benchmark` e receber um parâmetro do tipo `*testing.B`.

Considere novamente a função `Sum`:

```
1 package calculator
2
3 func Sum(a, b int) int {
4     return a + b
5 }
```

O benchmark correspondente pode ser escrito da seguinte forma:

```
1 package calculator
2
3 import "testing"
4
5 func BenchmarkSum(b *testing.B) {
6     for i := 0; i < b.N; i++ {
7         Sum(2, 3)
8     }
9 }
```

Observe que a função é executada dentro de um laço controlado por `b.N`. Esse valor é definido automaticamente pela ferramenta de testes. O Go executa o benchmark repetidas vezes, ajustando `b.N` até obter uma medição estável.

A execução dos benchmarks é realizada com o comando:

```
go test -bench=.
```

A saída será semelhante a:

```
goos: linux
goarch: amd64
pkg: example/calculator
cpu: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz
BenchmarkSum-12      10000000000          0.2233 ns/op
PASS
ok      example/calculator  0.428s
```

O campo ns/op representa o tempo médio gasto por operação, expresso em nanossegundos. No exemplo acima, cada chamada medida pelo benchmark levou, em média, 0.2233 ns.

Quando o pacote também possui testes, o comando `go test -bench=.` executa os testes antes dos benchmarks. Para executar apenas os benchmarks, é comum combinar `-bench` com `-run=^$`:

```
go test -run=^$ -bench=.
```

Também é possível medir a quantidade de memória alocada durante a execução adicionando a opção `-benchmem`:

```
go test -run=^$ -bench=. -benchmem
```

Nesse caso, a saída inclui informações adicionais:

```
BenchmarkSum-16      10000000000          0.2222 ns/op          0 B/op          0 allocs/op
```

Os novos campos possuem o seguinte significado:

- B/op: quantidade média de bytes alocados por operação.
- allocs/op: número médio de alocações de memória realizadas por operação.

Essas métricas são particularmente importantes quando se deseja otimizar algoritmos que manipulam grandes volumes de dados ou são executados com alta frequência.

Em alguns benchmarks, pode ser necessário preparar dados antes de iniciar a medição. Nesse caso, use `b.ResetTimer()` para remover o tempo de preparação do resultado.

```
func BenchmarkSumWithSetup(b *testing.B) {
    a := 2
    c := 3

    b.ResetTimer()

    for i := 0; i < b.N; i++ {
        Sum(a, c)
    }
}
```

O código antes de `b.ResetTimer()` prepara o cenário. Apenas o trecho executado depois disso entra na medição principal.

Embora benchmarks sejam uma ferramenta poderosa, seus resultados devem ser interpretados com cautela. Diferenças muito pequenas podem ser influenciadas pelo sistema operacional, pela carga da máquina e por recursos do processador, como cache e escalonamento de frequência.

Por isso, benchmarks são mais úteis quando comparam implementações executadas sob as mesmas condições.

Na próxima seção veremos como medir a cobertura dos testes, identificando quais partes do código são exercitadas durante sua execução.

16.4 Coverage

Criar testes é um passo importante, mas também é útil saber quais partes do código são executadas durante esses testes. Essa informação é conhecida como cobertura de testes (*test coverage*).

O Go fornece esse recurso de forma integrada por meio do comando `go test`, permitindo identificar quais funções, blocos e instruções foram exercitados pelos testes automatizados.

Para gerar um relatório simples de cobertura no pacote atual, utilize:

```
go test -cover
```

A saída será semelhante a:

```
PASS
coverage: 87.5% of statements
ok      example/calculator    0.003s
```

Nesse exemplo, aproximadamente 87,5% das instruções do pacote foram executadas durante os testes.

Também é possível medir a cobertura de todos os pacotes a partir do diretório atual:

```
go test -cover ./...
```

Esse comando é útil em projetos com múltiplos pacotes, pois mostra a cobertura individual de cada um deles.

Para gerar um relatório detalhado contendo as informações de cobertura, utilize:

```
go test -coverprofile=coverage.out
```

Esse comando cria um arquivo chamado `coverage.out`, que pode ser analisado posteriormente por outras ferramentas do próprio Go.

Para visualizar um relatório no terminal, utilize:

```
go tool cover -func=coverage.out
```

A saída apresenta a cobertura de cada função individualmente:

example/calculator/calculator.go:3:	Sum	100.0%
example/calculator/calculator.go:7:	Sub	0.0%
example/p01.go:8:	main	0.0%
total:	(statements)	33.3%

Outra possibilidade é gerar uma visualização em HTML:

```
go tool cover -html=coverage.out
```

Será aberta uma página destacando o código-fonte com diferentes cores:

- Verde: instruções executadas pelos testes.
- Vermelho: instruções que não foram executadas.

Essa visualização facilita a identificação de trechos do código que ainda não possuem testes.

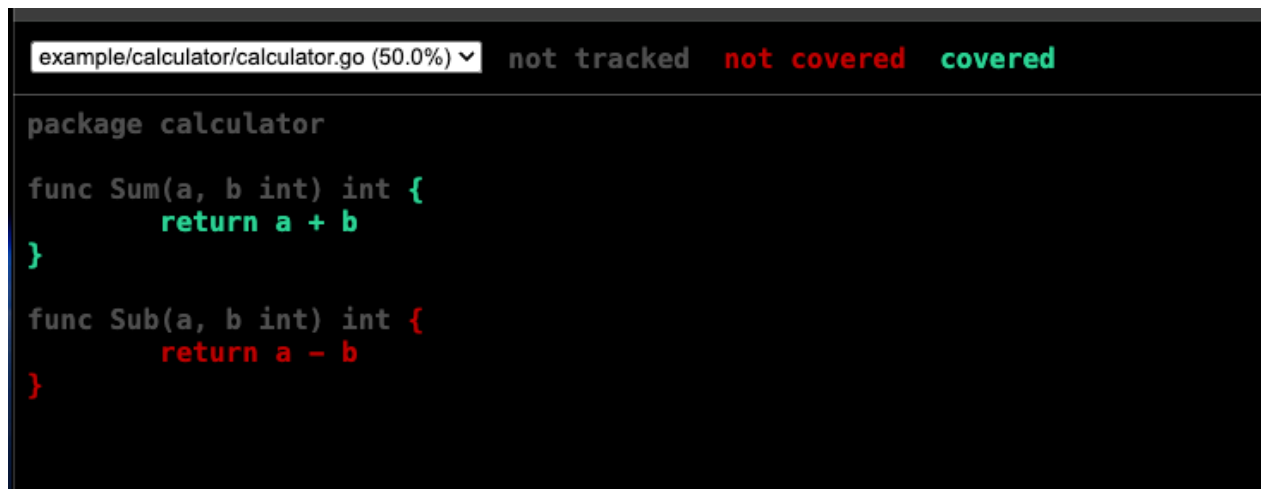


Figura 16.1: Test cover

Embora uma alta cobertura seja desejável, ela não garante que o software esteja livre de defeitos. É possível atingir 100% de cobertura com testes que executam todas as linhas, mas verificam poucos comportamentos relevantes.

Por exemplo, um teste pode chamar uma função e aumentar a cobertura sem conferir corretamente o resultado produzido por ela. Nesse caso, a linha foi executada, mas o comportamento não foi validado de forma significativa.

O objetivo da cobertura de testes não é alcançar um número mágico, mas servir como um indicador para revelar quais partes do código ainda não foram exercitadas. Em geral, é mais importante escrever testes que validem corretamente os comportamentos esperados do que simplesmente aumentar o percentual de cobertura.

Com isso, concluímos os principais recursos oferecidos pelo pacote `testing`. Utilizando testes unitários, *Table Driven Tests*, benchmarks e relatórios de cobertura, é possível construir aplicações mais confiáveis e manter sua evolução com maior segurança ao longo do tempo.

16.5 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos de testes automatizados apresentados neste capítulo. Procure escrever o código de produção e os testes de forma incremental, executando `go test` a cada nova função implementada.

Ao testar uma função, pense em comportamentos, não apenas em linhas de código. Inclua casos válidos, casos inválidos e entradas que possam revelar erros de implementação.

16.5.1 Antes de iniciar os exercícios

Crie um diretório para praticar os exercícios e inicialize um módulo Go:

```
mkdir practice-tests
cd practice-tests
go mod init practice-tests
```

Os pacotes criados dentro desse projeto poderão ser testados com:

```
go test ./...
```

Quando um exercício pedir um pacote específico, crie um diretório com o nome indicado. Por exemplo, um pacote chamado `textutil` pode ser organizado assim:

```
textutil/
├── textutil.go
└── textutil_test.go
```

16.5.2 Exercícios

1. Crie um pacote chamado `mathutil` contendo as seguintes funções:

```
func Max(a, b int) int
func Min(a, b int) int
func Abs(n int) int
```

Escreva testes unitários para cada função.

Inclua casos com:

- números positivos;
- números negativos;
- zero;
- valores iguais.

Execute os testes com:

```
go test ./...
```

2. Reescreva os testes do exercício anterior utilizando *Table Driven Tests*.

Cada caso de teste deve possuir um nome, os valores de entrada e o resultado esperado.

Utilize `t.Run` para executar cada caso como um subteste.

Depois, execute os testes em modo verboso:

```
go test -v ./...
```

Observe como os nomes dos subtestes aparecem na saída.

3. Crie um pacote chamado `brdocs` contendo a função:

```
func OnlyDigits(value string) string
```

Essa função deve retornar uma nova string contendo apenas os dígitos presentes em `value`.

Exemplos:

```
OnlyDigits("123") = "123"  
OnlyDigits("123.456.789-00") = "12345678900"  
OnlyDigits("04.252.011/0001-10") = "04252011000110"  
OnlyDigits("abc") = ""
```

Escreva testes utilizando *Table Driven Tests*.

Inclua casos com:

- string vazia;
- string contendo apenas dígitos;
- string contendo letras e símbolos;
- CPF formatado;
- CNPJ formatado.

4. No pacote `brdocs`, implemente uma função para validar CPF:

```
func IsCPF(value string) bool
```

A função deve aceitar CPFs com ou sem pontuação.

Regras:

- após remover caracteres que não são dígitos, o CPF deve conter exatamente 11 dígitos;
- CPFs com todos os dígitos iguais devem ser rejeitados;
- os dois dígitos verificadores devem ser validados.

Exemplos de entradas válidas:

```
529.982.247-25  
52998224725
```

Exemplos de entradas inválidas:

```
529.982.247-26  
111.111.111-11  
123
```

Escreva testes utilizando *Table Driven Tests*.

Inclua pelo menos:

- um CPF válido com pontuação;
- um CPF válido sem pontuação;
- um CPF com dígito verificador incorreto;
- um CPF com todos os dígitos iguais;
- uma entrada curta demais;
- uma entrada longa demais.

5. No pacote `brdocs`, implemente também:

```
func IsCNPJ(value string) bool
```

A função deve aceitar CNPJs com ou sem pontuação.

Regras:

- após remover caracteres que não são dígitos, o CNPJ deve conter exatamente 14 dígitos;
- CNPJs com todos os dígitos iguais devem ser rejeitados;
- os dois dígitos verificadores devem ser validados.

Exemplos de entradas válidas:

```
04.252.011/0001-10  
04252011000110
```

Exemplos de entradas inválidas:

```
04.252.011/0001-11  
00.000.000/0000-00  
123
```

Escreva testes utilizando *Table Driven Tests*.

Inclua pelo menos:

- um CNPJ válido com pontuação;
- um CNPJ válido sem pontuação;
- um CNPJ com dígito verificador incorreto;
- um CNPJ com todos os dígitos iguais;
- uma entrada curta demais;
- uma entrada longa demais.

6. Adicione benchmarks ao pacote `brdocs`.

Crie benchmarks para as funções `IsCPF` e `IsCNPJ`:

```
func BenchmarkIsCPF(b *testing.B)  
func BenchmarkIsCNPJ(b *testing.B)
```

Cada benchmark deve executar a função correspondente dentro de um laço controlado por `b.N`.

Execute:

```
go test -run=^$ -bench=. ./...
```

Depois, execute novamente incluindo informações de alocação:

```
go test -run=^$ -bench=. -benchmem ./...
```

Observe os valores de ns/op, B/op e allocs/op.

7. Gere um relatório de cobertura dos testes do pacote brdocs.

Execute:

```
go test -cover ./...
```

Em seguida, gere um arquivo de cobertura:

```
go test -coverprofile=coverage.out ./...
```

Visualize a cobertura por função:

```
go tool cover -func=coverage.out
```

Analise o resultado e identifique se alguma função ou ramificação importante ficou sem teste.

Se encontrar pontos descobertos, adicione novos casos de teste e execute os comandos novamente.

8. Organize o pacote brdocs como uma pequena biblioteca reutilizável.

Ao final, o pacote deve expor as seguintes funções:

```
func OnlyDigits(value string) string
func IsCPF(value string) bool
func IsCNPJ(value string) bool
```

Crie também um pequeno programa no pacote main que importe practice-tests/brdocs e exiba o resultado da validação de alguns valores.

Exemplo de uso:

```
1 package main
2
3 import (
4     "fmt"
5
6     "practice-tests/brdocs"
7 )
8
9 func main() {
10     fmt.Println(brdocs.IsCPF("529.982.247-25"))
11     fmt.Println(brdocs.IsCNPJ("04.252.011/0001-10"))
12 }
```

Depois de organizar o pacote, execute novamente:

```
go test ./...  
go test -cover ./...  
go test -run=^$ -bench=. ./...
```

O objetivo deste exercício é praticar o ciclo completo: implementar uma biblioteca pequena, validar seus comportamentos com testes automatizados, medir desempenho com benchmarks e observar a cobertura dos testes.

Capítulo 17

Interfaces Avançadas

Nos capítulos anteriores, vimos que interfaces definem contratos entre tipos e que sua implementação em Go é implícita. Um tipo satisfaz uma interface simplesmente por possuir os métodos exigidos por ela. Esse modelo simples é um dos motivos pelos quais a linguagem favorece código desacoplado, testável e fácil de manter.

Entretanto, existem situações em que apenas conhecer o conceito básico de interfaces não é suficiente. Em aplicações reais, às vezes precisamos lidar com valores cujo tipo concreto só será conhecido em tempo de execução, recuperar esse tipo com segurança, tratar diferentes tipos de forma controlada ou registrar explicitamente que uma struct deve satisfazer determinada interface.

Para esses cenários, Go oferece recursos como `any`, *type assertions*, *type switches* e *compile-time interface assertions*. Eles são simples de usar, mas pedem moderação. Quando aparecem no lugar certo, ajudam a escrever APIs flexíveis e seguras; quando aparecem em excesso, costumam indicar que a abstração escolhida precisa ser repensada.

Neste capítulo, estudaremos quando usar `any`, como recuperar tipos concretos com *type assertions*, como organizar decisões com *type switches* e como pedir ao compilador que verifique uma implementação de interface. Ao final, reuniremos boas práticas para projetar interfaces pequenas, expressivas e alinhadas à filosofia da linguagem Go.

17.1 `any`

Desde a versão 1.18, Go possui o identificador pré-declarado `any`, utilizado para representar um valor de qualquer tipo.

Na prática, `any` é apenas um alias para a interface vazia, `interface{}`. Durante muitos anos, `interface{}` foi a forma usada em Go para indicar que um valor poderia ter qualquer tipo concreto.

As duas declarações a seguir são equivalentes:

```
var value interface{}
```

```
var value any
```

O uso de `any` foi introduzido para tornar o código mais legível, principalmente após a chegada dos genéricos. Atualmente, quando a intenção é aceitar um valor de tipo arbitrário, `any` costuma deixar essa intenção mais clara que `interface{}`.

Como qualquer tipo satisfaz a interface vazia, valores de diferentes naturezas podem ser atribuídos a uma variável do tipo `any`:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var value any
7
8     value = 10
9     fmt.Println(value)
10
11    value = "Go"
12    fmt.Println(value)
13
14    value = 3.14
15    fmt.Println(value)
16
17    value = true
18    fmt.Println(value)
19 }
```

A saída será:

```
10
Go
3.14
true
```

Isso não significa que `any` transforme Go em uma linguagem sem tipos. O valor continua tendo um tipo concreto em tempo de execução. O que acontece é que, enquanto ele está armazenado em uma variável do tipo `any`, o compilador conhece apenas a interface vazia.

Por esse motivo, operações específicas do tipo concreto não podem ser realizadas diretamente.

O exemplo a seguir não compila:

```
var value any = "Go"

fmt.Println(len(value))
```

O compilador produz um erro semelhante a:


```
invalid argument: value (variable of interface type any) for built-in len
```

Embora o valor armazenado seja uma `string`, a variável `value` tem tipo estático `any`. Antes de utilizar o valor como uma `string`, é necessário recuperar seu tipo concreto. Esse processo pode ser realizado por meio de uma *type assertion*, assunto da próxima seção.

Um bom exemplo de uso de `any` aparece no pacote `fmt`. A função `fmt.Println` precisa aceitar qualquer quantidade de valores, de qualquer tipo:

```
func Println(a ...any) (n int, err error)
```

Essa assinatura faz sentido porque `fmt.Println` não depende de um tipo específico do domínio da aplicação. Sua responsabilidade é receber valores arbitrários e produzir uma representação textual para cada um deles.

Funções semelhantes do pacote `fmt`, como `fmt.Print`, `fmt.Printf`, `fmt.Sprint` e `fmt.Sprintf`, também utilizam `...any` para representar listas variáveis de argumentos.

Um uso comum de `any` aparece em estruturas que precisam representar dados de formato flexível, como JSON genérico:

```
var data map[string]any
```

Nesse caso, cada chave do mapa pode apontar para valores de tipos diferentes, como `string`, `float64`, `bool`, `[]any` ou outro `map[string]any`.

Apesar de útil, `any` deve ser usado com cuidado. Quando uma função realmente precisa de um comportamento específico, normalmente é melhor declarar uma interface que expresse esse comportamento.

Por exemplo, se uma função precisa receber valores que fornecem sua própria representação textual, uma interface como `fmt.Stringer` comunica melhor a intenção do que `any`.

```
func Label(value fmt.Stringer) string {  
    return fmt.Sprintf("value: %s", value)  
}
```

Nesse caso, a função não quer aceitar qualquer valor possível. Ela exige um valor cuja saída textual seja definida pelo método `String`, usado automaticamente pelo pacote `fmt` durante a formatação.

Com a introdução dos genéricos, muitas situações que anteriormente exigiam `interface{}` ou `any` passaram a ser resolvidas por parâmetros de tipo. Assim, `any` permanece mais adequado para casos em que o tipo realmente é desconhecido ou pode variar durante a execução, como serialização, reflexão, logs estruturados e processamento genérico de dados.

17.2 Type Assertions

Quando um valor é armazenado em uma variável do tipo `any` ou em outra interface, o compilador deixa de conhecer seu tipo concreto. Em algumas situações, entretanto, precisamos recuperar esse tipo para utilizar operações específicas.

Em Go, esse processo é realizado por meio de uma *type assertion*.

A sintaxe é:

```
value.(T)
```

Nesse caso, `value` deve ser uma expressão de tipo interface, e `T` representa o tipo que esperamos obter.

Considere o exemplo:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var value any = "Go"
7
8     text := value.(string)
9
10    fmt.Println(text)
11    fmt.Println(len(text))
12 }
```

Nesse caso, a *type assertion* informa ao compilador que o valor armazenado em `value` deve ser tratado como uma `string`. Após a conversão, todas as operações válidas para esse tipo podem ser utilizadas.

Se o tipo esperado não corresponder ao tipo real armazenado na interface, ocorrerá um `panic`.

```
1 package main
2
3 func main() {
4     var value any = 10
5
6     text := value.(string)
7
8     println(text)
9 }
```

A execução produz um erro semelhante a:

```
panic: interface conversion: interface {} is int, not string
```

Essa forma é útil apenas quando temos certeza sobre o tipo armazenado. Quando essa certeza não existe, a forma segura é preferível:

```
value, ok := expression.(T)
```

O segundo valor indica se a conversão foi realizada com sucesso.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var value any = 10
7
8     text, ok := value.(string)
9
10    fmt.Printf("%q\n", text)
11    fmt.Println(ok)
12 }
```

A saída será:

```
""
false
```

Observe que `text` recebe o valor zero da `string`, enquanto `ok` indica que a conversão falhou.

Normalmente, a verificação é utilizada da seguinte forma:

```
if text, ok := value.(string); ok {
    fmt.Println(len(text))
} else {
    fmt.Println("o valor não é uma string")
}
```

Esse padrão evita *panics* e é a forma recomendada de utilizar *type assertions* quando não há garantia sobre o tipo armazenado.

Uma *type assertion* também pode verificar se o valor armazenado implementa outra interface. No exemplo a seguir, verificamos se um valor implementa `fmt.Stringer`:

```
1 package main
2
3 import "fmt"
4
5 type Product struct {
6     Name string
7 }
8
9 func (p Product) String() string {
10     return p.Name
11 }
12
13 func main() {
14     var value any = Product{"Notebook"}
15
16     if s, ok := value.(fmt.Stringer); ok {
17         fmt.Println(s.String())
18     }
19 }
```

Nesse caso, a verificação não pergunta se o valor concreto é exatamente `fmt.Stringer`. Ela pergunta se o tipo concreto armazenado em `value` satisfaz a interface `fmt.Stringer`.

Isso permite recuperar uma visão mais específica do valor, baseada em comportamento.

Type assertions são úteis, mas devem ser utilizadas com moderação. Em muitos casos, a necessidade frequente de recuperar o tipo concreto indica que a abstração escolhida não é a mais adequada.

Sempre que possível, prefira trabalhar diretamente com interfaces que exponham o comportamento necessário. Deixe as *type assertions* para situações em que o tipo realmente só pode ser conhecido durante a execução.

Quando um valor pode assumir muitos tipos diferentes, uma sequência de *type assertions* tende a ficar repetitiva. Para esses casos, Go oferece o *type switch*, assunto da próxima seção.

17.3 Type Switch

Quando um valor armazenado em uma interface pode assumir muitos tipos diferentes, usar várias *type assertions* em sequência torna o código repetitivo e difícil de ler.

Para esses casos, Go oferece o *type switch*, uma construção semelhante ao `switch` tradicional, mas que seleciona o bloco de código com base no tipo concreto armazenado em uma interface.

A sintaxe utiliza `.(type)`:

```
switch v := value.(type) {  
case string:  
    // ...  
case int:  
    // ...  
}
```

Essa forma especial só pode ser usada dentro de um `type switch`. Ela não é uma expressão comum e não pode ser utilizada fora dessa construção.

Considere o exemplo:

```
1 package main  
2  
3 import "fmt"  
4  
5 func Print(value any) {  
6     switch v := value.(type) {  
7     case int:  
8         fmt.Printf("Inteiro: %d\n", v)  
9  
10    case float64:  
11        fmt.Printf("Float: %.2f\n", v)  
12  
13    case string:  
14        fmt.Printf("Texto: %s\n", v)  
15  
16    default:  
17        fmt.Printf("Tipo desconhecido: %T\n", v)  
18    }  
19 }  
20  
21 func main() {  
22     Print(10)  
23     Print(3.14)  
24     Print("Go")  
25     Print(true)  
26 }
```

A saída será:

```
Inteiro: 10  
Float: 3.14  
Texto: Go  
Tipo desconhecido: bool
```

Cada cláusula `case` representa um tipo. Quando o tipo concreto corresponde a um dos casos, a variável `v` passa a ter aquele tipo dentro do bloco correspondente.

No case `int`, por exemplo, `v` é um `int`. No case `string`, `v` é uma `string`. No `default`, `v` mantém o mesmo tipo da expressão original.

Assim como no `switch` tradicional, apenas o primeiro caso compatível é executado.

Também é possível agrupar vários tipos em um mesmo case:

```
switch v := value.(type) {
case int, int8, int16, int32, int64:
    fmt.Printf("Inteiro: %v\n", v)

case float32, float64:
    fmt.Printf("Ponto flutuante: %v\n", v)

default:
    fmt.Println("Outro tipo")
}
```

Quando um case agrupa vários tipos, a variável `v` não assume um tipo específico como `int` ou `int64`. Dentro desse bloco, ela mantém o tipo da interface original. Por isso, normalmente usamos operações válidas para todos os valores, como impressão com `%v`.

O *type switch* também funciona com interfaces. No exemplo abaixo, verificamos se um valor implementa determinadas interfaces da biblioteca padrão:

```
switch v := value.(type) {
case fmt.Stringer:
    fmt.Println(v.String())

case error:
    fmt.Println(v.Error())

default:
    fmt.Printf("%v\n", v)
}
```

Nesse caso, o teste não é realizado contra um tipo concreto, mas contra interfaces satisfeitas pelo valor armazenado.

A ordem dos casos pode importar quando mais de um case é compatível. Se um valor implementar duas interfaces listadas no `type switch`, será executado o primeiro caso correspondente.

Embora o *type switch* seja uma ferramenta útil, seu uso frequente pode indicar que o código está excessivamente dependente dos tipos concretos em vez de seus comportamentos. Em muitas situações, é mais idiomático definir uma interface contendo os métodos necessários e deixar que cada implementação forneça seu próprio comportamento.

Em geral, o *type switch* é mais apropriado em funções de infraestrutura, bibliotecas de serialização, sistemas de logging, reflexão e outras situações nas quais o tipo concreto realmente só pode ser conhecido durante a execução.

17.4 Compile-time Interface Assertions

Em Go, a implementação de interfaces é implícita. Um tipo satisfaz uma interface simplesmente por possuir todos os métodos exigidos, sem a necessidade de declarar explicitamente essa relação.

Essa característica torna o código mais simples, mas também significa que o compilador só verifica a relação entre um tipo e uma interface quando essa relação é usada em algum ponto do programa.

Considere o exemplo:

```
type Writer interface {
    Write([]byte) (int, error)
}

type File struct{}

func (f File) Write(p []byte) (int, error) {
    return len(p), nil
}
```

O tipo `File` satisfaz `Writer`, pois possui o método `Write` com a assinatura esperada. Entretanto, se `File` nunca for atribuído a uma variável do tipo `Writer`, passado para uma função que recebe `Writer` ou retornado como `Writer`, essa relação pode não ficar explícita no código.

Em projetos maiores, especialmente bibliotecas e APIs públicas, às vezes queremos declarar essa intenção de forma explícita e pedir ao compilador que faça a verificação.

Para isso, utiliza-se uma *compile-time interface assertion*:

```
var _ Writer = File{}
```

Essa declaração informa ao compilador que um valor do tipo `File` deve ser atribuível à interface `Writer`.

O identificador `_` descarta o valor. Ele é usado porque não queremos criar uma variável real para ser utilizada depois; queremos apenas forçar a verificação de tipos durante a compilação.

Caso todos os métodos da interface estejam implementados corretamente, o código compila normalmente.

Se algum método estiver ausente ou possuir uma assinatura incompatível, ocorrerá um erro de compilação.

Por exemplo, suponha que `Write` tenha sido implementado incorretamente:

```
type File struct{}

func (f File) Write(p []byte) {
}
```

A declaração:

```
var _ Writer = File{}
```

produzirá um erro semelhante a:

```
cannot use File{} as Writer value in variable declaration:
    File does not implement Writer
        (wrong type for method Write)
```

Esse tipo de verificação não possui custo em tempo de execução. Seu objetivo é solicitar ao compilador que valide a implementação da interface.

Quando a interface é implementada por meio de um *pointer receiver*, a verificação deve utilizar um ponteiro:

```
type Writer interface {
    Write([]byte) (int, error)
}

type File struct{}

func (f *File) Write(p []byte) (int, error) {
    return len(p), nil
}

var _ Writer = (*File)(nil)
```

Nesse exemplo, `File` não implementa `Writer`, mas `*File` sim. Isso acontece porque o método `Write` foi definido com receiver `*File`.

A expressão `(*File)(nil)` não cria um valor útil para execução do programa. Ela apenas fornece ao compilador o tipo `*File` para que a verificação seja realizada.

Quando os métodos são definidos com *value receiver*, como `func (f File) Write(...)`, tanto `File{}` quanto `(*File)(nil)` podem satisfazer a interface. Quando os métodos são definidos com *pointer receiver*, normalmente apenas o ponteiro satisfaz a interface.

As *compile-time interface assertions* são comuns em bibliotecas e frameworks, onde uma alteração acidental na assinatura de um método pode fazer com que um tipo deixe de implementar determinada interface. Ao realizar essa verificação durante a compilação, erros são detectados imediatamente, antes mesmo da execução dos testes.

Em aplicações pequenas, esse recurso costuma ser desnecessário, pois o compilador verificará naturalmente as interfaces à medida que forem utilizadas. Em projetos maiores e APIs públicas, entretanto, essa técnica pode documentar uma intenção importante e adicionar uma camada extra de segurança sem impacto no desempenho da aplicação.

17.5 Boas Práticas

Interfaces são um dos recursos mais importantes da linguagem Go, mas sua força está mais na simplicidade do que na quantidade de abstrações criadas. Interfaces bem projetadas tornam o código desacoplado e fácil de testar; interfaces mal posicionadas escondem tipos concretos, dificultam a leitura e aumentam a manutenção.

As recomendações a seguir refletem práticas comuns na biblioteca padrão e na comunidade Go.

17.5.1 Prefira interfaces pequenas

Em Go, interfaces costumam representar um comportamento pequeno e bem definido.

A biblioteca padrão contém diversos exemplos:

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

type Closer interface {
    Close() error
}
```

Interfaces pequenas são mais fáceis de implementar, reutilizar, testar e combinar.

Quando necessário, várias interfaces podem ser compostas para formar contratos maiores.

```
type ReadWriter interface {
    io.Reader
    io.Writer
}
```

Essa abordagem costuma ser preferível à criação antecipada de interfaces extensas contendo muitos métodos.

17.5.2 Aceite interfaces e retorne structs

Uma recomendação bastante conhecida na comunidade Go é:

Accept interfaces, return structs.

Isso significa que funções podem receber interfaces quando precisam apenas de um comportamento específico, permitindo que diferentes implementações sejam utilizadas.

Por outro lado, retornos normalmente devem utilizar tipos concretos, oferecendo ao chamador acesso à funcionalidade completa do valor retornado.

```
func Save(w io.Writer, data []byte) error {  
    _, err := w.Write(data)  
    return err  
}
```

Nesse exemplo, `Save` não precisa saber se está escrevendo em um arquivo, em um buffer de memória ou em uma conexão de rede. Ela precisa apenas de algo capaz de executar `Write`.

Já uma função construtora normalmente deve retornar o tipo concreto:

```
func NewBuffer() *bytes.Buffer {  
    return &bytes.Buffer{}  
}
```

Retornar uma interface nesse caso esconderia métodos úteis de `bytes.Buffer` sem trazer um benefício claro.

Essa recomendação não é uma regra absoluta. Em alguns casos, retornar uma interface faz sentido, principalmente quando o pacote deseja ocultar detalhes de implementação. Ainda assim, a decisão deve ser intencional.

17.5.3 Defina interfaces onde elas são utilizadas

Em vez de criar interfaces no pacote que fornece uma implementação, geralmente é preferível defini-las no pacote consumidor.

Considere:

```
type UserRepository interface {  
    FindByID(id int) (User, error)  
}
```

Essa interface normalmente pertence ao código que precisa consultar usuários, e não necessariamente ao pacote responsável por acessar o banco de dados.

Dessa forma, o consumidor declara apenas o comportamento de que precisa, e a implementação permanece livre para evoluir sem impor abstrações desnecessárias aos demais pacotes.

Em Go, não é necessário criar uma interface antes de criar a implementação. Muitas vezes, a interface aparece naturalmente depois que existe um segundo uso, um teste ou uma fronteira clara entre pacotes.

17.5.4 Evite utilizar `any` desnecessariamente

O tipo `any` deve ser utilizado apenas quando o tipo concreto realmente é desconhecido.

Sempre que possível, prefira um tipo específico:

```
func Print(name string)
```

ou uma interface representando o comportamento desejado:

```
func Save(w io.Writer)
```

O uso excessivo de `any` reduz a verificação realizada pelo compilador e normalmente exige *type assertions* durante a execução. Quando isso acontece em muitos pontos do código, parte da segurança de tipos da linguagem deixa de ser aproveitada.

`any` é útil em situações como decodificação de JSON genérico, estruturas de log, integração com APIs externas ou código de infraestrutura que realmente precisa aceitar valores variados. Fora desses casos, tipos concretos e interfaces específicas costumam comunicar melhor a intenção do programa.

17.5.5 Evite *type assertions* frequentes

Quando uma função realiza diversas *type assertions* ou utiliza grandes *type switches*, isso pode indicar que a abstração escolhida não é adequada.

Em muitos casos, é preferível definir uma interface contendo o comportamento necessário e permitir que cada tipo forneça sua própria implementação.

Em vez de:

```
switch v := value.(type) {  
case Circle:  
    ...  
case Rectangle:  
    ...  
}
```

considere:

```
type Shape interface {  
    Area() float64  
}
```

Assim, o código trabalha diretamente com o comportamento, sem depender dos tipos concretos.

Type assertions e *type switches* continuam sendo úteis em pontos específicos, como tratamento de erros, adaptação entre APIs ou leitura de dados genéricos. O problema aparece quando eles passam a substituir uma interface simples ou um método bem definido.

17.5.6 Utilize *compile-time interface assertions* em bibliotecas

Projetos pequenos normalmente não precisam de verificações explícitas de interfaces.

Entretanto, bibliotecas e APIs públicas frequentemente utilizam:

```
var _ bankaccountdomain.Repository = (*Repository)(nil)
```

Esse padrão garante que alterações futuras não façam um tipo deixar de implementar uma interface sem que o compilador detecte o problema.

Em código de aplicação, use esse recurso com moderação. Se a relação entre o tipo e a interface já é exercitada naturalmente pelo código ou pelos testes, a declaração explícita pode ser apenas ruído.

17.5.7 Interfaces representam comportamento

Uma interface deve descrever o que um tipo é capaz de fazer, e não o que ele é.

Por esse motivo, nomes terminados em *-er* são bastante comuns na biblioteca padrão:

- Reader
- Writer
- Closer
- Formatter
- Stringer

Esses nomes descrevem comportamentos, tornando a intenção da interface clara para quem lê o código.

Evite interfaces com nomes genéricos demais, como *Manager*, *Handler* ou *Service*, quando o contrato real não estiver claro. Um bom nome ajuda a revelar o comportamento esperado.

17.5.8 Não crie interfaces cedo demais

Uma prática comum em algumas linguagens é criar uma interface para quase toda struct. Em Go, isso geralmente não é necessário.

Se existe apenas uma implementação concreta e nenhum consumidor precisa substituir esse comportamento, uma interface pode ser prematura.

Comece com tipos concretos. Quando surgir a necessidade de desacoplar um pacote, escrever um teste com uma dependência falsa ou aceitar diferentes implementações para o mesmo comportamento, extraia uma interface pequena no ponto onde ela será usada.

Seguindo essas recomendações, as interfaces permanecem pequenas, reutilizáveis e fáceis de implementar. Essa simplicidade é uma das principais características da linguagem Go e contribui para a construção de APIs claras, desacopladas e de fácil manutenção.

17.6 Desafios

Os exercícios a seguir têm como objetivo consolidar os conceitos apresentados neste capítulo: *any*, *type assertions*, *type switches*, verificações de interface em tempo de compilação e boas práticas de modelagem.

Use esses recursos com moderação. Em alguns exercícios, a proposta é justamente perceber quando um *type switch* pode ser substituído por uma interface mais simples e expressiva.

17.6.1 Exercícios

1. Crie uma função chamada `Describe` que receba um valor do tipo `any` e retorne uma descrição textual desse valor.

A função deve utilizar um *type switch* para tratar pelo menos os seguintes tipos:

- `string`;
- `int`;
- `float64`;
- `bool`;
- `nil`;
- qualquer outro tipo.

Exemplos de retorno:

```
Describe("Go") = "texto: Go"
Describe(42) = "inteiro: 42"
Describe(3.14) = "decimal: 3.14"
Describe(true) = "booleano: true"
Describe(nil) = "valor nulo"
Describe([]int{1, 2, 3}) = "tipo desconhecido: []int"
```

Depois, escreva uma função `main` que chame `Describe` com valores de tipos diferentes e exiba os resultados.

2. Crie uma função chamada `Label` que receba um valor do tipo `any`.

Se o valor implementar a interface `fmt.Stringer`, a função deve retornar o resultado de `String()`.

Caso contrário, deve retornar uma representação genérica usando `fmt.Sprintf("%v", value)`.

Use uma *type assertion* segura para verificar se o valor implementa `fmt.Stringer`.

Em seguida, crie uma struct chamada `Product`:

```
type Product struct {
    Name  string
    Price float64
}
```

Implemente o método `String()` para `Product` e teste a função `Label` com:

- um `Product`;
- uma `string`;
- um `int`;
- um valor `nil`.

3. Crie uma interface chamada `Notifier` com o método:

```
Notify(message string) error
```

Em seguida, crie duas structs:

```
type EmailNotifier struct{}
type SMSNotifier struct{}
```

Implemente o método `Notify` para ambas.

Depois, adicione verificações de interface em tempo de compilação:

```
var _ Notifier = EmailNotifier{}
var _ Notifier = SMSNotifier{}
```

Altere temporariamente a assinatura de um dos métodos `Notify` e observe o erro produzido pelo compilador.

Depois, corrija a assinatura e confirme que o programa volta a compilar.

4. Refatore um código baseado em *type switch* para uma solução baseada em interface.

Considere o seguinte programa:

```
1 package main
2
3 import "fmt"
4
5 type Circle struct {
6     Radius float64
7 }
8
9 type Rectangle struct {
10     Width float64
11     Height float64
12 }
13
14 func PrintArea(shape any) {
15     switch s := shape.(type) {
16     case Circle:
17         fmt.Println(3.14 * s.Radius * s.Radius)
18     case Rectangle:
19         fmt.Println(s.Width * s.Height)
20     default:
21         fmt.Println("forma desconhecida")
22     }
23 }
```

Refatore o código criando uma interface:

```
type Shape interface {  
    Area() float64  
}
```

Implemente `Area()` em `Circle` e `Rectangle`.

Depois, altere `PrintArea` para receber `Shape` em vez de `any`.

Ao final, reflita sobre a diferença entre as duas versões:

- em qual delas o compilador consegue verificar melhor o uso correto dos tipos?
- em qual delas é mais fácil adicionar uma nova forma geométrica?
- em qual delas o código comunica melhor sua intenção?

O objetivo deste exercício é perceber que `any` e *type switches* são úteis em alguns contextos, mas que interfaces pequenas costumam ser mais idiomáticas quando o comportamento esperado é conhecido.

